

A Survey of Large Language Models

Wayne Xin Zhao, Kun Zhou*, Junyi Li*, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie and Ji-Rong Wen[†]

Abstract—Ever since the Turing Test was proposed in the 1950s, humans have explored the mastering of language intelligence by machine. Language is essentially a complex, intricate system of human expressions governed by grammatical rules. It poses a significant challenge to develop capable artificial intelligence (AI) algorithms for comprehending and grasping a language. As a major approach, *language modeling* has been widely studied for language understanding and generation in the past two decades, evolving from statistical language models to neural language models. Recently, pre-trained language models (PLMs) have been proposed by pre-training Transformer models over large-scale corpora, showing strong capabilities in solving various natural language processing (NLP) tasks. Since the researchers have found that model scaling can lead to an improved model capacity, they further investigate the scaling effect by increasing the parameter scale to an even larger size. Interestingly, when the parameter scale exceeds a certain level, these enlarged language models not only achieve a significant performance improvement, but also exhibit some special abilities (e.g., in-context learning) that are not present in small-scale language models (e.g., BERT). To discriminate the language models in different parameter scales, the research community has coined the term *large language models (LLM)* for the PLMs of significant size (e.g., containing tens or hundreds of billions of parameters). Recently, the research on LLMs has been largely advanced by both academia and industry, and a remarkable progress is the launch of ChatGPT (a powerful AI chatbot developed based on LLMs), which has attracted widespread attention from society. The technical evolution of LLMs has been making an important impact on the entire AI community, which would revolutionize the way how we develop and use AI algorithms. Considering this rapid technical progress, in this survey, we review the recent advances of LLMs by introducing the background, key findings, and mainstream techniques. In particular, we focus on four major aspects of LLMs, namely pre-training, adaptation tuning, utilization, and capacity evaluation. Furthermore, we also summarize the available resources for developing LLMs and discuss the remaining issues for future directions. This survey provides an up-to-date review of the literature on LLMs, which can be a useful resource for both researchers and engineers.

Index Terms—Large Language Models; Emergent Abilities; Adaptation Tuning; Utilization; Alignment; Capacity Evaluation



-
- Version: v18 (major update on March 7, 2026).
 - GitHub link: <https://github.com/RUCAIBox/LLMSurvey>
 - Chinese book link: lmbook-zh.github.io
 - [†] Corresponding Author.
 - * K. Zhou and J. Li contribute equally to this work.
 - The authors are mainly with Gaoling School of Artificial Intelligence and School of Information, Renmin University of China, Beijing, China; Jian-Yun Nie is with DIRO, Université de Montréal, Canada. Contact e-mail: batmanfly@gmail.com
 - The authors of this survey paper reserve all the copyrights of the figures/tables, and any use of these materials for publication purpose must be officially granted by the survey authors.

Contents

1	Introduction	5
2	Overview	7
2.1	Background for LLMs	7
2.2	Technical Evolution of GPT-series Models	10
3	Resources of LLMs	14
3.1	Publicly Available Model Checkpoints or APIs	14
3.2	Commonly Used Corpora for Pre-training	16
3.3	Commonly Used Datasets for Fine-tuning	17
3.3.1	Instruction Tuning Datasets	17
3.3.2	Alignment Datasets	18
3.4	Library Resource	18
4	Pre-training	19
4.1	Data Collection and Preparation	19
4.1.1	Data Source	19
4.1.2	Data Preprocessing	20
4.1.3	Data Scheduling	22
4.1.4	Summary of Data Preparation	23
4.2	Architecture	24
4.2.1	Typical Architectures	24
4.2.2	Detailed Configuration	26
4.2.3	Pre-training Tasks	28
4.2.4	Decoding Strategy	28
4.2.5	Summary and Discussion	29
4.3	Model Training	30
4.3.1	Optimization Setting	30
4.3.2	Scalable Training Techniques	30
5	Post-training of LLMs	32
5.1	Instruction Tuning	32
5.1.1	Formatted Instance Construction	32
5.1.2	Instruction Tuning Strategies	34
5.1.3	The Effect of Instruction Tuning	35
5.1.4	Empirical Analysis for Instruction Tuning	36
5.2	Alignment Tuning	38
5.2.1	Background and Criteria for Alignment	38
5.2.2	Collecting Human Feedback	38
5.2.3	Reinforcement Learning from Human Feedback	39
5.2.4	Alignment without RLHF	41
5.2.5	Remarks on SFT and RLHF	43
5.3	Parameter-Efficient Model Adaptation	44
5.3.1	Parameter-Efficient Fine-Tuning Methods	44
5.3.2	Parameter-Efficient Fine-Tuning on LLMs	45
6	Utilization	45
6.1	Prompting	45
6.1.1	Prompt Creation	45
6.1.2	Prompt Optimization	47
6.2	In-Context Learning	50
6.2.1	ICL Formulation	50
6.2.2	Demonstration Design	51
6.2.3	Underlying Mechanism	52
6.3	Chain-of-Thought Prompting	52
6.3.1	Basic CoT Prompting Approach	52
6.3.2	Improved CoT Prompting Strategies	53
6.3.3	Further Discussion on CoT Prompting	54
6.4	Planning	54
6.4.1	The Overall Framework	55

6.4.2	Plan Generation	55
6.4.3	Feedback Acquisition	55
6.4.4	Plan Refinement	55
7	Capacity and Evaluation	56
7.1	Basic Ability	56
7.1.1	Language Generation	56
7.1.2	Knowledge Utilization	58
7.1.3	Complex Reasoning	60
7.2	Advanced Ability	61
7.2.1	Human Alignment	62
7.2.2	Interaction with External Environment	62
7.2.3	Tool Manipulation	62
7.3	Benchmarks and Evaluation Approaches	63
7.3.1	Comprehensive Evaluation Benchmarks	63
7.3.2	Evaluation Approaches	63
7.4	Empirical Evaluation	65
7.4.1	Experimental Settings	65
7.4.2	Results Analysis and Findings	68
8	Applications	69
8.1	LLM for Research Community	69
8.1.1	LLM for Classic NLP Tasks	69
8.1.2	LLM for Information Retrieval	71
8.1.3	LLM for Recommender Systems	71
8.1.4	Multimodal Large Language Model	72
8.1.5	KG-Enhanced LLM	74
8.1.6	LLM for Evaluation	75
8.2	LLM for Specific Domains	76
9	Advanced Topics	77
9.1	Long Context Modeling	77
9.1.1	Scaling Position Embeddings	77
9.1.2	Adapting Context Window	78
9.1.3	Long Text Data	78
9.2	LLM-empowered Agent	78
9.2.1	Overall Framework.	79
9.2.2	Applications	79
9.2.3	Discussion	79
9.3	Analysis and Optimization for Model Training	80
9.3.1	Estimation of Training Memory Consumption	80
9.3.2	Memory Optimization Methods	81
9.3.3	Efficiency Optimization Methods	81
9.4	Analysis and Optimization for Model Inference	81
9.4.1	Analysis of Inference Efficiency	82
9.4.2	System-level Optimization	83
9.4.3	Algorithm-level Optimization	83
9.5	Model Compression	84
9.5.1	Quantization Methods	84
9.5.2	Other Model Compression Methods	86
9.5.3	Open-source Libraries	86
9.6	Retrieval-Augmented Generation	87
9.7	Hallucination	88
9.7.1	Definition of Hallucination	89
9.7.2	Source of Hallucination	89
9.7.3	Hallucination Detection	90
9.7.4	Hallucination Mitigation	90
9.8	Complex Reasoning	91
9.8.1	Overview and Analysis	91
9.8.2	Construction of Long CoT Data	92
9.8.3	Training Methods	92
9.8.4	Extended Discussion	94

"The limits of my language mean the limits of my world."
—Ludwig Wittgenstein

LANGUAGE is a prominent ability in human beings to express and communicate, which develops in early childhood and evolves over a lifetime [3, 4]. Machines, however, cannot naturally grasp the abilities of understanding and communicating in the form of human language, unless equipped with powerful artificial intelligence (AI) algorithms. It has been a longstanding research challenge to achieve this goal, to enable machines to read, write, and communicate like humans [5].

Technically, *language modeling (LM)* is one of the major approaches to advancing language intelligence of machines. In general, LM aims to model the generative likelihood of word sequences, so as to predict the probabilities of future (or missing) tokens. The research of LM has received extensive attention in the literature, which can be divided into four major development stages:

- **Statistical language models (SLM).** SLMs [6–9] are developed based on *statistical learning* methods that rose in the 1990s. The basic idea is to build the word prediction model based on the Markov assumption, e.g., predicting the next word based on the most recent context. The SLMs with a fixed context length n are also called n -gram language models, e.g., bigram and trigram language models. SLMs have been widely applied to enhance task performance in information retrieval (IR) [10, 11] and natural language processing (NLP) [12–14]. However, they often suffer from the curse of dimensionality: it is difficult to accurately estimate high-order language models since an exponential number of transition probabilities need to be estimated. Thus, specially designed smoothing strategies such as back-off estimation [15] and Good–Turing estimation [16] have been introduced to alleviate the data sparsity problem.

- **Neural language models (NLM).** NLMs [1, 17, 18] characterize the probability of word sequences by neural networks, e.g., multi-layer perceptron (MLP) and recurrent neural networks (RNNs). As a remarkable contribution, the work in [1] introduced the concept of *distributed representation* of words and built the word prediction function conditioned on the aggregated context features (i.e., the distributed word vectors). By extending the idea of learning effective features for text data, a general neural network approach was developed to build a unified, end-to-end solution for various NLP tasks [2]. Furthermore, word2vec [19, 20] was proposed to build a simplified shallow neural network for learning distributed word representations, which were demonstrated to be very effective across a variety of NLP tasks. These studies have initiated the use of language models for representation learning (beyond word sequence modeling), having an important impact on the field of NLP.

- **Pre-trained language models (PLM).** As an early attempt, ELMo [21] was proposed to capture *context-aware word representations* by first pre-training a bidirectional LSTM (biLSTM) network (instead of learning fixed word representations) and then fine-tuning the biLSTM network according to specific downstream tasks. Furthermore, based on the highly parallelizable Transformer architecture [22] with self-attention mechanisms, BERT [23] was proposed by pre-training bidirectional language models with specially designed pre-training tasks on large-scale unlabeled corpora. These pre-trained context-aware word representations are very effective as general-purpose semantic fea-

tures, which have largely raised the performance bar of NLP tasks. This study has inspired a large number of follow-up work, which sets the "*pre-training and fine-tuning*" learning paradigm. Following this paradigm, a great number of studies on PLMs have been developed, introducing either different architectures [24, 25] (e.g., GPT-2 [26] and BART [24]) or improved pre-training strategies [27–29]. In this paradigm, it often requires fine-tuning the PLM for adapting to different downstream tasks.

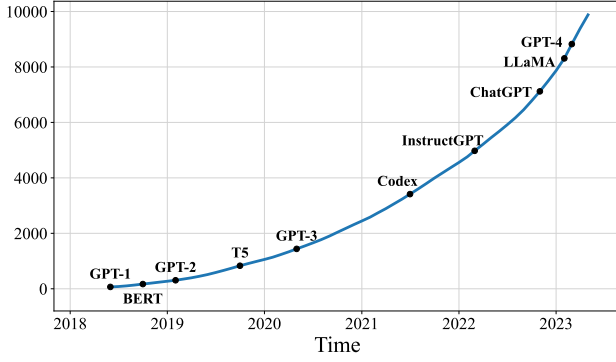
- **Large language models (LLM).** Researchers find that scaling PLM (e.g., scaling model size or data size) often leads to an improved model capacity on downstream tasks (i.e., following the scaling law [30]). A number of studies have explored the performance limit by training an ever larger PLM (e.g., the 175B-parameter GPT-3 and the 540B-parameter PaLM). Although scaling is mainly conducted in model size (with similar architectures and pre-training tasks), these large-sized PLMs display different behaviors from smaller PLMs (e.g., 330M-parameter BERT and 1.5B-parameter GPT-2) and show surprising abilities (called *emergent abilities* [31]) in solving a series of complex tasks. For example, GPT-3 can solve few-shot tasks through *in-context learning*, whereas GPT-2 cannot do well. Thus, the research community coins the term "*large language models (LLM)*"¹ for these large-sized PLMs [32–35], which attract increasing research attention (See Figure 1). A remarkable application of LLMs is *ChatGPT*² that adapts the LLMs from the GPT series for dialogue, which presents an amazing conversation ability with humans. We can observe a sharp increase of the arXiv papers that are related to LLMs after the release of ChatGPT in Figure 1.

As discussed before, language model is not a new technical concept specially for LLMs, but has evolved with the advance of artificial intelligence over the decades. Early language models mainly aim to model and generate text data, while latest language models (e.g., GPT-4) focus on complex task solving. From *language modeling* to *task solving*, it is an important leap in scientific thinking, which is the key to understand the development of language models in the research history. From the perspective of task solving, the four generations of language models have exhibited different levels of model capacities. In Figure 2, we describe the evolution process of language models in terms of the task solving capacity. At first, statistical language models mainly assisted in some specific tasks (e.g., retrieval or speech tasks), in which the predicted or estimated probabilities can enhance the performance of task-specific approaches. Subsequently, neural language models focused on learning task-agnostic representations (e.g., features), aiming to reduce the efforts for human feature engineering. Furthermore, pre-trained language models learned context-aware representations that can be optimized according to downstream tasks. For the latest generation of language model, LLMs are enhanced by exploring the scaling effect on model capacity, which can be considered as general-purpose task solvers. To summarize, in the evolution process, the task scope that can be solved by language models have been greatly extended, and the task performance attained by language models have been significantly enhanced.

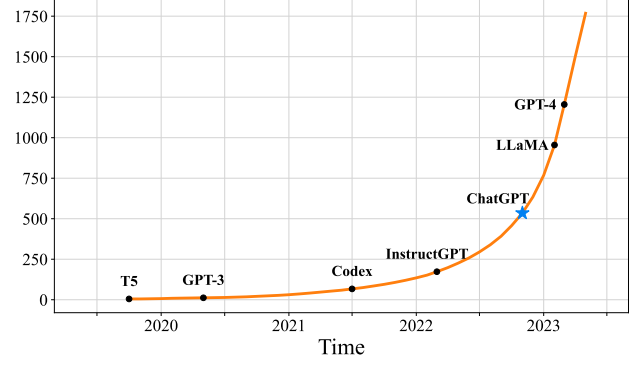
In the existing literature, PLMs have been widely discussed and surveyed [36–39], while LLMs are seldom reviewed in a

1. Note that a LLM is not necessarily more capable than a small PLM, and emergent abilities may not occur in some LLMs.

2. <https://openai.com/blog/chatgpt/>



(a) Query="Language Model"



(b) Query="Large Language Model"

Fig. 1: The trends of the cumulative numbers of arXiv papers that contain the keyphrases “*language model*” (since June 2018) and “*large language model*” (since October 2019), respectively. The statistics are calculated using exact match by querying the keyphrases in title or abstract by months. We set different x-axis ranges for the two keyphrases, because “*language models*” have been explored at an earlier time. We label the points corresponding to important landmarks in the research progress of LLMs. A sharp increase occurs after the release of ChatGPT: the average number of published arXiv papers that contain “*large language model*” in title or abstract goes from 0.40 per day to 8.58 per day (Figure 1(b)).

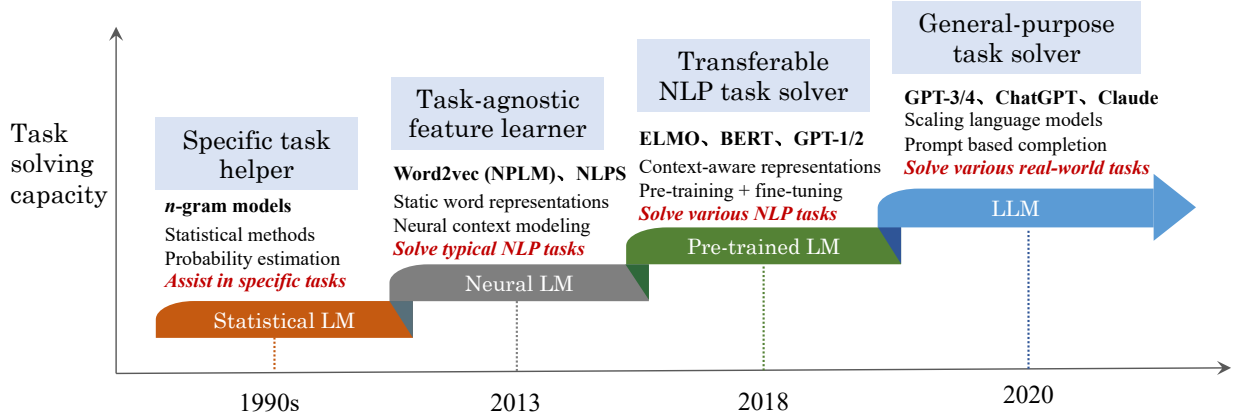


Fig. 2: An evolution process of the four generations of language models (LM) from the perspective of task solving capacity. Note that the time period for each stage may not be very accurate, and we set the time mainly according to the publish date of the most representative studies at each stage. For neural language models, we abbreviate the paper titles of two representative studies to name the two approaches: NPLM [1] (“*A neural probabilistic language model*”) and NLPS [2] (“*Natural language processing (almost) from scratch*”). Due to the space limitation, we don’t list all representative studies in this figure.

systematic way. To motivate our survey, we first highlight three major differences between LLMs and PLMs. First, LLMs display some surprising emergent abilities that may not be observed in previous smaller PLMs. These abilities are key to the performance of language models on complex tasks, making AI algorithms **unprecedentedly** powerful and effective. Second, LLMs would revolutionize the way that humans develop and use AI algorithms. Unlike small PLMs, the major approach to accessing LLMs is through the prompting interface (e.g., GPT-4 API). Humans have to understand how LLMs work and format their tasks in a way that LLMs can follow. Third, the development of LLMs no longer draws a clear distinction between research and engineering. The training of LLMs requires extensive practical experiences in large-scale data processing and distributed parallel training. To develop capable LLMs, researchers have to solve complicated engineering issues, working with engineers or being engineers.

Nowadays, LLMs are posing a significant impact on the AI community, and the advent of ChatGPT and GPT-4 leads to the rethinking of the possibilities of artificial general intelligence (AGI). OpenAI has published a technical article entitled “*Planning for AGI and beyond*”, which discusses the short-term and long-term plans to approach AGI [40], and a more recent paper has argued that GPT-4 might be considered as an early version of an AGI system [41]. The research areas of AI are being revolutionized by the rapid progress of LLMs. In the field of NLP, LLMs can serve as a general-purpose language task solver (to some extent), and the research paradigm has been shifting towards the use of LLMs. In the field of IR, traditional search engines are challenged by the new information seeking way through AI chatbots (i.e., ChatGPT), and *New Bing*³ presents an initial attempt that enhances the search results based on LLMs. In the field of CV, the researchers try to develop ChatGPT-

3. <https://www.bing.com/new>

like vision-language models that can better serve multimodal dialogues [42–45], and GPT-4 [46] has supported multimodal input by integrating the visual information. This new wave of technology would potentially lead to a prosperous ecosystem of real-world applications based on LLMs. For instance, Microsoft 365 is being empowered by LLMs (*i.e.*, Copilot) to automate the office work, and OpenAI supports the use of plugins in ChatGPT for implementing special functions.

Despite the progress and impact, the underlying principles of LLMs are still not well explored. Firstly, it is mysterious why emergent abilities occur in LLMs, instead of smaller PLMs. As a more general issue, there lacks a deep, detailed investigation of the key factors that contribute to the superior abilities of LLMs. It is important to study when and how LLMs obtain such abilities [47]. Although there are some meaningful discussions about this problem [31, 47], more principled investigations are needed to uncover the “secrets” of LLMs. Secondly, it is difficult for the research community to train capable LLMs. Due to the huge demand of computation resources, it is very costly to carry out repetitive, ablating studies for investigating the effect of various strategies for training LLMs. Indeed, LLMs are mainly trained by industry, where many important training details (*e.g.*, data collection and cleaning) are not revealed to the public. Thirdly, it is challenging to align LLMs with human values or preferences. Despite the capacities, LLMs are also likely to produce toxic, fictitious, or harmful contents. It requires effective and efficient control approaches to eliminating the potential risk of the use of LLMs [46].

Faced with both opportunities and challenges, it needs more attention on the research and development of LLMs. In order to provide a basic understanding of LLMs, this survey conducts a literature review of the recent advances in LLMs from four major aspects, including *pre-training* (how to pre-train a capable LLM), *adaptation* (how to effectively adapt pre-trained LLMs for better use), *utilization* (how to use LLMs for solving various downstream tasks) and *capability evaluation* (how to evaluate the abilities of LLMs and existing empirical findings). We thoroughly comb the literature and summarize the key findings, techniques, and methods of LLMs. For this survey, we also create a GitHub project website by collecting the supporting resources for LLMs, at the link <https://github.com/RUCAIBox/LLMSurvey>. We are also aware of several related review articles on PLMs or LLMs [32, 36, 38, 39, 43, 48–54]. These papers either discuss PLMs or some specific (or general) aspects of LLMs. Compared with them, we focus on the techniques and methods to develop and use LLMs and provide a relatively comprehensive reference to important aspects of LLMs.

The remainder of this survey is organized as follows: Section 2 introduces the background for LLMs and the evolution of GPT-series models, followed by the summarization of available resources for developing LLMs in Section 3. Sections 4, 5, 6, and 7 review and summarize the recent progress from the four aspects of pre-training, adaptation, utilization, and capacity evaluation, respectively. Then, Section 8 discusses the practical guide for prompt design, and Section 9 reviews the applications of LLMs in several representative domains. Finally, we conclude the survey in Section 10 by summarizing the major findings and discuss the remaining issues for future work.

2 Overview

In this section, we present an overview about the background of LLMs and then summarize the technical evolution of the GPT-series models.

2.1 Background for LLMs

Typically, *large language models* (LLMs) refer to Transformer language models that contain hundreds of billions (or more) of parameters⁴, which are trained on massive text data [32], such as GPT-3 [55], PaLM [56], Galactica [35], and LLaMA [57]. LLMs exhibit strong capacities to understand natural language and solve complex tasks (via text generation). To have a quick understanding of how LLMs work, this part introduces the basic background for LLMs, including scaling laws, emergent abilities and key techniques.

Formulation of Scaling Laws for LLMs. Currently, LLMs are mainly built upon the Transformer architecture [22], where multi-head attention layers are stacked in a very deep neural network. Existing LLMs adopt similar Transformer architectures and pre-training objectives (*e.g.*, language modeling) as small language models. However, LLMs significantly extend the model size, data size, and total compute (orders of magnification). Extensive research has shown that scaling can largely improve the model capacity of LLMs [26, 55, 56]. Thus, it is useful to establish a quantitative approach to characterizing the scaling effect. Next, we introduce two representative scaling laws for Transformer language models [30, 34].

• **KM scaling law**⁵. In 2020, Kaplan et al. [30] (the OpenAI team) firstly proposed to model the power-law relationship of model performance with respect to three major factors, namely model size (N), dataset size (D), and the amount of training compute (C), for neural language models. Given a compute budget c , they empirically presented three basic formulas for the scaling law⁶:

$$\begin{aligned} L(N) &= \left(\frac{N_c}{N}\right)^{\alpha_N}, & \alpha_N &\sim 0.076, N_c \sim 8.8 \times 10^{13} \\ L(D) &= \left(\frac{D_c}{D}\right)^{\alpha_D}, & \alpha_D &\sim 0.095, D_c \sim 5.4 \times 10^{13} \\ L(C) &= \left(\frac{C_c}{C}\right)^{\alpha_C}, & \alpha_C &\sim 0.050, C_c \sim 3.1 \times 10^8 \end{aligned} \quad (1)$$

where $L(\cdot)$ denotes the cross entropy loss in nats, and a follow-up study [58] from OpenAI has shown that the language modeling loss can be decomposed into two parts, namely *irreducible loss* (the entropy of the true data distribution) and *reducible loss* (an estimate of the KL divergence between the true and model distributions). The three laws were derived by fitting the model

4. In existing literature, there is no formal consensus on the minimum parameter scale for LLMs, since the model capacity is also related to data size and total compute. In this survey, we take a slightly loose definition of LLMs, and mainly focus on discussing language models with a model size larger than 10B.

5. Since there was not a model trained following this law in the original paper, we took the last names of the two co-first authors to name this scaling law.

6. Here, N_c , D_c and C_c are measured in the number of non-embedding parameters, the number of training tokens and the number of FP-days, respectively. According to the original paper [30], C_c and C should be denoted by C_c^{min} and C_{min} , corresponding to the optimal use of compute. We use the simplified notations for ease of discussions.

performance with varied data sizes (22M to 23B tokens), model sizes (768 to 1.5B non-embedding parameters) and training compute, under some assumptions (e.g., the analysis of one factor should be not bottlenecked by the other two factors). They showed that the model performance has a strong dependence relation on the three factors.

- **Chinchilla scaling law.** As another representative study, Hoffmann et al. [34] (the Google DeepMind team) proposed an alternative form for scaling laws to instruct the compute-optimal training for LLMs. They conducted rigorous experiments by varying a larger range of model sizes (70M to 16B) and data sizes (5B to 500B tokens), and fitted a similar scaling law yet with different coefficients as below [34]:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (2)$$

where $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$ and $\beta = 0.28$. By optimizing the loss $L(N, D)$ under the constraint $C \approx 6ND$, they showed that the optimal allocation of compute budget to model size and data size can be derived as follows:

$$N_{opt}(C) = G \left(\frac{C}{6} \right)^a, \quad D_{opt}(C) = G^{-1} \left(\frac{C}{6} \right)^b, \quad (3)$$

where $a = \frac{\alpha}{\alpha+\beta}$, $b = \frac{\beta}{\alpha+\beta}$ and G is a scaling coefficient that can be computed by A , B , α and β . As analyzed in [34], given an increase in compute budget, the KM scaling law favors a larger budget allocation in model size than the data size, while the Chinchilla scaling law argues that the two sizes should be increased in equal scales, i.e., having similar values for a and b in Equation (3).

Discussion on Scaling Laws. After introducing the formulations, we continue to discuss scaling law in the following two aspects, to enhance its understanding:

- **Predictable scaling.** In practice, scaling law can be used to instruct the training of LLMs, and it has been proven feasible to reliably estimate the performance of larger models based on that of smaller models, called *predictable scaling* [46]. The benefits of predictable scaling for training LLMs are mainly twofold. Firstly, for large models, it is infeasible to rigorously examine various training tricks or variants, and it would be very helpful if experiences gained from small models could also apply to large models. For instance, small proxy models can be trained to find the optimal schedule of the data mixture for large models [59]. Secondly, the training of large-scale models takes a long time, often suffering from issues such as training loss spike, and scaling law can be employed to monitor the training status of LLMs, e.g., identifying abnormal performance at an early time. Despite that scaling law characterizes a smooth trend of performance increase (or loss decrease), it also indicates that *diminishing returns*⁷ might occur as model scaling. An empirical study [58] from the OpenAI team has shown that representation quality or semantic content can still effectively improve even if approaching the point of diminishing returns (i.e., approaching the irreducible loss) [58]. This finding suggests that training large models are promising for improving the performance of downstream tasks. To further explore scaling effect, a potential issue is that the amount of available data for training LLMs is actually limited. With the ever-increasing model scale, the public text data would be soon

“exhausted” for LLMs [60]. Thus, it will be meaningful to study how scaling laws apply to a data-constrained regime [61], where data repetition or augmentation might be useful to alleviate data scarcity.

- **Task-level predictability.** Existing research of scaling laws are mostly conducted in terms of language modeling loss (e.g., per-token cross-entropy loss in nats [30]), while in practice we are more concerned about the performance of LLMs on actual tasks. Thus, a basic problem is that how the decrease of language modeling loss translates into the improvement of task performance [58]. Intuitively, a model with a smaller language modeling loss tends to yield a better performance on downstream tasks, since language modeling loss can be considered as a general measure of the overall model capacity. GPT-4 [46] has reported that some capabilities (e.g., coding ability) can be accurately predicted via scaling law. Despite that, readers should be aware that a direct decrease in language modeling loss does not always indicate an improvement of model performance on downstream tasks. Specially, the phenomenon of *inverse scaling* would occur for some tasks, where task performance surprisingly becomes worse as the language modeling loss decreases [62]. Overall, it is more difficult to explore and characterize task-level scaling laws, since it might be also dependent on task-related information (task metric, task difficulty, etc.). Furthermore, some capacities (e.g., in-context learning [55]) are unpredictable according to the scaling law, which can be observed only when the model size exceeds a certain level (as discussed below).

Emergent Abilities of LLMs. In the literature [31], *emergent abilities* of LLMs are formally defined as “the abilities that are not present in small models but arise in large models”, which is one of the most prominent features that distinguish LLMs from previous PLMs. It further introduces a notable characteristic when emergent abilities occur [31]: performance rises significantly above random when the scale reaches a certain level. By analogy, such an emergent pattern has close connections with the phenomenon of *phase transition* in physics [31, 63]. In principle, emergent abilities can be defined in relation to some complex tasks [31, 64], while we are more concerned with general abilities that can be applied to solve a variety of tasks. Here, we briefly introduce three typical emergent abilities for LLMs and representative models that possess such an ability⁸.

- **In-context learning.** The in-context learning (ICL) ability is formally introduced by GPT-3 [55]: assuming that the language model has been provided with a natural language instruction and/or several task demonstrations, it can generate the expected output for the test instances by completing the word sequence of input text, without requiring additional training or gradient update⁹. Among the GPT-series models, the 175B GPT-3 model exhibited a strong ICL ability in general, but not the GPT-1 and GPT-2 models. Such an ability also depends on the specific downstream task. For example, the ICL ability can emerge on the arithmetic tasks (e.g., the 3-digit addition and subtraction) for

8. It is difficult to accurately examine the critical size for emergent abilities of LLMs (i.e., the minimum size to possess an ability), since it might vary for different models or tasks. Also, existing studies often test emergent abilities on very limited model sizes for a specific LLM. For example, PaLM is often tested with three sizes of 8B, 62B and 540B. It is unclear about the model performance of the untested sizes.

9. In a recent study [65], it also shows that in-context learning implicitly performs meta-optimization through the attention mechanism.

7. https://en.wikipedia.org/wiki/Diminishing_returns

the 13B GPT-3, but 175B GPT-3 even cannot work well on the Persian QA task [31].

- *Instruction following.* By fine-tuning with a mixture of multi-task datasets formatted via natural language descriptions (called *instruction tuning*), LLMs are shown to perform well on unseen tasks that are also described in the form of instructions [28, 66, 67]. With instruction tuning, LLMs are enabled to follow the task instructions for new tasks without using explicit examples, thus having an improved generalization ability. According to the experiments in [67], instruction-tuned LaMDA-PT [68] started to significantly outperform the untuned one on unseen tasks when the model size reached 68B, but not for 8B or smaller model sizes. A recent study [69] found that a model size of 62B is at least required for PaLM to perform well on various tasks in four evaluation benchmarks (*i.e.*, MMLU, BBH, TyDiQA and MGSM), though a much smaller size might suffice for some specific tasks (*e.g.*, MMLU).

- *Step-by-step reasoning.* For small language models, it is usually difficult to solve complex tasks that involve multiple reasoning steps, *e.g.*, mathematical word problems. In contrast, with the chain-of-thought (CoT) prompting strategy [33], LLMs can solve such tasks by utilizing the prompting mechanism that involves intermediate reasoning steps for deriving the final answer. This ability is speculated to be potentially obtained by training on code [33, 47]. An empirical study [33] has shown that CoT prompting can bring performance gains (on arithmetic reasoning benchmarks) when applied to PaLM and LaMDA variants with a model size larger than 60B, while its advantage over the standard prompting becomes more evident when the model size exceeds 100B. Furthermore, the performance improvement with CoT prompting seems to be also varied for different tasks, *e.g.*, GSM8K > MAWPS > SWAMP for PaLM [33].

How Emergent Abilities Relate to Scaling Laws. In existing literature [30, 31, 34], scaling laws and emergent abilities provide two perspectives to understand the advantage of large models over small models. In general, scaling law (often measured by *language modeling loss*) describes predictable performance relation with the potential effect of diminishing returns, while emergent abilities (often measured by *task performance*) are unpredictable but very profitable once such abilities actually emerge. Since the two perspectives reflect different performance trends (continuous improvement *v.s.* sharp performance leap), they might lead to misaligned findings or observations. There are also extensive debates on the rationality of emergent abilities. A popular speculation is that emergent abilities might be partially attributed to the evaluation setting for special tasks (*e.g.*, the discontinuous evaluation metrics) [70, 71]: when evaluation metrics are altered accordingly, the sharpness of the emergent ability curve would disappear. However, the performance of LLMs on most tasks are perceived by users naturally in a discontinuous way. For instance, end users prefer a reliable code generated by LLMs that can successfully pass the test case, but are less interested in selecting a better code with fewer errors between two failed ones. More recently, a study [72] proposes a new evaluation setting that can enlarge the resolution of task metrics, making task performance more predictable. Despite these efforts, more fundamental research (*e.g.*, *grokking*¹⁰) about the working

mechanism of LLMs is still in need to understand the emergence of certain abilities. The subtle relation between scaling law and emergent abilities can be explained by analogy with the ability acquisition of human¹¹. Take the speaking ability as an example. For children, language development (especially infants) can be also considered as a multi-level process where “emergent abilities” occur. Specially, the language ability would relatively stable within a time interval, but qualitative change only occurs when evolving into another ability level (*e.g.*, from speaking simple words to speaking simple sentences). Such a learning process is essentially not *smooth* and *stable* (*i.e.*, language ability does not develop at a constant rate over time), though a child actually grows every day. It is interesting that young parents would be often surprised by unexpected progress of the speaking ability exhibited by their babies.

Key Techniques for LLMs. It has been a long way that LLMs evolve into the current state: *general* and *capable* learners. In the development process, a number of important techniques are proposed, which largely improve the capacity of LLMs. Here, we briefly list several important techniques that (potentially) lead to the success of LLMs, as follows.

- *Scaling.* As discussed in previous parts, there exists an evident scaling effect in Transformer language models: larger model/data sizes and more training compute typically lead to an improved model capacity [30, 34]. As two representative models, GPT-3 and PaLM explored the scaling limits by increasing the model size to 175B and 540B, respectively. Since compute budget is usually limited, scaling laws can be further employed to conduct a more compute-efficient allocation of the compute resources. For example, Chinchilla (with more training tokens) outperforms its counterpart model Gopher (with a larger model size) by increasing the data scale with the same compute budget [34]. In addition, data scaling should be with careful cleaning process, since the quality of pre-training data plays a key role in the model capacity.

- *Training.* Due to the huge model size, it is very challenging to successfully train a capable LLM. Distributed training algorithms are needed to learn the network parameters of LLMs, in which various parallel strategies are often jointly utilized. To support distributed training, several optimization frameworks have been released to facilitate the implementation and deployment of parallel algorithms, such as DeepSpeed [74] and Megatron-LM [75–77]. Also, optimization tricks are also important for training stability and model performance, *e.g.*, restart to overcome training loss spike [56] and mixed precision training [78]. More recently, GPT-4 [46] proposes to develop special infrastructure and optimization methods that reliably predict the performance of large models with much smaller models.

- *Ability eliciting.* After being pre-trained on large-scale corpora, LLMs are endowed with potential abilities as general-purpose task solvers. These abilities might not be explicitly exhibited when LLMs perform some specific tasks. As the technical approach, it is useful to design suitable task instructions or specific in-context learning strategies to elicit such abilities. For instance, chain-of-thought prompting has been shown to be useful to solve complex reasoning tasks by including intermediate reasoning steps. Furthermore, we can perform instruction tuning on LLMs with task descriptions expressed in natural language, for

10. Grokking refers that “a pattern in the data, improving generalization performance from random chance level to perfect generalization”, quoted from the original paper [73].

11. This explanation is only for ease of understanding, and there is not direct evidence to connect the two points.

improving the generalizability of LLMs on unseen tasks. These eliciting techniques mainly correspond to the emergent abilities of LLMs, which may not show the same effect on small language models.

- *Alignment tuning.* Since LLMs are trained to capture the data characteristics of pre-training corpora (including both high-quality and low-quality data), they are likely to generate toxic, biased, or even harmful content for humans. It is necessary to align LLMs with human values, e.g., *helpful*, *honest*, and *harmless*. For this purpose, InstructGPT [66] designs an effective tuning approach that enables LLMs to follow the expected instructions, which utilizes the technique of *reinforcement learning with human feedback* [66, 79]. It incorporates human in the training loop with elaborately designed labeling strategies. ChatGPT is indeed developed on a similar technique to InstructGPT, which shows a strong alignment capacity in producing high-quality, harmless responses, e.g., rejecting to answer insulting questions.

- *Tools manipulation.* In essence, LLMs are trained as text generators over massive plain text corpora, thus performing less well on the tasks that are not best expressed in the form of text (e.g., numerical computation). In addition, their capacities are also limited to the pre-training data, e.g., the inability to capture up-to-date information. To tackle these issues, a recently proposed technique is to employ external tools to compensate for the deficiencies of LLMs [80, 81]. For example, LLMs can utilize the calculator for accurate computation [80] and employ search engines to retrieve unknown information [81]. More recently, ChatGPT has enabled the mechanism of using external plugins (existing or newly created apps)¹², which are by analogy with the “eyes and ears” of LLMs. Such a mechanism can broadly expand the scope of capacities for LLMs.

In addition, many other factors (e.g., the upgrade of hardware) also contribute to the success of LLMs. Currently, we limit our discussion to the major technical approaches and key findings for developing LLMs.

2.2 Technical Evolution of GPT-series Models

Due to the excellent capacity in communicating with humans, ChatGPT has ignited the excitement of the AI community since its release. ChatGPT is developed based on the powerful GPT model with specially optimized conversation capacities. Considering the ever-growing interest in ChatGPT and GPT models, we add a special discussion about the technical evolution of the GPT-series models, to briefly summarize the progress how they have been developed in the past years. Meanwhile, we drew a schematic diagram depicting the technological evolution of the GPT-series models in Figure 4. The basic principle underlying GPT models is to compress the world knowledge into the decoder-only Transformer model by language modeling, such that it can recover (or memorize) the semantics of world knowledge and serve as a general-purpose task solver. Two key points to the success are (I) training decoder-only Transformer language models that can *accurately predict the next word* and (II) *scaling up the size of language models*. Overall, the research of OpenAI on LLMs can be roughly divided into the following stages¹³.

12. <https://openai.com/blog/chatgpt-plugins>

13. Note that the discussion of this part can be somewhat subjective. The overall viewpoints and summaries are made based on the understanding of the survey authors by reading the papers, blog articles, interview reports and APIs released by OpenAI.

Early Explorations. According to one interview with Ilya Sutskever¹⁴ (a co-founder and chief scientist of OpenAI), the idea of approaching intelligent systems with language models was already explored in the early days of OpenAI, while it was attempted with recurrent neural networks (RNN) [121]. With the advent of Transformer, OpenAI developed two initial GPT models, namely GPT-1 [122] and GPT-2 [26], which can be considered as the foundation to more powerful models subsequently i.e., GPT-3 and GPT-4.

- *GPT-1.* In 2017, the Transformer model [22] was introduced by Google, and the OpenAI team quickly adapted their language modeling work to this new neural network architecture. They released the first GPT model in 2018, i.e., GPT-1 [122], and coined the abbreviation term *GPT* as the model name, standing for *Generative Pre-Training*. GPT-1 was developed based on a generative, decoder-only Transformer architecture, and adopted a hybrid approach of unsupervised pre-training and supervised fine-tuning. GPT-1 has set up the core architecture for the GPT-series models and established the underlying principle to model natural language text, i.e., predicting the next word.

- *GPT-2.* Following a similar architecture of GPT-1, GPT-2 [26] increased the parameter scale to 1.5B, which was trained with a large webpage dataset WebText. As claimed in the paper of GPT-2, it sought to perform tasks via unsupervised language modeling, without explicit fine-tuning using labeled data. To motivate the approach, they introduced a probabilistic form for multi-task solving, i.e., $p(\text{output}|\text{input}, \text{task})$ (similar approaches have been adopted in [123]), which predicts the output conditioned on the input and task information. To model this conditional probability, language text can be naturally employed as a unified way to format input, output and task information. In this way, the process of solving a task can be cast as a word prediction problem for generating the solution text. Further, they introduced a more formal claim for this idea: “Since the (task-specific) supervised objective is the same as the unsupervised (language modeling) objective but only evaluated on a subset of the sequence, the global minimum of the unsupervised objective is also the global minimum of the supervised objective (for various tasks)” [26]¹⁵. A basic understanding of this claim is that each (NLP) task can be considered as the word prediction problem based on a subset of the world text. Thus, unsupervised language modeling could be capable in solving various tasks, if it was trained to have sufficient capacity in recovering the world text. These early discussion in GPT-2’s paper echoed in the interview of Ilya Sutskever by Jensen Huang: “What the neural network learns is some representation of the process that produced the text. This text is actually a projection of the world...the more accurate you are in predicting the next word, the higher the fidelity, the more resolution you get in this process...”¹⁶.

Capacity Leap. Although GPT-2 is intended to be an “unsupervised multitask learner”, it overall has an inferior performance compared with supervised fine-tuning state-of-the-art methods. Because it has a relatively small model size, it has been widely fine-tuned in downstream tasks, especially the dialog tasks [124, 125]. Based on GPT-2, GPT-3 demonstrates a key

14. <https://hackernoon.com/an-interview-with-ilya-sutskever-co-founder-of-openai>

15. To better understand this sentence, we put some explanation words in parentheses.

16. <https://lifearchitect.ai/ilya/>

TABLE 1: Statistics of large language models (having a size larger than 10B in this survey) in recent years, including the capacity evaluation, pre-training data scale (either in the number of tokens or storage size) and hardware resource costs. In this table, we only include LLMs with a public paper about the technical details. Here, “Release Time” indicates the date when the corresponding paper was officially released. “Publicly Available” means that the model checkpoints can be publicly accessible while “Closed Source” means the opposite. “Adaptation” indicates whether the model has been with subsequent fine-tuning: IT denotes instruction tuning and RLHF denotes reinforcement learning with human feedback. “Evaluation” indicates whether the model has been evaluated with corresponding abilities in their original paper: ICL denotes in-context learning and CoT denotes chain-of-thought. “*” denotes the largest publicly available version.

	Model	Release Time	Size (B)	Base Model	Adaptation		Pre-train Data Scale	Latest Data Timestamp	Hardware (GPUs / TPUs)	Training Time	Evaluation	
					IT	RLHF					ICL	CoT
Publicly Available	T5 [82]	Oct-2019	11	-	-	-	1T tokens	Apr-2019	1024 TPU v3	-	✓	-
	mT5 [83]	Oct-2020	13	-	-	-	1T tokens	-	-	-	✓	-
	PanGu- α [84]	Apr-2021	13*	-	-	-	1.1TB	-	2048 Ascend 910	-	✓	-
	CPM-2 [85]	Jun-2021	198	-	-	-	2.6TB	-	-	-	-	-
	T0 [28]	Oct-2021	11	T5	✓	-	-	-	512 TPU v3	27 h	✓	-
	CodeGen [86]	Mar-2022	16	-	-	-	577B tokens	-	-	-	✓	-
	GPT-NeoX-20B [87]	Apr-2022	20	-	-	-	825GB	-	96 40G A100	-	✓	-
	Tk-Instruct [88]	Apr-2022	11	T5	✓	-	-	-	256 TPU v3	4 h	✓	-
	UL2 [89]	May-2022	20	-	-	-	1T tokens	Apr-2019	512 TPU v4	-	✓	✓
	OPT [90]	May-2022	175	-	-	-	180B tokens	-	992 80G A100	-	✓	-
	NLLB [91]	Jul-2022	54.5	-	-	-	-	-	-	-	✓	-
	CodeGeeX [92]	Sep-2022	13	-	-	-	850B tokens	-	1536 Ascend 910	60 d	✓	-
	GLM [93]	Oct-2022	130	-	-	-	400B tokens	-	768 40G A100	60 d	✓	-
	Flan-T5 [69]	Oct-2022	11	T5	✓	-	-	-	-	-	✓	✓
	BLOOM [78]	Nov-2022	176	-	-	-	366B tokens	-	384 80G A100	105 d	✓	-
	mT0 [94]	Nov-2022	13	mT5	✓	-	-	-	-	-	✓	-
	Galactica [35]	Nov-2022	120	-	-	-	106B tokens	-	-	-	✓	✓
	BLOOMZ [94]	Nov-2022	176	BLOOM	✓	-	-	-	-	-	✓	-
	OPT-IML [95]	Dec-2022	175	OPT	✓	-	-	-	128 40G A100	-	✓	✓
	LLaMA [57]	Feb-2023	65	-	-	-	1.4T tokens	-	2048 80G A100	21 d	✓	-
	Pythia [96]	Apr-2023	12	-	-	-	300B tokens	-	256 40G A100	-	✓	-
	CodeGen2 [97]	May-2023	16	-	-	-	400B tokens	-	-	-	✓	-
	StarCoder [98]	May-2023	15.5	-	-	-	1T tokens	-	512 40G A100	-	✓	✓
	LLaMA2 [99]	Jul-2023	70	-	✓	✓	2T tokens	-	2000 80G A100	-	✓	-
	Baichuan2 [100]	Sep-2023	13	-	✓	✓	2.6T tokens	-	1024 A800	-	✓	-
	QWEN [101]	Sep-2023	14	-	✓	✓	3T tokens	-	-	-	✓	-
	FLM [102]	Sep-2023	101	-	✓	-	311B tokens	-	192 A800	22 d	✓	-
	Skywork [103]	Oct-2023	13	-	-	-	3.2T tokens	-	512 80G A800	-	✓	-
Closed Source	GPT-3 [55]	May-2020	175	-	-	-	300B tokens	-	-	-	✓	-
	GShard [104]	Jun-2020	600	-	-	-	1T tokens	-	2048 TPU v3	4 d	-	-
	Codex [105]	Jul-2021	12	GPT-3	-	-	100B tokens	May-2020	-	-	✓	-
	ERNIE 3.0 [106]	Jul-2021	10	-	-	-	375B tokens	-	384 V100	-	✓	-
	Jurassic-1 [107]	Aug-2021	178	-	-	-	300B tokens	-	800 GPU	-	✓	-
	HyperCLOVA [108]	Sep-2021	82	-	-	-	300B tokens	-	1024 A100	13.4 d	✓	-
	FLAN [67]	Sep-2021	137	LaMDA-PT	✓	-	-	-	128 TPU v3	60 h	✓	-
	Yuan 1.0 [109]	Oct-2021	245	-	-	-	180B tokens	-	2128 GPU	-	✓	-
	Anthropic [110]	Dec-2021	52	-	-	-	400B tokens	-	-	-	✓	-
	WebGPT [81]	Dec-2021	175	GPT-3	-	✓	-	-	-	-	✓	-
	Gopher [64]	Dec-2021	280	-	-	-	300B tokens	-	4096 TPU v3	920 h	✓	-
	ERNIE 3.0 Titan [111]	Dec-2021	260	-	-	-	-	-	-	-	✓	-
	GLaM [112]	Dec-2021	1200	-	-	-	280B tokens	-	1024 TPU v4	574 h	✓	-
	LaMDA [68]	Jan-2022	137	-	-	-	768B tokens	-	1024 TPU v3	57.7 d	-	-
	MT-NLG [113]	Jan-2022	530	-	-	-	270B tokens	-	4480 80G A100	-	✓	-
	AlphaCode [114]	Feb-2022	41	-	-	-	967B tokens	Jul-2021	-	-	-	-
	InstructGPT [66]	Mar-2022	175	GPT-3	✓	✓	-	-	-	-	✓	-
	Chinchilla [34]	Mar-2022	70	-	-	-	1.4T tokens	-	-	-	✓	-
	PaLM [56]	Apr-2022	540	-	-	-	780B tokens	-	6144 TPU v4	-	✓	✓
	AlexaTM [115]	Aug-2022	20	-	-	-	1.3T tokens	-	128 A100	120 d	✓	✓
	Sparrow [116]	Sep-2022	70	-	-	✓	-	-	64 TPU v3	-	✓	-
	WeLM [117]	Sep-2022	10	-	-	-	300B tokens	-	128 A100 40G	24 d	✓	-
	U-PaLM [118]	Oct-2022	540	PaLM	-	-	-	-	512 TPU v4	5 d	✓	✓
	Flan-PaLM [69]	Oct-2022	540	PaLM	✓	-	-	-	512 TPU v4	37 h	✓	✓
	Flan-U-PaLM [69]	Oct-2022	540	U-PaLM	✓	-	-	-	-	-	✓	✓
	GPT-4 [46]	Mar-2023	-	-	✓	✓	-	-	-	-	✓	✓
	PanGu- Σ [119]	Mar-2023	1085	PanGu- α	-	-	329B tokens	-	512 Ascend 910	100 d	✓	-
	PaLM2 [120]	May-2023	16	-	✓	-	100B tokens	-	-	-	✓	✓

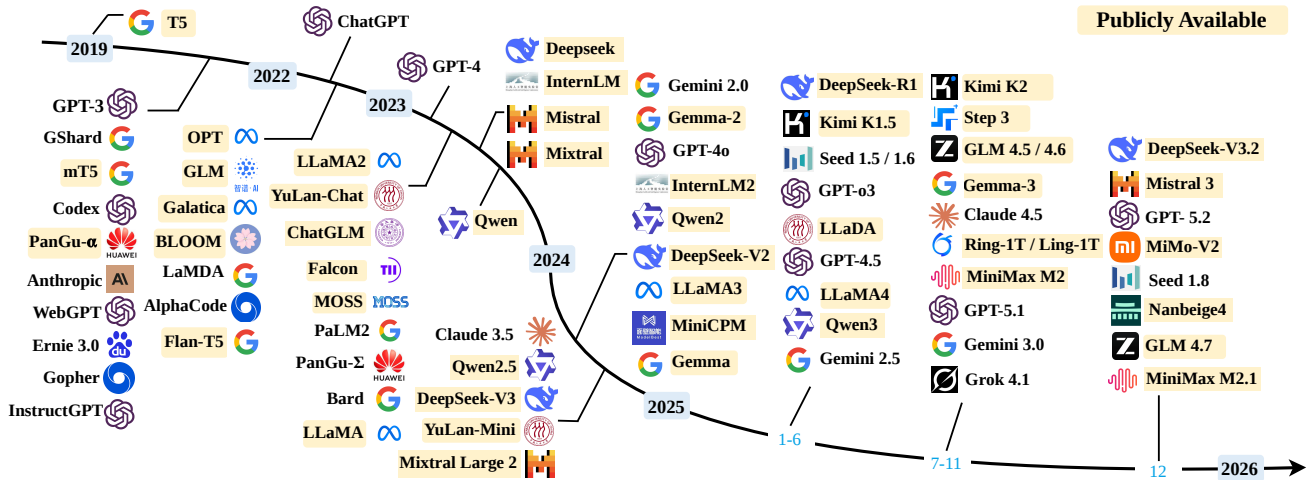


Fig. 3: A timeline of representative LLMs released in recent years. Models with publicly available checkpoints are highlighted in yellow.

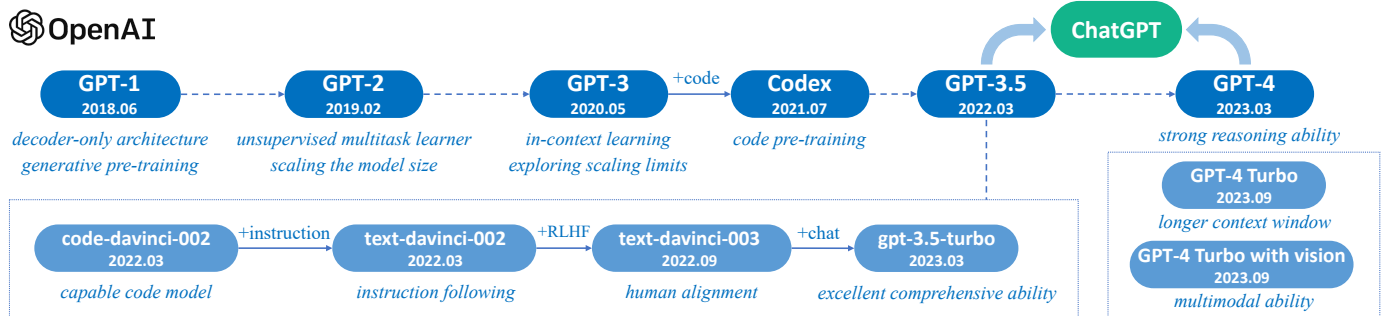


Fig. 4: A brief illustration for the technical evolution of GPT-series models. We plot this figure mainly based on the papers, blog articles and official APIs from OpenAI. Here, *solid lines* denote that there exists an explicit evidence (e.g., the official statement that a new model is developed based on a base model) on the evolution path between two models, while *dashed lines* denote a relatively weaker evolution relation.

capacity leap by scaling of the (nearly same) generative pre-training architecture.

- **GPT-3.** GPT-3 [55] was released in 2020, which scaled the model parameters to an ever larger size of 175B. In the GPT-3’s paper, it formally introduced the concept of in-context learning (ICL)¹⁷, which utilizes LLMs in a few-shot or zero-shot way. ICL can teach (or instruct) LLMs to understand the tasks in the form of natural language text. With ICL, the pre-training and utilization of LLMs converge to the same language modeling paradigm: pre-training predicts the following text sequence conditioned on the context, while ICL predicts the correct task solution, which can be also formatted as a text sequence, given the task description and demonstrations. GPT-3 not only demonstrates very excellent performance in a variety of NLP tasks, but also on a number of specially designed tasks that require the abilities of reasoning or domain adaptation. Although the GPT-3’s paper does not explicitly discuss the emergent abilities of LLMs, we can observe large performance leap that might transcend the basic scaling law [30], e.g., larger models have significantly stronger ICL ability (illustrated in the original Figure 1.2 of the GPT-3’s paper [55]). Overall, GPT-3 can be

17. GPT-2 essentially used ICL for unsupervised task learning, though it wasn’t called ICL at that time.

viewed as a remarkable landmark in the journey evolving from PLMs to LLMs. It has empirically proved that scaling the neural networks to a significant size can lead to a huge increase in model capacity.

Capacity Enhancement. Due to the strong capacities, GPT-3 has been the base model to develop even more capable LLMs for OpenAI. Overall, OpenAI has explored two major approaches to further improving the GPT-3 model, *i.e.*, training on code data and alignment with human preference, which are detailed as follows.

- **Training on code data.** A major limitation of the original GPT-3 model (pre-trained on plain text) lies in the lack of the reasoning ability on complex tasks, e.g., completing the code and solving math problems. To enhance this ability, Codex [105] was introduced by OpenAI in July 2021, which was a GPT model fine-tuned on a large corpus of GitHub code. It demonstrated that Codex can solve very difficult programming problems, and also lead to a significant performance improvement in solving math problems [126]. Further, a contrastive approach [127] to training text and code embedding was reported in January 2022, which was shown to improve a series of related tasks (*i.e.*, linear-probe classification, text search and code search). Actually, the GPT-

3.5 models are developed based on a code-based GPT model (*i.e.*, code-davinci-002), which indicates that training on code data is a very useful practice to improve the model capacity of GPT models, especially the reasoning ability. Furthermore, there is also a speculation that training on code data can greatly increase the chain-of-thought prompting abilities of LLMs [47], while it is still worth further investigation with more thorough verification.

- **Human alignment.** The related research of human alignment can be dated back to the year 2017 (or earlier) for OpenAI: a blog article entitled “learning from human preferences”¹⁸ was posted on the OpenAI blog describing a work that applied reinforcement learning (RL) to learn from the *preference comparisons* annotated by humans [79] (similar to the *reward training* step in the aligning algorithm of InstructGPT in Figure 12). Shortly after the release of this RL paper [79], the paper of the Proximal Policy Optimization (PPO) [128] was published in July 2017, which now has been the foundational RL algorithm for learning from human preferences [66]. Later in January 2020, GPT-2 was fine-tuned using the aforementioned RL algorithms [79, 128], which leveraged human preferences to improve the capacities of GPT-2 on NLP tasks. In the same year, another work [129] trained a summarization model for optimizing human preferences in a similar way. Based on these prior work, InstructGPT [66] was proposed in January 2022 to improve the GPT-3 model for human alignment, which formally established a three-stage *reinforcement learning from human feedback (RLHF)* algorithm. Note that it seems that the wording of “*instruction tuning*” has seldom been used in OpenAI’s paper and documentation, which is substituted by *supervised fine-tuning on human demonstrations* (*i.e.*, the first step of the RLHF algorithm [66]). In addition to improving the instruction following capacity, the RLHF algorithm is particularly useful to mitigate the issues of generating harm or toxic content for LLMs, which is key to the safe deployment of LLMs in practice. OpenAI describes their approach to alignment research in a technical article [130], which has summarized three promising directions: “training AI systems to use human feedback, to assist human evaluation and to do alignment research”.

These enhancement techniques lead to the improved GPT-3 models with stronger capacities, which are called GPT-3.5 models by OpenAI (see the discussion about the OpenAI API in Section 3.1).

The Milestones of Language Models. Based on all the exploration efforts, two major milestones have been achieved by OpenAI, namely ChatGPT [131] and GPT-4 [46], which have largely raised the capacity bar of existing AI systems.

- **ChatGPT.** In November 2022, OpenAI released the conversation model ChatGPT, based on the GPT models (GPT-3.5 and GPT-4). As the official blog article introduced [131], ChatGPT was trained in a similar way as InstructGPT (called “a sibling model to InstructGPT” in the original post), while specially optimized for dialogue. They reported a difference between the training of ChatGPT and InstructGPT in the data collection setup: human-generated conversations (playing both the roles of user and AI) are combined with the InstructGPT dataset in a dialogue format for training ChatGPT. ChatGPT exhibited superior capacities in communicating with humans: possessing a vast store of knowledge, skill at reasoning on mathematical problems, tracing the context accurately in multi-turn dialogues,

and aligning well with human values for safe use. Later on, the plugin mechanism has been supported in ChatGPT, which further extends the capacities of ChatGPT with existing tools or apps. So far, it seems to be the ever most powerful chatbot in the AI history. The launch of ChatGPT has a significant impact on the AI research in the future, which sheds light on the exploration of human-like AI systems.

- **GPT-4.** As another remarkable progress, GPT-4 [46] was released in March 2023, which extended the text input to multimodal signals. Overall, GPT-4 has stronger capacities in solving complex tasks than GPT-3.5, showing a large performance improvement on many evaluation tasks. A recent study [41] investigated the capacities of GPT-4 by conducting qualitative tests with human-generated problems, spanning a diverse range of difficult tasks, and showed that GPT-4 can achieve more superior performance than prior GPT models. Furthermore, GPT-4 responds more safely to malicious or provocative queries, due to a six-month iterative alignment (with an additional safety reward signal in the RLHF training). In the technical report, OpenAI has emphasized how to safely develop GPT-4 and applied a number of intervention strategies to mitigate the possible issues of LLMs, such as hallucinations, privacy and overreliance. For example, they introduced the mechanism called *red teaming* [132] to reduce the harm or toxic content generation. As another important aspect, GPT-4 has been developed on a well-established deep learning infrastructure with improved optimization methods. They introduced a new mechanism called *predictable scaling* that can accurately predict the final performance with a small proportion of compute during model training.

- **GPT-4V, GPT-4 turbo, and beyond.** Based on the work done for GPT-4 [46], OpenAI further released GPT-4V in September 2023, which focused on the safe deployment of the vision capabilities of GPT-4. In the GPT-4V’s system card [133], it has extensively discussed the assessment and mitigation of risks related to visually augmented inputs. Specially, GPT-4V exhibited strong vision capacities in various application scenarios, showing the great potential as a powerful multimodal learning system. More recently, in November 2023, OpenAI released an upgraded generation of GPT-4 model at DevDay, named *GPT-4 Turbo*, with a series of technical improvements. GPT-4 Turbo is featured by the improved model capacity (more capable than GPT-4), the extended knowledge source (up to April 2023), long context window (up to 128k tokens), optimized model performance (cheaper price), and other useful functionality updates (function call, reproducible outputs, etc.). At the same time, Assistants API was launched to ease the rapid development of agent-like assistants. With this API, developers can easily create goal-oriented assistants within their applications, by leveraging specific instruction, extra knowledge and tool use. Furthermore, multimodal capacities (see, hear, and speak) were also enhanced in this new release, supported by GPT-4 Turbo with vision, DALL·E 3, Text-to-speech (TTS), and Listen to voice samples. These improvements have greatly extended the capacity scope and enhanced the task performance of GPT models. More importantly, the application ecosystem will be greatly strengthened with the technology upgrade in improved models, APIs, and functionalities.

Despite the huge progress, there are still limitations with these superior LLMs, *e.g.*, generating hallucinations with factual errors or potentially risky response within some specific context [46]. More limitations or issues of LLMs will be discussed in

18. <https://openai.com/research/learning-from-human-preferences>

Section 7. It poses long-standing research challenges to develop more capable, safer LLMs. From the perspective of engineering, OpenAI has adopted an iterative deployment strategy [134] to develop the models and products by following a five-stage development and deployment life-cycle, which aims to effectively reduce the potential risks of using the models. In the following, we will dive into the technical details in order to have a specific understanding of how they have been developed.

3 Resources of LLMs

It is by no means an easy job to develop or reproduce LLMs, considering the challenging technical issues and huge demands of computation resources. A feasible way is to learn experiences from existing LLMs and reuse publicly available resources for incremental development or experimental study. In this section, we briefly summarize the publicly available resources for developing LLMs, including model checkpoints (or APIs), corpora and libraries.

3.1 Publicly Available Model Checkpoints or APIs

Given the huge cost of model pre-training, well-trained model checkpoints are critical to the study and development of LLMs for the research community. Due to space limitation, we can only selectively discuss several representative LLMs. In addition, for inference, we can directly employ public APIs to perform our tasks, without running the model locally. Next, we introduce the publicly available model checkpoints and APIs.

Publicly Available Model Checkpoints. To assist researchers in selecting a suitable model based on the resource budget and usage needs, we focus on discussing the model’s parameter size, data and computational resources required for training, the relevant technologies employed by the model, and its performance evaluation in downstream tasks. For more details of LLMs, see Table 1.

- *LLaMA*. The LLaMA series of models has gained immense popularity and widespread attention due to its openness and effectiveness. From LLaMA [57], LLaMA-2 [99], LLaMA-3 [135] to LLaMA-3.1 [136], continuous updates have been made and the development is still ongoing. With increased parameters (the largest version has 405B), more pre-training tokens (15T tokens), and an extended context window (128K), LLaMA-3.1 has significantly enhanced its capabilities, and it also integrates additional components that work in synergy with the model, including new security and safety tools. In evaluation, LLaMA-3.1 (405B version) achieves competitive performance against prominent closed-source LLMs, such as GPT-4, GPT-4o, and Claude 3.5 Sonnet in various benchmarks (e.g., MMLU, GSM8k, and HumanEval). The pre-training of LLaMA (65B version) involves 2,048 A100-80G GPUs, whereas LLaMA-3.1 (405B version) involves more than 16,000 H100 GPUs.

- *Mistral*. The Mistral series [137, 138] consist of Mistral (7B), Mistral NeMo (12B), Mistral Large 2 (123B), and Mistral (8×7B and 8×22B), which have been widely known for their strong performance on various mainstream benchmarks (e.g., MMLU and GSM8k). Mistral NeMo is featured with a long context window of 128K at the parameter scale of 12B. Although Mistral NeMo is trained with quantization awareness, it enables FP8 inference without sacrificing performance. Mistral Large 2 is the largest and most powerful model of the Mistral series, which

supports 11 natural languages and more than 80 programming languages. Mixtral is a kind of sparse Mixture-of-Experts (SMoE) model that activates only part of the parameters during inference, making it more efficient compared to dense models of the same size.

- *Gemma*. Gemma [139, 140] is a series of lightweight, strong, and open models, consisting of Gemma-1 (2B and 7B) and Gemma-2 (2B, 9B, and 27B). During the pre-training stage, Gemma-2 2B, 9B, and 27B versions are trained on 2T, 8T, and 13T primarily English tokens, respectively. The largest version of Gemma-2 is trained on 6144 TPUv5p chips. Gemma-2 has achieved excellent performance in multiple benchmarks (e.g., ARC-c, MMLU, and GSM8k).

- *Qwen*. Qwen [141, 142] is an open-source large model series consisting of Qwen (ranging from 7B to 72B), Qwen1.5 (ranging from 0.5B to 110B), Qwen2 (ranging from 0.5B to 72B), and Qwen2.5 (ranging from 0.5B to 72B). Qwen2.5 is the newest LLM collection of Qwen, which is pre-trained on up to 18T tokens. Compared to Qwen2, Qwen2.5 demonstrates a significant increase in knowledge retention, as well as notable advancements in coding and mathematical abilities. Qwen2.5 has also shown large improvements in instruction following, long texts generation (over 8K tokens), structured data understanding and generation (e.g., JSON).

- *GLM*. GLM [143] is a series of LLMs featuring comprehensive capabilities in both English and Chinese. GLM has been upgraded to its fourth-generation model, GLM-4, with a parameter scale of up to 9B, possesses excellent conversational abilities. It has achieved excellent performance in evaluations from multiple perspectives including semantics, mathematics, reasoning, code, and knowledge. In addition to the base model GLM-4-9B, it has open-sourced human preference-aligned model GLM-4-9B-Chat, and long context conversational model GLM-4-9B-Chat-1M.

- *Baichuan*. Baichuan is a series of open-source bilingual LLMs and the latest version is Baichuan-2. Both Baichuan and Baichuan-2 have two available parameter sizes (7B and 13B). Baichuan supports both Chinese and English, with pre-training data reaching 1.2 trillion tokens. Furthermore, Baichuan-2 expands its pre-training data to 2.6 trillion tokens. Baichuan-2 surpasses Baichuan in all evaluation benchmarks, demonstrating excellent multilingual capabilities and showing potential for vertical applications in the domains such as law and healthcare (e.g., JEC-QA [144] and MedQA [145]).

LLaMA Model Family. The collection of LLaMA models [57] were introduced by Meta AI in February, 2023, consisting of four sizes (7B, 13B, 30B and 65B). Since released, LLaMA has attracted extensive attention from both research and industry communities. LLaMA models have achieved very excellent performance on various open benchmarks, which have become the most popular open language models thus far. A large number of researchers have extended LLaMA models by either instruction tuning or continual pre-training. In particular, instruction tuning LLaMA has become a major approach to developing customized or specialized models, due to the relatively low computational costs. To effectively adapt LLaMA models in non-English languages, it often needs to extend the original vocabulary (trained mainly on English corpus) or fine-tune it with instructions or data in the target language. Among these extended models, Stanford Alpaca [146] is the first open instruct-following model fine-

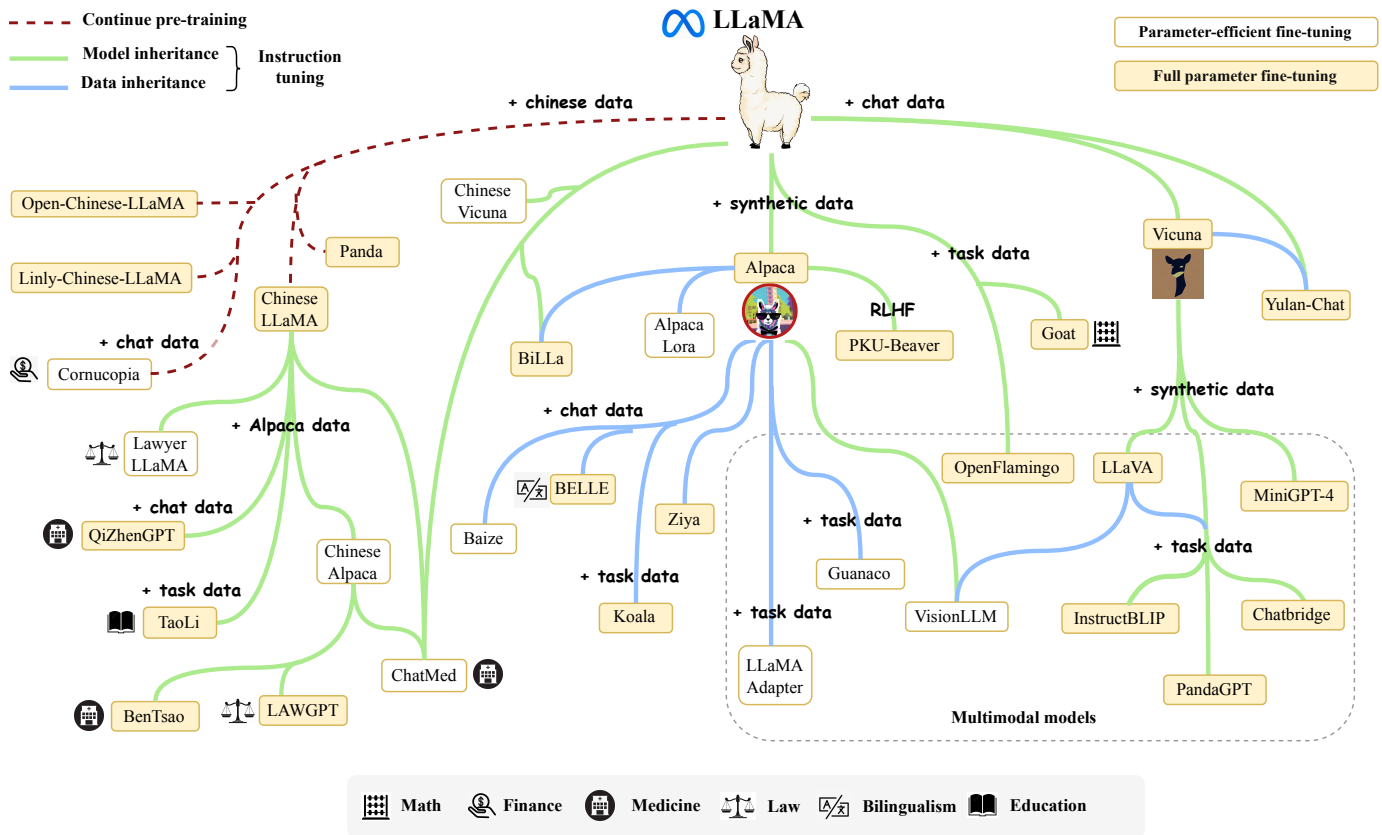


Fig. 5: An evolutionary graph of the research work conducted on LLaMA. Due to the huge number, we cannot include all the LLaMA variants in this figure, even much excellent work. To support incremental update, we share the source file of this figure, and welcome the readers to include the desired models by submitting the pull requests on our GitHub page.

tuned based on LLaMA (7B). It is trained by 52K instruction-following demonstrations generated via self-instruct [147] using text-davinci-003. The instruction data, named *Alpaca-52K*, and training code have been extensively adopted in subsequent work, such as Alpaca-LoRA [148] (a reproduction of Stanford Alpaca using LoRA [149]), Koala [150], and BELLE [151]. In addition, Vicuna [152] is another popular LLaMA variant, trained upon user-shared conversations collected from ShareGPT [153]. Due to the excellent performance and availability of the LLaMA model family, many multimodal models incorporate them as the base language models, to achieve strong language understanding and generation abilities. Compared with other variants, Vicuna is more preferred in multimodal language models, which have led to the emergence of a variety of popular models, including LLaVA [154], MiniGPT-4 [155], InstructBLIP [156], and PandaGPT [157]. The release of LLaMA has greatly advanced the research progress of LLMs. To summarize the research work conducted on LLaMA, we present a brief evolutionary graph in Figure 5.

Public API of LLMs. Instead of directly using the model copies, APIs provide a more convenient way for common users to use LLMs, without the need of running the model locally. As a representative interface for using LLMs, the APIs for the GPT-series models [46, 55, 66, 105] have been widely used for both academia and industry¹⁹. OpenAI has provided seven major interfaces to

the models in GPT-3 series: ada, babbage, curie, davinci (the most powerful version in GPT-3 series), text-ada-001, text-babbage-001, and text-curie-001. Among them, the first four interfaces can be further fine-tuned on the host server of OpenAI. In particular, babbage, curie, and davinci correspond to the GPT-3 (1B), GPT-3 (6.7B), and GPT-3 (175B) models, respectively [55]. In addition, there are also two APIs related to Codex [105], called code-cushman-001 (a powerful and multilingual version of the Codex (12B) [105]) and code-davinci-002. Further, GPT-3.5 series include one base model code-davinci-002 and three enhanced versions, namely text-davinci-002, text-davinci-003, and gpt-3.5-turbo. As more powerful alternatives, in this year, OpenAI has released the model interfaces for GPT-4 series, including gpt-4, gpt-4-32k, gpt-4-1106-preview (*i.e.*, GPT-4 Turbo) and gpt-4-vision-preview (*i.e.*, GPT-4 Turbo with vision, a multimodal model). It is worth noting that OpenAI has been maintaining and upgrading these model interfaces (gpt-3.5-turbo, gpt-4, gpt-4-32k), so the API name will actually point to the latest version. Currently, ChatGPT can be powered by either GPT-3.5 or GPT-4 models. Overall, one select the suitable model interface based on the specific application scenarios and response requirements. The detailed usage can be found on their project websites²⁰.

19. <https://platform.openai.com/docs/api-reference/introduction>

20. <https://platform.openai.com/docs/models/overview>

TABLE 2: Statistics of commonly-used data sources.

Corpora	Size	Source	Latest Update Time
BookCorpus [158]	5GB	Books	Dec-2015
Gutenberg [159]	-	Books	Dec-2021
C4 [82]	800GB	CommonCrawl	Apr-2019
CC-Stories-R [160]	31GB	CommonCrawl	Sep-2019
CC-NEWS [27]	78GB	CommonCrawl	Feb-2019
REALNEWS [161]	120GB	CommonCrawl	Apr-2019
OpenWebText [162]	38GB	Reddit links	Mar-2023
Pushift.io [163]	2TB	Reddit links	Mar-2023
Wikipedia [164]	21GB	Wikipedia	Mar-2023
BigQuery [165]	-	Codes	Mar-2023
the Pile [166]	800GB	Other	Dec-2020
ROOTS [167]	1.6TB	Other	Jun-2022

3.2 Commonly Used Corpora for Pre-training

In contrast to earlier PLMs, LLMs which consist of a significantly larger number of parameters require a higher volume of training data that covers a broad range of content. For this need, there are increasingly more accessible training datasets that have been released for research. In this section, we will briefly summarize several widely used corpora for training LLMs. Based on their content types, we categorize these corpora into five groups: web pages, books, Wikipedia, code, and others.

Web pages. Web pages are a primary data source for training language models.

- *CommonCrawl.* CommonCrawl [168] is one of the largest open-source web crawling databases, containing a petabyte-scale data volume, which has been widely used as training data for existing LLMs. As the whole dataset is very large, existing studies mainly extract subsets of web pages from it within a specific period or specific needs (e.g., extracting mathematical texts). However, due to the widespread existence of noisy and low-quality information in web page data, it is necessary to perform data preprocessing before usage. One commonly used toolkit for cleaning CommonCrawl is CC-Net [169], which is developed by Facebook and has been used in processing datasets like RedPajama-Data [170].

- *C4.* The Colossal Clean Crawled Corpus (C4) includes five variants²¹, namely en (806G), en.noclean (6T), real-newslike (36G), webtextlike (17G), and multilingual (38T). The en version has been utilized for pre-training T5 [82], LaMDA [68], Gopher [64], and UL2 [89]. The multilingual C4, also called mC4, has been used in mT5 [83].

- *RedPajama-Data.* RedPajama-Data [170] is a publicly available comprehensive web dataset, comprising 100 billion documents from Common Crawl. It has been cleaned, filtered, and deduplicated using the CCNet tool, resulting in approximately 30T tokens, which is available for download on Hugging Face. RedPajama-Data is a multilingual dataset that includes five languages: English, French, Spanish, German, and Italian. Additionally, it offers over 40 quality labels, making it feasible to filter or reweight the dataset according to specific criteria. The dataset is continuously updated and maintained, with all data processing scripts open-sourced on GitHub for convenient use.

- *RefinedWeb.* RefinedWeb [171] is a web dataset obtained through rigorous selection and deduplication based on data from Common Crawl, encompassing all Common Crawl web records

from 2008 to June 2023, totaling around 5T tokens. The open-source portion consists of 600B tokens, with a data size of approximately 500GB. After decompression, it requires 2.8TB of local storage space and is available for download on Hugging Face. This dataset serves as the primary training dataset for the open-source large language model Falcon.

- *WebText.* WebText [26] is a well-known corpus composed of highly upvoted links from Reddit, a social media platform that enables users to submit links and text posts, but it is not publicly available. As a surrogate, there is a readily accessible open-source alternative called OpenWebText [162].

Books & Academic Data. Books and academic data contains a wealth of world knowledge and linguistic information, serving as a high-quality corpus for model learning.

- *Book Data.* BookCorpus [158] is a commonly used dataset in previous small-scale models (e.g., GPT [122] and GPT-2 [26]), consisting of over 11,000 books covering a wide range of topics and genres (e.g., novels and biographies). Another large-scale book corpus is Project Gutenberg [159], consisting of over 70,000 literary books including novels, essays, poetry, drama, history, science, philosophy, and other types of works in the public domain. It is currently one of the largest open-source book collections, which is used in training of MT-NLG [113] and LLaMA [57]. As for Books1 [55] and Books2 [55] used in GPT-3 [55], they are much larger than BookCorpus but have not been publicly released so far.

- *Academic Data.* In addition to book data, scientific publication data such as paper is also important for model pre-training. arXiv Dataset [172] is a corpus of 1.7 million academic papers, covering a wide range of papers in the fields of physics, mathematics, and computer science. S2ORC [173] is a corpora that consists of 136M academic papers collected by Semantic Scholar. It also releases a derivative dataset peS2o [174], which contains about 42B tokens.

Wikipedia. Wikipedia [164] is an online encyclopedia containing a large volume of high-quality articles on diverse topics. Most of these articles are composed in an expository style of writing (with supporting references), covering a wide range of languages and fields. Typically, the English-only filtered versions of Wikipedia are widely used in most LLMs (e.g., GPT-3 [55], LaMDA [68], and LLaMA [57]). Wikipedia is available in multiple languages, so it can be used in multilingual settings.

Code. To collect code data, existing work mainly crawls open-source licensed codes from the Internet. Two major sources are public code repositories under open-source licenses (e.g., GitHub) and code-related question-answering platforms (e.g., StackOverflow). Google has publicly released the BigQuery dataset [165], which includes a substantial number of open-source licensed code snippets in various programming languages, serving as a representative code dataset. CodeGen has utilized BIGQUERY [86], a subset of the BigQuery dataset, for training the multilingual version of CodeGen (CodeGen-Multi). In addition, Hugging Face has collected and released a code dataset named The Stack [175], covering more than 30 programming languages. The Stack is continuously updated, and the v1.2 version has expanded to 358 programming languages. Based on this dataset, BigCode further processed it and released StarCoder [98], which is also the pre-training data of the model StarCoder.

21. <https://www.tensorflow.org/datasets/catalog/c4>

Mixed Data. In addition to the aforementioned specific types of datasets, different types of data have been combined to facilitate usage by researchers. The Pile [166] is a large-scale, diverse, and open-source text dataset consisting of over 800GB of data from multiple sources, including books, websites, codes, scientific papers, and social media platforms. It is constructed from 22 diverse high-quality subsets. The Pile dataset is widely used in models with different parameter scales, such as GPT-J (6B) [176], CodeGen (16B) [86], and Megatron-Turing NLG (530B) [113]. ROOTS [167] is composed of various smaller datasets (totally 1.61 TB of text) and covers 59 different languages (containing natural languages and programming languages), which have been used for training BLOOM [78]. Another mixture dataset is Dolma [177], which includes web text from Common Crawl, academic papers from Semantic Scholar, GitHub code, books, social media from Reddit, and Wikipedia data. Dolma consisting of 3T tokens of approximately 200TB of raw text and has been used to train OLMo [178].

In practice, it commonly requires a mixture of different data sources for pre-training LLMs (see Figure 6), instead of a single corpus. Therefore, existing studies commonly mix several ready-made datasets (e.g., C4, OpenWebText, and the Pile), and then perform further processing to obtain the pre-training corpus. Furthermore, to train the LLMs that are adaptive to specific applications, it is also important to extract data from relevant sources (e.g., Wikipedia and BigQuery) for enriching the corresponding information in pre-training data.

TABLE 3: A detailed list of available collections for instruction tuning.

Categories	Collections	Time	#Examples
Task	Nat. Inst. [179]	Apr-2021	193K
	FLAN [67]	Sep-2021	4.4M
	P3 [180]	Oct-2021	12.1M
	Super Nat. Inst. [88]	Apr-2022	5M
	MVPCorpus [181]	Jun-2022	41M
	xP3 [94]	Nov-2022	81M
	OIG[182]	Mar-2023	43M
Chat	HH-RLHF [183]	Apr-2022	160K
	HC3 [184]	Jan-2023	87K
	ShareGPT [153]	Mar-2023	90K
	Dolly [185]	Apr-2023	15K
	OpenAssistant [186]	Apr-2023	161K
Synthetic	Self-Instruct [147]	Dec-2022	82K
	Alpaca [187]	Mar-2023	52K
	Guanaco [188]	Mar-2023	535K
	Baize [189]	Apr-2023	158K
	BELLE [190]	Apr-2023	1.5M

TABLE 4: A list of available collections for alignment.

Dataset	Release Time	#Examples
Summarize from Feedback [129]	Sep-2020	193K
SHP [191]	Oct-2021	385K
WebGPT Comparisons [81]	Dec-2021	19K
Stack Exchange Preferences [192]	Dec-2021	10M
HH-RLHF [183]	Apr-2022	169K
Sandbox Alignment Data [193]	May-2023	169K
CValues [194]	Jul-2023	145K
PKU-SafeRLHF [195]	Oct-2023	330K

3.3 Commonly Used Datasets for Fine-tuning

After pre-training, it requires further fine-tuning LLMs to enhance the model capacity, which often involve two major steps, namely instruction tuning (supervised fine-tuning) and alignment tuning. In this section, we mainly focus on discussing the related available datasets for the two kinds of tuning approaches, and more algorithm details can be found in Section 5.

3.3.1 Instruction Tuning Datasets

After pre-training, instruction tuning (*a.k.a.*, supervised fine-tuning) is an important method to enhance or unlock specific abilities of LLMs (e.g., instruction following). In this part, we introduce several widely used datasets for instruction tuning, and categorize them into three main types based on the construction method of formatted instruction instances, namely NLP task datasets, daily chat datasets and synthetic datasets. We show their details in Table 3.

NLP Task Datasets. This kind of datasets are formatted based on collected NLP task datasets (e.g., text classification and summarization) with corresponding natural language task descriptions. In this category, P3 [196] and FLAN [67, 197] are two widely used datasets for instruction tuning.

- P3 [196] is composed of 170 English NLP datasets and 2,052 English prompt templates, where the input and output of each data example have been formatted with specific prompt templates for composing the training instance.

- FLAN [67] consists of 62 widely used NLP benchmarks in its original version. Recently, FLAN-v2 [197] is also proposed, which expands FLAN by mixing additional instruction datasets, including Muffin [67], NIV2 [88], T0-SF [28], and CoT [198–200]. Muffin contains 62 tasks from the original FLAN and additional 26 tasks, including conversation and code synthesis tasks. T0-SF is extracted from T0 [28] while ensuring no overlap with Muffin. NIV2 refers to the Natural-Instructions v2 dataset [88], and CoT [198–200] is a combination of nine reasoning tasks with corresponding chain-of-thought prompts and outputs.

Daily Chat Datasets. This kind of datasets are constructed based on real user conversations where queries are posed by humans and responses are mainly generated by human labelers or LLMs (e.g., ChatGPT, GPT-4). The conversation types include open-ended generation, question answering, brainstorming, and chatting. In this category, ShareGPT [153], OpenAssistant [186] and Dolly [185] are three commonly used datasets for LLM fine-tuning.

- ShareGPT [153] is collected from a data collection platform where users can upload their conversations with ChatGPT or GPT-4 through the ShareGPT API. Currently, this dataset consists of approximately 90,000 conversations, including real instructions or inquiries from human and responses from ChatGPT.

- OpenAssistant [186] is a multilingual corpus containing 66,497 real-world conversation trees between human and AI assistant. Each conversation tree consists of multiple nodes, and each node represents the information generated by a role in the dialogue. It spans 35 languages and includes 461,292 manually annotated quality ratings of responses.

- Dolly [185] is an English dataset comprising 15,000 human-generated data instances (prompt-response pairs) from Databricks. This dataset covers seven domains outlined in the InstructGPT [66], including brainstorming, classification,

closed-book quality assurance, generation, information extraction, open-book quality assurance, and summarization.

Synthetic Datasets. This kind of datasets are typically constructed by instructing LLMs, based on pre-defined guidance rules or methods. In this category, Self-Instruct-52K [147], Alpaca [146] and Baize [189] are three commonly used synthetic datasets for LLMs.

- **Self-Instruct-52K** [147] is an instruction dataset generated through the self-instruct [147] method, consisting of 82,000 instances with 52,000 instructions. Concretely, the authors construct 175 seed instances, and then iteratively prompt the LLM [55] to synthesize additional instructions based on randomly selected 8 instructions as reference. Subsequently, the LLM is further instructed to generate instance inputs and their corresponding outputs based on the synthetic instructions, and finally obtain the Self-Instruct-52K dataset.

- **Alpaca** [146] is also a synthetic dataset based on the self-instruct [147] method. It utilizes the text-davinci-003 model on the 175 seed datasets from Self-Instruct-52K to obtain 52,000 new instructions and corresponding inputs and outputs. Moreover, 60% of the examples are pure instructions without the input part in the final dataset.

- **Baize** [189] is an English multi-turn conversation corpus constructed using ChatGPT, comprising 111.5K instances. To create Baize, a method called “self-chat” [189] is purposed, where ChatGPT takes on the roles of both the user and the AI assistant in turns, generating information in a conversational format.

3.3.2 Alignment Datasets

Apart from instruction tuning, it is important to construct high-quality datasets for aligning LLMs with human values and preferences (e.g., helpfulness, honesty, and harmlessness). In this section, we introduce several widely used datasets for alignment tuning, including HH-RLHF [183], SHP [191], PKU-SafeRLHF [195], Stack Exchange Preferences [192] and Sandbox Alignment Data [193]. We show their details in Table 4.

- **HH-RLHF** [183] consists of around 169K instances, and can be divided into two parts that focus on the helpfulness and harmlessness of LLMs, respectively. Each instance is an open-ended conversation between a crowdworker and a chat model, about seeking assistance, advice, or task completion. The chat model provides two responses to each user query, and the more helpful or harmful responses will be chosen as the annotations.

- **SHP** [191] focuses on the helpfulness of responses. It comprises 385K collective human preferences over responses to questions/instructions across 18 diverse subject areas, spanning topics from cooking to legal advice. Each instance is a Reddit post containing a question or instruction and a pair of top-level comments, one of which is deemed as more preferable by Reddit users and the other one is deemed as less helpful. Different from HH-RLHF [183], the data in SHP consists of naturally occurring and human-written responses.

- **PKU-SafeRLHF** [195] encompasses more than 330K instances of expert comparison data, concentrating on the helpfulness and harmlessness. Each instance in the dataset includes a question and two responses, accompanied by safety labels for each response and two preference annotations between the two responses according to helpfulness and harmlessness. The harmlessness of a response indicates its classification as risk-neutral across all 14 harm categories, while the helpfulness of

a response is evaluated based on its effectiveness in addressing the question.

- **Stack Exchange Preferences** [192] focuses on the helpfulness of answers. It comprises about 10M questions and answers from Stack Overflow. Each instance consists of a question and more than two corresponding answers. Each answer is annotated with a score calculated based on its votes and a label denoting whether it is selected.

- **Sandbox Alignment Data** [193] is an alignment dataset containing feedback from LLMs rather than human. It comes from a virtual interaction environment called SANDBOX, where the model simulates social interactions with other models and revise responses according to the feedback from other models. The dataset contains 169K instances, and each instance consists of a societal query, several responses, and corresponding ratings from other models.

3.4 Library Resource

In this part, we briefly introduce a series of available libraries for developing LLMs.

- **Transformers** [201] is an open-source Python library for building models using the Transformer architecture, which is developed and maintained by Hugging Face. It has a simple and user-friendly API, making it easy to use and customize various pre-trained models. It is a powerful library with a large and active community of users and developers who regularly update and improve the models and algorithms.

- **DeepSpeed** [74] is a deep learning optimization library (compatible with PyTorch) developed by Microsoft, which has been used to train a number of LLMs, such as MT-NLG [113] and BLOOM [78]. It provides the support of various optimization techniques for distributed training, such as memory optimization (ZeRO technique, gradient checkpointing), and pipeline parallelism.

- **Megatron-LM** [75–77] is a deep learning library developed by NVIDIA for training large-scale language models. It also provides rich optimization techniques for distributed training, including model and data parallelism, mixed-precision training, and FlashAttention. These optimization techniques can largely improve the training efficiency and speed, enabling efficient distributed training across GPUs.

- **JAX** [202] is a Python library for high-performance machine learning algorithms developed by Google, allowing users to easily perform computations on arrays with hardware acceleration (e.g., GPU or TPU). It enables efficient computation on various devices and also supports several featured functions, such as automatic differentiation and just-in-time compilation.

- **Colossal-AI** [203] is a deep learning library developed by HPC-AI Tech for training large-scale AI models. It is implemented based on PyTorch and supports a rich collection of parallel training strategies. Furthermore, it can also optimize heterogeneous memory management with methods proposed by PatrickStar [204]. Recently, a ChatGPT-like model called ColossalChat [205] has been publicly released with two versions (7B and 13B), which are developed using Colossal-AI based on LLaMA [57].

- **BMTrain** [206] is an efficient library developed by OpenBMB for training models with large-scale parameters in a distributed manner, which emphasizes code simplicity, low resource, and high availability. BMTrain has already incorporated

several common LLMs (e.g., Flan-T5 [69] and GLM [93]) into its ModelCenter, where developers can use these models directly.

- **FastMoE** [207] is a specialized training library for MoE (i.e., mixture-of-experts) models. It is developed based on PyTorch, prioritizing both efficiency and user-friendliness in its design. FastMoE simplifies the process of transferring Transformer models to MoE models and supports both data parallelism and model parallelism during training.

- **vLLM** [208] is a fast, memory efficient, and easy-to-use library for LLM inference and serving. To enable fast inference, it is specially optimized with high serving throughput, effective attention memory management using PagedAttention [208], continuous batching, and optimized CUDA kernels. Furthermore, vLLM also supports various decoding algorithms, tensor parallelism and streaming outputs. To ease the integration with other systems, vLLM is friendly to the use of HuggingFace models, and also provide OpenAI-compatible API servers.

- **DeepSpeed-MII** [209] is also a memory efficient Python library developed by DeepSpeed [74]. It aims to democratize LLMs inference by prioritizing high throughput, low latency, and cost-effectiveness. DeepSpeed-MII achieves accelerated text generation inference by leveraging four essential technologies: blocked KV caching, continuous batching, dynamic SplitFuse, and high-performance CUDA Kernels. It currently supports over 13,000 models across three popular model architectures, such as LLaMA [57], Mistral [137], and OPT [90].

- **DeepSpeed-Chat** [210] is a fast, cost-effective, and easy-to-use system framework that enables the integration of the complete RLHF process during model training. It is featured by three major functionalities: (1) it simplifies the training and inference process for ChatGPT-like models, enabling using a simple script to implement multiple training or inference steps; (2) it replicates the training mode of InstructGPT [66] and provides a complete pipeline for three training steps (i.e., SFT, reward model fine-tuning, and RLHF); (3) it integrates the training engine and inference engine of Deepspeed into a unified hybrid engine (DeepSpeed HE) for RLHF training, which enables seamless switch between training and inference modes, and leveraging various optimizations from DeepSpeed Inference.

In addition to the above library resources, existing deep learning frameworks (e.g., PyTorch [211], TensorFlow [212], MXNet [213], PaddlePaddle [214], MindSpore [215] and OneFlow [216]) have also provided the support for parallel algorithms, which are commonly used for training large-scale models.

4 Pre-training

Pre-training establishes the basis of the abilities of LLMs. By pre-training on large-scale corpora, LLMs can acquire essential language understanding and generation skills [55, 56]. In this process, the scale and quality of the pre-training corpus are critical for LLMs to attain powerful capabilities. Furthermore, to effectively pre-train LLMs, model architectures, acceleration methods, and optimization techniques need to be well designed. In what follows, we first discuss the data collection and processing in Section 4.1, then introduce the commonly used model architectures in Section 4.2, and finally present the training techniques to stably and efficiently optimize LLMs in Section 4.3.

4.1 Data Collection and Preparation

Compared with small-scale language models, LLMs have a stronger demand for high-quality data for model pre-training,

and their model capacities largely rely on the pre-training corpus and how it has been preprocessed. In this part, we discuss the collection and processing of pre-training data, including data sources, preprocessing methods, and important analysis of how pre-training data affects the performance of LLMs.

4.1.1 Data Source

To develop a capable LLM, it is key to collect a large amount of natural language corpus from various data sources. Existing LLMs mainly leverage a mixture of diverse public textual datasets as the pre-training corpus. Figure 6 shows the distribution of the sources of pre-training data for a number of representative LLMs.

The source of pre-training corpus can be broadly categorized into two types: general data and specialized data. General data, such as webpages, books, and conversational text, is utilized by most LLMs [55, 56, 90] due to its large, diverse, and accessible nature, which can enhance the language modeling and generalization abilities of LLMs. In light of the impressive generalization capabilities exhibited by LLMs, there are also studies that extend their pre-training corpus to more specialized datasets, such as multilingual data, scientific data, and code, endowing LLMs with specific task-solving capabilities [35, 56, 86]. In what follows, we describe these two types of pre-training data sources and their effects on LLMs. For a detailed introduction to the commonly used corpus, one can refer to Section 3.2.

General Text Data. As we can see in Figure 6, the vast majority of LLMs adopt general-purpose pre-training data, such as webpages, books, and conversational text, which provides rich text sources on a variety of topics. Next, we briefly summarize three important kinds of general data.

- **Webpages.** Owing to the proliferation of the Internet, various types of data have been created, which enables LLMs to gain diverse linguistic knowledge and enhance their generalization capabilities [26, 82]. For convenient use of these data resources, a large amount of data is crawled from the web in previous work, such as CommonCrawl [168]. However, the crawled web data tends to contain both high-quality text, such as Wikipedia and low-quality text, like spam mail, thus it is important to filter and process webpages for improving the data quality.

- **Conversation text.** Conversation data can enhance the conversational competence of LLMs [90] and potentially improve their performance on a range of question-answering tasks [56]. Researchers can utilize subsets of public conversation corpus (e.g., PushShift.io Reddit corpus) [163, 217] or collect conversation data from online social media. Since online conversational data often involves discussions among multiple participants, an effective processing way is to transform a conversation into a tree structure, where the utterance is linked to the one it responds to. In this way, the multi-party conversation tree can be divided into multiple sub-conversations, which can be collected in the pre-training corpus. Furthermore, a potential risk is that the excessive integration of dialogue data into LLMs may result in a side effect [90]: declarative instructions and direct interrogatives are erroneously perceived as the beginning of conversations, thus leading to a decline in the efficacy of the instructions.

- **Books.** Compared to other corpus, books provide an important source of formal long texts, which are potentially beneficial for LLMs to learn linguistic knowledge, model long-term dependency, and generate narrative and coherent texts. To obtain open-source book data, existing studies usually adopt the

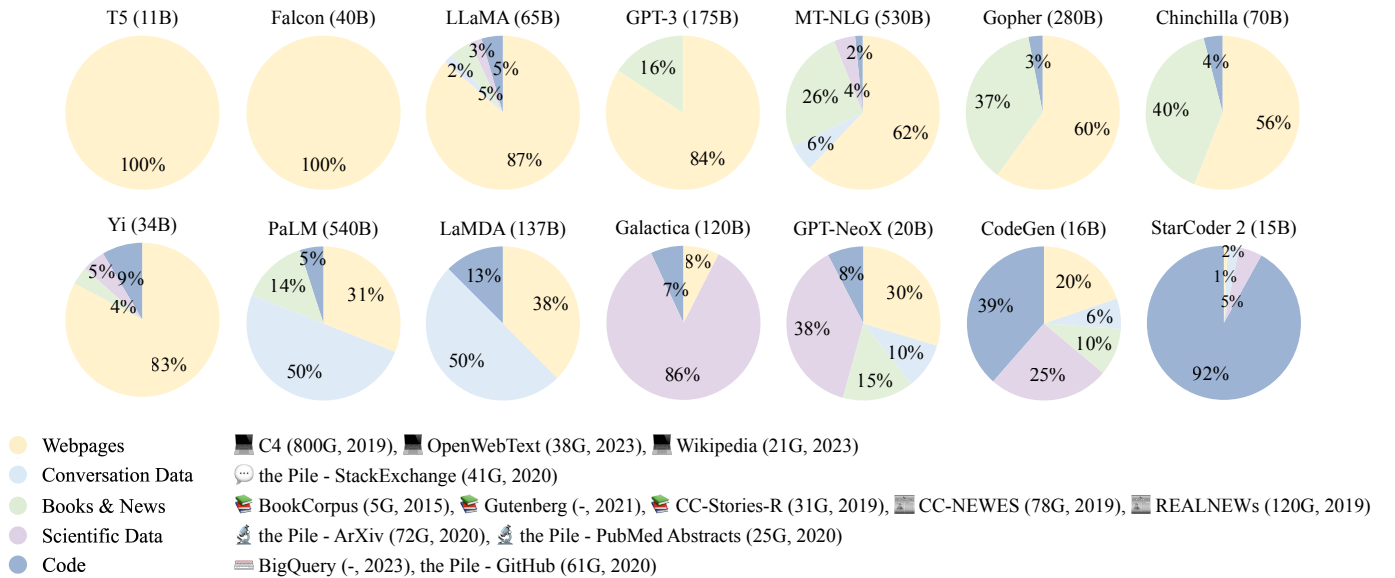


Fig. 6: Ratios of various data sources in the pre-training data for existing LLMs.

Books3 and Bookcorpus2 datasets, which are available in the Pile dataset [166].

Specialized Text Data. Specialized datasets are useful to improve the specific capabilities of LLMs on downstream tasks. Next, we introduce three kinds of specialized data.

- *Multilingual text.* In addition to the text in the target language, integrating a multilingual corpus can enhance the multilingual abilities of language understanding and generation. For example, BLOOM [78] and PaLM [56] have curated multilingual data covering 46 and 122 languages, respectively, within their pre-training corpora. FLM [102] mixes Chinese and English corpora in nearly equal proportions. These models demonstrate impressive performance in multilingual tasks, such as translation, multilingual summarization, and multilingual question answering, and achieve comparable or superior performance to the state-of-the-art models that are fine-tuned on the corpus in the target language(s).

- *Scientific text.* The exploration of science by humans has been witnessed by the increasing growth of scientific publications. In order to enhance the understanding of scientific knowledge for LLMs [35, 218], it is useful to incorporate a scientific corpus for model pre-training [35, 218]. By pre-training on a vast amount of scientific text, LLMs can achieve impressive performance in scientific and reasoning tasks [219]. To construct the scientific corpus, existing efforts mainly collect arXiv papers, scientific textbooks, math webpages, and other related scientific resources. Due to the complex nature of data in scientific fields, such as mathematical symbols and protein sequences, specific tokenization and preprocessing techniques are usually required to transform these different formats of data into a unified form that can be processed by language models.

- *Code.* Program synthesis has been widely studied in the research community [105, 220–223], especially the use of PLMs trained on code [176, 224]. However, it remains challenging for these PLMs (e.g., GPT-J [176]) to generate high-quality and accurate programs. Recent studies [105, 223] have found that training LLMs on a vast code corpus can lead to a substan-

tial improvement in the quality of the synthesized programs. The generated programs can successfully pass expert-designed unit-test cases [105] or solve competitive programming questions [114]. In general, two types of code corpora are commonly used for pre-training LLMs. The first source is from programming question answering communities like Stack Exchange [225]. The second source is from public software repositories such as GitHub [86, 105, 223], where code data (including comments and docstrings) are collected for utilization. Compared to natural language text, code is in the format of a programming language, corresponding to long-range dependencies and accurate execution logic [226]. A recent study [47] also speculates that training on code might be a source of complex reasoning abilities (e.g., chain-of-thought ability [33]). Furthermore, it has been shown that formatting reasoning tasks into code can help LLMs generate more accurate results [226].

4.1.2 Data Preprocessing

After collecting a large amount of text data, it is essential to preprocess the data for constructing the pre-training corpus, especially removing noisy, redundant, irrelevant, and potentially toxic data [56, 64, 227], which may largely affect the capacity and performance of LLMs. To facilitate the data processing, a recent study [228] proposes a useful data processing system for LLMs, named Data-Juicer, which provides over 50 processing operators and tools. In this part, we review the detailed data preprocessing strategies to improve the quality of the collected data [64, 78, 112]. A typical pipeline of preprocessing the pre-training data for LLMs has been illustrated in Figure 7.

Filtering and Selection. To remove low-quality data from the collected corpus, existing work generally adopts two approaches, namely classifier-based and heuristic-based. The former approach trains a selection classifier based on high-quality texts and leverages it to identify and filter out low-quality data. Typically, these methods train a binary classifier using positive instances that are: well-curated data (e.g., Wikipedia pages) [55, 56, 112], high-quality synthesized data [135, 229–231], or a combination

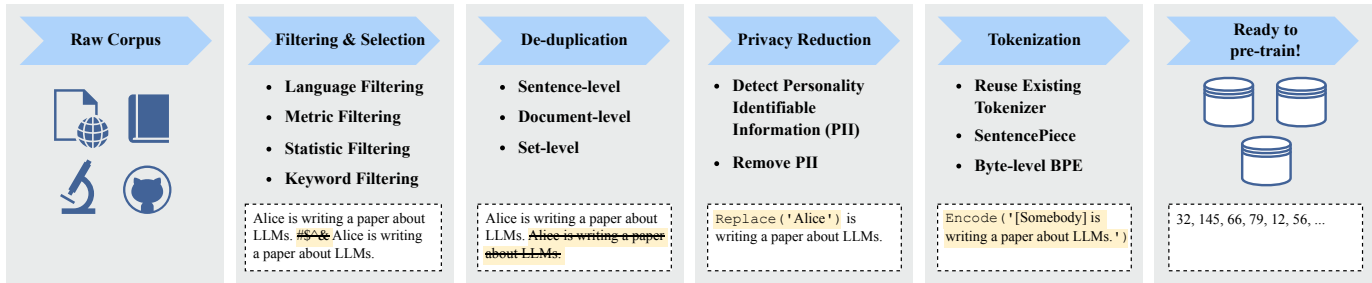


Fig. 7: An illustration of a typical data preprocessing pipeline for pre-training large language models.

of both. They sample candidate data as negative instances and predict the score that measures the quality of each data example. However, several studies [64, 112] find that a classifier-based approach may result in the unintentional removal of high-quality texts in dialectal, colloquial, and sociolectal languages, which potentially leads to bias in the pre-training corpus and diminishes the corpus diversity. As the second approach, several studies, such as BLOOM [78] and Gopher [64], employ heuristic-based approaches to eliminate low-quality texts through a set of well-designed rules, which can be summarized as follows:

- *Language based filtering.* If a LLM would be mainly used in the tasks of certain languages, the text in other languages can be filtered.
- *Metric based filtering.* Evaluation metrics about the generated texts, e.g., perplexity, can be employed to detect and remove unnatural sentences.
- *Statistic based filtering.* Statistical features of a corpus, e.g., the punctuation distribution, symbol-to-word ratio, and sentence length, can be utilized to measure the text quality and filter the low-quality data.
- *Keyword based filtering.* Based on specific keyword set, the noisy or useless elements in the text, such as HTML tags, hyperlinks, boilerplates, and offensive words, can be identified and removed.

In addition to the above methods, LLMs (especially relatively small models) can be also employed for data selection, either by computing perplexity [232] or directly prompting LLMs [233] for measuring the sample importance. However, using LLMs is unavoidably computationally intensive for large-scale data selection.

De-duplication. Existing work [234] has found that duplicate data in a corpus would reduce the diversity of language models, which may cause the training process to become unstable and thus affect the model performance. Therefore, it is necessary to de-duplicate the pre-training corpus. Specially, de-duplication can be performed at different granularities, including sentence-level, document-level, and dataset-level de-duplication. First, low-quality sentences that contain repeated words and phrases should be removed, as they may introduce repetitive patterns in language modeling [235]. At the document level, existing studies mostly rely on the overlap ratio of surface features (e.g., words and n -grams overlap) between documents to detect and remove duplicate documents containing similar contents [57, 64, 78, 236]. Furthermore, to avoid the dataset contamination problem, it is also crucial to prevent the overlap

between the training and evaluation sets [56], by removing the possible duplicate texts from the training set. It has been shown that the three levels of de-duplication are useful to improve the training of LLMs [56, 237], which should be jointly used in practice.

Privacy Reduction. Thus, it is necessary to remove the *personally identifiable information (PII)* from the pre-training corpus. One direct and effective approach is to employ rule-based methods, such as keyword spotting, to detect and remove PII such as names, addresses, and phone numbers [167]. Furthermore, researchers also find that the vulnerability of LLMs under privacy attacks can be attributed to the presence of duplicate PII data in the pre-training corpus [238]. Therefore, de-duplication can also reduce privacy risks to some extent.

Tokenization. Tokenization is also a crucial step for data preprocessing. It aims to segment raw text into sequences of individual tokens, which are subsequently used as the inputs of LLMs. In traditional NLP research (e.g., sequence labeling with conditional random fields [239]), word-based tokenization is the predominant approach, which is more aligned with human's language cognition. However, word-based tokenization can yield different segmentation results for the same input in some languages (e.g., Chinese word segmentation), generate a huge word vocabulary containing many low-frequency words, and also suffer from the "out-of-vocabulary" issue. Thus, several neural network models employ *character* as the minimum unit to derive the word representation (e.g., a CNN word encoder in ELMo [21]). Recently, *subword tokenizers* have been widely used in Transformer based language models, typically including Byte-Pair Encoding tokenization, WordPiece tokenization and Unigram tokenization. HuggingFace has maintained an excellent online NLP course on tokenizer²² with running examples, and we refer to the beginners to this course. Next, we briefly describe the three representative tokenization methods.

- *Byte-Pair Encoding (BPE) tokenization.* BPE was originally proposed as a general data compression algorithm in 1994 [240], and then adapted to NLP for tokenization [241]. It starts with a set of basic symbols (e.g., the alphabets and boundary characters), and iteratively combine frequent pairs of two consecutive tokens in the corpus as new tokens (called *merge*). For each merge, the selection criterion is based on the co-occurrence frequency of two contiguous tokens: the top frequent pair would be selected. The merge process continues until it reaches the pre-defined size. Further, Byte-level BPE has been used to improve the tokenization quality for multilingual corpus (e.g., the text

22. <https://huggingface.co/learn/nlp-course/chapter6>

containing non-ASCII characters) by considering *bytes* as the basic symbols for merge. Representative language models with this tokenization approach include GPT-2, BART, and LLaMA.

- **WordPiece tokenization.** WordPiece was a Google internal subword tokenization algorithm. It was originally proposed by Google in developing voice search systems [242]. Then, it was used in the neural machine translation system in 2016 [243], and was adopted as the word tokenizer for BERT in 2018 [23]. WordPiece has a very similar idea with BPE by iteratively merging consecutive tokens, whereas taking a slightly different selection criterion for the merge. To conduct the merge, it first trains a language model and employs it to score all possible pairs. Then, at each merge, it selects the pair that leads to the most increase in the likelihood of training data. Since Google has't released the official implementation of the WordPiece algorithm, HuggingFace gives a more intuitive selection measure in its online NLP course: a pair is scored by dividing the co-occurrence count by the product of the occurrence counts of two tokens in the pair based on training corpus.

- **Unigram tokenization.** Unlike BPE and WordPiece, Unigram tokenization [244] starts with a sufficiently large set of possible substrings or subtokens for a corpus, and iteratively removes the tokens in the current vocabulary until the expected vocabulary size is reached. As the selection criterion, it calculates the yielded increase in the likelihood of training corpus by assuming that some token was removed from current vocabulary. This step is conducted based on a trained unigram language model. To estimate the unigram language model, it adopts an expectation-maximization (EM) algorithm: at each iteration, we first find the currently optimal tokenization of words based on the old language model, and then re-estimate the probabilities of unigrams to update the language model. During this procedure, dynamic programming algorithms (*i.e.*, the Viterbi algorithm) are used to efficiently find the optimal decomposition way of a word given the language model. Representative models that adopt this tokenization approach include T5 and mBART.

Although it is expedient to leverage an existing tokenizer (*e.g.*, OPT [90] and GPT-3 [55] utilize the tokenizer of GPT-2 [26]), using a tokenizer specially designed for the pre-training corpus can be highly beneficial [78], especially for the corpus that consists of diverse domains, languages, and formats. Therefore, recent LLMs often train the customized tokenizers specially for the pre-training corpus with the SentencePiece library [245], which includes Byte-level BPE and Unigram tokenization. A note is that normalization techniques in BPE, such as NFKC [246], may degrade the tokenization performance [34, 64, 78]. When extending existing LLMs (*i.e.*, continual pre-training or instruction tuning), we should be also aware of the potential side effect with customized tokenizers. For example, LLaMA trains the BPE tokenizer based on a pre-training corpus mainly consisting of English texts, and the derived vocabulary might be less capable in processing non-English data, *e.g.*, taking longer inference latency to generate Chinese texts.

Discussion on Effect of Data Quality. For pre-training, the quality of pre-training data is vital to the model capacities of LLMs. Existing work has shown that pre-training on the low-quality corpus, such as noisy, toxic, and duplicate data, would largely hurt the performance of models [64, 234, 236, 238]. Recent studies, such as T5 [82], GLaM [112], and Gopher [64], have investigated the influence of data quality on the LLMs'

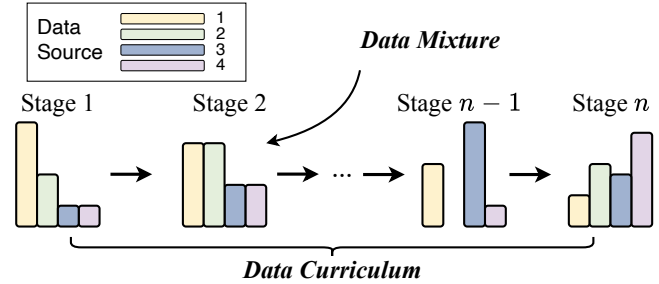


Fig. 8: An illustration of data scheduling for pre-training LLMs.

capacities. By comparing the performance of models trained on the filtered and unfiltered corpus, they have reached the similar conclusion that pre-training LLMs on cleaned data can improve the model performance. More specifically, the duplication of data may result in “*double descent*” (referring to the phenomenon of performance initially deteriorating and subsequently improving) [234, 247], or even overwhelm the training process [234]. In addition, it has been shown that duplicate data degrades the ability of LLMs to copy from the context, which might further affect the generalization capacity of LLMs using in-context learning [234]. Therefore, as suggested in [56, 64, 78, 227], it is essential to utilize preprocessing methods like quality filtering, toxic filtering and deduplication to carefully clean the pre-training corpus (as illustrated in Section 4.1.2), to improve stability of the training process and avoid affecting the model performance.

4.1.3 Data Scheduling

After data preprocessing, it is essential to design suitable strategies to schedule these multi-source data for pre-training a capable LLM. Generally, two key aspects should be paid close attention for data scheduling: the proportion of each data source (*data mixture*), and the order in which each data source is scheduled for training (*data curriculum*). Next, we discuss the two aspects in detail. An illustration of data scheduling has been presented in Figure 8.

Data Mixture. Since each kind of data source is closely related to the development of certain capacities for LLMs (referring to the discussions in Section 4.1), it is important to set a suitable distribution to mix these data. The data mixture is generally set in a global level (*i.e.*, the distribution of the entire pre-training data), and can be also locally set to varied proportions at different training stages. During pre-training, data samples from different sources would be selected according to the mixture proportions: more data will be sampled from a data source with a larger weight. Typically, existing LLMs such as LLaMA [57] may employ upsampling or downsampling on the full data of each source to create specific data mixtures as pre-training data. As Figure 6 illustrates, existing LLMs use different data mixtures to construct the pre-training data. As a representative model, the pre-training data of LLaMA [57] mainly consists of webpages (over 80%), alongside 6.5% of code-heavy data from GitHub and StackExchange, 4.5% from books, and 2.5% of scientific data sourced from arXiv, which has become an important reference for training general-purpose LLMs. Furthermore, special data mixtures can be used to facilitate different purposes. For example, Falcon [171] is trained on pure webpages, and CodeGen [86] largely increases the amount of code data. In practice, data mixture is often deter-

mined empirically, and we summarize several common strategies for finding an effective data mixture as follows:

- *Increasing the diversity of data sources.* Recent studies have empirically shown that training on excessive data about a certain domain would degrade the generalization capability of LLMs on other domains [35, 64]. In contrast, increasing the data source heterogeneity (e.g., including diverse data sources) is critical for improving the downstream performance of LLMs [227, 248, 249]. To further examine the effect of different data sources, some studies have conducted ablation experiments by removing each data source one by one, and pre-train LLMs with specially curated datasets [227]. It has been shown that dropping data sources with high heterogeneity (e.g., webpages) impacts LLM’s abilities more severely than dropping sources with low heterogeneity (e.g., academic corpus).

- *Optimizing data mixtures.* In addition to manually setting the data mixtures, several studies have proposed to optimize the data mixtures for improving the model pre-training [59, 250]. Given the target downstream tasks, one can select pre-training data with either higher proximity in the feature space [250] or those that provide positive influences on downstream task performance [251]. Further, to reduce the reliance of target tasks, DoReMi [59] first trains a small reference model using given initial domain weights, and then trains another small proxy model, upweighting the domains on which the greatest discrepancies in likelihood between the two models are observed. Finally, the learned domain weights of the proxy model are applied to train a much larger LLM. In a more simple way, one can train several small language models with different data mixtures, and select the data mixture that leads to the most desirable performance. However, an assumption made in this approach is, when trained in a similar way, small models would resemble with large models in model abilities or behaviors, which may not always hold in practice.

- *Specializing the targeted abilities.* The model capacities of LLMs heavily rely on data selection and mixture, and one can boost the proportions of specific data sources to enhance certain model abilities [64, 227]. For example, the mathematical reasoning and coding abilities can be specially enhanced by training with more mathematical texts and code data, respectively. Furthermore, experimental results on the LAMBADA dataset [252] show that increasing the proportion of books data can improve the model capacity in capturing long-term dependencies from text, and increasing the proportion of the C4 dataset [82] leads to performance improvement on the C4 validation dataset [64]. Generally, it is important to identify more implicit relations between data sources and model abilities. To enhance specific skills such as mathematics and coding in LLMs, or to develop specialized LLMs, a practical way is to employ a multi-stage training approach, e.g., general and skill-specific data can be scheduled at two consecutive stages. This approach of training LLMs on varying sources or proportions of data across multiple stages is also known as “data curriculum”, which will be introduced below.

Data Curriculum. After preparing the data mixture, it is important to schedule the order that specific data is presented to LLMs for pre-training. It has been shown that, in some cases, to learn a certain skill, learning in a skill-set sequence (e.g., basic skills → target skill) outperforms direct learning from a corpus focused solely on the target skill [253, 254]. Following the idea of

curriculum learning [255], *data curriculum* has been proposed and widely used in model pre-training [253, 254, 256, 257]. It aims to organize different parts of pre-training data for LLMs in a specific order, e.g., starting with easy/general examples and progressively introducing more challenging/specialized ones. More generally, it can broadly refer to the adaptive adjustment of data proportions for different sources during pre-training. Existing work about data curriculum mainly focuses on continual pre-training, such as specialized coding LLMs (e.g., CodeLLaMA [254]) or long context LLMs (e.g., LongLLaMA [257]). However, it still lacks of more detailed report about data curriculum for general-purpose LLMs (e.g., LLaMA) in the literature. To determine data curriculum, a practical approach is to monitor the development of key abilities of LLMs based on specially constructed evaluation benchmarks, and then adaptively adjust the data mixture during pre-training. Next, we take three common abilities as examples to introduce how the concept of data curriculum²³ applies in continual pre-training.

- *Coding.* To improve the coding ability of LLMs, CodeLLaMA [254] is developed based on LLaMA 2 [99] (2T general tokens → 500B code-heavy tokens), aiming to improve the code generation ability and retain natural language understanding skills. CodeLLaMA also provides a version that is further specialized to a certain programming language, namely CodeLLaMA-Python (2T general tokens → 500B code-heavy tokens → 100B Python-heavy tokens).

- *Mathematics.* Llemma [258] is proposed to enhance the mathematical capacities of general-purpose LLMs. It is developed based on CodeLLaMA. Although CodeLLaMA [254] mainly focuses on the coding ability, experiments have shown that it performs better than its base model LLaMA 2 on mathematics benchmarks [258]. Based on CodeLLaMA, Llemma is continually trained on mixtures of scientific papers, web data containing mathematical text and code (2T general tokens → 500B code-heavy tokens → 50~200B math-heavy tokens). Note that the pre-training data of Llemma also contains 5% general domain data as a form of regularization.

- *Long context.* Long context modeling is an important ability for LLMs, and many studies have explored extending the context windows of LLMs via continually training [254, 257]. With modifications on position embeddings (i.e., position interpolation) of RoPE-based LLMs [57, 99, 259], CodeLLaMA further extends the context window of LLaMA 2 (2.5T tokens with 4K context window → 20B tokens with 16K context window). LongLLaMA [257] also achieves longer context window with the help of external memory and a unique training objective (1T tokens with 2K context window → 10B tokens with 8K context window).

4.1.4 Summary of Data Preparation

In this part, we summarize the general procedure and key points to prepare pre-training data for LLMs, which are detailed in the following three aspects.

- *Data collection.* It is suggested to include diverse data sources in the pre-training data. Although Falcon [171] shows that webpages alone can be employed to train powerful LLMs, a more typical approach is to also incorporate diverse high-quality

23. We utilize the symbol “→” to represent the data order in data curriculum. For example, “2T webpage tokens → 500B code tokens” means that the LLM is firstly trained with 2T webpage tokens and subsequently with 500B code data tokens.

text like code, books, scientific papers, etc. If a LLM is specialized with a certain skill, the proportion of corresponding data source should be increased accordingly. For example, Gopher [64] and Chinchilla [34] are trained with approximately 40% of data from books. PaLM [44] and LaMDA [68] use approximately 50% conversational data.

- **Data cleaning.** After data collection, it is crucial to clean the raw corpus to enhance its quality as possible. First, deduplication is commonly used in existing work [99, 171, 248]. Second, low-quality text, toxic content, and data with privacy concerns should be removed at different granularities (e.g., document, passage or sentence). In practice, both heuristic and classifier-based methods can be employed for quality and toxicity filtering (e.g., CCNet [260], fastText [261], and Data-Juicer [262]). Third, with the cleaned data, one can further unify or specify the format for pre-training data, and perform the tokenization by training the tokenizer on the filtered and deduplicated corpus with libraries like SentencePiece [245].

- **Data scheduling.** With the preprocessed data, the next step is to determine the data mixture and the specific order of data for pre-training LLMs. To determine both settings, a practical way is to first train several small language models with multiple candidate plans and then select a good plan among them [59]. Overall, it is more difficult to find a suitable data curriculum. In practice, one can monitor the performance of intermediate model checkpoints on specific evaluation benchmarks, and dynamically tune the data mixture and distribution during pre-training. In this process, it is also useful to explore the potential relations between data sources and model abilities to instruct the design of data curriculum.

4.2 Architecture

In this section, we review the architecture design of LLMs, i.e., mainstream architecture, pre-training objective, and detailed configuration. Table 5 presents the model cards of several representative LLMs with public details.

4.2.1 Typical Architectures

Due to the excellent parallelizability and capacity, the Transformer architecture [22] has become the de facto backbone to develop various LLMs, making it possible to scale language models to hundreds or thousands of billions of parameters. In general, the mainstream architectures of existing LLMs can be roughly categorized into three major types, namely encoder-decoder, causal decoder, and prefix decoder, as shown in Figure 9.

Encoder-decoder Architecture. The vanilla Transformer model is built on the encoder-decoder architecture [22], which consists of two stacks of Transformer blocks as the encoder and decoder, respectively. The encoder adopts stacked multi-head self-attention layers to encode the input sequence for generating its latent representations, while the decoder performs cross-attention on these representations and autoregressively generates the target sequence. Encoder-decoder PLMs (e.g., T5 [82] and BART [24]) have shown effectiveness on a variety of NLP tasks. So far, there are only a small number of LLMs that are built based on the encoder-decoder architecture, e.g., Flan-T5 [69]. We leave a detailed discussion about the architecture selection in Section 4.2.5.

Causal Decoder Architecture. The causal decoder architecture incorporates the unidirectional attention mask, to guarantee that

each input token can only attend to the past tokens and itself. The input and output tokens are processed in the same fashion through the decoder. As representative language models of this architecture, the GPT-series models [26, 55, 122] are developed based on the causal-decoder architecture. In particular, GPT-3 [55] has successfully demonstrated the effectiveness of this architecture, also showing an amazing in-context learning capability of LLMs. Interestingly, GPT-1 [122] and GPT-2 [26] do not exhibit such superior abilities as those in GPT-3, and it seems that scaling plays an important role in increasing the model capacity of this model architecture. So far, the causal decoders have been widely adopted as the architecture of LLMs by various existing LLMs, such as OPT [90], BLOOM [78], and Gopher [64]. Note that both the causal decoder and prefix decoder discussed next belong to decoder-only architectures. When mentioning “decoder-only architecture”, it mainly refers to the causal decoder architecture in existing literature, unless specified.

Prefix Decoder Architecture. The prefix decoder architecture (a.k.a., non-causal decoder [263]) revises the masking mechanism of causal decoders, to enable performing bidirectional attention over the prefix tokens [264] and unidirectional attention only on generated tokens. In this way, like the encoder-decoder architecture, the prefix decoders can bidirectionally encode the prefix sequence and autoregressively predict the output tokens one by one, where the same parameters are shared during encoding and decoding. Instead of pre-training from scratch, a practical suggestion is to continually train causal decoders and then convert them into prefix decoders for accelerating convergence [29], e.g., U-PaLM [118] is derived from PaLM [56]. Existing representative LLMs based on prefix decoders include GLM-130B [93] and U-PaLM [118].

Mixture-of-Experts. For the above three types of architectures, we can further extend them via the mixture-of-experts (MoE) scaling, in which a subset of neural network weights for each input are sparsely activated, e.g., Switch Transformer [25] and GLaM [112]. The major merit is that MoE is a flexible way to scale up the model parameter while maintaining a constant computational cost [25]. It has been shown that substantial performance improvement can be observed by increasing either the number of experts or the total parameter size [265]. Despite the merits, training large MoE models may suffer from instability issues due to the complex, hard-switching nature of the routing operation. To enhance the training stability of MoE-based language models, techniques such as selectively using high-precision tensors in the routing module or initializing the model with a smaller range have been introduced [25]. More recently, there is widespread speculation that GPT-4 has been developed based on the MoE architecture, but without official verification.

Emergent Architectures. The conventional Transformer architecture typically suffers from quadratic computational complexity with respect to sequence length, resulting in a high processing cost for dealing with long inputs. To improve efficiency, recent studies aim to devise new architectures for language modeling, most based on parameterized state space models (SSM) [266], which can be viewed as a combination of RNN and CNN. On the one hand, SSM can generate outputs recursively like RNN, meaning that they only need to refer to the single previous state

TABLE 5: Model cards of several selected LLMs with public configuration details. Here, PE denotes position embedding, #L denotes the number of layers, #H denotes the number of attention heads, d_{model} denotes the size of hidden states, and MCL denotes the maximum context length during training.

Model	Category	Size	Normalization	PE	Activation	Bias	#L	#H	d_{model}	MCL
GPT3 [55]	Causal decoder	175B	Pre LayerNorm	Learned	GeLU	✓	96	96	12288	2048
PanGU- α [84]	Causal decoder	207B	Pre LayerNorm	Learned	GeLU	✓	64	128	16384	1024
OPT [90]	Causal decoder	175B	Pre LayerNorm	Learned	ReLU	✓	96	96	12288	2048
PaLM [56]	Causal decoder	540B	Pre LayerNorm	RoPE	SwiGLU	×	118	48	18432	2048
BLOOM [78]	Causal decoder	176B	Pre LayerNorm	ALiBi	GeLU	✓	70	112	14336	2048
MT-NLG [113]	Causal decoder	530B	-	-	-	-	105	128	20480	2048
Gopher [64]	Causal decoder	280B	Pre RMSNorm	Relative	-	-	80	128	16384	2048
Chinchilla [34]	Causal decoder	70B	Pre RMSNorm	Relative	-	-	80	64	8192	-
Galactica [35]	Causal decoder	120B	Pre LayerNorm	Learned	GeLU	×	96	80	10240	2048
LaMDA [68]	Causal decoder	137B	-	Relative	GeLU	-	64	128	8192	-
Jurassic-1 [107]	Causal decoder	178B	Pre LayerNorm	Learned	GeLU	✓	76	96	13824	2048
LLaMA [57]	Causal decoder	65B	Pre RMSNorm	RoPE	SwiGLU	×	80	64	8192	2048
LLaMA 2 [99]	Causal decoder	70B	Pre RMSNorm	RoPE	SwiGLU	×	80	64	8192	4096
Falcon [171]	Causal decoder	40B	Pre LayerNorm	RoPE	GeLU	×	60	64	8192	2048
GLM-130B [93]	Prefix decoder	130B	Post DeepNorm	RoPE	GeLU	✓	70	96	12288	2048
T5 [82]	Encoder-decoder	11B	Pre RMSNorm	Relative	ReLU	×	24	128	1024	512

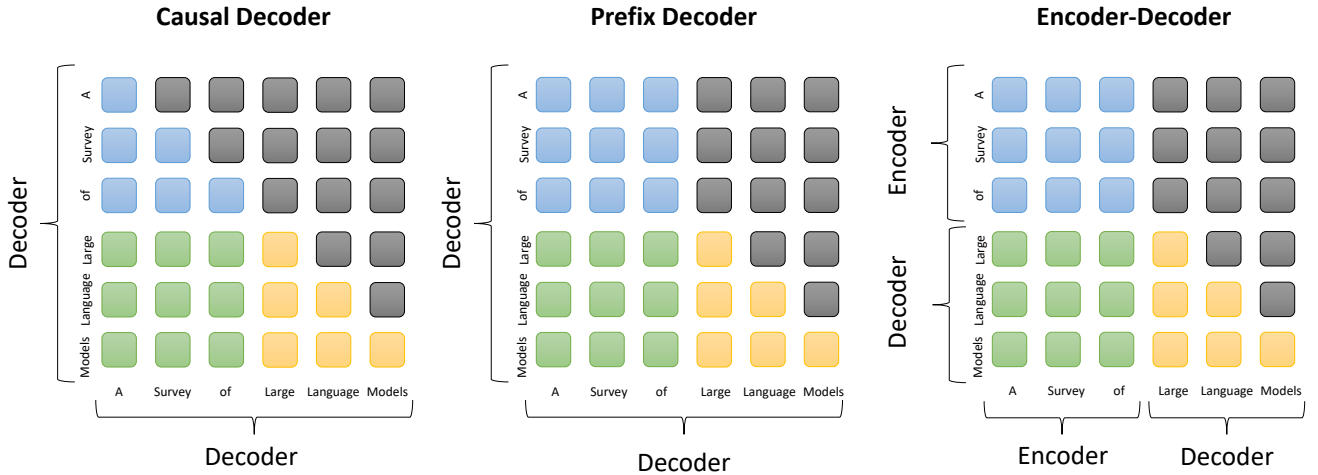


Fig. 9: A comparison of the attention patterns in three mainstream architectures. Here, the blue, green, yellow and grey rounded rectangles indicate the attention between prefix tokens, attention between prefix and target tokens, attention between target tokens, and masked attention respectively.

TABLE 6: Comparison of parallelism and complexity of different models. T represents sequence length, H represents the dimension of the input representation, N represents the dimension after compression in SSMs, and M represents the number of layers in each Hyena module.

Model	Decoding Complexity	Training Complexity
Transformer	$O(H(T + H))$	$O(TH(T + H))$
SSM	$O(H(N^2 + H))$	$O(TH(\log T + N^2 + H))$
Mamba	$O(H(N^2 + H))$	$O(TH(N^2 + H))$
RWKV	$O(H^2)$	$O(TH^2)$
RetNet	$O(H^2)$	$O(TH^2)$
Hyena	$O(MH(T + H))$	$O(TM(\log T + H))$

during decoding. It makes the decoding process more efficient as it eliminates the need to revisit all previous states as in conventional Transformers. On the other hand, these models have the capability to encode an entire sequence in parallel like Transformers via convolution computation. Thus, they can

benefit from the parallelism of GPUs with techniques such as Parallel Scan [267, 268], FFT [269, 270], and Chunkwise Recurrent [271]. Despite the high computation efficiency of SSMs, their performance still lags behind Transformer. Thus, several variants of SSM have been proposed, including Mamba [272], RetNet [271], RWKV [273], and Hyena [269].

- **Mamba.** Mamba [272] aims to selectively filter out or re-member information during state update. It replaces the original fixed parameters of SSM layers with functions of the input, selectively filtering out information of the previous state and the current input depending on the current input. Compared with traditional SSMs, Mamba has demonstrated improved text modeling capacities.

- **RWKV.** RWKV [273] combines the advantages of Transformer and RNN. It employs time-mixing modules, *i.e.*, RNN with gating, and channel-mixing modules that are special feedforward neural networks [273]. Within these modules, token shift, a linear combination of the current and previous token, is used instead of the token representation as the input.

- *RetNet*. RetNet [271] proposes multi-scale retention (MSR) to replace the attention module in Transformer. Similar to linear attention, in the MSR module, the input is first mapped into query, key, and value, and the product of key and value is employed to update the state. Then, the query is used to project the state into the output. Similar to traditional SSMs, RetNet keeps the parallel and recurrent computation capacity at the same time.

- *Hyena*. Hyena employs long convolution to replace the attention module. In the long convolution module, the filters based on relative positions are used to aggregate information at different positions into the middle representations, and gating functions are employed to further project intermediate representations into the final output. However, due to the long convolution, Hyena can not infer like RNN and must explicitly access all previous states.

4.2.2 Detailed Configuration

Since the launch of Transformer [22], various improvements have been proposed to enhance its training stability, performance, and computational efficiency. In this part, we will discuss the corresponding configurations for four major parts of the Transformer, including normalization, position embeddings, activation functions, and attention and bias. To make this survey more self-contained, we present the detailed formulations for these configurations in Table 7.

Normalization Methods. Training instability is a challenging issue for pre-training LLMs. To alleviate this issue, normalization is a widely adopted strategy to stabilize the training of neural networks. In the vanilla Transformer [22], LayerNorm [275] is employed. Recently, several advanced normalization techniques have been proposed as alternatives to LayerNorm, *e.g.*, RMSNorm, and DeepNorm.

- *LayerNorm*. In the early research, BatchNorm [284] is a commonly used normalization method. However, it is difficult to deal with sequence data of variable lengths and small-batch data. Thus, LayerNorm [275] is introduced to conduct layerwise normalization. Specifically, the mean and variance over all activations per layer are calculated to re-center and re-scale the activations.

- *RMSNorm*. To improve the training speed of LayerNorm (LN), RMSNorm [276] is proposed by re-scaling the activations with only the root mean square (RMS) of the summed activations, instead of the mean and variance. Related research has demonstrated its superiority in training speed and performance on Transformer [285]. Representative models that adopt RMSNorm include Gopher [64] and Chinchilla [34].

- *DeepNorm*. DeepNorm is proposed by Microsoft [277] to stabilize the training of deep Transformers. With DeepNorm as residual connections, Transformers can be scaled up to 1,000 layers [277], which has shown the advantages of stability and good performance. It has been adopted by GLM-130B [93].

Normalization Position. In addition to the normalization method, normalization position also plays a crucial role in the LLMs. There are generally three choices for the normalization position, *i.e.*, post-LN, pre-LN, and sandwich-LN.

- *Post-LN*. Post-LN is used in the vanilla Transformer [22], which is placed between residual blocks. However, existing work has found that the training of Transformers with post-LN tends to be unstable due to the large gradients near the output

layer [286]. Thus, post-LN is rarely employed in existing LLMs except combined with other strategies (*e.g.*, combining post-LN with pre-LN in GLM-130B [93]).

- *Pre-LN*. Different from post-LN, pre-LN [287] is applied before each sub-layer, and an additional LN is placed before the final prediction. Compared with post-LN, the Transformers with pre-LN are more stable in training. However, it performs worse than the variants with post-LN [288]. Despite the decreasing performance, most LLMs still adopt pre-LN due to the training stability. However, one exception is that pre-LN has been found unstable in GLM when training models more than 100B parameters [93].

- *Sandwich-LN*. Based on pre-LN, Sandwich-LN [274] adds extra LN before the residual connections to avoid the value explosion issues in Transformer layer outputs. However, it has been found that Sandwich-LN sometimes fails to stabilize the training of LLMs and may lead to the collapse of training [93].

Activation Functions. To obtain good performance, activation functions also need to be properly set in feed-forward networks. In existing LLMs, GeLU activations [289] are widely used. Specially, in the latest LLMs (*e.g.*, PaLM and LaMDA), variants of GLU activation [281, 290] have also been utilized, especially the SwiGLU and GeGLU variants, which often achieve better performance in practice [285]. However, compared with GeLU, they require extra parameters (about 50%) in the feed-forward networks [291].

Position Embeddings. Since the self-attention modules in Transformer are permutation equivariant, position embeddings (PE) are employed to inject absolute or relative position information for modeling sequences.

- *Absolute position embedding*. In the vanilla Transformer [22], absolute position embeddings are employed. At the bottoms of the encoder and the decoder, the absolute positional embeddings are added to the input embeddings. There are two variants of absolute position embeddings proposed in the vanilla Transformer [22], *i.e.*, sinusoidal and learned position embeddings, where the latter is commonly used in existing pre-trained language models.

- *Relative position embedding*. Unlike absolute position embeddings, relative positional embeddings are generated according to the offsets between keys and queries [292]. A popular variant of relative PE was introduced in Transformer-XL [293, 294]. The calculation of attention scores between keys and queries has been modified to introduce learnable embeddings corresponding to relative positions. T5 [82] further simplified relative positional embeddings, which was subsequently adopted by Gopher [64]. Specifically, it adds learnable scalars to the attention scores, where the scalars are calculated based on the distances between the positions of the query and the key. Compared with the absolute PE, Transformers with relative position embedding can generalize to sequences longer than those sequences for training, *i.e.*, extrapolation [283].

- *Rotary position embedding*. Rotary position embedding (RoPE) [282] sets specific rotatory matrices based on the absolute position of each key or query. The scores between keys and queries can be computed with relative position information (Table 7). RoPE combines each consecutive pair of elements in query and key vectors as a *dimension*, so there are $d/2$ dimensions for an original d -length embedding. For each dimension $i \in \{1, \dots, d/2\}$, the pair of involved elements will rotate based on the rotation angle $t \cdot \theta_i$, where t denotes the position index

TABLE 7: Detailed formulations for the network configurations. Here, Sublayer denotes a FFN or a self-attention module in a Transformer layer, d denotes the size of hidden states, $\mathbf{p}_i$ denotes position embedding at position i , A_{ij} denotes the attention score between a query and a key, r_{i-j} denotes a learnable scalar based on the offset between the query and the key, and $\mathbf{R}_{\Theta,t}$ denotes a rotary matrix with rotation degree $t \cdot \Theta$.

Configuration	Method	Equation
Normalization position	Post Norm [22]	$\text{Norm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$
	Pre Norm [26]	$\mathbf{x} + \text{Sublayer}(\text{Norm}(\mathbf{x}))$
	Sandwich Norm [274]	$\mathbf{x} + \text{Norm}(\text{Sublayer}(\text{Norm}(\mathbf{x})))$
Normalization method	LayerNorm [275]	$\frac{\mathbf{x} - \mu}{\sigma} \cdot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$
	RMSNorm [276]	$\frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \gamma, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$
	DeepNorm [277]	$\text{LayerNorm}(\alpha \cdot \mathbf{x} + \text{Sublayer}(\mathbf{x}))$
Activation function	ReLU [278]	$\text{ReLU}(\mathbf{x}) = \max(\mathbf{x}, \mathbf{0})$
	GeLU [279]	$\text{GeLU}(\mathbf{x}) = 0.5\mathbf{x} \otimes [1 + \text{erf}(\mathbf{x}/\sqrt{2})], \quad \text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$
	Swish [280]	$\text{Swish}(\mathbf{x}) = \mathbf{x} \otimes \text{sigmoid}(\mathbf{x})$
	SwiGLU [281]	$\text{SwiGLU}(\mathbf{x}_1, \mathbf{x}_2) = \text{Swish}(\mathbf{x}_1) \otimes \mathbf{x}_2$
	GeGLU [281]	$\text{GeGLU}(\mathbf{x}_1, \mathbf{x}_2) = \text{GeLU}(\mathbf{x}_1) \otimes \mathbf{x}_2$
Position embedding	Absolute [22]	$\mathbf{x}_i = \mathbf{x}_i + \mathbf{p}_i$
	Relative [82]	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{x}_j^T \mathbf{W}_k^T + r_{i-j}$
	RoPE [282]	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{R}_{\Theta, i-j} \mathbf{x}_j^T \mathbf{W}_k^T = (\mathbf{W}_q \mathbf{x}_i \mathbf{R}_{\Theta, i}) (\mathbf{W}_k \mathbf{x}_j \mathbf{R}_{\Theta, j})^T$
	ALiBi [283]	$A_{ij} = \mathbf{W}_q \mathbf{x}_i \mathbf{x}_j^T \mathbf{W}_k^T - m(i-j)$

and θ_i is the basis in the dimension. Following sinusoidal position embeddings [22], RoPE defines the *basis* θ_i as an exponentiation of the *base* b (set to 10000 by default):

$$\Theta = \{\theta_i = b^{-2(i-1)/d} | i \in \{1, 2, \dots, d/2\}\}. \quad (4)$$

Furthermore, a recent study [295] defines the distance required to rotate one cycle (2π) for each dimension as wavelength:

$$\lambda_i = 2\pi b^{2(i-1)/d} = 2\pi/\theta_i. \quad (5)$$

Due to the excellent performance and the long-term decay property, RoPE is widely adopted in the latest LLMs, e.g., PaLM [56] and LLaMA [57]. Based on RoPE, xPos [296] further improves the translation invariance and length extrapolation of Transformer. At each dimension of the rotation angle vector, xPos adds a special exponential decay that is smaller when the basis is larger. It can alleviate the unstable phenomenon during training as the distance increases.

- **ALiBi.** ALiBi [283] is proposed to improve the extrapolation of Transformer. Similar to relative position embedding, it biases attention scores with a penalty based on the distances between keys and queries. Different from the relative positional embedding methods like T5 [82], the penalty scores in ALiBi are pre-defined without any trainable parameters. Empirical results in [283] have shown that ALiBi has a better extrapolation performance on sequences that are longer than those for training than several popular position embedding methods such as sinusoidal PE [22], RoPE [282], and T5 bias [82]. In addition, it has been shown that ALiBi can also improve training stability in BLOOM [78].

Attention. Attention mechanism is a critical component of Transformer. It allows the tokens across the sequence to interact with each other and compute the representations of the input and output sequence.

- **Full attention.** In the vanilla Transformer [22], the attention mechanism is conducted in a pairwise way, considering the relations between all token pairs in a sequence. It adopts scaled

dot-product attention, in which the hidden states are mapped into queries, keys, and values. Additionally, Transformer uses multi-head attention instead of single attention, projecting the queries, keys, and values with different projections in different heads. The concatenation of the output of each head is taken as the final output.

- **Sparse attention.** A crucial challenge of full attention is the quadratic computational complexity, which becomes a burden when dealing with long sequences. Therefore, various efficient Transformer variants are proposed to reduce the computational complexity of the attention mechanism [297, 298]. For instance, locally banded sparse attention (*i.e.*, Factorized Attention [299]) has been adopted in GPT-3 [55]. Instead of the whole sequence, each query can only attend to a subset of tokens based on the positions.

- **Multi-query/grouped-query attention.** Multi-query attention refers to the attention variant where different heads share the same linear transformation matrices on the keys and values [300]. It achieves higher inference speed with only a minor sacrifice in model quality. Representative models with multi-query attention include PaLM [56] and StarCoder [98]. To make a trade-off between multi-query attention and multi-head attention, grouped-query attention (GQA) [301] has been explored. In GQA, heads are assigned into different groups, and those heads that belong to the same group will share the same transformation matrices. Specially, GQA has been adopted and empirically tested in the recently released LLaMA 2 model [99].

- **FlashAttention.** Different from most existing approximate attention methods that trade-off model quality to improve the computing efficiency, FlashAttention [302] proposes to optimize the speed and memory consumption of attention modules on GPUs from an IO-aware perspective. There exist different levels of memory on modern GPUs, e.g., SRAM with a fast IO and HBM with a relatively slow IO. FlashAttention organizes the input into blocks and introduces necessary recomputation, both to make better use of the fast memory SRAM. Implemented as a fused kernel in CUDA, FlashAttention has been integrated into

PyTorch [211], DeepSpeed [74], and Megatron-LM [75]. The updated version FlashAttention-2 [303] further optimizes the work partitioning of GPU thread blocks and warps, leading to around $2\times$ speedup when compared to the original FlashAttention.

- **PagedAttention.** It has been observed when LLM are deployed on servers, GPU memory is largely occupied by cached attention key and value tensors (called *KV cache*). The major reason is that the input lengths are often varied, leading to fragmentation and over-reservation issues. Inspired by the classic paging technique in operating systems, PagedAttention has been proposed to improve the memory efficiency and throughput of deployed LLMs [304]. In detail, PagedAttention partitions each sequence into subsequences, and the corresponding KV caches of these subsequences are allocated into non-contiguous physical blocks. The paging technique increases the GPU utilization and enables efficient memory sharing in parallel sampling.

To put all these discussions together, we summarize the suggestions from existing literature for detailed configuration. For stronger generalization and training stability, it is suggested to choose the pre RMSNorm for layer normalization, and SwiGLU or GeGLU as the activation function. In addition, LN may not be used immediately after embedding layers, which is likely to incur performance degradation. As for position embeddings, RoPE or ALiBi is a better choice since it performs better on long sequences.

4.2.3 Pre-training Tasks

Pre-training plays a key role that encodes general knowledge from large-scale corpus into the massive model parameters. For training LLMs, there are two commonly used pre-training tasks, namely language modeling and denoising autoencoding.

Language Modeling. The language modeling task (LM) is the most commonly used objective to pre-train decoder-only LLMs, e.g., GPT3 [55] and PaLM [56]. Given a sequence of tokens $\mathbf{x} = \{x_1, \dots, x_n\}$, the LM task aims to autoregressively predict the target tokens x_i based on the preceding tokens $x_{<i}$ in a sequence. A general training objective is to maximize the following likelihood:

$$\mathcal{L}_{LM}(\mathbf{x}) = \sum_{i=1}^n \log P(x_i | \mathbf{x}_{<i}). \quad (6)$$

Since most language tasks can be cast as the prediction problem based on the input, these decoder-only LLMs might be potentially advantageous to implicitly learn how to accomplish these tasks in a unified LM way. Some studies have also revealed that decoder-only LLMs can be naturally transferred to certain tasks by autoregressively predicting the next tokens [26, 55], without fine-tuning. An important variant of LM is the *prefix language modeling* task, which is designed for pre-training models with the prefix decoder architecture. The tokens within a randomly selected prefix would not be used in computing the loss of prefix language modeling. With the same amount of tokens seen during pre-training, prefix language modeling performs slightly worse than language modeling, since fewer tokens in the sequence are involved for model pre-training [29].

Denoising Autoencoding. In addition to conventional LM, the denoising autoencoding task (DAE) has also been widely used to pre-train language models [24, 82]. The inputs $\mathbf{x}_{\setminus \tilde{\mathbf{x}}}$ for DAE task are corrupted text with randomly replaced spans. Then, the

I am sleepy. I start a pot of _ _ _ _ _					
coffee	0.661	strong	0.008	soup	0.005
water	0.119	black	0.008	...	...
tea	0.057	hot	0.007	happy	4.3e-6
rice	0.017	oat	0.006	Boh	4.3e-6
chai	0.012	beans	0.006	...	...

Fig. 10: The probability distribution over the vocabulary in descending order for the next token of the context “*I am sleepy. I start a pot of*”. For ease of discussion, this example is given in word units instead of subword units.

language models are trained to recover the replaced tokens $\tilde{\mathbf{x}}$. Formally, the training objective of DAE is denoted as follows:

$$\mathcal{L}_{DAE}(\mathbf{x}) = \log P(\tilde{\mathbf{x}} | \mathbf{x}_{\setminus \tilde{\mathbf{x}}}). \quad (7)$$

However, the DAE task seems to be more complicated in implementation than LM task. As a result, it has not been widely used to pre-train large language models. Existing LLMs that take DAE as pre-training objectives include T5 [82] and GLM-130B [93]. These models are mainly trained to recover the replaced spans in an autoregressive way.

Mixture-of-Denoisers. Mixture-of-Denoisers (MoD) [89], also known as UL2 loss, was introduced as a unified objective for pre-training language models. MoD regards both LM and DAE objectives as different types of denoising tasks, namely S-denoiser (LM), R-denoiser (DAE, short span and low corruption), and X-denoiser (DAE, long span or high corruption). Among the three denoising tasks, S-denoiser is similar to the conventional LM objective (Equation (6)), while R-denoiser and X-denoiser are similar to DAE objectives (Equation (7)) but differ from each other in the lengths of spans and ratio of corrupted text. For input sentences started with different special tokens (i.e., $\{[R], [S], [X]\}$), the model will be optimized using the corresponding denoisers. MoD has been applied in the latest PaLM 2 model [120].

4.2.4 Decoding Strategy

After the LLMs have been pre-trained, it is essential to employ a specific decoding strategy to generate the appropriate output from the LLMs.

Background. We start the discussion with the prevalent decoder-only architecture, and introduce the auto-regressive decoding mechanism. Since such LLMs are pre-trained based on the language modeling task (Equation 6), a basic decoding method is *greedy search* that predicts the most likely token at each step based on the previously generated tokens, formally modeled as:

$$x_i = \arg \max_x P(x | \mathbf{x}_{<i}), \quad (8)$$

where x_i is the token with the highest probability at i -th step of generation conditioned on the context $\mathbf{x}_{<i}$. For instance in Figure 10, when predicting the next token of the sentence “*I am sleepy. I start a pot of*”, greedy search selects the token “*coffee*” which has the highest probability at the current step. Greedy search can achieve satisfactory results in text generation tasks (e.g., machine translation and text summarization), in which the output is highly dependent on the input [305]. However, in terms of open-ended generation tasks (e.g., story generation and

dialog), greedy search sometimes tends to generate awkward and repetitive sentences [306].

As another alternative decoding strategy, sampling-based methods are proposed to randomly select the next token based on the probability distribution to enhance the randomness and diversity during generation:

$$x_i \sim P(x|x_{<i}). \quad (9)$$

For the example in Figure 10, sampling-based methods will sample the word “coffee” with higher probability while also retaining the possibilities of selecting the rest words, “water”, “tea”, “rice”, etc.

Not limited to the decoder-only architecture, these two decoding methods can be generally applied to encoder-decoder models and prefix decoder models in a similar way.

Improvement for Greedy Search. Selecting the token with the highest probability at each step may result in overlooking a sentence with a higher overall probability but a lower local estimation. Next, we introduce several improvement strategies to alleviate this issue.

- *Beam search*. Beam search [307] retains the sentences with the n (beam size) highest probabilities at each step during the decoding process, and finally selects the generated response with the top probability. Typically, the beam size is configured within the range of 3 to 6. However, opting for a larger beam size might result in a decline in performance [308].

- *Length penalty*. Since beam search favours shorter sentences, imposing length penalty (*a.k.a.*, length normalization) is a commonly used technique [309] to overcome this issue, which normalizes the sentence probability according to the sentence length (divided by an exponential power α of the length).

Besides, some researchers [310] propose to penalize the generation of previously generated tokens or n -grams to alleviate the issue of repetitive generation. In addition, diverse beam search [311] can be leveraged to produce a set of diverse outputs based on the same input.

Improvement for Random Sampling. Sampling-based methods sample the token over the whole vocabulary, which may select wrong or irrelevant tokens (*e.g.*, “happy” and “Boh” in Figure 10) based on the context. To improve the generation quality, several strategies have been proposed for mitigating or preventing the selection of words with exceedingly low probabilities.

- *Temperature sampling*. To modulate the randomness of sampling, a practical method is to adjust the temperature coefficient of the softmax function for computing the probability of the j -th token over the vocabulary:

$$P(x_j|x_{<j}) = \frac{\exp(l_j/t)}{\sum_{j'} \exp(l_{j'}/t)}, \quad (10)$$

where $l_{j'}$ is the logits of each word and t is the temperature coefficient. Reducing the temperature t increases the chance of selecting words with high probabilities while decreases the chances of selecting words with low probabilities. When t is set to 1, it becomes the default random sampling; when t is approaching 0, it is equivalent to greedy search. In addition, when t goes to infinity, it degenerates to uniform sampling.

- *Top- k sampling*. Different from temperature sampling, top- k sampling directly truncates the tokens with lower probability and only samples from the tokens with the top k highest probabilities [312]. For example in Figure 10, top-5 sampling will sample

from the words “coffee”, “water”, “tea”, “rice”, and “chai” from their re-scaled probabilities.

- *Top- p sampling*. Since top- k sampling does not consider the overall possibility distribution, a constant value of k may be not be suitable for different contexts. Therefore, top- p sampling (*a.k.a.*, nucleus sampling) is proposed by sampling from the smallest set having a cumulative probability above (or equal to) p [306]. In practice, the smallest set can be constructed by gradually adding tokens from the vocabulary sorted in descending order of generative probability, until their cumulative value exceeds p .

Recently, researchers have also explored other sampling strategies for LLMs. For instance, η -sampling [313] further improves top- p sampling by introducing a dynamic threshold based on the probability distribution. Furthermore, *contrastive search* [314] and *typical sampling* [315] can be utilized to improve the generation coherence during decoding. Since it has been found that large models tend to assign higher probability to important tokens compared to small models, *contrastive decoding* [316] utilizes a larger LM (*e.g.*, OPT-13B) and a smaller LM (*e.g.*, OPT-125M) to measure their log-likelihood differences. Subsequently, tokens are sampled based on the delta value of the probability distribution, thereby amplifying the impact of important tokens. Based on this contrastive idea, DoLa [317] further extends this approach to contrasting the logits across different layers of a single LLM, as higher layers tend to assign more weight to important tokens.

Practical Settings. In practice, existing libraries (*e.g.*, Transformers [201]) and public APIs of LLMs (*e.g.*, OpenAI) have supported various decoding strategies to serve different scenarios of text generation. Next, we present the decoding settings of several representative LLMs:

- *T5* [82] utilizes greedy search as the default setting and applies beam search (beam size of 4) with a length penalty of 0.6 for translation and summarization tasks.

- *GPT-3* [55] employs beam search with a beam size of 4 and a length penalty of 0.6 for all generation tasks.

- *Alpaca* [146] utilizes sampling-based strategies with top- k ($k = 50$), top- p ($p = 0.9$), and temperature of 0.7 for open-ended generation.

- *LLaMA* [57] applies diverse decoding strategies tailored to specific tasks. For instance, it employs the greedy search for question answering tasks while utilizes a sampling strategy with the temperature settings of 0.1 (pass@1) and 0.8 (pass@100) for code generation.

- *OpenAI API* supports several basic decoding strategies, including greedy search (by setting temperature to 0), beam search (with the setting `best_of`), temperature sampling (with the setting `temperature`), nucleus sampling (with the setting `top_p`). It also introduce parameters `presence_penalty` and `frequency_penalty` to control the repetition degree of generation. According to the OpenAI’s document, their APIs would produce different outputs even if the input and the hyper-parameters are the same. Setting temperature to 0 can yield more deterministic outputs, albeit with a slight chance of variability.

4.2.5 Summary and Discussion

The choice of architecture and pre-training tasks may incur different inductive biases for LLMs, which would lead to different model capacities. In this part, we discuss one open issue about the architecture choice for LLMs.

Why does Predicting the Next Word Works?

The essence of decoder-only architecture is to *accurately predict the next word* for reconstructing the pre-training data. Till now, there has been no formal study that theoretically demonstrates its advantage over other architectures. An interesting explanation was from Ilya Sutskever during the interview held by Jensen Huang^a. The original transcript from the interview was copied below^b:

Say you read a detective novel. It's like complicated plot, a storyline, different characters, lots of events, mysteries like clues, it's unclear. Then, let's say that at the last page of the book, the detective has gathered all the clues, gathered all the people and saying, "okay, I'm going to reveal the identity of whoever committed the crime and that person's name is". Predict that word. ...
Now, there are many different words. But predicting those words better and better, the understanding of the text keeps on increasing. GPT-4 predicts the next word better.

a. <https://www.nvidia.com/en-us/on-demand/session/gtcspring23-S52092/>

b. <https://lifaearchitect.ai/ilya/>

Architecture Choice. In earlier literature of pre-trained language models, there are lots of discussions on the effects of different architectures [29, 89]. However, most LLMs are developed based on the causal decoder architecture, and there still lacks a theoretical analysis on its advantage over the other alternatives. Next, we briefly summarize existing discussions on this issue.

- By pre-training with the LM objective, it seems that causal decoder architecture can achieve a superior zero-shot and few-shot generalization capacity. Existing research has shown that without multi-task fine-tuning, the causal decoder has better zero-shot performance than other architectures [29]. The success of GPT-3 [55] has demonstrates that the large causal decoder model can be a good few-shot learner. In addition, instruction tuning and alignment tuning discussed in Section 5 have been proven to further enhance the capability of large causal decoder models [66, 67, 69].

- Scaling law has been widely observed in causal decoders. By scaling the model size, the dataset size, and the total computation, the performance of causal decoders can be substantially improved [30, 55]. Thus, it has become an important strategy to increase the model capacity of the causal decoder via scaling. However, more detailed investigation on encoder-decoder models is still lacking, and more efforts are needed to investigate the performance of encoder-decoder models at a large scale.

More research efforts about the discussions on architectures and pre-training objectives are in need to analyze how the choices of the architecture and pre-training tasks affect the capacity of LLMs, especially for encoder-decoder architectures. Despite the effectiveness of decoder-only architecture, it is also suggested to make more diverse exploration on architecture design. Besides

the major architecture, the detailed configuration of LLM is also worth attention, which has been discussed in Section 4.2.2.

4.3 Model Training

In this part, we review the important settings, techniques, or tricks for training LLMs.

4.3.1 Optimization Setting

For parameter optimization of LLMs, we present the commonly used settings for batch training, learning rate, optimizer, and training stability.

Batch Training. For language model pre-training, existing work generally sets the batch size to a large number (e.g., 2,048 examples or 4M tokens) to improve the training stability and throughput. For LLMs such as GPT-3 and PaLM, they have introduced a new strategy that dynamically increases the batch size during training, ultimately reaching a million scale. Specifically, the batch size of GPT-3 is gradually increasing from 32K to 3.2M tokens. Empirical results have demonstrated that the dynamic schedule of batch size can effectively stabilize the training process of LLMs [56].

Learning Rate. Existing LLMs usually adopt a similar learning rate schedule with the warm-up and decay strategies during pre-training. Specifically, in the initial 0.1% to 0.5% of the training steps, a linear warm-up schedule is employed for gradually increasing the learning rate to the maximum value that ranges from approximately 5×10^{-5} to 1×10^{-4} (e.g., 6×10^{-5} for GPT-3). Then, a cosine decay strategy is adopted in the subsequent steps, gradually reducing the learning rate to approximately 10% of its maximum value, until the convergence of the training loss.

Optimizer. The Adam optimizer [318] and AdamW optimizer [319] are widely utilized for training LLMs (e.g., GPT-3), which are based on adaptive estimates of lower-order moments for first-order gradient-based optimization. Commonly, its hyper-parameters are set as follows: $\beta_1 = 0.9$, $\beta_2 = 0.95$ and $\epsilon = 10^{-8}$. Meanwhile, the Adafactor optimizer [320] has also been utilized in training LLMs (e.g., PaLM and T5), which is a variant of the Adam optimizer specially designed for conserving GPU memory during training. The hyper-parameters of the Adafactor optimizer are set as: $\beta_1 = 0.9$ and $\beta_2 = 1.0 - k^{-0.8}$, where k denotes the number of training steps.

Stabilizing the Training. During the pre-training of LLMs, it often suffers from the training instability issue, which may cause the model collapse. To address this issue, weight decay and gradient clipping have been widely utilized, where existing studies [55, 78, 90, 93, 113] commonly set the threshold of gradient clipping to 1.0 and weight decay rate to 0.1. However, with the scaling of LLMs, the training loss spike is also more likely to occur, leading to unstable training. To mitigate this problem, PaLM [56] and OPT [90] use a simple strategy that restarts the training process from an earlier checkpoint before the occurrence of the spike and skips over the data that may have caused the problem. Further, GLM [93] finds that the abnormal gradients of the embedding layer usually lead to spikes, and proposes to shrink the embedding layer gradients to alleviate it.

4.3.2 Scalable Training Techniques

As the model and data sizes increase, it has become challenging to efficiently train LLMs under a limited computational

TABLE 8: Detailed optimization settings of several existing LLMs.

Model	Batch Size (#tokens)	Learning Rate	Warmup	Decay Method	Optimizer	Precision Type	Weight Decay	Grad Clip	Dropout
GPT3 (175B)	32K→3.2M	6×10^{-5}	yes	cosine decay to 10%	Adam	FP16	0.1	1.0	-
PanGu- α (200B)	-	2×10^{-5}	-	-	Adam	-	0.1	-	-
OPT (175B)	2M	1.2×10^{-4}	yes	manual decay	AdamW	FP16	0.1	-	0.1
PaLM (540B)	1M→4M	1×10^{-2}	no	inverse square root	Adafactor	BF16	$1/r^2$	1.0	0.1
BLOOM (176B)	4M	6×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	0.0
MT-NLG (530B)	64 K→3.75M	5×10^{-5}	yes	cosine decay to 10%	Adam	BF16	0.1	1.0	-
Gopher (280B)	3M→6M	4×10^{-5}	yes	cosine decay to 10%	Adam	BF16	-	1.0	-
Chinchilla (70B)	1.5M→3M	1×10^{-4}	yes	cosine decay to 10%	AdamW	BF16	-	-	-
Galactica (120B)	2M	7×10^{-6}	yes	linear decay to 10%	AdamW	-	0.1	1.0	0.1
LaMDA (137B)	256K	-	-	-	-	BF16	-	-	-
Jurassic-1 (178B)	32 K→3.2M	6×10^{-5}	yes	-	-	-	-	-	-
LLaMA (65B)	4M	1.5×10^{-4}	yes	cosine decay to 10%	AdamW	-	0.1	1.0	-
LLaMA 2 (70B)	4M	1.5×10^{-4}	yes	cosine decay to 10%	AdamW	-	0.1	1.0	-
Falcon (40B)	2M	1.85×10^{-4}	yes	cosine decay to 10%	AdamW	BF16	0.1	-	-
GLM (130B)	0.4M→8.25M	8×10^{-5}	yes	cosine decay to 10%	AdamW	FP16	0.1	1.0	0.1
T5 (11B)	64K	1×10^{-2}	no	inverse square root	AdaFactor	-	-	-	0.1
ERNIE 3.0 Titan (260B)	-	1×10^{-4}	-	-	Adam	FP16	0.1	1.0	-
PanGu- Σ (1.085T)	0.5M	2×10^{-5}	yes	-	Adam	FP16	-	-	-

resource. Especially, two primary technical issues are required to be resolved, *i.e.*, increasing training throughput and loading larger models into GPU memory. In this part, we review several widely used approaches in existing work to address the above two challenges, namely 3D parallelism [75, 321, 322] and mixed precision training [323], and also give general suggestions about how to utilize them for training.

3D Parallelism. 3D parallelism is actually a combination of three commonly used parallel training techniques, namely data parallelism, pipeline parallelism [321, 322], and tensor parallelism [75]²⁴. We next introduce the three parallel training techniques.

- *Data parallelism.* Data parallelism is one of the most fundamental approaches to improving the training throughput. It replicates the model parameters and optimizer states across multiple GPUs and then distributes the whole training corpus into these GPUs. In this way, each GPU only needs to process the assigned data for it, and performs the forward and backward propagation to obtain the gradients. The computed gradients on different GPUs will be further aggregated to obtain the gradients of the entire batch for updating the models in all GPUs. In this way, as the calculations of gradients are independently performed on different GPUs, the data parallelism mechanism is highly scalable, enabling the way that increases the number of GPUs to improve training throughput. Furthermore, this technique is simple in implementation, and most of existing popular deep learning libraries have already implemented data parallelism, such as TensorFlow and PyTorch.

- *Pipeline parallelism.* Pipeline parallelism aims to distribute the different layers of a LLM into multiple GPUs. Especially, in the case of a Transformer model, pipeline parallelism loads consecutive layers onto the same GPU, to reduce the cost of transmitting the computed hidden states or gradients between GPUs. However, a naive implementation of pipeline parallelism may result in a lower GPU utilization rate as each GPU has to wait for the previous one to complete the computation, leading to the unnecessary cost of *bubbles overhead* [321]. To reduce these bubbles in pipeline parallelism, GPipe [321] and PipeDream [322]

propose the techniques of padding multiple batches of data and asynchronous gradient update to improve the pipeline efficiency.

- *Tensor parallelism.* Tensor parallelism is also a commonly used technique that aims to decompose the LLM for multi-GPU loading. Unlike pipeline parallelism, tensor parallelism focuses on decomposing the tensors (the parameter matrices) of LLMs. For a matrix multiplication operation $Y = XA$ in the LLM, the parameter matrix A can be split into two submatrices, A_1 and A_2 , by column, which can be expressed as $Y = [XA_1, XA_2]$. By placing matrices A_1 and A_2 on different GPUs, the matrix multiplication operation would be invoked at two GPUs in parallel, and the final result can be obtained by combining the outputs from the two GPUs through across-GPU communication. Currently, tensor parallelism has been supported in several open-source libraries, *e.g.*, Megatron-LM [75], and can be extended to higher-dimensional tensors. Also, Colossal-AI has implemented tensor parallelism for higher-dimensional tensors [324–326] and proposed sequence parallelism [327] especially for sequence data, which can further decompose the attention operation of the Transformer model.

Mixed Precision Training. In previous PLMs (*e.g.*, BERT [23]), 32-bit floating-point numbers, also known as FP32, have been predominantly used for pre-training. In recent years, to pre-train extremely large language models, some studies [323] have started to utilize 16-bit floating-point numbers (FP16), which reduces memory usage and communication overhead. Additionally, as popular NVIDIA GPUs (*e.g.*, A100) have twice the amount of FP16 computation units as FP32, the computational efficiency of FP16 can be further improved. However, existing work has found that FP16 may lead to the loss of computational accuracy [64, 78], which affects the final model performance. To alleviate it, an alternative called *Brain Floating Point (BF16)* has been used for training, which allocates more exponent bits and fewer significant bits than FP16. For pre-training, BF16 generally performs better than FP16 on representation accuracy [78].

Overall Training Suggestion. In practice, the above training techniques, especially 3D parallelism, are often jointly used to improve the training throughput and large model loading. For instance, researchers have incorporated 8-way data parallelism, 4-

²⁴. Model parallelism is a more broader term that includes tensor parallelism and pipeline parallelism in some work [75].

way tensor parallelism, and 12-way pipeline parallelism, enabling the training of BLOOM [78] on 384 A100 GPUs. Currently, open-source libraries like DeepSpeed [74], Colossal-AI [203], and Alpaca [328] can well support the three parallel training methods. To reduce the memory redundancy, ZeRO, FSDP, and activation recomputation techniques [77, 329] can be also employed for training LLMs, which have already been integrated into DeepSpeed, PyTorch, and Megatron-LM. In addition, the mixed precision training technique such as BF16 can be also leveraged to improve the training efficiency and reduce GPU memory usage, while it requires necessary support on hardware (e.g., A100 GPU). Because training large models is a time-intensive process, it would be useful to forecast the model performance and detect abnormal issues at an early stage. For this purpose, GPT-4 [46] has recently introduced a new mechanism called *predictable scaling* built on a deep learning stack, enabling the performance prediction of large models with a much smaller model, which might be quite useful for developing LLMs. In practice, one can further leverage the supporting training techniques of mainstream deep learning frameworks. For instance, PyTorch supports the data parallel training algorithm FSDP [330] (i.e., fully sharded data parallel), which allows for partial offloading of training computations to CPUs if desired.

5 Post-training of LLMs

After pre-training, LLMs can acquire the general abilities for solving various tasks. However, an increasing number of studies have shown that LLM’s abilities can be further adapted according to specific goals. In this section, we introduce two major approaches to adapting pre-trained LLMs, namely instruction tuning and alignment tuning. The former approach mainly aims to enhance (or unlock) the abilities of LLMs, while the latter approach aims to align the behaviors of LLMs with human values or preferences. Further, we will also discuss efficient tuning and quantization for model adaptation in resource-limited settings. In what follows, we will introduce the four parts in detail.

5.1 Instruction Tuning

In essence, instruction tuning is the approach to fine-tuning pre-trained LLMs on a collection of formatted instances in the form of natural language [67], which is highly related to supervised fine-tuning [66] and multi-task prompted training [28]. In order to perform instruction tuning, we first need to collect or construct instruction-formatted instances. Then, we employ these formatted instances to fine-tune LLMs in a supervised learning way (e.g., training with the sequence-to-sequence loss). After instruction tuning, LLMs can demonstrate superior abilities to generalize to unseen tasks [28, 67, 69], even in a multilingual setting [94].

A recent survey [331] presents a systematic overview of the research on instruction tuning. In comparison to that, we mainly focus on the effect of instruction tuning on LLMs and provide detailed guidelines or strategies for instance collection and tuning. In addition, we also discuss the use of instruction tuning for satisfying the real needs of users, which has been widely applied in existing LLMs, e.g., InstructGPT [66] and GPT-4 [46].

5.1.1 Formatted Instance Construction

Generally, an instruction-formatted instance consists of a task description (called an *instruction*), an optional input, the corresponding output, and a small number of demonstrations (optional). As important public resources, existing studies have released a large number of labeled data formatted in natural language (see the list of available resources in Table 3) as introduced in Section 3.3.1. Next, we introduce four major methods for constructing formatted instances (see an illustration in Figure 11) and then discuss several key factors for instance construction.

Formatting NLP Task Datasets. Before instruction tuning was proposed, several early studies [181, 332, 333] collected the instances from a diverse range of traditional NLP tasks (e.g., text summarization, text classification, and translation) to create supervised multi-task training datasets. As a major source of instruction tuning instances, it is convenient to format these multi-task training datasets with natural language task descriptions. Specifically, recent work [28, 66, 67, 88] augments the labeled datasets with human-written task descriptions, which instructs LLMs to understand the tasks by explaining the task goal. For example, in Figure 11(a), a task description “*Please answer this question*” is added for each example in the question-answering task. After instruction tuning, LLMs can generalize well to other unseen tasks by following their task descriptions [28, 67, 69]. In particular, it has been shown that instructions are the crucial factor in task generalization ability for LLMs [67]: by fine-tuning the model on labeled datasets with the task descriptions removed, it results in a dramatic drop in model performance. To better generate labeled instances for instruction tuning, a crowd-sourcing platform, PromptSource [180] has been proposed to effectively create, share, and verify the task descriptions for different datasets. To enrich the training instances, several studies [28, 181, 334] also try to invert the input-output pairs of existing instances with specially designed task descriptions for instruction tuning. For instance, given a question-answer pair, we can create a new instance by predicting the answer-conditioned question (e.g., “*Please generate a question based on the answer:*”).

Formatting Daily Chat Data. Despite that a large number of training instances have been formatted with instructions, they mainly come from public NLP datasets, either lacking instruction diversity or mismatching with real human needs [66]. To overcome this issue, InstructGPT [66] proposes to take the queries that real users have submitted to the OpenAI API as the task descriptions. Additionally, to enrich the task diversity, human labelers are also asked to compose the instructions for real-life tasks, including open-ended generation, open question answering, brainstorming, and chatting. Then, they let another group of labelers directly answer these instructions as the output. Finally, they pair one instruction (i.e., the collected user query) and the expected output (i.e., the human-written answer) as a training instance. Note that InstructGPT also employs these real-world tasks formatted in natural language for alignment tuning (discussed in Section 5.2). Further, GPT-4 [46] has designed potentially high-risk instructions and guided the model to reject these instructions through supervised fine-tuning for safety concerns. Considering the absence of high-quality public chat data, several studies have also collected users’ chat requests as input data, and then utilized ChatGPT or GPT-4 to generate

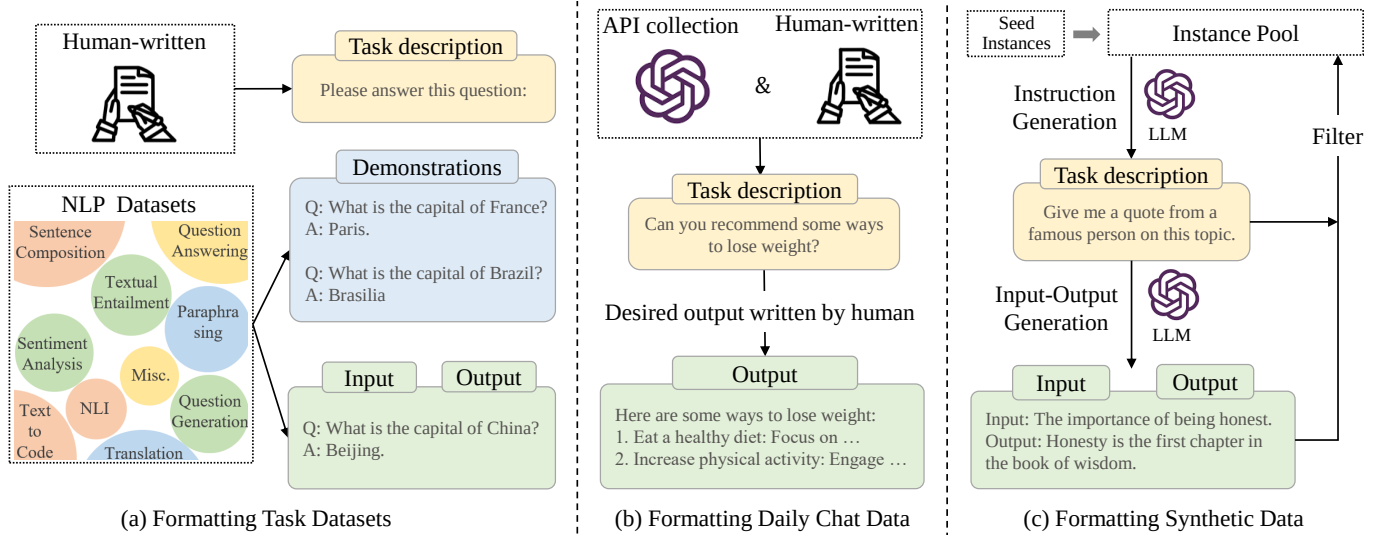


Fig. 11: An illustration of instance formatting and three different methods for constructing the instruction-formatted instances.

responses as output data. A notable example of such a dataset is the conversational data from ShareGPT [153]. Additionally, Dolly [185] and OpenAssistant [186] have further released their conversation data, which has been carefully labeled by human annotators to attain a high level of quality.

Formatting Synthetic Data. To reduce the burden of human annotation or manual collection, several semi-automated approaches [147] have been proposed for constructing instances by feeding existing instances into LLMs to synthesize diverse task descriptions and instances. As illustrated in Figure 11(c), the Self-Instruct method only needs 175 instances as the initial task pool. Then, they randomly select a few instances from the pool as demonstrations and prompt a LLM to generate new instructions and corresponding input-output pairs. After the quality and diversity filtering, newly generated instances would be added into the task pool. Hence, the synthetic method is an effective and economical way to generate large-scale instruction data for LLMs. However, the instances generated by the Self-Instruct method might be simplistic or lack the diversity. To improve the quality of synthetic instructions, WizardLM [335] introduces Evol-Instruct by proposing in-depth and in-breadth evolving to enrich the complexity and diversity of the instances. Furthermore, Self-Align [336] establishes multiple human-aligned principles to filter the synthesized instances. It then employs these instances to train a LLM in order to yield more aligned instances. To enhance the quality of the instance output, researchers directly adopt human-written texts as the output and synthesize corresponding instructions using ICL examples [337].

Key Factors for Instruction Dataset Construction. The quality of instruction instances has an important impact on the performance of the model. Here, we discuss some essential factors for instance construction.

- *Scaling the instructions.* It has been widely shown that scaling the number of tasks can largely enhance the generalization ability of LLMs [28, 67, 88]. With the increasing of the task number, the model performance initially shows a continuous growth pattern, while the gain becomes negligible when it reaches a certain level [69, 88]. A plausible speculation is that a certain

number of representative tasks can provide relatively sufficient knowledge and adding more tasks may not bring additional gains [69]. Also, it is beneficial to enhance the diversity of the task descriptions in several aspects, such as length, structure, and creativity [28]. As for the number of instances per task, it has been found that a small number of instances can usually saturate the generalization performance of the model to perform a specific task [67, 69]. Specially, several recent work [338, 339] has explored the effect of fine-tuning with a small amount of high-quality instruction data (e.g., one or a few thousand instances), showing very promising results on the evaluation tasks. In contrast, another line of studies continue to explore the scaling effect of instruction data [340, 341]. For example, Orca [340] scales up the synthesized instances to 5 million with step-by-step explanations, and it achieves superior performance across a wide range of tasks.

- *Formatting design.* As an important factor, the design of natural language format also highly impacts the generalization performance of LLMs [88]. Typically, we can add task descriptions and optional demonstrations to the input-output pairs of existing datasets, where the task description is the most key part for LLMs to understand the task [88]. Further, it can lead to substantial improvements by using an appropriate number of exemplars as demonstrations [69], which also alleviates the model sensitivity to instruction engineering [67, 69]. However, incorporating other components (e.g., things to avoid, reasons, and suggestions) into instructions may have a negligible or even adverse effect on the performance of LLMs [88, 179]. Recently, to elicit the step-by-step reasoning ability of LLMs, some work [69] proposes to include chain-of-thought (CoT) examples for some reasoning datasets, such as arithmetic reasoning. It has been shown that fine-tuning LLMs with both CoT and non-CoT examples can lead to a good performance across various reasoning tasks, including those that require multi-hop reasoning ability (e.g., commonsense question answering and arithmetic reasoning) as well as those without the need for such a reasoning way (e.g., sentiment analysis and extractive question answering) [69, 95].

- *Instruction quality improvement.* Data quality is very important for the performance of instruction tuning, and a surge

of work has been proposed to further improve the quality of existing instruction datasets. Typically, these methods mostly rely on carefully designed prompts, to guide LLMs to refine or rewrite the given instruction. WizardLM [335] aims to complexify and diversify the Alpaca dataset [187] by devising prompts to widen and deepen the required knowledge of given instructions. It also crafts the filter strategy to remove the low-quality instructions. To further provide fine-grained knowledge guidance, recent work also involves the knowledge taxonomy into the input prompt, e.g., knowledge key points [342] and the human-AI conversation topic taxonomy [343]. To guarantee the instruction quality, early methods mainly employ close-source API or powerful open-source LLMs, which would take a huge cost for large-scale instructions synthesis. Considering this issue, recent studies widely explore the potential of relatively small models for data synthesis. For instance, JiuZhang3.0 [344] fine-tunes a 7B language model to synthesize questions by distilling the knowledge from GPT-4, and then utilizes it to synthesize massive high-quality instructions based on pre-training corpus. Such a way can achieve better performance on mathematical reasoning tasks than baseline methods, with only 20% data synthesis cost.

- **Instruction selection.** As a surge of instruction datasets are proposed, it is non-trivial to select the high-quality ones from them to construct the training dataset. Generally, existing work either leverages quality estimation metrics or employs LLMs as the judge model to rank all the instruction instances, and then selects those with relatively higher scores. Concretely, for metrics, perplexity and other heuristic measurements (e.g., length) [345] have been widely used in practice, e.g., we can consider removing high-perplexity or very short instructions, which might correspond to low-quality ones. To better estimate the effect of an instruction for the LLM capability, more complex metrics (e.g., IFD [346]) have also been proposed, which are computed by combining multiple simple metrics. Additionally, diversity-aware sampling methods have been introduced to ensure the overall coverage of representative instruction data [347]. Besides, when downstream task data is available, cross-instance gradient similarity can be employed to measure the value of training instances for the target task. LESS [348] computes gradients for both downstream validation and training instruction data, to evaluate the contribution of instruction data based on extensions of influence function [349].

To summarize, diversity and quality of instructions are important factors to consider when scaling the number of instances [338]. As the capacities of LLMs improve, data synthesis methods have become the mainstream approach for generating large amount of instruction data. Following this trend, there are increasingly more automatically generated instruction datasets available, and selection and refining methods are key to effectively use these datasets. To help readers understand how different factors affect instruction tuning, we conduct an empirical study by experimenting with multiple specially constructed instruction datasets in Section 5.1.4.

5.1.2 Instruction Tuning Strategies

Unlike pre-training, instruction tuning is often more efficient since only a moderate number of instances are used for training. Since instruction tuning can be considered as a supervised training process, its optimization is different from pre-training in several aspects [69], such as the training objective (i.e., sequence-to-sequence loss) and optimization configuration (e.g., smaller

batch size and learning rate), which require special attention in practice. In addition to these optimization configurations, there are also four important aspects to consider for instruction tuning:

Balancing the Data Distribution. Since instruction tuning involves a mixture of different tasks, it is important to balance the proportion of different tasks during fine-tuning. A widely used method is the *examples-proportional mixing* strategy [82], i.e., combining all the datasets and sampling each instance equally from the mixed datasets. Furthermore, increasing the sampling ratio of high-quality collections (e.g., FLAN [67] and P3 [180]) can generally lead to performance improvement according to recent findings [69, 95]. Further, it is common to set a *maximum cap* to control the maximum number of examples that a dataset can contain during instruction tuning [82], which is set to prevent larger datasets from overwhelming the entire distribution [82, 95]. In practice, the maximum cap is typically set to several thousands or tens of thousands according to different datasets [67, 69]. Recently, it has been empirically found that existing instruction datasets (Table 3) mainly focus on enhancing LLMs’ capabilities in certain aspects, and a single dataset alone cannot lead to a comprehensive enhancement in model capacity [350]. Therefore, it is often suggested to use a mixture of existing instruction datasets to achieve a balanced improvement in different capacities, including NLP task data (e.g., FLAN v2 [351]), chat data (e.g., ShareGPT [153]), and synthetic data (e.g., GPT4-Alpaca [352]).

Combining Instruction Tuning and Pre-Training. To make the tuning process more effective and stable, OPT-IML [95] incorporates pre-training data during instruction tuning, which can be regarded as regularization for model tuning. Further, instead of using a separate two-stage process (*pre-training* then *instruction tuning*), some studies attempt to train a model from scratch with a mixture of pre-training data (i.e., plain texts) and instruction tuning data (i.e., formatted datasets) using multi-task learning [82]. Specifically, GLM-130B [93] and Galactica [35] integrate instruction-formatted datasets as a small proportion of the pre-training corpora to pre-train LLMs, which potentially achieves the advantages of pre-training and instruction tuning at the same time.

Multi-stage Instruction Tuning. For instruction tuning, there are two kinds of important instruction data, namely task-formatted instructions and daily chat instructions. Generally, the former has a significantly larger volume than the latter. It is important to balance the training with the two kinds of instruction data. In addition to carefully mixing different instruction data, we can also adopt a multi-stage instruction tuning strategy [341], where LLMs are first fine-tuned with large-scale task-formatted instructions and subsequently fine-tuned on daily chat ones. To avoid the capacity forgetting issue, it is also useful to add an amount of task-formatted instructions at the second stage. Actually, such a multi-stage tuning strategy can be also applied to other settings for instruction tuning. For example, we can schedule different fine-tuning stages with progressively increased levels on difficulty and complexity, and gradually improve the capacities of LLMs to follow complex instructions.

Other Practical Tricks. In practice, there are also several useful strategies and tricks that are helpful to improve the fine-tuning performance of LLMs. We list several representative ones as

TABLE 9: Basic statistics of the required number of GPUs, tuning time, batch size (denoted as BS) per device (full tuning and LoRA tuning), and inference rate (the number of generated tokens per second). Our experiments are conducted based on two Linux servers having 8 A800-80G SXM4 GPUs with 6 NVSwitch and 8 3090-24G GPUs, respectively. The major difference between A800 and A100 lies in the NVLink interconnect speed. Thus, our estimations about training and inference efficiency would be slightly improved for A100, while the rest memory consumption would remain the same. For full tuning experiments, we use data parallel training, ZeRO Stage 3, BF16, and gradient checkpointing. Additionally, the LoRA tuning can be executed on one 80G GPU utilizing INT8 quantization with the rank setting set to 16. All the experiments are conducted with Alpaca-52K dataset by training LLaMA models three epochs. The max sequence length for both training settings is set to 512. The inference experiments are performed with the batch size set to 1.

Models	A800 Full Tuning			A800 LoRA Tuning			A800 Inference (16-bit)		3090 Inference (16-bit)		3090 Inference (8-bit)	
	#GPU	BS	Time	#GPU	BS	Time	#GPU	#Token/s	#GPU	#Token/s	#GPU	#Token/s
LLaMA (7B)	2	8	3.0h	1	80	3.5h	1	36.6	1	24.3	1	7.5
LLaMA (13B)	4	8	3.1h	1	48	5.1h	1	26.8	2	9.9	1	4.5
LLaMA (30B)	8	4	6.1h	1	24	14.3h	1	17.7	4	3.8	2	2.6
LLaMA (65B)	16	2	11.2h	1	4	60.6h	2	8.8	8	2.0	4	1.5

follows:

- *Efficient training for multi-turn chat data.* Given a multi-turn chat example (the conversation between a user and chatbot), a straightforward fine-tuning way is to split it into multiple context-response pairs for training: a LLM is fine-tuned to generate the response based on the corresponding context for all splits (*i.e.*, at each utterance from the user). In such a fine-tuning way, it is apparent that there exist overlapping utterances in the split examples from a conversation. To save the training cost, Vicuna [152] has adopted an efficient way that feeds the whole conversation into the LLM, but relies on a loss mask that only computes the loss on the responses of the chatbot for training. It can significantly reduce the compute costs derived from the overlapped utterances.

- *Establishing self-identification for LLM.* To deploy LLMs for real-world applications, it is necessary to establish its identity and make LLMs aware of these identity information, such as name, developer and affiliation. A practical way is to create identity-related instructions for fine-tuning the LLM. It is also feasible to prefix the input with the self-identification prompt, *e.g.*, “*The following is a conversation between a human and an AI assistant called CHATBOTNAME, developed by DEVELOPER.*”, where CHATBOTNAME and DEVELOPER refer to the name and developer of the chatbot, respectively.

In addition to the above practical strategies and tricks, existing work has also used other tricks, *e.g.*, concatenating multiple examples into a single sequence to approach the max length [353].

5.1.3 The Effect of Instruction Tuning

In this part, we discuss the effect of instruction tuning on LLMs in three major aspects.

Performance Improvement. Despite being tuned on a moderate number of instances, instruction tuning has become an important way to improve or unlock the abilities of LLMs [69]. Recent studies have experimented with language models in multiple scales (ranging from 77M to 540B), showing that the models of different scales can all benefit from instruction tuning [69, 334], yielding improved performance as the parameter scale increases [94]. Further, smaller models with instruction tuning can even perform better than larger models without fine-tuning [28, 69]. Besides the model scale, instruction tuning demonstrates consistent improvements in various model archi-

tectures, pre-training objectives, and model adaptation methods [69]. In practice, instruction tuning offers a general approach to enhancing the abilities of existing language models [69] (including small-sized PLMs). Also, it is much less costly than pre-training, since the amount of instruction data required by LLMs is significantly smaller than pre-training data.

Task Generalization. Instruction tuning encourages the model to understand natural language instructions for task completion. It endows LLMs with the ability (often considered as an emergent ability) to follow human instructions [31] to perform specific tasks without demonstrations, even on unseen tasks [69]. A large number of studies have confirmed the effectiveness of instruction tuning to achieve superior performance on both seen and unseen tasks [95, 334]. Also, instruction tuning has been shown to be useful in alleviating several weaknesses of LLMs (*e.g.*, repetitive generation or complementing the input without accomplishing a certain task) [66, 69], leading to a superior capacity to solve real-world tasks for LLMs. Furthermore, LLMs trained with instruction tuning can generalize to related tasks across languages. For example, BLOOMZ-P3 [94] is fine-tuned based on BLOOM [78] using English-only task collection P3 [180]. Interestingly, BLOOMZ-P3 can achieve a more than 50% improvement in multilingual sentence completion tasks compared to BLOOM, which shows that instruction tuning can help LLMs acquire general task skills from English-only datasets and transfer such skills into other languages [94]. In addition, it has been found that using English-only instructions can produce satisfactory results on multilingual tasks [94], which helps reduce the effort of instruction engineering for a specific language.

Domain Specialization. Existing LLMs have showcased superior capabilities in traditional NLP tasks (*e.g.*, generation and reasoning) and daily questions. However, they may still lack domain knowledge to accomplish specific tasks, such as medicine, law, and finance (See Section 8 for a detailed discussion of LLMs in different applications). Instruction tuning is an effective approach to adapting existing general LLMs to be domain-specific experts. For instance, researchers propose to fine-tune Flan-PaLM [69] using medical datasets to create Med-PaLM [354], a medical knowledge assistant that achieves performance levels comparable to those of expert clinicians. Furthermore, a recent study [355] fine-tunes FLAN-T5 to support e-commerce recommender systems with natural language instructions, showing strong performance in a variety of recommendation tasks. There are also

several open-sourced medical models instruction-tuned based on LLaMA [57], such as BenTsao [356]. Also, researchers explore instruction tuning on law [357], finance [358], and arithmetic computation [359].

5.1.4 Empirical Analysis for Instruction Tuning

Fine-tuning LLMs with different instruction sets tend to lead to model variants with varied performance on downstream tasks. In this section, we will explore the effect of different types of instructions in fine-tuning LLMs (*i.e.*, LLaMA (7B) and LLaMA (13B)²⁵), as well as examine the usefulness of several instruction improvement strategies.

Instruction Datasets. According to the discussion in Section 5.1.1, we mainly consider three common kinds of instructions as follows:

- *Task-specific instructions.* For the first type of instructions, we adopt the most commonly-used multi-task instruction dataset, *FLAN-T5* [69], which contains 1,836 tasks and over 15M instructions by combining four data mixtures from prior work.
- *Daily chat instructions.* This type of instructions are conversations posed by users about daily life, which are more closely related to real-life scenarios. We adopt the ShareGPT instruction set, consisting of 63K real-user instructions. It has been used as the core instructions for Vicuna.
- *Synthetic instructions.* In addition to reusing existing instructions, we can also automatically synthesize massive instructions using LLMs. We adopt the popular synthetic instruction dataset Self-Instruct-52K [147], consisting of 52K instructions paired with about 82K instance inputs and outputs. These generated instructions have a similar data distribution as the human-written seed tasks (*e.g.*, grammar checking, brainstorming).

As the original FLAN-T5 dataset is very large (*i.e.*, over 15M), we randomly sample 80,000 instructions from it for conducting a fair comparison with other instruction datasets (*i.e.*, ShareGPT and Self-Instruct-52K) at a similar scale. In our experiments, we test on each individual instruction set to explore their own effects and also examine their combinatorial effects on model performance.

Improvement Strategies. Although real-world instructions from human users are more suitable for fine-tuning LLMs, it is difficult to collect them at a large scale. As alternatives to human-generated instructions, most existing research mainly adopts synthetic instructions generated by LLMs. However, there are some potential problems with synthetic instructions, such as poor topic diversity and uneven instruction difficulty (either too simple or too difficult). Thus, it is necessary to improve the quality of the synthetic instructions. Next, we summarize four major improvement strategies widely used in existing work as follows:

- *Enhancing the instruction complexity.* As discussed in existing work [335], enhancing the complexity of instructions can improve the model capacity of LLMs in following complex instructions, *e.g.*, including more task demands or requiring more reasoning steps. To validate this strategy, we follow WizardLM [335] by gradually increasing the complexity levels, *e.g.*, adding constraints, increasing reasoning steps, and complicating the input. We leverage the publicly released WizardLM-70K instructions [335] as the complexity-enhanced instruction dataset,

which has been generated via the above enhancement approach based on the Self-Instruct-52K dataset [335].

- *Increasing the topic diversity.* In addition to the complexity, improving the topic diversity of the instruction dataset can help elicit different abilities of LLMs on diverse tasks in real world [336]. However, it is difficult to directly control the self-instruct process for generating diverse instructions. Following YuLan-Chat [341], we employ ChatGPT to rewrite the instructions from Self-Instruct-52K dataset for adapting them into 293 topics via specific prompts. Finally, we obtain 70K instructions as the diversity-increased dataset.

- *Scaling the instruction number.* In addition to the above aspects, the number of instructions is also an important factor that may affect the model performance. Specially, using more instructions can extend the task knowledge and improve the ability of instruction following for LLMs [69]. To examine this strategy, we sample new instructions from the synthesized instruction set released from the MOSS project [360], as they are also synthesized using the same self-instruct method [147]. We mix them with the Self-Instruct-52K dataset to compose a larger one containing 220K instructions.

- *Balancing the instruction difficulty.* As the synthetic instructions tend to contain too easy or too hard ones, it is likely to result in training instability or even overfitting for LLMs. To explore the potential effects, we leverage the perplexity score of LLMs to estimate the difficulty of instructions and remove too easy or too hard instructions. To generate the same scale of instructions for fair comparison, we adopt a LLaMA (7B) model to compute the perplexity for the 220K instructions from the large instruction dataset, and then keep 70K instructions of moderate perplexity scores as the difficulty-balanced dataset.

Experimental Setup. To conduct the experiments on the effect of instruction data, we leverage these new instruction datasets for tuning LLaMA, a popular LLM backbone that has been widely used for instruction-tuning. We use the code from YuLan-Chat [341] for our experiments, and train LLaMA 7B and 13B on a server of 8 A800-80G GPUs. All the hyper-parameters settings remain the same as Stanford Alpaca. To better evaluate the instruction following ability of fine-tuned models, we consider two settings, namely *Chat setting* and *QA setting*. The chat setting mainly utilizes user instructions and queries from daily chat, whereas the QA setting mainly employs question answering examples from existing NLP datasets. The evaluation on the chat setting is conducted based on the AlpacaFarm evaluation set [361]. Instead of using a full pairwise comparison, we select the LLaMA 7B and 13B models fine-tuned on Self-Instruct-52K as the reference baselines, and then compare them with other fine-tuned LLaMA 7B and 13B models using different instructions, respectively. Since our focus is to examine the usefulness of different strategies to generate the instructions, the model fine-tuned on Self-Instruct-52K can serve as a good reference. Following AlpacaFarm [361], for each comparison, we employ ChatGPT to automatically annotate which response from two compared models each time is the best for the user query, and report the win rate (%) as the evaluation metric. For the QA setting, we select two benchmarks, MMLU [362] and BBH [363], and evaluate the accuracy based on their default settings by using heuristic rules to parse the answers from these LLMs.

For both instruction tuning and evaluation, we adopt the following prompt: “*The following is a conversation between*

25. Due to the limit of computational resources, we cannot conduct large-scale experiments on larger LLaMA variants right now, which would be scheduled in a future version.

TABLE 10: Results of instruction-tuning experiments (all in a single-turn conversation) based on the LLaMA (7B) and LLaMA (13B) model under the chat and QA setting. We employ four instruction improvement strategies on the Self-Instruct-52K dataset, *i.e.*, enhancing the complexity (*w/ complexity*), increasing the diversity (*w/ diversity*), balancing the difficulty (*w/ difficulty*), and scaling the instruction number (*w/ scaling*). *Since we select the LLaMA (7B)/(13B) model fine-tuned on Self-Instruct-52K as the baseline, we omit the win rate of the fine-tuned model with Self-Instruct-52K against itself.

Models	Dataset Mixtures	Instruction Numbers	Lexical Diversity	Chat		QA	
				AlpacaFarm	MMLU	BBH3k	
LLaMA (7B)	① FLAN-T5	80,000	48.48	23.77	38.58	32.79	
	② ShareGPT	63,184	77.31	81.30	38.11	27.71	
	③ Self-Instruct-52K	82,439	25.92	/*	37.52	29.81	
	② + ③	145,623	48.22	71.36	41.26	28.36	
	① + ② + ③	225,623	48.28	70.00	43.69	29.69	
	③ Self-Instruct-52K	82,439	25.92	/*	37.52	29.81	
	w/ complexity	70,000	70.43	76.96	39.73	33.25	
	w/ diversity	70,000	75.59	81.55	38.01	30.03	
	w/ difficulty	70,000	73.48	79.15	32.55	31.25	
	w/ scaling	220,000	57.78	51.13	33.81	26.63	
	① FLAN-T5	80,000	48.48	22.12	34.12	34.05	
	② ShareGPT	63,184	77.31	77.13	47.49	33.82	
LLaMA (13B)	③ Self-Instruct-52K	82,439	25.92	/*	36.73	25.43	
	② + ③	145,623	48.22	72.85	41.16	29.49	
	① + ② + ③	225,623	48.28	69.49	43.50	31.16	
	③ Self-Instruct-52K	82,439	25.92	/*	36.73	25.43	
	w/ complexity	70,000	70.43	77.94	46.89	35.75	
	w/ diversity	70,000	75.59	78.92	44.97	36.40	
	w/ difficulty	70,000	73.48	80.45	43.15	34.59	
	w/ scaling	220,000	57.78	58.12	38.07	27.28	

a human and an AI assistant. The AI assistant gives helpful, detailed, and polite answers to the user's questions.\n [Human]:{input}\n [AI]:". To reproduce our results, we release the code and data at the link: <https://github.com/RUCAIBox/LLMSurvey/tree/main/Experiments>.

Results and Analysis. The results using different instruction datasets based on 7B and 13B LLaMA are in Table 10. Next, we summarize and analyze our findings in detail.

- *Task-formatted instructions are more proper for the QA setting, but may not be useful for the chat setting.* By comparing the performance of instruction tuning using FLAN-T5 with that of ShareGPT and Self-Instruct-52K, we can observe that FLAN-T5 mostly achieves a better performance on QA benchmarks while underperforms ShareGPT on the chat setting. The reason is that FLAN-T5 is composed of a mixture of instructions and examples from existing NLP tasks, *e.g.*, translation and reading comprehension. As a result, LLaMA fine-tuned with FLAN-T5 performs better on QA tasks, but poorly on user queries. In contrast, ShareGPT consists of real-world human-ChatGPT conversations, which is able to better elicit LLaMA to follow user instructions in daily life, while may not be suitable for accomplishing the QA tasks.

- *A mixture of different kinds of instructions are helpful to improve the comprehensive abilities of LLMs.* After mixing the three kinds of instructions for fine-tuning, we can see that the derived LLaMA variant (with FLAN-T5, ShareGPT and Self-Instruct-52K) performs well in both task settings. In MMLU, the performance of LLaMA (7B) can surpass the ones using individual instruction set by a large margin, *i.e.*, 43.69 vs. 38.58 (FLAN-T5). It shows that mixing multiple sources of instruction datasets is helpful to improve the performance of instruction-

tuned LLMs, which scales the instruction number as well as increases the diversity.

- *Enhancing the complexity and diversity of instructions leads to an improved model performance.* By increasing the complexity and diversity of the Self-Instruct-52K dataset respectively, the chat and QA performance of LLaMA can be consistently improved, *e.g.*, from 37.52 to 39.73 in MMLU for LLaMA (7B). It demonstrates that both strategies are useful to improve the instruction following ability of LLMs. Further, we can see that improving the complexity yields a larger performance improvement on QA tasks. The reason is that the QA tasks mostly consist of difficult questions for evaluating LLMs, which can be better solved by LLMs that have learned complex instructions at the fine-tuning stage.

- *Simply increasing the number of instructions may not be that useful, and balancing the difficulty is not always helpful.* As the results shown in Table 10, balancing the difficulty and increasing the number of fine-tuning instructions are not very helpful in our experiments. Especially for scaling the instruction number, it even hurts the performance, *e.g.*, a decrease from 29.81 to 26.63 in BBH for LLaMA (7B). It shows that simply scaling the number of synthesized instructions without quality control may not be effective to improve the performance. Furthermore, fine-tuning with the instructions of moderate difficulty also performs well in the chat setting, while slightly decreasing the performance in the QA setting. A possible reason is that we filter complex and hard instructions with large perplexity scores, hurting the model performance in answering complex questions.

- *A larger model scale leads to a better instruction following performance.* By comparing the performance of LLaMA (7B) and LLaMA (13B) models fine-tuned with the same set of instruction data, we can see that LLaMA (13B) mostly achieves a better

performance. It indicates that scaling the model size is helpful for improving the instruction following capability. Besides, we can see that the QA performance has been improved a lot, e.g., from 38.11 to 47.49 in MMLU. It is likely because that the larger models generally have better knowledge utilization and reasoning capability [33, 55], which can accurately answer more complex questions.

Instruction Tuning Suggestions

To conduct instruction tuning on LLMs, one can prepare the computational resources according to the basic statistics about the required number of GPUs and tuning time in Table 9. After setting up the development environment, we recommend beginners to follow the code of Alpaca repository [187] for instruction tuning. Subsequently, one should select the base model and construct the instruction datasets as we discuss in this section. When computational resources for training are constrained, users can utilize LoRA for parameter-efficient tuning (see Section 5.3). As for inference, users can further use quantization methods to deploy LLMs on fewer or smaller GPUs (see Section 5.3).

5.2 Alignment Tuning

This part first presents the background of alignment with its definition and criteria, then focuses on the collection of human feedback data for aligning LLMs, and finally discusses the key technique of reinforcement learning from human feedback (RLHF) for alignment tuning.

5.2.1 Background and Criteria for Alignment

Background. LLMs have shown remarkable capabilities in a wide range of NLP tasks [55, 56, 67, 90]. However, these models may sometimes exhibit unintended behaviors, e.g., fabricating false information, pursuing inaccurate objectives, and producing harmful, misleading, and biased expressions [66, 364]. For LLMs, the language modeling objective pre-trains the model parameters by word prediction while lacking the consideration of human values or preferences. To avert these unexpected behaviors, human alignment has been proposed to make LLMs act in line with human expectations [66, 365]. However, unlike the original pre-training and adaptation tuning (e.g., instruction tuning), such an alignment requires considering very different criteria (e.g., helpfulness, honesty, and harmlessness). It has been shown that alignment might harm the general abilities of LLMs to some extent, which is called *alignment tax* in related literature [366].

Alignment Criteria. Recently, there is increasing attention on developing multifarious criteria to regulate the behaviors of LLMs. Here, we take three representative alignment criteria (i.e., helpful, honest, and harmless) as examples for discussion, which have been widely adopted in existing literature [66, 366]. In addition, there are other alignment criteria for LLMs from different perspectives including behavior, intent, incentive, and inner aspects [364], which are essentially similar (or at least with similar alignment techniques) to the above three criteria. It is also feasible to modify the three criteria according to specific needs, e.g., substituting honesty with correctness [116]. Next, we

give brief explanations about the three representative alignment criteria:

- **Helpfulness.** To be helpful, the LLM should demonstrate a clear attempt to assist users in solving their tasks or answering questions in a concise and efficient manner as possible. At a higher level, when further clarification is needed, the LLM should demonstrate the capability of eliciting additional relevant information through pertinent inquiries and exhibit suitable levels of sensitivity, perceptiveness, and prudence [366]. Realizing the alignment of helpful behavior is challenging for LLMs since it is difficult to precisely define and measure the intention of users [364].

- **Honesty.** At a basic level, a LLM aligned to be honest should present accurate content to users instead of fabricating information. Additionally, it is crucial for the LLM to convey appropriate degrees of uncertainty in its output, in order to avoid any form of deception or misrepresentation of information. This requires the model to know about its capabilities and levels of knowledge (e.g., “know unknowns”). According to the discussion in [366], honesty is a more objective criterion compared to helpfulness and harmlessness, hence honesty alignment could potentially be developed with less reliance on human efforts.

- **Harmlessness.** To be harmless, it requires that the language produced by the model should not be offensive or discriminatory. To the best of its abilities, the model should be capable of detecting covert endeavors aimed at soliciting requests for malicious purposes. Ideally, when the model was induced to conduct a dangerous action (e.g., committing a crime), the LLM should politely refuse. Nonetheless, *what behaviors* are deemed harmful and *to what extent* vary amongst individuals or societies [366] highly depend on who is using the LLM, the type of the posed question, and the context (e.g., time) at which the LLM is being used.

As we can see, these criteria are quite subjective, and are developed based on human cognition. Thus, it is difficult to directly formulate them as optimization objectives for LLMs. In existing work, there are many ways to fulfill these criteria when aligning LLMs. A promising technique is *red teaming* [367], which involves using manual or automated means to probe LLMs in an adversarial way to generate harmful outputs and then updates LLMs to prevent such outputs.

5.2.2 Collecting Human Feedback

During the pre-training stage, LLMs are trained using the language modeling objective on a large-scale corpus. However, it cannot take into account the subjective and qualitative evaluations of LLM outputs by humans (called *human feedback* in this survey). High-quality human feedback is extremely important for aligning LLMs with human preferences and values. In this part, we discuss how to select a team of human labelers for feedback data collection.

Human Labeler Selection. In existing work, the dominant method for generating human feedback data is human annotation [66, 116, 365]. This highlights the critical role of selecting appropriate human labelers. To provide high-quality feedback, human labelers are supposed to have a qualified level of education and excellent proficiency in English. For example, Sparrow [116] requires human labelers to be UK-based native English speakers who have obtained at least an undergraduate-level educational qualification. Even then, several studies [365] have found that

there still exists a mismatch between the intentions of researchers and human labelers, which may lead to low-quality human feedback and cause LLMs to produce unexpected output. To address this issue, InstructGPT [66] further conducts a screening process to filter labelers by assessing the agreement between human labelers and researchers. Specifically, researchers first label a small amount of data and then measure the agreement between themselves and human labelers. The labelers with the highest agreement will be selected to proceed with the subsequent annotation work. In some other work [368], “super raters” are used to ensure the high quality of human feedback. Researchers evaluate the performance of human labelers and select a group of well-performing human labelers (e.g., high agreement) as super raters. The super raters will be given priority to collaborate with the researchers in the subsequent study. When human labelers annotate the output of LLMs, it is helpful to specify detailed instructions and provide instant guidance for human labelers, which can further regulate the annotation of labelers.

Human Feedback Collection. In existing work, there are mainly three kinds of approaches to collecting feedback and preference data from human labelers.

- *Ranking-based approach.* In early work [365], human labelers often evaluate model-generated outputs in a coarse-grained manner (*i.e.*, only selecting the best) without taking into account more fine-grained alignment criteria. Nonetheless, different labelers may hold diverse opinions on the selection of the best candidate output, and this method disregards the unselected samples, which may lead to inaccurate or incomplete human feedback. To address this issue, subsequent studies [116] introduce the Elo rating system to derive the preference ranking by comparing candidate outputs. The ranking of outputs serves as the training signal that guides the model to prefer certain outputs over others, thus inducing outputs that are more reliable and safer.

- *Question-based approach.* Further, human labelers can provide more detailed feedback by answering certain questions designed by researchers [81], covering the alignment criteria as well as additional constraints for LLMs. Specially, in WebGPT [81], to assist the model in filtering and utilizing relevant information from retrieved documents, human labelers are required to answer questions with multiple options about whether the retrieved documents are useful for answering the given input.

- *Rule-based approach.* Many studies also develop rule-based methods to provide more detailed human feedback. As a typical case, Sparrow [116] not only selects the response that labelers consider the best but also uses a series of rules to test whether model-generated responses meet the alignment criteria of being helpful, correct, and harmless. In this way, two kinds of human feedback data can be obtained: (1) the response preference feedback is obtained by comparing the quality of model-generated output in pairs, and (2) the rule violation feedback is obtained by collecting the assessment from human labelers (*i.e.*, a score indicating to what extent the generated output has violated the rules). Furthermore, GPT-4 [46] utilizes a set of zero-shot classifiers (based on GPT-4 itself) as rule-based reward models, which can automatically determine whether the model-generated outputs violate a set of human-written rules.

In the following, we focus on a well-known technique, reinforcement learning from human feedback (RLHF), which has been widely used in the recent powerful LLMs such as Chat-

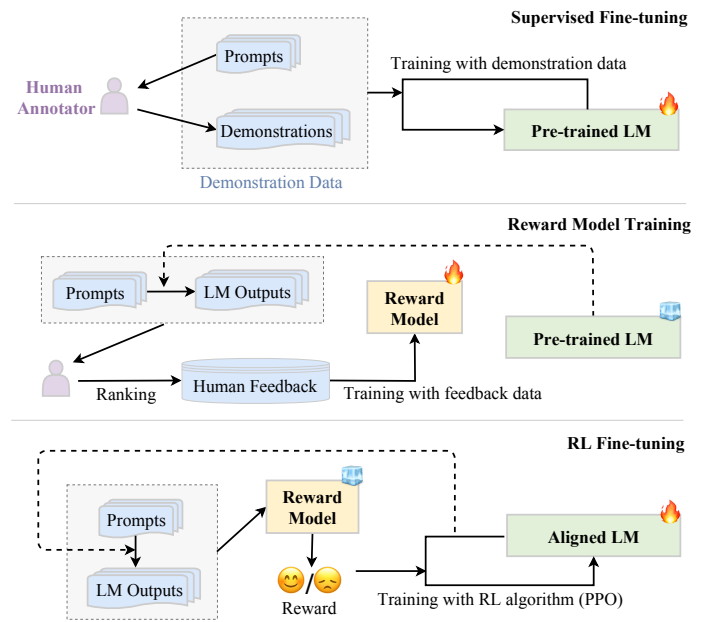


Fig. 12: The workflow of the RLHF algorithm.

GPT. As discussed below, the alignment criteria introduced in Section 5.2.1 can be fulfilled by learning from human feedback on the responses of LLMs to users’ queries.

5.2.3 Reinforcement Learning from Human Feedback

To align LLMs with human values, reinforcement learning from human feedback (RLHF) [79, 365] has been proposed to fine-tune LLMs with the collected human feedback data, which is useful to improve the alignment criteria (e.g., helpfulness, honesty, and harmlessness). RLHF employs reinforcement learning (RL) algorithms (e.g., Proximal Policy Optimization (PPO) [128]) to adapt LLMs to human feedback by learning a reward model. Such an approach incorporates humans in the training loop for developing well-aligned LLMs, as exemplified by InstructGPT [66].

RLHF System. The RLHF system mainly comprises three key components: a pre-trained LM to be aligned, a reward model learning from human feedback, and a RL algorithm training the LM. Specifically, the *pre-trained LM* is typically a generative model that is initialized with existing pre-trained LM parameters. For example, OpenAI uses 175B GPT-3 for its first popular RLHF model, InstructGPT [66], and DeepMind uses the 280 billion parameter model Gopher [64] for its GopherCite model [368]. Further, the *reward model (RM)* provides (learned) guidance signals that reflect human preferences for the text generated by the LM, usually in the form of a scalar value. The reward model can take on two forms: a fine-tuned LM or a LM trained de novo using human preference data. Existing work typically employs reward models having a parameter scale different from that of the aligned LM [66, 368]. For example, OpenAI uses 6B GPT-3 and DeepMind uses 7B Gopher as the reward model, respectively. Finally, to optimize the pre-trained LM using the signal from the reward model, a specific *RL algorithm* is designed for large-scale model tuning. Specifically, Proximal Policy Optimization (PPO) [128] is a widely used RL algorithm for alignment in existing work [66, 116, 368].

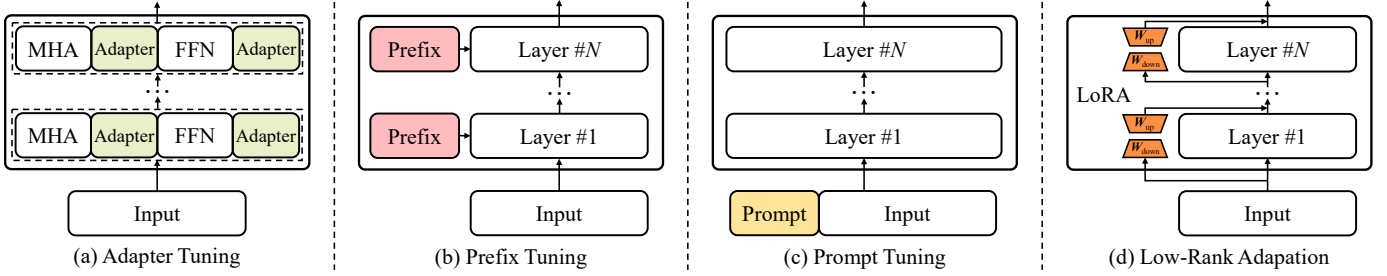


Fig. 13: An illustration of four different parameter-efficient fine-tuning methods. MHA and FFN denote the multi-head attention and feed-forward networks in the Transformer layer, respectively.

Key Steps for RLHF. Figure 12 illustrates the overall three-step process of RLHF [66] as introduced below.

- *Supervised fine-tuning.* To make the LM initially perform desired behaviors, it usually needs to collect a supervised dataset containing input prompts (instruction) and desired outputs for fine-tuning the LM. These prompts and outputs can be written by human labelers for some specific tasks while ensuring the diversity of tasks. For example, InstructGPT [66] asks human labelers to compose prompts (e.g., “List five ideas for how to regain enthusiasm for my career”) and desired outputs for several generative tasks such as open QA, brainstorming, chatting, and rewriting. Note that the first step is optional in specific settings or scenarios.

- *Reward model training.* The second step is to train the RM using human feedback data. Specifically, we employ the LM to generate a certain number of output texts using sampled prompts (from either the supervised dataset or the human-generated prompt) as input. We then invite human labelers to annotate the preference for these pairs. The annotation process can be conducted in multiple forms, and a common approach is to annotate by ranking the generated candidate texts, which can reduce the inconsistency among annotators. Then, the RM is trained to predict the human-preferred output. In InstructGPT, labelers rank model-generated outputs from best to worst, and the RM (i.e., 6B GPT-3) is trained to predict the ranking. Note that, in recent work [369], the annotation of preference on response pairs has been conducted by an AI agent (usually an aligned LLM) instead of humans, which is called “reinforcement learning from AI feedback (RLAIF)”. LLMs trained with typical RLHF algorithms tend to generate harmless responses with less helpfulness, which is called *evasion problem* [369]. To guarantee both the harmlessness and helpfulness, RLAIF generates the AI feedback based on pre-set alignment principles in instructions [369, 370], which can also reduce the efforts of human annotation.

- *RL fine-tuning.* At this step, aligning (i.e., fine-tuning) the LM is formalized as an RL problem. In this setting, the pre-trained LM acts as the policy that takes as input a prompt and returns an output text, the action space of it is the vocabulary, the state is the currently generated token sequence, and the reward is provided by the RM. To avoid eviating significantly from the initial (before tuning) LM, a penalty term is commonly incorporated into the reward function. For example, InstructGPT optimizes the LM against the RM using the PPO algorithm. For each input prompt, InstructGPT calculates the KL divergence between the generated results from the current LM and the initial LM as the penalty. It is noted that the second and final steps can be iterated in multiple turns for better aligning LLMs. Due to the

instability of the RL algorithm, recent work [371] replaces the RL tuning with another supervised fine-tuning by reusing the best ranked samples with higher rewards.

Practical Strategies for RLHF. Although RLHF is promising to effectively improve the alignment of LLMs with humans, it is practically challenging for researchers to successfully implement it. In this part, we focus on discussing several useful strategies and tricks for improving the effectiveness and efficiency of RLHF. Concretely, we focus on the effective training of reward models, efficient and effective RL training, respectively.

- *Effective reward model training.* Despite that InstructGPT used a small reward model (6B GPT model), increasing work [99] has shown it is often more effective to use a large reward model (e.g., equal or greater than the original model size), since large reward models generally perform better in judging the quality of the LLM generated outputs. In LLaMa 2 [99], pretrained chat model checkpoints are used to initialize the reward model, they argue that such an approach can effectively reduce the information mismatch between the model to be aligned and the reward model by sharing the same pre-training knowledge. Whereas, it is common to encounter the overfitting problem when training large-scale reward models. As a simple yet effective solution, existing work [372, 373] has introduced the LM loss on the preferred response of the input prompt from the human-annotated alignment dataset as a regularizer, which alleviates the overfitting of the reward model on the binary classification task. In addition, as there are multiple criteria for alignment (e.g., helpfulness and honesty), it is often difficult to train a single reward model that can satisfy all the alignment criteria. Therefore, it is useful to train multiple reward models that focus on different alignment criteria [99], and compute the final reward based on the produced ones from them via special combination strategies (e.g., mean pooling and weighted sum). Such a way enables more flexible rules or standards on multiple criteria, e.g., relaxing the requirement on helpfulness while posing more strict limits on harmfulness.

- *Effective RL training.* As the RL training process tends to be unstable and hyper-parameter sensitive, it is suggested that the language model should be well supervised fine-tuned before RL training, so as to reaching a good model capacity. A commonly-used way is to fine-tune the LLM on its best outputs of the prompts (referred to as *rejection sampling* or *best-of-N*) from the alignment dataset until convergence before RL. Given a prompt, the LLM would first produce N outputs via the sampling algorithm, and then the best candidate from the model will be selected by the reward model for learning. After fine-tuning the LLM on the best samples until convergence, the

RL process will be performed to further improve the performance. LLaMA 2 [99] has successively trained five versions of RLHF models, where the LLM has been progressively improved with the improvement of the reward models. In this way, the collected prompts and annotations of human preference data can better reflect the issues of the current model checkpoint, thus making special tuning to address these issues. In addition, LLaMA 2 also adds samples from prior iterations into the subsequent ones, to alleviate the possible capacity regression issue during iterative optimization.

- *Efficient RL training.* As the RL training requires to iterate the inference process of both the LLM and reward models, it would greatly increase the total memory and computation cost, especially for larger reward models and LLMs. As a practical trick, we can deploy the reward model on a separate server, and invoke the corresponding API to work with the LLM on its own server. In addition, as RLHF requires the LLM to generate multiple candidate outputs, instead of calling the sample decoding procedure for multiple times, it is more efficient to utilize the beam search decoding algorithm²⁶. It only needs to perform one-pass decoding for response generation, meanwhile such a strategy can also enhance the diversity of the generated candidate responses.

Process-Supervised RLHF. In existing literature of RLHF [374], the supervision approach for RL training generally takes two major forms, either using outcome-supervision signals or process-supervision signals. The outcome-supervised RLHF employs a quantitative score to assess the quality of the whole text generated by LLMs. In contrast, process-supervised RLHF offers an evaluation of each individual component (e.g., sentence, word, or reasoning step) within the generated content, which leverage fine-grained supervision signals to guide the training, helping LLMs refine the undesired generation contents [374, 375]. In what follows, we discuss two key aspects of process-supervised RLHF.

- *Obtaining Fine-grained Supervision Signals.* Compared with outcome rewards, it is more difficult to obtain fine-grained supervision signals. OpenAI has released a fine-grained annotation dataset named PRM800k [375] consisting of 12K process-annotated mathematical problems (i.e., MATH dataset [376]) and 75K solutions generated by LLMs of these problems, where each reasoning step of mathematical problems is labeled as *positive*, *negative* or *neutral* in PRM800k. Considering the cost and efficiency of the human annotation process, several methods aim to automatically annotate the correctness of intermediate reasoning steps, e.g., using powerful LLMs to directly replace human annotators [377] or Monte Carlo tree search [378]. After obtaining fine-grained supervision signals, existing work typically leverages them to train process-supervised reward models (PRM) [375, 379], which can produce step-level rewards (e.g., sentence based or token based rewards) during the RLHF procedure. Furthermore, rather than leveraging the discriminative model to produce the rewards, RLMEC [380] utilizes a generative reward model trained on rewriting tasks with the minimum editing constraint, to provide token-level rewards. In addition, for the downstream tasks where fine-grained supervision signals are difficult to collected, outcome-supervision signals can also be utilized to perform process-supervised RLHF [381].

- *Utilizing the PRMs.* To effectively leverage process-supervision signals from PRMs, existing work mainly utilizes these fine-grained signals to evaluate individual parts within the LLM responses and then guides LLMs to adjust their generation behaviors to maximize the received reward of the response. Concretely, expert iteration [382, 383], an effective RL algorithm, has been utilized to improve the base policy via learning from expert policy [374]. Typically, expert iteration contains two main stages: policy improvement and distillation [374]. In the policy improvement stage, expert policy processes the systematic search procedure to produce the samples under the guidance of PRMs. Subsequently, during the distillation stage, the samples generated by expert policy in the first stage are utilized to improve the base policy through supervised fine-tuning. In addition to expert iteration, PRMs can also be utilized to re-rank the candidates of the final answers generated by LLMs [375] or to select better intermediate reasoning steps during step by step reasoning [379, 384].

5.2.4 Alignment without RLHF

Although RLHF has achieved great success in aligning the behaviors of LLMs with human values and preferences, it also suffers from notable limitations. First, RLHF needs to train multiple LMs including the model being aligned, the reward model, and the reference model at the same time, which is tedious in algorithmic procedure and memory-consuming in practice. Besides, the commonly-used PPO algorithm in RLHF is rather complex and often sensitive to hyper-parameters. As an alternative, increasing studies explore to directly optimize LLMs to adhere to human preferences, using supervised fine-tuning without reinforcement learning [338].

Overview. The basic idea of non-RL alignment approaches is to directly fine-tune LLMs with *supervised learning* on high-quality *alignment dataset*. It basically assumes that response feedback or golden rules to avert unsafe behaviors have been injected or included in the specially curated alignment dataset, so that LLMs can directly learn aligned behaviors from these demonstration data via suitable fine-tuning strategies. Thus, to implement this approach, two key issues are the construction of alignment dataset and the design of fine-tuning loss. For the first issue, the alignment dataset can be automatically constructed by an aligned LLMs according to human-written safety principles [336] or refining existing examples using edits operations [385]. In addition, we can also reuse existing reward models to select high-rated responses from existing human feedback data [371]. For the second issue, non-RL alignment approaches mainly fine-tune LLMs in a supervised learning way (the same as the original instruction tuning loss) on a high-quality alignment dataset, meanwhile auxiliary learning objectives can be used to enhance the alignment performance, e.g., ranking responses or contrasting instruction-response pairs.

Alignment Data Collection. The construction of alignment data is important to effectively align the behaviors of LLMs with human preferences. To collect high-quality alignment data, some work tries to reuse existing reward models to select high-rated responses, and others explore to leverage powerful LLMs (e.g., ChatGPT) or build a simulated environment to generate synthetic alignment examples. Next, we will discuss these three lines of research.

- *Reward model based approaches.* The reward model in RLHF has been trained to measure the alignment degree on

26. https://huggingface.co/docs/transformers/v4.31.0/en/main_classes/text_generation#transformers.GenerationMixin.group_beam_search

the responses of LLMs. It is straightforward to leverage existing reward models to select high-quality responses as alignment data for subsequent fine-tuning. Based on this idea, RAFT [371] adopts reward models trained on human preference data to rank the responses of LLMs and collect those with higher rewards for supervised fine-tuning. In addition, the reward model can be also used to score model responses and assign them to different quality groups. Quark [386] sorts the responses of LLMs into different quantiles based on the reward scores. Each quantile is attached with a special reward token to represent the reward level of the quantile. Conditioned on the highest-reward tokens, LLMs are subsequently prompted to generate high-quality responses. Given an initial answer and the corresponding human feedback, ILF [387] first adopts LLMs to generate refined answers, then utilizes the reward model to select the answer that best matches the feedback for further training. As valuable resources for aligning LLMs, several reward models have been released, including DeBERTa-base/large/xxlarge from OpenAssistant²⁷, Moss-7B from Fudan²⁸, and Flan-T5-xl from Stanford²⁹.

- *LLM based generative approaches.* Reward models help to select aligned data from model responses. However, training reward models itself necessitates substantial high-quality human-labeled data, which is typically expensive and in short supply. In addition, although existing reward models can be reused, they might not be able to accurately capture the nonalignment behaviors in another separately trained LLM. Therefore, some work explores leveraging powerful LLMs to automatically generate human-aligned data. As a representative work, constitutional AI [369] proposes that human supervision comes from a set of principles (*i.e.*, natural language instructions) governing AI behaviors. Based on these principles, LLMs will critique their own harmful responses and revise them repeatedly into finally aligned responses. Similarly, Self-Align [336] first adopts self-instruct [147] to generate instructions focusing on covering diverse topics. Then, the model is also prompted with multiple human-written principles that describe the rules of expected model behaviors (also with several in-context exemplars), to generate helpful, ethical, and reliable responses as alignment data. To mitigate the limit that the original SFT method can only learn from positive responses, FIGA [388] develops an improved supervised alignment approach, where both negative (the original output of low quality) and positive (the refined output by LLMs) responses are leveraged in a contrastive way, to enable LLMs to deeply understand what fine-grained revisions actually lead to good response.

- *LLM based interactive approaches.* Most existing approaches train LLMs in isolation, where LLMs are not present in actual environments to improve themselves through external feedback signals. As a comparison, humans learn social norms and values from interactions with others in social environments [389]. To mimic such a learning approach, Stable Alignment [193] builds a simulated interaction environment consisting of a number of LLM agents, where AI agents keep interacting with and each other, receiving feedback on improvement. Once a central agent receives an instruction, it produces a response and shares it with nearby agents. These critic agents generate feedback comprising ratings about the response and revision

suggestions. Then the central agent would revise the original response following these suggestions. Such an alignment approach can be also extended to real-world environment with humans.

Supervised Alignment Tuning. After obtaining alignment data, it is also key to design suitable fine-tuning strategies for direct alignment. A straightforward approach is to optimize LLMs using the conventional sequence-to-sequence objective based on the alignment data. In addition to the conventional optimization objective, several studies further explore auxiliary losses that enhance the learning from the alignment data.

- *Primary training objective.* Since the alignment data typically consists of an input instruction and an output response, the primary training loss is still the traditional cross-entropy loss for sequence-to-sequence learning. Based on this loss, many studies propose a number of improvement variants for enhancing the supervised alignment tuning. For example, CoH [390] constructs the training data by prepending “A helpful answer:” and “An unhelpful answer:” to the annotated good and bad responses, respectively, and only compute losses for those response tokens with special masking. Quark [386] sorts model responses into different quantiles with varying alignment quality, it prepends a special reward token to each model response to represent the reward level of the response. These studies basically adopt the maximum likelihood objective, and employ instruction prefixes to guide the learning of human preference.

- *Direct preference optimization.* To better mimic the learning approach of RLHF in a supervised learning way, DPO [391] proposes to reparameterize the response rewards using the policy model (*i.e.*, the language model being optimized), and then the original reward modeling objective can be reformulated only based on the policy model. In this way, DPO removes the explicit reward modeling step, and optimizing the new learning objective that only involves the policy model is equivalent to optimizing the rewards. Based on DPO, existing work has proposed several improvement strategies for enhancing the effectiveness or efficiency, *e.g.*, decomposing the optimization of positive responses and negative responses into two independent components [392] or removing the probability of the reference model in the objective function [393]. Furthermore, FIGA [388] designs a token-level contrastive loss that aims to encourage desirable tokens, penalize undesirable ones, and disregard trivial tokens. Despite the effectiveness, recent work has also revealed that DPO may have inherent limitations in several aspects. First, based on the analysis about the magnitude and gradient directions, recent work reveals that DPO might have difficulty in well balancing the learning of positive instances and negative instances [394]. In addition, as the reference model provides the reward scores for itself in DPO algorithm, a weak reference model would also influence the alignment performance [395], which can be enhanced by improved learning strategies [396] or well-trained policy model [395].

- *Auxiliary optimization objectives.* Besides the primary cross-entropy loss, several studies propose auxiliary training loss to enhance the learning from the alignment data. First, since the responses of each instruction can be scored by the reward model, the ranking loss can be used to train the model to preserve the ranking order of these responses. For example, RRHF [397] samples responses from multiple sources, including model-generated responses, such as those derived from the model itself, ChatGPT, and GPT-4, as well as human-written responses, spanning

27. <https://huggingface.co/OpenAssistant>

28. <https://github.com/OpenLLMLab/MOSS-RLHF>

29. <https://huggingface.co/stanfordnlp/SteamSHP-flan-t5-xl>

both high-quality and low-quality instances. To align with the scores from reward models, it further optimizes the ranking loss by encouraging the model to have a higher conditional log probability for the response with a higher ranking. Moreover, SLiC-HF [398] proposes to assess the similarity between model outputs and human preference via the distance in the latent space, and introduces specific calibration and regularization loss to calibrate the candidate sequences based on human-preference data. Similarly, the difference between positive and negative responses from the reward model can be employed to construct the regularization loss [399], to enhance the discrimination between positive and negative responses by LLMs. Second, to enhance the relatedness between the response and the instruction, some work adopts contrastive learning to push up the probability of correct instruction-response pairs while pushing down incorrect instruction-response pairs. Specifically, for an output response, the proposed approach in [400] contrasts the target instruction to the other irrelevant instructions. By doing so, it can enable the model to learn the right correlation between instructions and responses.

5.2.5 Remarks on SFT and RLHF

As discussed in Section 5.1, instruction tuning is the process of training pre-trained language models with formatted demonstration data (instructions paired with desired outputs). At early exploration, instruction data was mainly collected from NLP tasks [67], while it has been now extended to more diverse supervision data that pairs input and output texts (e.g., the utterances of open-ended dialogues). Training with such paired texts is also called *supervised fine-tuning (SFT)* in the context of LLMs [66]. In this part, we mainly use the abbreviation *SFT* for discussion but not instruction tuning, due to the simplicity and popularity.

Since SFT and RLHF are two major adaptation tuning methods for LLMs, it is important to understand the connections and difference between them. Next, we make some discussions on this issue³⁰.

Overall Comparison with RL Formulation. Following the discussion in Section 5.2.3 (the part related to RL training), the text generation problem can be formulated as a decision-making process based on RL. Taking a prompt as input, the task of a LLM is to generate a text completion that appropriately responds to the prompt. This task would be completed step by step. At each step, an agent (*i.e.*, LLM) will perform an action (*i.e.*, generating a token) according to the policy (*i.e.*, the generative probability distribution of LLM) conditioned on the current state (currently generated token sequence and other available context information). It is expected that a high-quality output text would be produced by the LLM, which can earn a large reward score based on the entire response. Overall, RLHF and SFT can be considered as two different training approaches to optimizing the above decision making process for LLMs. Specially, RLHF firstly learns the reward model, and then employs it to improve the LLM with RL training (e.g., PPO). As a comparison, SFT adopts a teacher-forcing approach, which directly optimizes the likelihood of a demonstration output. Such a token-level training way essentially does *behavior cloning* (a special algorithm of imitation

learning [401]): it utilizes the expert’s action (*i.e.*, the target token at each step) as the supervision label and directly learns to imitate the demonstrations from experts without specifying a reward model as in typical RL algorithms. To learn the desired policies, SFT adopts a “local” optimization way (*i.e.*, token-level loss) based on demonstration data, while RLHF takes a “global” optimization way (*i.e.*, text-level loss) by involving human preference. More theoretical analysis about imitation learning and reinforcement learning can be referred to the related RL literature [401, 402].

Pros and Cons of SFT. SFT has been shown to be an effective approach to boosting the performance of LLMs on various benchmarks [67, 69, 152, 187], which can largely enhance the task generalization ability and flexibly endow specific functions (e.g., establishing the chatbot’s identity). More discussions about the usefulness of SFT can be found in Section 5.1.3. It has been widely recognized that SFT mainly *unlocks* the abilities but not *inject* new abilities into LLMs. Thus, it might become problematic when one tries to stimulate the non-endogenous abilities of LLMs via SFT. As a concrete scenario, it would potentially advocate the hallucination behaviors when demonstration data is beyond the knowledge or ability scope of LLMs, e.g., training a LLM to answer questions about its unknown facts. An interesting viewpoint from John Schulman’s talk on RLHF [403] is that distilling superior models to train less capable models (e.g., prompting GPT-4 to generate the response as fine-tuning data) might increase the possibilities of generating the hallucinated texts, thus likely affecting the factual accuracy of LLMs. Furthermore, as a behavior cloning method, SFT aims to imitate the behaviors (without explorations) of the experts who construct the demonstration data. However, there often exist variations among different annotators on the writing styles, quality, and preferences of demonstration data, which tends to affect the learning performance of SFT. Thus, high-quality instruction data (but not the quantity) is the primary factor for effective training of LLMs during the SFT stage [99].

Pros and Cons of RLHF. RLHF was early explored in the literature of deep RL [79], then borrowed to improve the capacity of language models (e.g., summarization [129]), and subsequently adopted as the fundamental technique to develop InstructGPT [66]. Recently, increasing evidence [99, 369] has demonstrated the effectiveness of RLHF in mitigating the harmful responses and enhancing the model capacity. Specially, LLaMA 2 has demonstrated that RLHF can improve both the helpfulness and harmlessness scores [99], and attributed this to a better human-LLM synergy for data annotation. They explain this reason in two major aspects as follows. First, since human annotators mainly provide preference annotations for RLHF, it can largely alleviate the discrepancies of annotators as that in SFT. Secondly, preference annotation is much easier than writing the demonstration data, and annotators can even judge the quality of more superior generations than those they create, making it possible to explore a broader state space beyond what can be demonstrated by human annotators. Another key point is that RLHF essentially encourages LLMs to learn correct policies by contrasting the self-generated responses (discriminating between good and bad responses). It no longer forces the model to imitate external demonstration data, and thus can mitigate the

30. This part would be somehow subjective, mainly based on the authors’ opinions and experiences. Comments or corrections are welcome to enhance this part.

hallucination issues with SFT as discussed above³¹. Actually, RLHF has been demonstrated to be an important approach to reduce the hallucination behaviors in GPT-4 [46]. However, RLHF inherits the drawbacks of classic RL algorithms, e.g., sample inefficiency and training instability. When adapted to LLMs, RLHF further relies on a strong SFT model as initial model checkpoint for efficiently achieving good performance. In addition, human annotators are involved in a complex iterative optimization process, in which a number of important details (e.g., the prompt selection, the schedule of reward model training and PPO training, and the settings of hyper-parameters) have important impact on the whole model performance.

Overall, SFT is particularly useful to increase the model capacity of pre-trained model checkpoints right after pre-training, while RLHF is promising to further improve the model capacity of SFT models. However, RLHF has been difficult to implement, and far from well explored (according to public literature), and more improvements (e.g., efficient and reliable annotation [369] and simplified optimization [391]) are still needed for further research.

5.3 Parameter-Efficient Model Adaptation

In the above, we have discussed the approaches of instruction tuning and alignment tuning to adapt LLMs according to specific goals. Since LLMs consist of a huge amount of model parameters, it would be costly to perform the full-parameter tuning. In this section, we will discuss how to conduct efficient tuning on LLMs. We first review several representative parameter-efficient fine-tuning methods for Transformer language models, and then summarize existing work on parameter-efficient fine-tuned LLMs.

5.3.1 Parameter-Efficient Fine-Tuning Methods

In existing literature, parameter-efficient fine-tuning [149, 404, 405] has been an important topic that aims to reduce the number of trainable parameters while retaining a good performance as possible. In what follows, we briefly review four parameter-efficient fine-tuning methods for Transformer language models, including adapter tuning, prefix tuning, prompt tuning and LoRA. The illustration of these four methods are shown in Figure 13.

Adapter Tuning. Adapter tuning incorporates small neural network modules (called *adapter*) into the Transformer models [406]. To implement the adapter module, a bottleneck architecture has been proposed in [406, 407], which first compresses the original feature vector into a smaller dimension (followed by a nonlinear transformation) and then recovers it to the original dimension. The adapter modules would be integrated into each Transformer layer, typically using a serial insertion after each of the two core parts (i.e., attention layer and feed-forward layer) of a Transformer layer. Alternatively, parallel adapters [408] can be also used in Transformer layers, where it places two adapter modules in parallel with the attention layer and feed-forward layer accordingly. During fine-tuning, the adapter modules would be optimized according to the specific task goals, while the parameters of the original language model are frozen in this process. In this way, we can effectively reduce the number of trainable parameters during fine-tuning.

31. In RLHF, it seems to be also important that reward models should be aware of the knowledge or ability of a LLM to be aligned. For example, LLaMA 2 adopts pre-trained chat model checkpoints to initialize reward models [99].

Prefix Tuning. Prefix tuning [404] prepends a sequence of prefixes, which are a set of trainable continuous vectors, to each Transformer layer in language models. These prefix vectors are task-specific, which can be considered as virtual token embeddings. To optimize the prefix vectors, a reparameterization trick [404] has been proposed by learning a MLP function that maps a smaller matrix to the parameter matrix of prefixes, instead of directly optimizing the prefixes. It has been shown that this trick is useful for stable training. After optimization, the mapping function would be discarded, and only the derived prefix vectors are kept to enhance task-specific performance. Since only the prefix parameters would be trained, it can lead to a parameter-efficient model optimization. Similar to prefix tuning, p-tuning v2 [409] incorporates layer-wise prompt vectors into the Transformer architecture specially for natural language understanding, which also utilizes multi-task learning for jointly optimizing shared prompts. It has been shown to be useful in improving the model performance of different parameter scales on natural language understanding tasks.

Prompt Tuning. Different from prefix tuning, prompt tuning [405, 410] mainly focuses on incorporating trainable prompt vectors at the input layer³². Based on the discrete prompting methods [412, 413], it augments the input text by including a group of soft prompt tokens (either in a free form [410] or a prefix form [405]), and then takes the prompt-augmented input to solve specific downstream tasks. In implementation, task-specific prompt embeddings are combined with the input text embeddings, which are subsequently fed into language models. P-tuning [410] has proposed a free form to combine the context, prompt and target tokens, which can be applied to the architectures for both natural language understanding and generation. They further learn the representations of soft prompt tokens by a bidirectional LSTM. Another representative approach [405] named *prompt tuning* directly prepends prefix prompts to the input. During training, only the prompt embeddings would be learned according to task-specific supervisions. Since this method only includes a small number of trainable parameters at the input layer, it has been found that the performance highly relies on the model capacity of the underlying language models [405].

Low-Rank Adaptation (LoRA). LoRA [149] imposes the low-rank constraint for approximating the update matrix at each dense layer, so as to reduce the trainable parameters for adapting to downstream tasks. Consider the case of optimizing a parameter matrix $\mathbf{W}$. The update process can be written in a general form as: $\mathbf{W} \leftarrow \mathbf{W} + \Delta\mathbf{W}$. The basic idea of LoRA is to freeze the original matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ while approximating the parameter update $\Delta\mathbf{W}$ by low-rank decomposition matrices, i.e., $\Delta\mathbf{W} = \mathbf{A} \cdot \mathbf{B}^\top$, where $\mathbf{A} \in \mathbb{R}^{m \times k}$ and $\mathbf{B} \in \mathbb{R}^{n \times k}$ are the trainable parameters for task adaptation and $k \ll \min(m, n)$ is the reduced rank. The major merit of LoRA is that it can largely save the memory and storage usage (e.g., VRAM). Further, one can only keep a single large model copy, while maintaining

32. Here, prompt tuning denotes a category of related efficient tuning methods exemplified by the work [405, 410, 411], instead of a specific method as used in [405]. Indeed, the prefix based tuning methods [404, 409] can be also considered as prompting methods, which are called *deep prompting tuning* in [409]. In this survey, prompt tuning specially refer to the methods that only include the prompt tokens at the input layer, in the context of LLMs. We assign p-tuning v2 [409] to the category of prefix tuning, because it incorporates layerwise prompts in language models.

a number of task-specific low-rank decomposition matrices for adapting to different downstream tasks. Further, several studies have also discussed how to set the rank in a more principled approach, e.g., importance score based allocation [414] and search-free optimal rank selection [415].

Besides the above methods, there is extensive research on efficient tuning of Transformer language models. However, a more comprehensive discussion of efficient tuning is beyond the scope of this article, which can be found in the related papers on this topic [408, 416].

5.3.2 Parameter-Efficient Fine-Tuning on LLMs

With the rising of LLMs, efficient tuning has attracted increasing research attention for developing a more lightweight adaptation approach in downstream tasks.

In particular, LoRA [149] has been widely applied to open-source LLMs (e.g., LLaMA and BLOOM) for parameter-efficient fine-tuning. Among these research attempts, LLaMA and its variants have gained much attention for parameter-efficient tuning. For example, Alpaca-LoRA [148] has been trained using LoRA as a lightweight tuned version of Alpaca [146] (a fine-tuned 7B LLaMA model with 52K human demonstrations of instruction following). There are extensive explorations of Alpaca-LoRA ranging in different languages or model sizes, which can be found in the collection page³³. A recent study LLaMA-Adapter [417] inserts learnable prompt vectors into each Transformer layer, in which zero-initialized attention has been proposed to improve the training by mitigating the influence of under-fitted prompt vectors. They also extend this approach to a multi-modal setting, e.g., visual question answering.

Further, an empirical study [407] has been conducted to examine the effect of different tuning methods on language models. They compare four efficient tuning methods including serial adapter tuning [406], parallel adapter tuning [408, 418], and LoRA [149], on three open-source LLMs, namely GPT-J (6B), BLOOM (7.1B) and LLaMA (7B), for evaluation. Based on the experimental results on six math reasoning datasets, they show that these efficient-tuning methods under-perform the reference baseline GPT-3.5 on difficult tasks, while achieving a comparable performance on simple tasks. Overall, LoRA performs relatively well among these comparison methods, using significantly fewer trainable parameters.

As an important resource, the library *PEFT* [419] (standing for parameter-efficient fine-tuning) has been released on GitHub³⁴. It has included several widely used efficient tuning methods, including LoRA [149]/AdaLoRA [414], prefix-tuning [404, 409], P-Tuning [410], and prompt-tuning [405]. Further, it supports a number of language models such as GPT-2 and LLaMA, and also covers several representative vision Transformer models (e.g., ViT and Swin Transformer).

As discussed in Section 5.3.1, there have been a large number of efficient tuning methods proposed in the existing literature. However, most of these approaches are tested on small-sized pre-trained language models, instead of the LLMs. So far, there still lacks a thorough investigation on the effect of different efficient tuning methods on large-sized language models at different settings or tasks.

6 Utilization

After pre-training or adaptation tuning, a major approach to using LLMs is to design suitable *prompting* strategies for solving various tasks. In existing literature, task-specific prompts can be effectively learned through manual creation and automatic optimization. A representative prompting method is *in-context learning* [50, 55], which formulates the task description and/or demonstrations in the form of natural language text. In addition, *chain-of-thought prompting* [33] can be employed to enhance in-context learning by involving a series of intermediate reasoning steps in prompts. Furthermore, *planning* [432] is proposed for solving complex tasks, which first breaks them down into smaller sub-tasks and then generates a plan of action to solve these sub-tasks one by one. We summarize representative work for these prompting approaches in Table 11. Next, we will elaborate on the details of the four techniques.

6.1 Prompting

As discussed in previous work [36], prompting is the major approach to utilizing LLMs for solving various tasks. Since the quality of prompts will largely influence the performance of LLMs in specific tasks, there have been a series of studies proposed to generate suitable task prompts through manual creation or automatic optimization, which will be introduced in this section.

6.1.1 Prompt Creation

The process of manually creating a suitable prompt is also called *prompt engineering* [445, 446]. A well-designed prompt is very helpful to elicit the abilities of LLMs for accomplishing specific tasks. In this part, we will first introduce the key components of prompts and discuss several principles for prompt design. Then, we evaluate ChatGPT with different prompts to show the results on several representative tasks. We are aware that there have been several existing papers [446, 447] and websites [448–450] that present the suggestions and guidelines to design good prompts. As a comparison, we mainly aim to discuss the key factors (ingredients and principles) that are useful for prompt creation, and provide experimental results and analysis on popular tasks as the reference to the beginners.

Key Ingredients. Typically, there are four key ingredients that depict the functionality of a prompt for eliciting the abilities of LLMs to complete the tasks, including task description, input data, contextual information, and prompt style. To have an intuitive understanding of our discussion, we also present three prompt examples for question answering, meta-review generation, and text-to-SQL in Table 13.

- **Task description.** A task description is typically a specific instruction that LLMs are expected to follow. In general, one should clearly describe the task goal in natural language. For the tasks with special input or output format, detailed clarifications are often needed, and one can further utilize keywords to highlight the special settings for better guiding LLMs in task completion.

- **Input data.** In common cases, it is straightforward to describe input data (e.g., an instance to be responded by LLMs) in natural language. For special input data, such as knowledge graph and table, it is necessary to apply an appropriate and

33. <https://github.com/tloen/alpaca-lora>

34. <https://github.com/huggingface/peft>

TABLE 11: Typical LLM utilization methods and their key points for ICL, CoT, and planning. Note that the key points only highlight the most important technical contribution.

Approach	Representative Work	Key Point
In-context Learning (ICL)	KATE [420] EPR [421] SG-ICL [422] APE [423] Structured Prompting [424] GlobalE & LocalE [425]	Demonstration selection (similar; k-NN) Demonstration selection (dense retrieval; constrative learning) Demonstration selection (LLM as the demonstration generator) Demonstration format (automatic generation & selection) Demonstration format (grouped context encoding; rescaled attention) Demonstration order (entropy-based metric; probing set generation with LLM)
Chain-of-thought Prompting (CoT)	Complex CoT [426] Auto-CoT [427] Selection-Inference [428] Self-consistency [429] DIVERSE [430] Rationale-augmented ensembles [431]	Demonstration (complexity-based selection) Demonstration (automatic generation) Generation (alternate between selection and inference) Generation (diverse paths; self-ensemble) Generation (diverse paths); Verification (step-wise voting) Generation (rationale sampling)
Planning	Least-to-most prompting [432] DECOMP [433] PS [434] Faithful CoT [435] PAL [436] HuggingGPT [437] AdaPlanner [438] TIP [439] RAP [440] ChatCoT [441] ReAct [442] Reflexion [443] Tree of Thoughts [444]	Plan generation (text-based; problem decomposition) Plan generation (text-based; problem decomposition) Plan generation (text-based) Plan generation (code-based) Plan generation (code-based; Python) Plan generation (code-based; models from HuggingFace) Plan refinement (skill memory) Feedback acquisition (visual perception) Feedback acquisition (LLM as the world model); Plan refinement (Monte Carlo Tree Search) Feedback acquisition (tool); Plan refinement (conversation between LLM and tools) Feedback acquisition (tool); Plan refinement (synergizing reasoning and acting) Feedback acquisition (text-based self-reflection); Plan refinement (dynamic memory) Feedback acquisition (vote comparison); Plan refinement (tree-based search)

convenient way to make them readable for LLMs. For structured data, linearization is commonly used to transform the original records (e.g., knowledge triples) into sequences [451] due to the simplicity. Further, the programming language (e.g., executable code) has also been utilized to formulate the structured data, which can also support using external tools (e.g., program executor) to produce the precise results [452, 453].

- *Contextual information.* In addition to the task description and input data, contextual or background information is also essential for specific tasks. For example, retrieved documents are highly useful for open-domain question answering as supporting evidence. Both the quality of the retrieved documents and their relevance to the question have an impact on the generated answers [454]. Thus, it needs to include such information in a proper prompt pattern or expression format. Furthermore, in-context task exemplars are also helpful for eliciting LLMs to accomplish a complex task, which can better depict the task goal, the special output formats, and the mapping relation between input and output.

- *Prompt style.* For different LLMs, it is important to design a suitable prompt style for eliciting their abilities to solve specific tasks. Overall, one should express the prompt as a clear question or detailed instruction that can be well understood and answered. In some cases, it is also useful to add the prefix or suffix to better guide LLMs. For example, using the prefix “*Let us think step by step*” can help elicit LLMs perform step-by-step reasoning, and using the prefix “*You are an expert on this task (or in this domain)*” can boost the performance of LLMs in some specific tasks. Further, for chat-based LLMs (e.g., ChatGPT), instead of directly feeding a long or complex task prompt, it is suggested to decompose it into multiple prompts for the sub-tasks and then feed them into LLMs via a multi-turn conversation [441].

Design Principles. Based on the key ingredients of prompts, we summarize several critical design principles that can help create more effective prompts for solving various tasks.

- *Expressing the task goal clearly.* Task descriptions should not be ambiguous or unclear, which likely lead to inaccurate or inappropriate responses. This highlights the need for clear and unambiguous directives when utilizing these models [66]. A clear and detailed description should contain various elements to explain a task, including task objective, input/output data (e.g., “*Given a long document, I want you to generate a concise summary.*”), and the response constraints (e.g., “*the length of the summary cannot exceed 50.*”). By providing a well-clarified task description, LLMs can more effectively understand the target task and generate the desired output.

- *Decomposing into easy, detailed sub-tasks.* To solve complex tasks, it is important to decompose the difficult task into several more easier, detailed sub-tasks for helping LLMs accomplish the goal step by step, which is closely related to the planning technique in Section 6.4. For example, following the suggestion [447], we can explicitly list the sub-tasks in the form of multiple numbered items (e.g., “*Braid a coherent narrative by performing the following tasks: 1.; 2.; 3.*”). By decomposing a target task into sub-tasks, LLMs can focus on solving easier sub-tasks and finally achieve more accurate results for complex tasks.

- *Providing few-shot demonstrations.* As discussed in Section 6.2, LLMs can benefit from in-context learning for solving complex tasks, where the prompts contain a small number of task examples of the desired input-output pairs, i.e., few-shot demonstrations. Few-shot demonstrations can help LLMs learn the semantic mapping between input and output without parameter tuning. In practice, it is suggested that one should generate a few high-quality demonstrations for the target task, which would highly benefit the final task performance.

- *Utilizing model-friendly format.* Since LLMs are pretrained on specially constructed datasets, there are some prompt formats that can make LLMs better understand the instruction. For example, as the OpenAI documentation suggests, we can use ### or "" as a stop symbol to separate the instruction and context,

which can be better understood by LLMs. As a general guideline, most existing LLMs perform a task better in English, thus it is useful to employ English instructions to solve difficult tasks based on machine translation.

- **Adopting role-playing strategies.** Since LLMs are pretrained on extensive corpora containing diverse characters and dialogues, they possess an inherent ability for role-playing. This feature can be harnessed through specific prompts to enhance the corresponding capacity for some specific domains [455]. For instance, when solving a math problem, we can use a prompt prefix like “*You are an expert in mathematics*”. This enables LLMs to solve the problem from an expert’s perspective, thereby leveraging their pretrained knowledge more effectively. By guiding LLMs with role-playing prompts, they can often generate more reasonable and accurate solutions.

Useful Tips. In addition to the design principles, we also present a collection of useful prompt tips based on existing work or our empirical experiences in Table 12. Note that these tips are suggested in a general manner, it does not indicate that they are the best prompts for the corresponding tasks. This part will be continuously updated with more guidelines or tips. We welcome readers to contribute to this collection of prompt tips. We present the detailed procedure to contribute to the prompt tips, at the link: <https://github.com/RUCAIBox/LLMSurvey/tree/main/Prompts>.

Empirical Analysis. We further conduct empirical studies to present the impact of prompts on task performance. To conduct the experiments, we select a variety of tasks that span language generation, knowledge utilization, complex reasoning, structure data generation, and information retrieval. For each task, we manually write a prompt that follows general guidelines introduced above. Note that the tested prompts may not be the optimal for these tasks, since they mainly aim to help readers understand how to write an effective prompt for solving different tasks. Also, we add a simplified prompt as the comparison for most tasks. Following the experimental settings in Section 7.4, we examine the 3-shot performance of ChatGPT on complex reasoning tasks (Colored Objects and GSM8k), and zero-shot performance on other tasks. We report the experimental results in Table 17, where we also include the supervised performance in existing papers as reference.

- **Carefully designed prompts can boost the zero-shot or few-shot performance of ChatGPT.** By comparing the results of using different prompts on the same task, we can see that using the carefully designed prompts can achieve better performance than the simpler ones. In the carefully designed prompts, we provide a more clearly expressed task description (e.g., WMT and WikiFact), or use a model-friendly format (e.g., GSM8k and OBQA). For example, for WikiFact task, the prompt with a more detailed task description leads to a performance increase from 29.25 to 31.21.

- **More complex tasks can benefit more from careful prompt engineering on ChatGPT.** In the WikiFact and Colored Objects tasks, the designed prompts have greatly improved the performance of ChatGPT, i.e., from 23.61 to 28.47 on WikiFact and from 53.20 to 66.75 on Colored Objects. It indicates the necessity of prompt engineering for LLMs to perform well on complex tasks, since these tasks typically have specific output formats or require background knowledge. Our example prompts provide more detailed task description (e.g., output format and task

goal), which can help ChatGPT better understand the complex task requirement for fulfilling it.

- **For mathematical reasoning tasks, it is more effective to design specific prompts based on the format of programming language.** For GSM8k, the designed prompt employs code-formatted few-shot demonstrations to convert this mathematical reasoning task into code generation task, which can leverage the strong code synthesis ability of ChatGPT for solving mathematical problems. Further, with the help of an external program executor, we are able to obtain more precise results instead of using LLMs for arithmetic operation. As we can see, the performance is boosted from 78.47 to 79.30 on GSM8k, indicating the usefulness of programming language in mathematical reasoning tasks.

- **In knowledge utilization and complex reasoning tasks, ChatGPT with proper prompts achieves comparable performance or even outperforms the supervised baselines methods.** In knowledge utilization and complex reasoning tasks, ChatGPT with proper zero-shot or few-shot prompts can achieve comparable performance or even outperform the supervised methods, e.g., 31.21 (ChatGPT) v.s. 34.20 (supervised baseline) on WikiFact. Despite that, ChatGPT still performs worse than supervised baseline models on some specific tasks (e.g., ARC and WikiFact), since these supervised models have been specially optimized with task-specific data.

- **Through suitable prompt engineering, LLMs can handle some non-traditional NLP tasks.** With the help of specific prompts, ChatGPT can also accomplish non-traditional NLP tasks, i.e., the general recommendation and conversational recommendation. A key point is that these tasks can be well expressed or described in natural language. However, the performance of ChatGPT is still far from the referenced performance in these tasks, as LLMs cannot directly fit these tasks, which require specific domain knowledge and task adaptation [355, 456].

6.1.2 Prompt Optimization

Although manually creating task prompts is more intuitive, it is time consuming and, more importantly, models are highly sensitive to the crafted prompts—improper prompts will lead to low task performance (as shown in Table 17). Therefore, a large body of studies propose automatic optimization approaches for discrete prompts and continuous prompts to achieve the optimal performance [404, 413]. In this part, we will detail these studies from two perspectives, i.e., discrete prompts and continuous prompts.

Discrete Prompt Optimization. Discrete prompt is typically composed of a sequence of natural language tokens. Despite that the form is simple and flexible, optimizing prompts in discrete space is a challenging problem due to the combinatorial huge search space. To automatically search effective prompts for downstream tasks, existing studies propose a wide spectrum of discrete prompt optimization approaches, which are detailed as follows.

- **Gradient-based approaches.** This kind of approaches aims to optimize the prompt search process by maximizing the output likelihood via gradient update [413, 458–461]. As a representative work, Auto-Prompt [413] proposes a gradient-guided method to greedily search the optimal token for each position of the prompt, leveraging the gradient approximated by the change in the log-likelihood when replacing a prompt token with another candidate token from vocabulary. However, such a search process can be

TABLE 12: A collection of useful tips for designing prompts that are collected from online notes [446–449] and experiences from our authors, where we also show the related ingredients and principles (introduced in Section 6.1.1). We abbreviate principles as Prin. and list the IDs of the related principles for each prompt. ①: expressing the task goal clearly; ②: decomposing into easy, detailed sub-tasks; ③: providing few-shot demonstrations; ④: utilizing model-friendly format.

Ingredient	Collected Prompts	Prin.
Task Description	T1. Make your prompt as detailed as possible , e.g., "Summarize the article into a short paragraph within 50 words. The major storyline and conclusion should be included, and the unimportant details can be omitted."	①
	T2. It is helpful to let the LLM know that it is an expert with a prefixed prompt , e.g., "You are a sophisticated expert in the domain of computer science."	①
	T3. Tell the model more what it should do , but not what it should not do.	①
	T4. To avoid the LLM to generate too long output, you can just use the prompt: "Question: Short Answer: ". Besides, you can also use the following suffixes, "in a or a few words", "in one of two sentences".	①
Input Data	I1. For the question required factual knowledge, it is useful to first retrieve relevant documents via the search engine, and then concatenate them into the prompt as reference.	④
	I2. To highlight some important parts in your prompt, please use special marks , e.g., quotation ("") and line break (\n). You can also use both of them for emphasizing.	④
Contextual Information	C1. For complex tasks, you can clearly describe the required intermediate steps to accomplish it, e.g., "Please answer the question step by step as: Step 1 - Decompose the question into several sub-questions, . . ."	②
	C2. If you want LLMs to provide the score for a text, it is necessary to provide a detailed description about the scoring standard with examples as reference.	①
	C3. When LLMs generate text according to some context (e.g., making recommendations according to purchase history), instructing them with the explanation about the generated result conditioned on context is helpful to improve the quality of the generated text.	②
	C4. An approach similar to tree-of-thoughts but can be done in one prompt : e.g., "Imagine three different experts are answering this question. All experts will write down one step of their thinking, then share it with the group of experts. Then all experts will go on to the next step, etc. If any expert realizes they're wrong at any point then they leave. The question is	②
Demonstration	D1. Well-formatted in-context exemplars are very useful, especially for producing the outputs with complex formats.	③
	D2. For few-shot chain-of-thought prompting, you can also use the prompt "Let's think step-by-step", and the few-shot examples should be separated by "\n" instead of full stop.	①③
	D3. You can also retrieve similar examples in context to supply the useful task-specific knowledge for LLMs. To retrieve more relevant examples, it is useful to first obtain the answer of the question, and then concatenate it with the question for retrieval.	③④
	D4. The diversity of the in-context exemplars within the prompt is also useful. If it is not easy to obtain diverse questions, you can also seek to keep the diversity of the solutions for the questions.	③
	D5. When using chat-based LLMs, you can decompose in-context exemplars into multi-turn messages , to better match the human-chatbot conversation format. Similarly, you can also decompose the reasoning process of an exemplars into multi-turn conversation.	③
	D6. Complex and informative in-context exemplars can help LLMs answer complex questions.	③
	D7. As a symbol sequence can typically be divided into multiple segments (e.g., $i_1, i_2, i_3 \rightarrow i_1, i_2 \text{ and } i_2, i_3$), the preceding ones can be used as in-context exemplars to guide LLMs to predict the subsequent ones, meanwhile providing historical information.	②③
	D8. Order matters for in-context exemplars and prompts components. For very long input data, the position of the question (first or last) may also affect the performance.	③
	D9. If you can not obtain the in-context exemplars from existing datasets, an alternative way is to use the zero-shot generated ones from the LLM itself.	③
Other Designs	O1. Let the LLM check its outputs before draw the conclusion, e.g., "Check whether the above solution is correct or not."	②
	O2. If the LLM can not well solve the task, you can seek help from external tools by prompting the LLM to manipulate them. In this way, the tools should be encapsulated into callable APIs with detailed description about their functions, to better guide the LLM to utilize the tools.	④
	O3. The prompt should be self-contained , and better not include pronouns (e.g., it and they) in the context.	①
	O4. When using LLMs for comparing two or more examples, the order affects the performance a lot.	①
	O5. Before the prompt, assigning a role for the LLM is useful to help it better fulfill the following task instruction, e.g., "I want you to act as a lawyer".	①
	O6. OpenAI models can perform a task better in English than other languages. Thus, it is useful to first translate the input into English and then feed it to LLMs.	④
	O7. For multi-choice questions, it is useful to constrain the output space of the LLM. You can use a more detailed explanation or just imposing constraints on the logits.	①
	O8. For sorting based tasks (e.g., recommendation), instead of directly outputting the complete text of each item after sorting, one can assign indicators (e.g., ABCD) to the unsorted items and instruct the LLMs to directly output the sorted indicators.	①

extremely expensive since it needs to evaluate each candidate token for each position of the prompt, leading to a number of additional forward passes. Therefore, an improved gradient method [458] has been proposed by transforming discrete tokens into continuous embeddings and computing the gradient on continuous space during optimization.

- **RL-based approaches.** Since discrete prompts are difficult to be learned through gradient back-propagation, a number of studies propose to formulate the discrete prompt optimization as a reinforcement learning (RL) problem and leverage RL algorithms for optimization [462–465]. For example, RLPrompt [462] trains a policy network to generate desired prompts with multiple reward functions. In this approach, several effective reward stabilization strategies are also proposed to enhance the RL training

efficiency. Compared to previous work that requires sufficient data for training, TEMPERA [463] proposes to edit prompts at test time by utilizing a pre-trained RL agent to sequentially edit different parts of a manually-written initial prompt. Although these methods are simple and effective, they explore a manually defined edit space (e.g., add, swap and delete) and focus on modifying the original prompt, which limits the flexibility of prompt search. In contrast, PRewrite [465] employs RL to train a prompt rewriter for generating new prompts instead of modification, which does not impose any restrictions in the prompt rewriting and offers improved flexibility in the action space.

- **Edit-based approaches.** For the above methods, gradient-based and RL-based tuning can be extremely computationally demanding for ever larger models, and may not be feasible for

TABLE 13: Example instructions collected from [447, 457]. The **blue** text denotes the task description, the **red** text denotes the contextual information, the **green** text denotes the demonstrations, and the **gold** text denotes the prompt style.

Use the provided articles delimited by triple quotes to answer questions. If the answer cannot be found in the articles, write "I could not find an answer."
Articles: ""Joao Moutinho is a Portuguese footballer who last played as a central midfielder for Premier League club Wolverhampton Wanderers and the Portugal national team.""
Question: Is the following sentence plausible? 'Joao Moutinho was out at third.'
Answer: Let's think step by step. Joao Moutinho is a soccer player. Being out at third is part of baseball, not soccer. So the answer is No.
... <Demonstrations>
Articles: <insert articles, each delimited by triple quotes> Question: <insert question> Answer:
Prepare a meta-review by answering the following questions from the reviewer comments (provided after the questions).
1. Based on the reviewer's comments, what are the core contributions made by this manuscript? 2. What are the common strengths of this work, as mentioned by multiple reviewers? 3. What are the common weaknesses of this work, as highlighted by multiple reviewers? 4. What suggestions would you provide for improving this paper? 5. What are the missing references mentioned by the individual reviews?
The review texts are below: <insert three comments R_1, R_2, R_3 from the reviewers> Meta-review: <insert meta-review> ... <Demonstrations>
Provide justification for your response in detail by explaining why you made the choices you actually made. A good output should be coherent, highlight major strengths/issues mentioned by multiple reviewers, be less than 400 words in length, and finally, the response should be in English only.
The review texts are below: <insert three comments R_1, R_2, R_3 from the reviewers> Meta-review:
CREATE TABLE Highschooler (ID int primary key, name text, grade int); /* 3 example rows: SELECT * FROM Highschooler LIMIT 3; ID name grade 1234 Janie 8 5678 Mary 8 9012 Mike 9 */
Using valid SQLite, answer the following questions for the tables provided above.
Question: What is Kyle's id? SQL: SELECT ID FROM Highschooler WHERE name="Kyle"; ... <Demonstrations>
Question: <insert question> SQL:

API-based model calls (e.g., ChatGPT). Therefore, another line of work aims to directly edit existing prompts based on the task performance. Specifically, GPS [466] borrows an idea from the genetic algorithm and proposes a genetic prompt search method that utilizes a language model (i.e., T5) to edit prompts by taking the cloze task form. In addition to model based edit methods, human-defined operations can be also employed for prompt editing [467], including delete, swap, paraphrase, and addition. Based on these operations, they iteratively edit the prompts and greedily search for the best prompt guided by the model performance on a small pool of examples.

- *LLM-based approaches.* Due to the exceptional capacities of LLMs, an increasing number of studies directly leverage

LLMs as prompt generator [468–475]. Specifically, APE [468] utilizes an LLM to generate initial prompts, then selects the best prompt with the highest accuracy, and finally improves the best candidate through an iterative Monte Carlo search method. However, this method does not effectively constrain the prompt search space, which might likely lead to unstable results. To achieve good performance and fast convergence, one line of work utilizes heuristic methods (e.g., evolutionary algorithms [473, 474] and adversarial learning [475]) for prompt optimization. Another line of work draws an analogy to gradient-based model optimizers for LLM-based prompt optimization. For example, APO [469] instructs the LLM to generate text feedback on how to refine an old prompt into new improved

prompts and then execute textual gradient descent. However, their search in the prompt space might be inefficient without fully considering the whole refinement trace of previous prompts, thus potentially leading to sub-optimal results. Therefore, some recent studies [470, 471] incorporate the previous prompts with their scores to instruct LLMs for progressively generating better new prompts. To further design formalized guidelines about the design of prompt optimizers, GPO [472] conducts a systematic analogy for LLM-based prompt optimizers with gradient-based model optimizers. It further develops a more formal LLM-based prompt optimization framework, which extensively borrows the idea of machine learning optimization. Specifically, it retrieves relevant prompts from the previous prompts and utilizes the generation-based refinement strategy to perform the update. In order to avoid large variation at each iteration, GPO further adopts a cosine-based decay strategy to control the edit distance. However, these approaches still struggle in exploring the vast space of effective prompts. Inspired by human-like trial-and-error, prompt optimization is further formulated as a strategic planning problem [476] and uses Monte Carlo tree search to navigate the vast prompt space.

Continuous Prompt Optimization. Different from discrete prompts, continuous prompts consist of a set of continuous embeddings, which can be directly optimized through the gradient update based on the loss of downstream tasks. Note that continuous prompt optimization has been mainly studied in PLMs, but draws limited attention in era of LLMs due to their massive magnitudes of parameters. We include the discussion of this part for content completeness. In prior work, most studies typically rely on supervised learning to train continuous prompts based on task data. Furthermore, in data-scarce scenarios, transfer learning methods can be employed to alleviate the lack of labeled data on target tasks. These two approaches are detailed below.

- *Prompt learning with sufficient data.* In this approach, most existing methods regard continuous prompts as trainable model parameters and then leverage supervised learning to optimize the continuous prompts by minimizing the cross-entropy loss based on sufficient downstream task data [404, 405, 409, 477]. As discussed in Section 5.3.1, prefix tuning [404] prepends a sequence of prefixes (*i.e.*, a set of trainable continuous vectors) to each Transformer layer in language models, while prompt tuning [405] only incorporates trainable prompt vectors at the input layer. By fixing the large-scale parameters of LLMs and only tuning continuous prompt vector, this kind of approaches can be extremely parameter-efficient (Section 5.3). However, these approaches are typically independent of the inputs, lacking sufficient consideration of input semantics. Therefore, the authors in [477] propose context tuning, where the continuous prompts are derived based on the input text and learned through the downstream task losses.

- *Prompt transferring with scarce data.* Supervised learning approaches demand in sufficient training data to learn optimal continuous prompts, which may not work well in data-scarce domains and tasks. To address this problem, SPoT [478] proposes a prompt-based transfer learning approach, which first learns a single continuous prompt for several representative source tasks and then uses this prompt to initialize the prompt for a target task. However, this approach leverages the same prompt for solving all instances of the target task. For a single task, even a well-learned prompt may not be suitable for all the data instances

from a large population. To address this issue, an improved method [479] designs an adaptive attention mechanism during the prompt transfer process to derive the target prompts, considering both task- and instance-level information. The prompt transfer paradigm can leverage the knowledge of data-sufficient source tasks encoded in source prompts for solving data-scarce target tasks.

6.2 In-Context Learning

As a special prompting form, in-context learning (ICL) is first proposed along with GPT-3 [55], which has become a typical approach to utilizing LLMs.

6.2.1 ICL Formulation

As stated in [55], ICL uses a formatted natural language prompt, consisting of the task description and/or a few task examples as demonstrations. Figure 14 presents an illustration of ICL. First, starting with a task description, a few examples are selected from the task dataset as demonstrations. Then, they are combined in a specific order to form natural language prompts with specially designed templates. Finally, the test instance is appended to the demonstration as the input for LLMs to generate the output. Based on task demonstrations, LLMs can recognize and perform a new task without explicit gradient update.

Formally, let $D_k = \{f(x_1, y_1), \dots, f(x_k, y_k)\}$ represent a set of demonstrations with k examples, where $f(x_k, y_k)$ is the prompt function that transforms the k -th task example into natural language prompts. Given the task description I , demonstration D_k , and a new input query x_{k+1} , the prediction of the output $\hat{y}_{k+1}$ generated from LLMs can be formulated as follows³⁵:

$$\text{LLM}(I, \underbrace{f(x_1, y_1), \dots, f(x_k, y_k)}_{\text{demonstrations}}, \underbrace{f(x_{k+1}, \text{---})}_{\text{input answer}}) \rightarrow \hat{y}_{k+1}. \quad (11)$$

where the actual answer y_{k+1} is left as a blank to be predicted by the LLM. Since the performance of ICL heavily relies on demonstrations, it is important to properly design them in the prompts. According to the construction process in Equation (11), we focus on three major aspects of formatting demonstrations in the prompts, including how to select examples that make up demonstrations, format each example into the prompt with the function $f(\cdot)$, and arrange demonstrations in a reasonable order.

A comprehensive review of ICL has been presented in the survey paper [50], and we suggest the readers referring to it for a more general, detailed discussion on this topic. Compared with this survey, we specially focus on the discussion of applying ICL to LLMs in two major aspects, *i.e.*, demonstration design and the underlying mechanism of ICL. Also, ICL has a close connection with instruction tuning (discussed in Section 5.1) in that both utilize natural language to format the task or instances. However, instruction tuning needs to fine-tune LLMs for adaptation, while ICL only prompts LLMs for utilization. Furthermore, instruction tuning can enhance the ICL ability of LLMs to perform target tasks, especially in the zero-shot setting (only using task descriptions) [69].

35. When ICL was introduced in the GPT-3's paper [55], it was originally defined to be a combination of the task description and demonstration examples, wherein either component is dispensable. Following this definition, when a LLM is required to solve an unseen task by using only task descriptions, it can be also considered to perform ICL for task solving, whereas the ICL ability can be enhanced by instruction tuning.

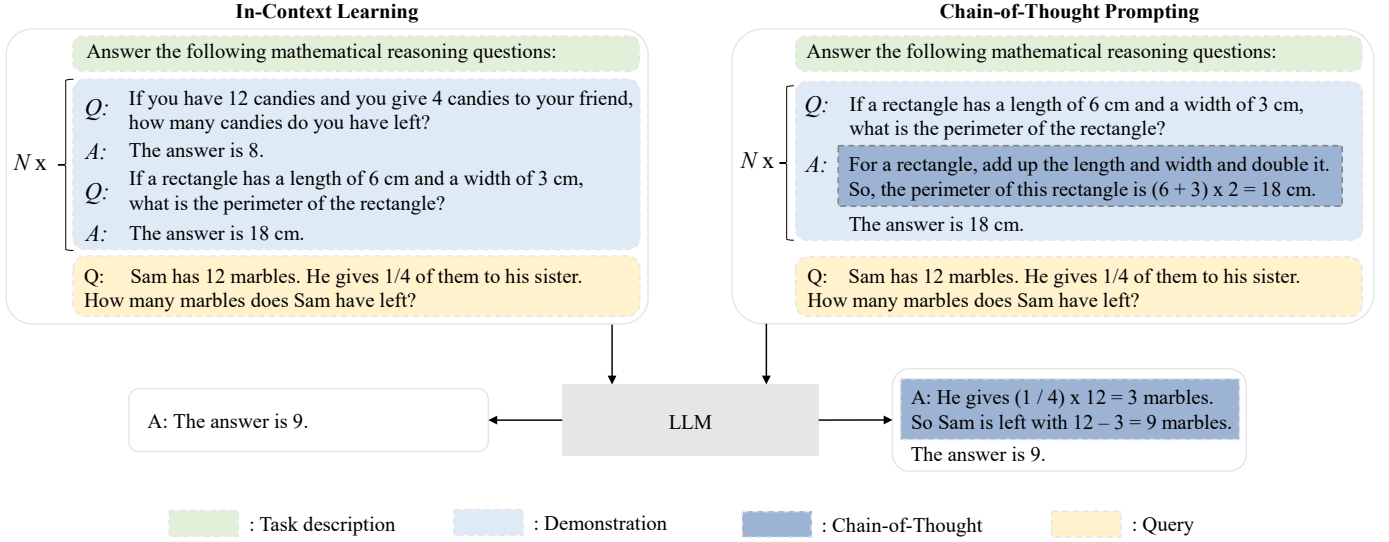


Fig. 14: A comparative illustration of in-context learning (ICL) and chain-of-thought (CoT) prompting. ICL prompts LLMs with a natural language description, several demonstrations, and a test query, while CoT prompting involves a series of intermediate reasoning steps in prompts.

6.2.2 Demonstration Design

Several studies have shown that the effectiveness of ICL is highly affected by the design of demonstrations [425, 480, 481]. Following the discussion in Section 6.2.1, we will introduce the demonstration design of ICL from three major aspects, *i.e.*, demonstration selection, format, and order.

Demonstration Selection. The performance of ICL tends to have a large variance with different demonstration examples [420], so it is important to select a subset of examples that can effectively leverage the ICL capability of LLMs. There are two main demonstration selection approaches, namely heuristic and LLM-based approaches:

- **Heuristic approaches.** Due to their simplicity and low costs, existing work widely adopts heuristic methods to select demonstrations. Several studies employ a k -NN based retriever to select examples that are semantically relevant to the query [420, 482]. However, they perform the selection individually for each example, rather than evaluating the example set as a whole. To resolve this issue, diversity-based selection strategies are proposed to choose the most representative set of examples for specific tasks [483, 484]. Furthermore, in [485], both relevance and diversity are taken into consideration when selecting demonstrations.

- **LLM-based approaches.** Another line of work selects demonstrations by making use of LLMs. For example, LLMs can be utilized to directly measure the informativeness of each example according to the performance gain after adding the example [486]. In addition, EPR [421] proposes a two-stage retrieval approach that first recalls similar examples with an unsupervised method (*e.g.*, BM25) and then ranks them using a dense retriever (trained with positive and negative examples labeled by LLMs). As an alternative approach, the task of demonstration selection can be formulated into a RL problem, where LLMs serve as the reward function to provide feedback for training the policy model [487]. Since LLMs perform well for text annotation [488], some recent studies employ LLM itself as the demonstration generator without human intervention [489].

To summarize, as discussed in [490], the selected demonstration examples in ICL should contain sufficient information about the task to solve as well as be relevant to the test query, for the above two selection approaches.

Demonstration Format. After selecting task examples, the next step is to integrate and format them into a natural language prompt for LLMs. A straightforward method is to instantiate a pre-defined template with the corresponding input-output pairs [36]. To construct more informative templates, recent studies consider adding task descriptions [69] or enhancing the reasoning capability of LLMs with chain-of-thought prompts [33]. For instance, in [179], the authors collect a large-scale dataset with task descriptions written by humans. After tuning with this dataset, the performance on seen tasks can be boosted, and LLMs can also generalize to unseen tasks to some extent. To reduce the annotation costs, a semi-automated approach has been proposed in [147] by employing a seed set consisting of human-written task descriptions to guide LLMs to generate task descriptions for new tasks. Since it is costly to manually annotate demonstration formats for different tasks, some work also studies how to automatically generate high-quality ones. As two representative methods, Auto-CoT [427] leverages LLMs with the zero-shot prompt “*Let’s think step by step*” for generating intermediate reasoning steps, while least-to-most prompting [432] first queries LLMs to perform problem decomposition and then utilizes LLMs to sequentially solve sub-problems based on the intermediate answers to previously solved ones.

Demonstration Order. LLMs are shown to sometimes suffer from the recency bias, *i.e.*, they are prone to repeat answers that are near the end of demonstrations [481]. Thus, it is important to arrange demonstrations (*i.e.*, task examples) in a reasonable order. Early work proposes several heuristic methods to quickly find a good order. For example, demonstrations can be directly organized according to their similarity to the query in the embedding space [420]: the more similar, the closer to the end. In addition, global and local entropy metrics can be used

to score different demonstration orders [425]. To integrate more task information, some recent studies propose to minimize the code length required to compress and transmit task labels, which is inspired by information theory [491]. However, these methods need additional labeled data as the validation set to evaluate the performance of specific demonstration orders. To eliminate this need, the authors in [425] propose to sample the validation data from the LLM itself.

6.2.3 Underlying Mechanism

After pre-training, LLMs can exhibit intriguing ICL capability without being updated. In what follows, we discuss two key questions about the ICL ability of LLMs, *i.e.*, “*how does pre-training affect the ICL ability*” and “*how do LLMs perform ICL during inference*”.

How Pre-Training Affects ICL? ICL is first proposed in GPT-3 [55], and it has been shown that the ICL ability becomes more significant with a larger model size. Further, some studies reveal that small-scale PLMs can also demonstrate a strong ICL ability by continual pre-training [492] or fine-tuning [493] on specially designed training tasks, which typically involve additional task examples in the input during the training process. It suggests that the design of training tasks is an important influence factor on the ICL capability of LLMs. Besides training tasks, recent studies have also investigated the relationship between ICL and pre-training corpora [490, 494]. For example, ICL can be theoretically explained as the product of pre-training on documents that exhibit long-range coherence [490]. Further, another study [494] theoretically analyzes that when scaling parameters and data, LLMs based on next-word prediction can emerge the ability of ICL by learning from the compositional structure (*e.g.*, how words and phrases are combined to form larger linguistic units like sentences) present in language data.

How LLMs Perform ICL? At the inference stage, researchers focus on analyzing how the ICL capability operates based on given demonstrations since no explicit learning or updating is involved. According to the discussion in [495], there are two main ways for LLMs to utilize demonstrations: task recognition and task learning.

- **Task recognition.** In the first way, LLMs recognize the task from demonstrations and utilize the prior knowledge obtained from pre-training to solve new test tasks. A Probably Approximately Correct (PAC) framework [496] has been proposed to assess the learnability of ICL. It assumes that there exists a latent variable representing the task in the pre-training data, and LLMs have been shown to be capable of capturing this variable from demonstrations, enabling them to recognize the task in ICL. Also, the interpretation of ICL as task recognition is supported by several empirical studies [480, 497]. For example, it has been observed that replacing the inputs or labels of demonstrations with random ones sampled from the input or label space does not seriously hurt the performance of LLMs, indicating that LLMs mainly recognize the target task from demonstrations instead of learning from them [480, 495]. Similarly, LLMs can exhibit decent performance even if the prompt template is irrelevant or misleading [497].

- **Task learning.** In the second way, LLMs learn new tasks unseen in the pre-training stage only through demonstrations. Specially, task learning is analyzed mainly from the perspective of gradient descent and considered as implicit fine-tuning [65, 498].

Then, ICL can be explained as follows: by means of forward computation, LLMs generate meta-gradients with respect to demonstrations and implicitly perform gradient descent via the attention mechanism. Experiments also show that certain attention heads in LLMs are capable of performing task-agnostic atomic operations (*e.g.*, copying and prefix matching), which are closely related to the ICL ability [499]. Furthermore, some studies abstract ICL as an algorithm learning process [500]. For example, the authors in [500] find that LLMs essentially encode implicit models through their parameters during pre-training. With the examples provided in ICL, LLMs can implement learning algorithms such as gradient descent or directly compute the closed-form solution to update these models during forward computation. Under this explanation framework, it has been shown that LLMs can effectively learn simple linear functions and even some complex functions like decision trees with ICL [500].

As discussed in a recent study [495], LLMs exhibit the abilities of both task recognition and task learning in ICL, but the two abilities seem to be possessed with different model scales. As shown in the experiments [495], the ability of task recognition is easier to obtain, and even a small LM with only 350M parameters can exhibit this ability, while task learning can only emerge for LLMs with at least 66B parameters. Another study [501] also supports this finding with specially designed experiments. They set up the tasks with flipped and semantically unrelated labels in the experiment, which require task learning when performing ICL. The results suggest that small LMs tend to disregard the labels and mainly depend on their prior knowledge to accomplish the task, while LLMs have the ability to surpass their prior knowledge and acquire new knowledge from demonstrations, resulting in better outcomes. Furthermore, to improve the task learning ability, Meta-In-Context Learning [502] proposes to include multiple related tasks instead of just a single one in the prompt. In addition, Symbol Tuning [503] fine-tunes LLMs on demonstrations with semantically unrelated labels (*e.g.*, foo/bar instead of positive/negative for sentiment analysis), forcing LLMs to learn the task from demonstrations instead of relying on prior knowledge.

6.3 Chain-of-Thought Prompting

Chain-of-Thought (CoT) prompting [33, 504] is an improved prompting strategy to boost the performance of LLMs on complex reasoning tasks, such as arithmetic reasoning [505], commonsense reasoning [506], and symbolic reasoning [33]. Instead of simply constructing the prompts with input-output pairs like ICL, CoT prompting further incorporates intermediate reasoning steps, which serve as the bridge between inputs and outputs. Figure 14 presents an illustration of CoT. In the following part, we will first elaborate on the basic CoT prompting approach and its improved strategies, then discuss when and why CoT prompting works.

6.3.1 Basic CoT Prompting Approach

CoT prompting is first proposed as an extension of ICL [33], which augments each demonstration (*input, output*) as (*input, CoT, output*). A *CoT* is a series of intermediate reasoning steps for connecting the *input* and *output*. With these augmented demonstrations, LLMs can follow them to generate CoTs and the answer for a new input. However, unlike (*input, output*) pairs in ICL, CoTs are difficult to obtain and usually require

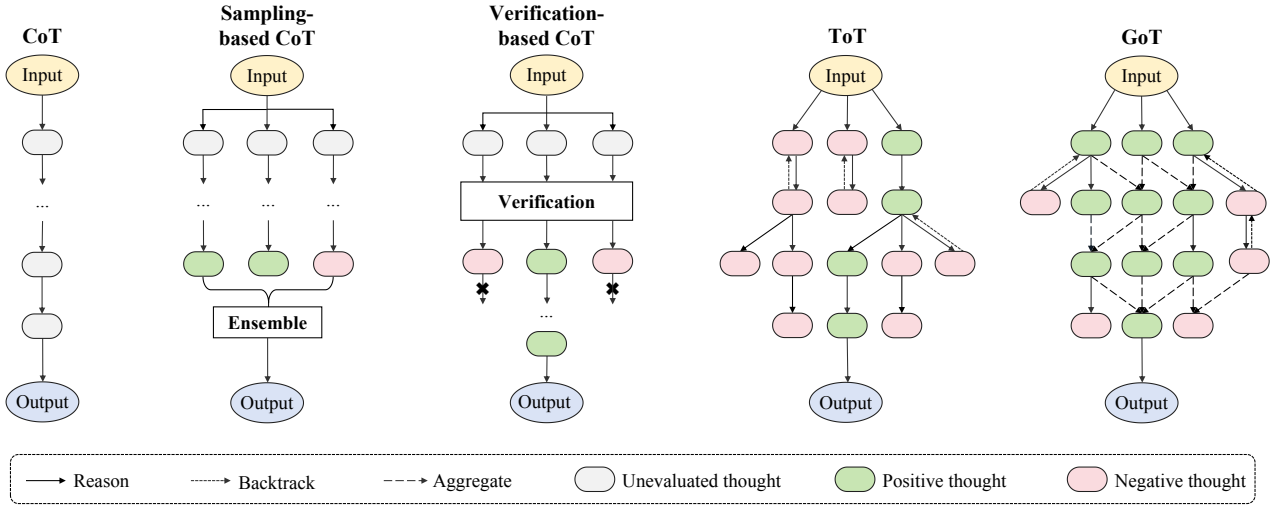


Fig. 15: An illustration of the evolution of CoT prompting strategies. It begins with the basic CoT approach and progresses to enhanced CoT generation techniques, including sampling-based and verification-based methods. Finally, it extends to variations of the chain structure, such as trees and graphs. Here, “thought” refers to an intermediate reasoning step as stated in [33, 444].

human annotation. Fortunately, it has been found that LLMs can be triggered to generate CoTs through simple instructions like “*Let’s think step by step.*” [507], making CoT prompting easy to use. There are also alternative magic prompts that can elicit the ability of CoT reasoning and further improve the performance of LLMs, such as “*Take a deep breath and work on this problem step-by-step.*” [470].

As illustrated in Figure 15, the generation process of CoT follows a chain structure in the basic CoT prompting approach, where LLMs generate CoTs step by step. Typically, CoT takes the format of natural language text. However, textual CoTs may not work well on complex tasks that require rigorous logic for reasoning. Considering this, some work uses code [508, 509] due to its structured and precise nature. Furthermore, the authors in [510] propose to dynamically select text or code as the format of CoTs to combine their advantages.

6.3.2 Improved CoT Prompting Strategies

Despite the performance improvement in complex reasoning tasks, CoT prompting still suffers from problems like incorrect reasoning and instability. In this part, we first introduce how to design better CoT prompts and enhanced CoT generation strategies, and then introduce the extension of the basic chain structure of CoT. Figure 15 illustrates the evolution of representative CoT prompting strategies.

Better Prompt Design. Since CoT prompting relies on prompts to elicit the reasoning capabilities of LLMs, the design of prompts is critical to its performance. As a direct approach, it is shown that using diverse CoTs (*i.e.*, multiple reasoning paths for each problem) can effectively enhance the performance [430]. Another intuitive idea is that prompts with more complex reasoning paths are more likely to elicit the reasoning ability of LLMs [426], which can result in higher accuracy in generating correct answers. However, all these approaches rely on annotated CoT datasets, which limits their use in practice. To overcome this limitation, magic instructions such as “*Let’s think step by step*” can be used to automatically construct CoTs by prompting LLMs [427].

Enhanced CoT Generation. Since LLMs are prone to producing incorrect reasoning steps and exhibiting instability in the generation process, there are a number of studies [429, 511] to improve the generation of CoT. In this part, we will introduce two typical approaches to enhancing the generation of CoT: sampling- and verification-based methods.

- **Sampling-based methods.** LLMs are known to suffer from instability during inference, which can lead to unfaithfulness in the generated reasoning steps. To address this issue, some work proposes to sample multiple reasoning paths instead of using greedy decoding. As a representative solution, self-consistency [429] first generates several reasoning paths and then takes an ensemble over the corresponding answers, selecting the most consistent one through majority voting. However, such a method can still lead to wrong answers when most of the reasoning paths are misled. Considering this, the authors in [426] only vote on the k most complex reasoning paths based on their observation that reasoning paths with higher complexity (*e.g.*, more reasoning steps) usually have better performance. Furthermore, MCR [512] proposes referring to the steps from other reasoning paths when generating the next step, and performs reasoning across multiple reasoning paths to generate the final answer.

- **Verification-based methods.** The sequential nature of reasoning steps in CoTs can lead to the accumulation of errors in the generated CoTs when certain steps are incorrect. To mitigate this problem, recent studies propose to verify the correctness of generated reasoning steps with either trained verifiers or LLMs themselves. For example, DIVERSE [511] trains solution-level and step-level verifiers respectively to examine the reasoning steps at different granularities. Another approach [513] utilizes LLMs to verify the correctness of reasoning steps through step-by-step self-verification with a specially designed reasoning format. In addition, several studies propose backward reasoning for verification: it first deduces the necessary question conditions [514, 515] or variables [516] from the model’s predictions, and then compares them with the original ones.

Reasoning Structure Extension. Despite the generality, the chain reasoning structure of basic CoT prompting limits its effectiveness in solving complex tasks, which require exploration like foresight and backtracking during inference. Therefore, many studies have been devoted to extending the reasoning structure by designing more intricate thought processes, e.g., tree- and graph-structured reasoning.

- *Tree-structured reasoning.* This approach (exemplified by Tree of Thoughts (ToT) [444, 517]) formulates the reasoning process in a hierarchical tree structure, where intermediate thoughts are nodes. In this way, it enables LLMs to explore multiple reasoning paths in parallel and further supports the operation of lookahead and backtracking to facilitate more comprehensive decisions. In addition, TouT [518] takes the uncertainty of intermediate thoughts into account for thought evaluation based on Monte Carlo Dropout.

- *Graph-structured reasoning.* Although the tree structure facilitates parallel reasoning, it also imposes restrictions on the reasoning process. With more complex topological structures, graphs offer greater flexibility in reasoning, enabling the characterization of more intricate relationships and interactions. For instance, Graph of Thoughts (GoT) [519, 520] conceptualizes the reasoning process as an arbitrary graph, where vertices denote intermediate thoughts and edges denote the interdependence between these thoughts. Compared with ToT, it can further utilize thoughts from other reasoning paths when generating new thoughts. However, such an approach requires a large number of interactions with LLMs, making the thought exploration process highly inefficient. To reduce potentially meaningless thought exploration, XoT [521] further proposes to guide the search of thoughts with pre-trained policy and value networks.

6.3.3 Further Discussion on CoT Prompting

In this part, we present discussions regarding two fundamental questions related to CoT prompting, *i.e.*, “when does CoT prompting work for LLMs” and “why can LLMs perform CoT reasoning”.

When CoT Prompting Works For LLMs? Since CoT reasoning is an emergent ability [31], it only has a positive effect on sufficiently large models (typically containing 10B or more parameters [33]) but not on small models. Moreover, since CoT prompting augments the standard prompting with intermediate reasoning steps, it is mainly effective for the tasks that require step-by-step reasoning [33], e.g., arithmetic reasoning, common-sense reasoning, and symbolic reasoning. Whereas, for other tasks that do not rely on complex reasoning, CoT prompting might lead to worse performance than standard prompting [431], e.g., MNLI-m/mm, SST-2, and QQP from GLUE [279]. Interestingly, it seems that the performance gain brought by CoT prompting could be significant only when standard prompting yields poor results [33].

Why LLMs Can Perform CoT Reasoning? As the second question, we discuss the underlying mechanism of CoT prompting in the following two aspects.

- *The source of CoT reasoning ability.* Regarding the source of CoT reasoning capability, it is widely hypothesized that it can be attributed to training on code since models trained on it show a strong reasoning ability [47, 522, 523]. Intuitively, code data is well organized with algorithmic logic and programming flow, which may be useful to improve the reasoning performance

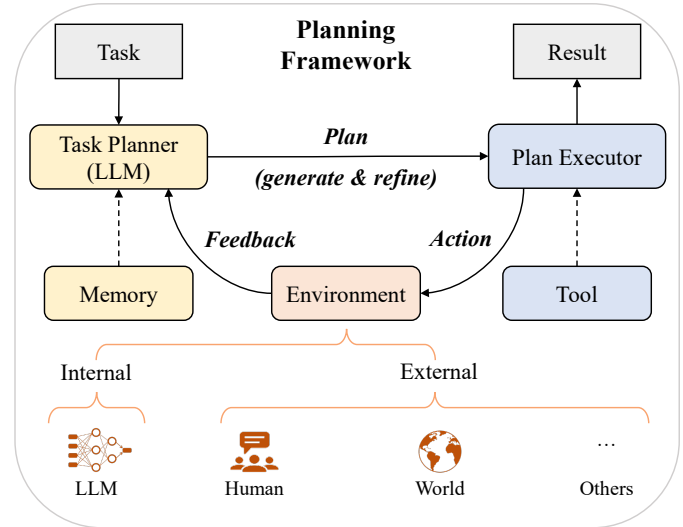


Fig. 16: An illustration of the formulation for prompt based planning by LLMs for solving complex tasks.

of LLMs. However, this hypothesis still lacks publicly reported evidence of ablation experiments (*with* and *without* training on code). In addition, instruction tuning seems not to be the key reason for obtaining the CoT reasoning ability, since it has been empirically shown that instruction tuning on non-CoT data does not improve the performance on held-out CoT reasoning benchmarks [69].

- *The effect of CoT prompting components.* The major distinction between CoT prompting and standard prompting is the incorporation of reasoning paths prior to the final answer. Thus, some researchers investigate the effects of different components in the reasoning paths. Specifically, a recent study identifies three key components in CoT prompting, namely *symbols* (e.g., numerical quantities in arithmetic reasoning), *patterns* (e.g., equations in arithmetic reasoning), and *text* (*i.e.*, the rest of tokens that are not symbols or patterns) [524]. It is shown that the latter two parts (*i.e.*, patterns and text) are essential to the model performance, and removing either one would lead to a significant performance drop. However, the correctness of symbols and patterns does not seem critical. Further, there exists a symbiotic relationship between text and patterns: the text helps LLMs to generate useful patterns, and patterns aid LLMs to understand tasks and generate texts that help solve them [524].

In summary, CoT prompting provides a general and flexible approach to eliciting the reasoning ability of LLMs. There are also some preliminary attempts to extend this technique to solve multimodal [525] and multilingual tasks [526].

6.4 Planning

Prompting with ICL and CoT is a conceptually simple yet general approach to solving various tasks. However, this approach struggles with complex tasks like mathematical reasoning [527] and multi-hop question answering [528]. As an enhanced approach, prompt-based planning has been proposed to break down complex tasks into smaller sub-tasks and generate a plan of actions to accomplish the task.

6.4.1 The Overall Framework

In this part, we first formulate the general planning paradigm of LLMs for solving complex tasks, which is illustrated in Figure 16.

In this paradigm, there are typically three components: *task planner*, *plan executor*, and *environment*³⁶. Specifically, task planner, which is played by LLMs, aims to generate the whole plan to solve a target task. The plan can be presented in various forms, e.g., an action sequence in the form of natural language [432] or an executable program written in programming language [436]. The LLM-based task planner can be enhanced with the memory mechanism for plan storage and retrieval, which is helpful for long-horizon tasks. Then, plan executor is responsible for executing the actions in the plan. It can be implemented by models like LLMs for textual tasks [434] or by tools like code interpreters for coding tasks [443]. Furthermore, environment refers to where the plan executor carries out the actions, which can be set differently according to specific tasks, e.g., the LLM itself [529] or an external virtual world like Minecraft [530]. It provides *feedback* about the execution result of the action to the task planner, either in the form of natural language [443] or from other multimodal signals [439].

For solving a complex task, the task planner first needs to clearly understand the task goal and generate a reasonable plan based on the reasoning of LLMs (See Section 6.4.2). Then, the plan executor acts according to the plan in the environment, and the environment will produce feedback for the task planner (See Section 6.4.3). The task planner can further incorporate the feedback obtained from the environment to refine its initial plan and iteratively perform the above process to get better results as the task solution (See Section 6.4.4).

6.4.2 Plan Generation

Plan generation focuses on directly generating action sequences by prompting LLMs. Based on the format of the generated plans, existing work can be divided into two groups: text-based and code-based approaches.

Text-based Approaches. It is straightforward for LLMs to generate plans in the form of natural language. In this approach, LLMs are prompted to generate a sequence of actions for the plan executor to perform and solve the complex task. For example, Plan-and-Solve [434] adds explicit instructions like “devise a plan” to directly prompt the LLM for planning in a zero-shot manner, while Self-planning [531] and DECOMP [433] add demonstrations in the prompt to guide the LLM to devise a plan through ICL. Following this way, some work further considers incorporating extra tools or models when planning. For example, ToolFormer [80] first annotates a pre-training corpus with potential API calls using LLMs, and then fine-tunes LLMs on it, so that LLMs can learn when and how to call APIs and incorporate the results returned by APIs during generation. HuggingGPT [437] introduces the models available in HuggingFace and regards LLMs as the controller to select suitable models based on their descriptions and aggregate their results as the final solution.

Code-based Approaches. Although text-based approaches sound intuitive, they cannot guarantee faithful execution of the

plan, which may lead to failure even when the plan is sound. To address this issue, code-based approaches have been proposed to generate more verifiable plans in the form of executable code in programming languages, e.g., Python or PDDL. In this way, LLMs are first prompted to generate the program and then utilize a deterministic solver to execute it. For example, Faithful CoT [435] and PAL [436] decompose a reasoning task into two stages: at the first stage, the LLM generates a plan conditioned on the query; at the second stage, a deterministic solver executes the plan to derive the final answer. Furthermore, code-based approaches can be applied to embodied agents in a similar way. For example, PROG PROMPT [532] and LLM+P [533] first utilize LLMs to generate plans in the form of python functions or PDDL files, and then leverage a virtual agent or classical planner to solve the problem according to the code-based plans.

6.4.3 Feedback Acquisition

After executing the generated plan, the environment would produce the feedback signal to the LLM-based task planner, which can be used to refine its initial plan for better results. In existing work, there are typically two sources of feedback from the environment, depending on their relationship with the LLM-based task planner: internal (*i.e.*, the LLM itself) and external (*e.g.*, tools or virtual worlds) feedback.

Internal Feedback. The LLM itself can be utilized as a feedback provider. One straightforward way is to directly evaluate the quality of the generated plans through prompting. For example, RAP [440] evaluate the likelihood that each candidate plan can lead to task success, while Tree of Thoughts [529] proposes to vote across plans by making comparisons between them. Further, LLMs can provide feedback based on the intermediate results from the plan executor. For example, Reflexion [443] utilizes LLMs to transform sparse result signals (*e.g.*, success or failure) into concrete text-based feedback (*e.g.*, “*You should recommend comedies that the user mentions in the query instead of horror movies*”) and stores this feedback in long-term memory for future planning.

External Feedback. In addition to LLMs, external objects can also provide feedback signals. For example, tools like code interpreters are widely used in programming tasks to provide real-time error messages [443], models like stable diffusion [534] can be used in multimodal tasks to provide visual perception [439], and virtual worlds like Minecraft can provide immersive experiences [530]. Besides, some work (*e.g.*, Generative Agents [535]) explores multi-agent collaboration in simulated environments, where each agent receives feedback not only from interaction with the environment but also from communication with other agents.

6.4.4 Plan Refinement

With access to feedback from the environment, the task planner can accordingly refine its current plan and iteratively go through the “*planning – execution – refinement*” loop for better results. In this part, we summarize three major refinement approaches in existing work.

Reasoning. The feedback data from the environment may not be directly suitable to be utilized by LLMs for plan refinement, *e.g.*, containing irrelevant information or taking a non-language form. To solve this, some work adds the explicit reasoning process

36. Despite the similarity with RL, our formulation decouples the planning and execution phases, whereas in RL, they are typically interleaved in the agent. This paradigm is defined in a general yet slightly loose way, and it mainly aims to help readers understand the key idea underlying the planning approaches of LLMs.

to extract critical information from feedback [441, 442]. For example, React [442] prompts LLMs with demonstrations to generate reasoning traces over feedback. It has been widely used in autonomous agent projects, such as AutoGPT [536], which can automatically reason over the observed feedback to revise the initial plan for solving various user requests. However, these approaches typically fix the order of reasoning and planning. To support flexible switching between the two processes for better performance, ChatCoT [441] further unifies the tool-augmented reasoning process into a multi-turn conversation between the LLM-based task planner and the tool-based environment.

Backtracking. Early methods mainly consider planning forward actions while maintaining the existing plan, thus likely leading to local optimal plans based on a short-term evaluation. To solve this, Tree of Thoughts [529] allows backtracking with search algorithms like breadth-first and depth-first search to make global planning. It refines the plan step by step by backtracking to the last state in the initial plan and choosing the next unexplored action. Furthermore, some studies [439, 537] utilize feedback signals to revise the entire plan. For example, DEPS [537] selects a better plan according to feedback signals, while TIP [439] adds feedback signals to prompts for the LLM-based planner to revise each step in the initial plan.

Memorization. In order to handle long-horizon tasks, it has become a key approach to aid plan refinement with *long-term memory* in addition to utilizing the *short-term memory* of LLMs through ICL. For example, Reflexion [443] stores the feedback from self-reflection into the memory, so previous feedback can be retrieved for plan refinement. Generative Agents [535] designs the memory stream mechanism as the core component of agents for action planning and reflection. Further, the skill library mechanism [438, 530] is proposed to store successful plans in the library, which can be reused and synthesized as complex plans for novel tasks. To implement the long-term memory mechanism, tools like vector databases (e.g., milvus [538]) can be used to encode plans or feedbacks into high-dimensional vectors for efficient storage and retrieval at a large scale. MemoryBank [539] further proposes the memory updating mechanism to allow memory forgetting and strengthening following the Ebbinghaus Forgetting Curve theory.

7 Capacity and Evaluation

To examine the effectiveness and superiority of LLMs, a surge of tasks and benchmarks have been proposed for conducting empirical ability evaluation and analysis. In this section, we first introduce three types of basic ability evaluation of LLMs for language generation and understanding, then present several advanced ability evaluations with more complicated settings or goals, and finally discuss existing benchmarks, evaluation approaches, and empirical analysis.

7.1 Basic Ability

In this part, we mainly focus on three basic types of ability evaluation for LLMs, *i.e.*, language generation, knowledge utilization, and complex reasoning. It is noted that we do not intend to have complete coverage of all the related tasks, but instead only focus on the most widely discussed or studied tasks for LLMs. Next, we introduce these tasks in detail.

7.1.1 Language Generation

According to the task definition, existing tasks about language generation can be roughly categorized into language modeling, conditional text generation, and code synthesis tasks. Note that code synthesis is not a typical NLP task, we include it for discussion because it can be directly solved by a number of LLMs (trained on code data) in a similar generation approach as natural language text.

Language Modeling. As the most fundamental ability of LLMs, *language modeling* aims to predict the next token based on the previous tokens [1], which mainly focuses on the capacity of basic language understanding and generation. For evaluating such an ability, typical language modeling datasets that existing work uses include Penn Treebank [540], WikiText-103 [541], and the Pile [166], where the metric of *perplexity* is commonly used for evaluating the model performance under the zero-shot setting. Empirical studies [55, 93] show that LLMs bring substantial performance gains over the previous state-of-the-art methods on these evaluation datasets. To better test the modeling capacity of long-range dependencies in text, the LAMBADA dataset [252] has been introduced, where LLMs are required to predict the last word of sentences based on a paragraph of context. Then, the accuracy and perplexity of the predicted last words are employed to evaluate LLMs. As shown in existing work, the performance on the language modeling tasks typically follows the scaling law [30], which means that scaling language models would improve the accuracy and reduce the perplexity.

Conditional Text Generation. As an important topic in language generation, conditional text generation [48] focuses on generating texts satisfying specific task demands based on the given conditions, typically including machine translation [626], text summarization [550], and question answering [559]. To measure the quality of the generated text, automatic metrics (e.g., Accuracy, BLEU [627] and ROUGE [628]) and human ratings have been typically used for evaluating the performance. Due to the powerful language generation capabilities, LLMs have achieved remarkable performance on existing datasets and benchmarks. For instance, GPT-4 exhibits comparable performance as commercial translation products, even for the translation task of languages that are with significant linguistic distance [629]. On news summarization tasks (*i.e.*, CNN/DM and XSUM), LLMs also demonstrate comparable performance with human freelance writers [630]. Despite the rapid progress on model capacity, there are increasing concerns on the feasibility of existing automatic metrics to faithfully assess the performance of LLMs in conditional text generation tasks [630–632]. As the alternatives to automatic metrics, recent studies also propose to incorporate LLMs as generation evaluators to examine the quality of the generated content [152, 633, 634]. Moreover, researchers also explore more challenging language generation tasks for LLMs, such as structured data generation [451] and long text generation [46, 635, 636].

Code Synthesis. In addition to generating high-quality natural language text, existing LLMs also show strong abilities to generate formal language, especially computer programs (*i.e.*, code) that satisfy specific conditions, called *code synthesis* [637]. Unlike natural language generation, as the generated code can be directly checked by execution with corresponding compilers or interpreters, existing work mostly evaluates the quality of

TABLE 14: Representative basic and advanced abilities and corresponding representative datasets for evaluating.

Level	Ability	Task	Dataset
Basic	Language Generation	Language Modeling	Penn Treebank [540], WikiText-103 [541], the Pile [166], LAMBADA [252]
		Conditional Text Generation	WMT’14,16,19,20,21,22 [542–547], Flores-101 [548], DiaBLa [549], CNN/DailyMail [550], XSum [551], WikiLingua [552] OpenDialKG [553]
			Code Synthesis
	Knowledge Utilization	Closed-Book QA	Natural Questions [556], ARC [557], TruthfulQA [558], Web Questions [559], TriviaQA [560], PIQA [561], LC-quad2.0 [562], GrailQA [563], KQApro [564], CWQ [565], MKQA [566], ScienceQA [567]
		Open-Book QA	Natural Questions [556], OpenBookQA [568], ARC [557], TriviaQA [560], Web Questions [559], MS MARCO [569], QASC [570], SQuAD [571], WikiMovies [572]
		Knowledge Completion	WikiFact [573], FB15k-237 [574], Freebase [575], WN18RR [576], WordNet [577], LAMA [578], YAGO3-10 [579], YAGO [580]
	Complex Reasoning	Knowledge Reasoning	CSQA [506], StrategyQA [199], HotpotQA [581], ARC [557], BoolQ [582], PIQA [561], SIQA [583], HellaSwag [584], WinoGrande [585], COPA [586], OpenBookQA [568], ScienceQA [567], proScript [587], ProPara [588], ExplaGraphs [589], ProofWriter [590], EntailmentBank [591], ProOntoQA [592]
		Symbolic Reasoning	CoinFlip [33], ReverseList [33], LastLetter [33], Boolean Assignment [593], Parity [593], Colored Object [70], Penguins in a Table [70], Repeat Copy [436], Object Counting [436]
		Mathematical Reasoning	MATH [362], GSM8k [198], SVAMP [594], MultiArith [595], ASDiv [505], MathQA [596], AQUA-RAT [597], MAWPS [598], DROP [599], NaturalProofs [600], PISA [601], miniF2F [602], ProofNet [603]
	Advanced	Human Alignment	Honestness
Helpfulness			HH-RLHF [183]
Harmlessness			HH-RLHF [183], Crows-Pairs [605] WinoGender [606], RealToxicityPrompts [607]
Interaction with External Environment		Household	VirtualHome [608], BEHAVIOR [609], ALFRED [610],ALFWorld [611]
		Website Environment	WebShop [612], Mind2Web [613]
		Open World	MineRL [614], MineDojo [615]
Tool Manipulation		Search Engine	HotpotQA [581], TriviaQA [560], Natural Questions [556]
		Code Executor	GSM8k [198], TabMWP [616], Date Understanding [70]
		Calculator	GSM8k [198], MATH [362], CARP [617]
		Model Interface	GPT4Tools [618], Gorilla [619]
	Data Interface	WebQSP [620], MetaQA [621], WTQ [622] WikiSQL [623], TabFact [624], Spider [625]	

the generated code from LLMs by calculating the pass rate against the test cases, *i.e.*, $\text{pass}@k$ ³⁷. Recently, several code benchmarks focusing on functional correctness are proposed to assess the code synthesis abilities of LLMs, such as APPS [376], HumanEval [105], and MBPP [223]. Typically, they consist of diverse programming problems, with text specification and test cases for correctness checking. To improve such an ability, it is key to fine-tuning (or pre-training) LLMs on code data, which can effectively adapt LLMs to code synthesis tasks [86]. In addition, existing work has proposed new strategies to generate code, *e.g.*, sampling multiple candidate solutions [223] and planning-guided decoding [638], which can be considered as the imitation of bug-fixing and code-planning processes by programmers. Impressively, LLMs have recently shown competitive performance with humans by achieving a ranking of the top 28% among users on the

programming contest platform Codeforces [114]. Further, GitHub Copilot has been released to assist programming in coding IDEs (*e.g.*, Visual Studio and JetBrains IDEs), which can support a variety of languages including Python, JavaScript, and Java. A viewpoint article entitled “*The End of Programming*” [639] in Communications of the ACM has discussed the impact of AI programming in the field of computer science, emphasizing an important shift towards the highly adaptive LLM as a new atomic unit of computation.

Major Issues. Although LLMs have achieved splendid performance in generating human-like text, they are susceptible to suffering from two major issues in language generation as discussed below.

- *Unreliable generation evaluation.* With the advancement of language generation ability of LLMs, existing studies find that the generated texts from LLMs have reached a comparable quality to the reference texts on a variety of text generation tasks. However,

37. Given k programs generated by the LLM, $\text{pass}@k$ is computed as 1 when at least one program passes all test cases, or else 0

due to the intrinsic weakness of existing evaluation benchmarks, there exists pronounced inconsistency between human evaluation and automatic reference-based metrics [630–632, 640]. For example, in OpenDialKG [553], ChatGPT underperforms a fine-tuned GPT-2 on BLEU and ROUGE-L metrics, while earning more favor from human judgment [640]. Furthermore, existing work argues that even human evaluation may not be robust enough [630, 631, 641, 642]. In some cases, it is difficult to achieve a high level of consensus among human annotators [631], and there is also a large gap between the annotation quality of crowdworkers and experts [641, 642]. Thus, how to conduct reliable evaluation for language generation tasks in the era of LLMs has become a fundamental yet challenging research topic. Recently, increasing research work proposes to leverage LLMs to improve the evaluation quality of the generated texts. Specially, LLMs can be used to improve the evaluation quality of existing metrics. For example, Para-Ref [643] augments various automatic metrics by leveraging LLMs to paraphrase existing references into semantically equivalent references with diverse expressions. Further, LLMs are widely employed as the evaluators of text generation in a reference-free manner, including evaluating a single prediction [633, 634, 644] or comparing several candidates [152, 645–647]. Nevertheless, LLMs may expose bias (e.g., order bias or preference for LLM-generated texts over human-written texts) as language generation evaluators, demonstrating disparities when compared to human evaluation [634, 648, 649].

Unreliable Generation Evaluation

LLMs have been capable of generating texts with a comparable quality to human-written texts, which however might be underestimated by automatic reference-based metrics. As an alternative evaluation approach, LLMs can serve as language generation evaluators to evaluate a single text, compare multiple candidates, and improve existing metrics. However, this evaluation approach still needs more inspections and examinations in real-world tasks.

- **Underperforming specialized generation.** Although LLMs have learned general language patterns to generate coherent text, their proficiency in generation might be constrained when dealing with a specialized domain or task. For instance, a language model that has been trained on general web articles may face challenges when generating a medical report which involves many medical jargon and methods. Intuitively, domain knowledge should be critical for model specialization. However, it is not easy to inject such specialized knowledge into LLMs. As discussed in recent analyses [47, 650], when LLMs are trained to exhibit some specific ability that allows them to excel in some areas, they might struggle in others. Such an issue is related to *catastrophic forgetting* [651, 652] in training neural networks, which refers to the conflict phenomenon of integrating new and old knowledge. Similar cases also occur in human alignment of LLMs, where “*alignment tax*” [66] (e.g., a potential loss in the in-context learning ability) has to be paid for aligning to human values and needs. Moreover, due to the limitations of sequence modeling architecture, LLMs still face challenges in the understanding and generation of structured data. Consequently, they often fall behind task-specific models on complex structured data tasks, such as knowledge-base question answering and semantic

parsing [451, 653]. Therefore, it is important to develop effective model specialization methods that can flexibly adapt LLMs to various task scenarios, meanwhile retaining the original abilities as possible.

Underperforming Specialized Generation

LLMs may fall short in mastering generation tasks that require domain-specific knowledge or generating structured data. It is non-trivial to inject specialized knowledge into LLMs, meanwhile maintaining the original abilities of LLMs.

7.1.2 Knowledge Utilization

Knowledge utilization is an important ability of intelligent systems to accomplish knowledge-intensive tasks (e.g., common-sense question answering and fact completion) based on supporting factual evidence. Concretely, it requires LLMs to properly utilize the rich factual knowledge from the pre-training corpus or retrieve external data when necessary. In particular, question answering (QA) and knowledge completion have been two commonly used tasks for evaluating this ability. According to the test tasks (question answering or knowledge completion) and evaluation settings (*with* or *without* external resources), we categorize existing knowledge utilization tasks into three types, namely closed-book QA, open-book QA³⁸, and knowledge completion.

Closed-Book QA. Closed-book QA tasks [654] test the acquired factual knowledge of LLMs from the pre-training corpus, where LLMs should answer the question only based on the given context without using external resources. For evaluating this ability, there are several datasets that can be leveraged, including Natural Questions [556], Web Questions [559], and TriviaQA [560], where the accuracy metric is widely adopted. Empirical results have revealed that LLMs can perform well in this setting and even match the performance of state-of-the-art open-domain QA systems [56]. Also, the performance of LLMs on closed-book QA tasks shows a scaling law pattern in terms of both model size and data size: scaling the parameters and training tokens can increase the capacity of LLMs and help them learn (or memorize) more knowledge from the pre-training data [56]. Further, under a similar parameter scale, LLMs with more pre-training data relevant to the evaluated tasks would achieve better performance [81]. Also, the closed-book QA setting provides a testbed for probing the accuracy of the factual knowledge encoded by LLMs. However, as shown in existing work [55], LLMs might perform less well on QA tasks relying on fine-grained knowledge, even when it exists in the pre-training data.

Open-Book QA. Unlike closed-book QA, in open-book QA tasks, LLMs can extract useful evidence from the external knowledge base or document collections, and then answer the question based on the extracted evidence [655–658]. Typical open-book QA datasets (e.g., Natural Questions [556], OpenBookQA [568],

38. In this part, open-book QA refers to the QA tasks that require to extract and utilize useful information from external knowledge resources, as the antithesis of closed-book QA (only using the encoded information from pre-training corpus). Note that there is a dataset also named OpenBookQA [568], which follows the settings of open-book QA tasks by extracting and utilizing external science facts.

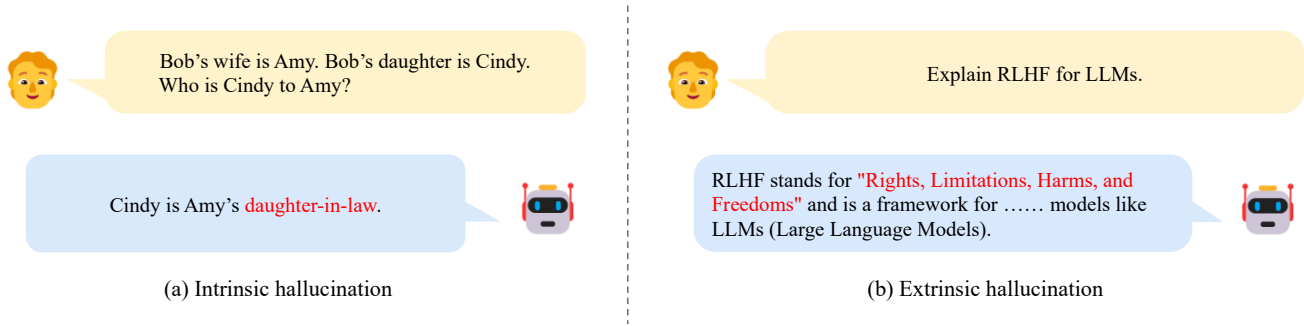


Fig. 17: Examples of intrinsic and extrinsic hallucination for a public LLM (access date: March 19, 2023). As an example of intrinsic hallucination, the LLM gives a conflicting judgment about the relationship between Cindy and Amy, which contradicts the input. For extrinsic hallucination, in this example, the LLM seems to have an incorrect understanding of the meaning of RLHF (reinforcement learning from human feedback), though it can correctly understand the meaning of LLMs (in this context).

and SQuAD [571]) have overlap with closed-book QA datasets, but they incorporate external data sources, *e.g.*, Wikipedia. The metrics of accuracy and F1 score are widely used in open-book QA tasks for evaluation. To select relevant knowledge from external resources, LLMs are often paired with a text retriever (or even a search engine), which is trained independently or jointly with LLMs [81, 655, 659]. Also, previous work [660–662] has indicated that retrievers can assist LLMs in verifying and rectifying the reasoning path. In evaluation, existing studies mainly focus on testing how LLMs utilize the extracted knowledge to answer the question and show that the retrieved evidence can largely improve the accuracy of the generated answers, even enabling a smaller LLM to outperform $10\times$ larger ones [655, 659]. Further, open-book QA tasks can be also employed to evaluate the recency of knowledge information. Pre-training or retrieving from outdated knowledge resources may cause LLMs to generate incorrect answers for time-sensitive questions [655].

Knowledge Completion. In knowledge completion tasks, LLMs might be (to some extent) considered as a knowledge base [578], which can be leveraged to complete or predict the missing parts of knowledge units (*e.g.*, knowledge triples). Such tasks can probe and evaluate *how much* and *what kind of* knowledge LLMs have learned from the pre-training data. Existing knowledge completion tasks can be roughly divided into knowledge graph completion tasks (*e.g.*, FB15k-237 [574] and WN18RR [576]) and fact completion tasks (*e.g.*, WikiFact [573]), which aim to complete the triples from a knowledge graph and incomplete sentences about specific facts, respectively. Empirical studies have revealed that it is difficult for existing LLMs to accomplish knowledge completion tasks related to specific relation types [522]. As shown in the evaluation results on WikiFact, LLMs perform well on several frequent relations that occur in the pre-training data (*e.g.*, currency and author), while not well on rare ones (*e.g.*, discoverer_or_inventor and place_of_birth). Interestingly, under the same evaluation settings (*e.g.*, in-context learning), InstructGPT (*i.e.*, text-davinci-002) outperforms GPT-3 in all subsets of WikiFact.

Major Issues. Although LLMs have achieved key progress in capturing and utilizing knowledge information, they suffer from two major issues as discussed below.

- **Hallucination.** In generating factual texts, a challenging issue is *hallucination generations* [640, 663], where the generated information is either in conflict with the existing source (*intrinsic*

hallucination) or cannot be verified by the available source (*extrinsic hallucination*), which are illustrated by two examples in Figure 17. Hallucination widely occurs in existing LLMs, even the most superior LLMs such as GPT-4 [46]. Furthermore, existing work shows that LLMs encounter difficulties in recognizing the hallucinated content in text [604], even the powerful ChatGPT. Additionally, beyond language tasks, a recent study has shown that large vision-language models (LVLM) also face challenges with hallucination, *i.e.*, generating objects that are not present in the accompanying images [664]. In essence, LLMs seem to “unconsciously” utilize the knowledge in task solving, which still lack an ability to accurately control the use of internal or external knowledge. Hallucinations would mislead LLMs to generate undesired outputs and mostly degrade the performance, leading to potential risks when deploying LLMs in real-world applications. To alleviate this problem, alignment tuning strategies (as discussed in Section 5.2) have been widely utilized in existing work [66], which rely on tuning LLMs on high-quality data or using human feedback. Moreover, the integration of external tools for the provision of credible information sources can help alleviate the hallucination issue [81, 604, 661]. Another line of research work leverages uncertainty estimation of LLMs to identify hallucinations [665, 666]. For instance, considering that hallucinated facts are prone to exhibit inconsistency across different sampled outputs, SelfCheckGPT [666] detects hallucination by measuring information inconsistency within sampled outputs. For the evaluation of the hallucination problem, a set of hallucination detection tasks have been proposed, *e.g.*, TruthfulQA [558] for detecting human falsehood mimicked by models. More recently, HaluEval [604] creates a large-scale LLM-generated and human-annotated hallucinated samples to evaluate the ability of language models to recognize hallucination in both task-specific and general scenarios.

Hallucination

LLMs are prone to generate untruthful information that either conflicts with the existing source or cannot be verified by the available source. Even the most powerful LLMs such as ChatGPT face great challenges in migrating the hallucinations of the generated texts. This issue can be partially alleviated by special approaches such as alignment tuning and tool utilization.

- **Knowledge recency.** As another major challenge, LLMs would encounter difficulties when solving tasks that require the latest knowledge beyond the training data. To tackle this issue, a straightforward approach is to regularly update LLMs with new data. However, it is very costly to fine-tune LLMs, and also likely to cause the catastrophic forgetting issue when incrementally training LLMs. Therefore, it is necessary to develop efficient and effective approaches that can integrate new knowledge into existing LLMs, making them up-to-date. Existing studies have explored how to utilize the external knowledge source (e.g., search engine) to complement LLMs, which can be either jointly optimized with LLMs [655] or used as a plug-and-play module [661]. For instance, ChatGPT utilizes a retrieval plugin to access up-to-date information sources [667]. By incorporating the extracted relevant information into the context [668–670], LLMs can acquire new factual knowledge and perform better on relevant tasks. However, such an approach seems to be still at a superficial level. In addition, existing studies also explore editing parameters of language models to update intrinsic knowledge [671–673]. Nevertheless, previous work [674] has shown that several parameter editing methods perform not well on LLMs, though they can improve the performance of small language models. Therefore, it is still difficult to directly amend intrinsic knowledge or inject specific knowledge into LLMs, which remains an open research problem [674]. Recently, a useful framework *EasyEdit* [675] has been released to facilitate the research of knowledge editing for LLMs.

Knowledge Recency

The parametric knowledge of LLMs is hard to be updated in a timely manner. Augmenting LLMs with external knowledge sources is a practical approach to tackling the issue. However, how to effectively update knowledge within LLMs remains an open research problem.

7.1.3 Complex Reasoning

Complex reasoning refers to the ability of understanding and utilizing supporting evidence or logic to derive conclusions or make decisions [51, 52]. According to the type of involved logic and evidence in the reasoning process, we consider dividing existing evaluation tasks into three major categories, namely knowledge reasoning, symbolic reasoning, and mathematical reasoning.

Knowledge Reasoning. The knowledge reasoning tasks rely on logical relations and evidence about factual knowledge to answer the given question. Existing work mainly uses specific datasets to evaluate the reasoning capacity of the corresponding type of knowledge, e.g., CSQA [506]/StrategyQA [199] for com-

monsense knowledge reasoning and ScienceQA [567] for science knowledge reasoning. In addition to the accuracy of the predicted results, existing work [567] has also evaluated the quality of the generated reasoning process, via automatic metrics (e.g., BLEU) or human evaluation. Typically, these tasks require LLMs to perform step-by-step reasoning based on factual knowledge, until reaching the answer to the given question. To elicit the step-by-step reasoning ability, chain-of-thought (CoT) prompting strategy [33] has been proposed for enhancing the complex reasoning capacity of LLMs. As discussed in Section 6.3, CoT involves the intermediate reasoning steps, which can be manually created [33] or automatically generated [676], into the prompts to guide LLMs to perform multi-step reasoning. Such a way largely improves the reasoning performance of LLMs, leading to new state-of-the-art results on several complex knowledge reasoning tasks [33, 56, 528]. Further, after reformulating knowledge reasoning tasks into code generation tasks, researchers have found that the performance of LLMs can be further improved [226], especially with the LLMs pre-trained on code. However, due to the complexity of knowledge reasoning tasks, the performance of current LLMs still lags behind human results on tasks such as commonsense reasoning [33, 56, 677]. As a common type of mistakes, LLMs might generate inaccurate intermediate steps, leading to a wrong final result. To address this issue, existing work has proposed special decoding or ensemble strategies to improve the accuracy of the whole reasoning chain [429, 430].

Symbolic Reasoning³⁹. The symbolic reasoning tasks mainly focus on manipulating the symbols in a formal rule setting to fulfill some specific goal [51], where the operations and rules may have never been seen by LLMs during pre-training. Existing work [33, 432, 507] commonly evaluates LLMs on the task of last letter concatenation and coin flip, where the evaluation examples require the same reasoning steps as the in-context examples (called *in-domain test*) or more steps (called *out-of-domain test*). For an example of the out-of-domain test, LLMs could only see the examples with two words in context, but it requires LLMs to concatenate the last letters of three or more words. Typically, the accuracy of the generated symbols is adopted to evaluate the performance of LLMs on these tasks. Thus, LLMs need to understand the semantic relations among the symbolic operations and their composition in complex scenarios. However, under the out-of-domain setting, as LLMs have not seen the complex compositions of symbolic operations and rules (e.g., twice the number of operations in context examples), it is hard for LLMs to capture their accurate meanings. To solve this issue, existing studies incorporate scratchpad [593, 678] and tutor [679] strategies to help LLMs better manipulate symbolic operations, for generating longer and more complex reasoning processes. Another line of research work utilizes the formal programming language to represent the symbolic operations and rules, which requires LLMs to generate code and perform the reasoning process by executing it with external interpreters. Such a way can decompose the complex reasoning process into code synthesis and program execution for LLMs and interpreters, respectively, leading to a simplified reasoning process with yet more accurate results [436].

39. Following [33], we mainly discuss symbolic reasoning tasks specially designed for evaluating LLMs. We do not consider symbolic reasoning methods in traditional NLP tasks, such as deducing logical rules from the knowledge graphs in KBQA.

Mathematical Reasoning. The mathematical reasoning tasks need to comprehensively utilize mathematical knowledge, logic, and computation for solving problems or generating proof statements. Existing mathematical reasoning tasks can be mainly categorized into math problem solving and automated theorem proving. For math problem solving tasks, SVAMP [594], GSM8k [198] and MATH [362] datasets are commonly used for evaluation, where LLMs need to generate accurate concrete numbers or equations to answer the mathematical problem. As these tasks also require multi-step reasoning, the CoT prompting strategy has been widely adopted for LLMs to improve the reasoning performance [33]. As another practical strategy, continually pre-training LLMs on large-scale mathematical corpora can largely boost their performance on mathematical reasoning tasks [35, 218, 680]. Further, since math problems in different languages share the same mathematical logic, researchers also propose a multilingual math word problem benchmark [526] to evaluate the multilingual mathematical reasoning capacity of LLMs. As another challenging task, automated theorem proving (ATP) [600, 602, 681] requires the reasoning model to strictly follow the reasoning logic and mathematical skills. To evaluate the performance on this task, PISA [601] and miniF2F [602] are two typical ATP datasets with the *proof success rate* as the evaluation metric. As a typical approach, existing work on ATP utilizes LLMs to aid the search for proofs using an interactive theorem prover (ITP), such as Lean, Metamath, and Isabelle [682–684]. A major limitation of ATP research is the lack of related corpora in formal language. To tackle it, several studies utilize LLMs to convert informal statements into formal proofs for augmenting new data [685] or generate drafts and proof sketches to reduce the search space of the proofs [686].

Major Issues. In spite of the advancements, LLMs still have several limitations in solving complex reasoning tasks.

- *Reasoning inconsistency.* With improved reasoning strategies (e.g., CoT prompting), LLMs can solve some complex reasoning tasks, by performing step-by-step reasoning based on the supporting logic and evidence. Despite the effectiveness, the *reasoning inconsistency* issue often occurs in the decomposed reasoning process. Concretely, LLMs may generate the correct answer following an invalid reasoning path, or produce a wrong answer after a correct reasoning process [33, 435], leading to inconsistency between the derived answer and the reasoning process. To alleviate this problem, existing work has proposed to guide the whole generation process of LLMs via external tools or models [430, 444, 638], to re-check the reasoning process and final answer for correcting the potential errors [687–689] or fine-tune LLMs with process-based feedback [690, 691]. For instance, *Tree of Thoughts (ToT)* [444] empowers LLMs to engage in the decision-making process by concurrently exploring and self-evaluating various reasoning paths. To refine the reasoning processes, Self-Refine [687] elicits feedback from LLMs on self-generated solutions, enabling the iterative refinement of solutions based on the feedback. Moreover, several studies improve the consistency in the reasoning chain of LLMs through the integration of process-based supervision during training [690, 691]. As a promising solution, recent approaches reformulate the complex reasoning tasks into code generation tasks, where the strict execution of the generated code ensures the consistency between the reasoning process and the outcome. Also, it has been revealed that there might exist inconsistency between tasks with similar

inputs, where small changes in the task description may cause the model to produce different results [49, 594]. To mitigate this problem, self-consistency [429] adopts the ensemble of multiple reasoning paths to enhance the decoding process of LLMs.

Reasoning Inconsistency

LLMs may generate the correct answer following an invalid reasoning path, or produce a wrong answer after a correct reasoning process, leading to inconsistency between the derived answer and the reasoning process. The issue can be alleviated by fine-tuning LLMs with process-level feedback, using an ensemble of diverse reasoning paths, and refining the reasoning process with self-reflection or external feedback.

- *Numerical computation.* For complex reasoning tasks, LLMs still face difficulties in the involved numerical computation, especially for the symbols that are seldom encountered during pre-training, such as arithmetic with large numbers [49, 679, 692]. To tackle this issue, a direct way is to tune LLMs on synthesized arithmetic problems [359, 693]. Also, a surge of studies improve the numerical computation performance by tracing intermediate calculation steps in training and inference stages [359, 678, 694], e.g., scratchpad tracing. In addition, existing work [80] has also incorporated external tools (e.g., calculator), especially for handling arithmetic operations. More recently, ChatGPT has provided a plugin mechanism to use external tools [667]. In this way, LLMs need to learn how to properly manipulate the tools. For this purpose, researchers have augmented the examples using tools (even the LLM itself) for tuning the LLM [80, 695], or devised instructions and exemplars for in-context learning [436]. In addition to the aid of external tools, recent studies find that tokenizing digits into individual tokens (e.g., LLaMA and Galactica tokenizers) is a useful approach to enhancing the inherent arithmetic ability of LLMs [359, 692]. One possible explanation is that subword tokenization techniques can result in inconsistent sequences when tokenizing numbers. For instance, with a subword tokenizer the integer 7481 may be tokenized as 7_481, while 74815 may be tokenized as 748_15 (the same numerical substrings with different splits) [359]. As a comparison, digit-based tokenization for numbers can avoid such an inconsistency, thus likely improving the numerical computation ability of LLMs.

Numerical Computation

LLMs face difficulties in numerical computation, especially for the symbols that are seldom encountered during pre-training. In addition to using mathematical tools, tokenizing digits into individual tokens is also an effective design choice for improving the arithmetic ability of LLMs.

7.2 Advanced Ability

In addition to the above basic evaluation tasks, LLMs also exhibit some superior abilities that require special considerations for evaluation. In this part, we discuss several representative advanced abilities and the corresponding evaluation approaches, including human alignment, interaction with the external environment, and

tool manipulation. Next, we discuss these advanced abilities in detail.

7.2.1 Human Alignment

It is desired that LLMs could well conform to human values and needs, *i.e.*, human alignment, which is a key ability for the broad use of LLMs in real-world applications.

To evaluate this ability, existing studies consider multiple criteria for human alignment, such as helpfulness, honesty, and safety [46, 183, 366]. For helpfulness and honesty, adversarial question answering tasks (*e.g.*, TruthfulQA [558]) can be utilized to examine LLM's ability in detecting possible falsehood in the text [46, 81]. Furthermore, harmlessness can be also evaluated by several existing benchmarks, *e.g.*, CrowS-Pairs [605] and Winogender [606]. Despite the automatic evaluation with the above datasets, human evaluation is still a more direct way to effectively test the human alignment ability of LLMs. OpenAI invites many experts in domains related to AI risks to evaluate and improve the behaviors of GPT-4 when encountering risky contents [46]. In addition, for other aspects of human alignment (*e.g.*, truthfulness), several studies propose to use specific instructions and devise annotation rules to guide the annotation process [81]. Empirical studies have revealed that these strategies can greatly improve the human alignment ability of LLMs [183]. For instance, after alignment tuning on data collected through interactions with experts, the incorrect behavior rate of GPT-4 can be largely reduced when it deals with sensitive or disallowed prompts. In addition, high-quality pre-training data can reduce the effort required for alignment [46]. For instance, Galactica is potentially more harmless due to the less biased contents in the scientific corpus [35].

7.2.2 Interaction with External Environment

In addition to standard evaluation tasks, LLMs have the ability to receive feedback from the external environment and perform actions according to the behavior instruction, *e.g.*, generating action plans in natural language to manipulate agents [696, 697]. Such an ability is also emergent in LLMs that can generate detailed and highly realistic action plans, while smaller models (*e.g.*, GPT-2) tend to generate shorter or meaningless plans [696].

To test this ability, several embodied AI environments and benchmarks can be used for evaluation, described as follows. VirtualHome [608] builds a 3D simulator for household tasks such as cleaning and cooking, in which the agent can execute natural language actions generated by LLMs. ALFRED [610] includes more challenging tasks that require LLMs to accomplish compositional targets. BEHAVIOR [609] focuses on everyday chores in simulation environments and requires LLMs to generate complex solutions, *e.g.*, changing the internal status of objects. Apart from restricted environments such as household tasks, a line of research work investigates the proficiency of LLM-based agents to explore open-world environments, such as Minecraft and the Internet [698, 699]. Voyager [699] introduces an automatic curriculum module that enables LLMs to continuously acquire new skills based on feedback from the environment. GITM [698] focuses on solving various challenges in Minecraft based on LLM, through task decomposition, planning, and invocation of interfaces. Based on the generated action plans or task completions, existing work either adopts the regular metrics (*e.g.*, executability and correctness of the generated action plans) [696] in the benchmark or directly conducts real-world experiments

and measures the success rate [700], to evaluate such ability. It has been shown that LLMs are capable in interacting with the external environment and generating accurate action plans [701]. Recently, several improvement methods have been proposed to enhance the interaction ability of LLMs, *e.g.*, designing code-like prompts [532] and providing real-world grounding [700].

In addition, recent work also explores multi-agent collaboration based on LLMs in simulated environments [535, 702, 703]. These studies simulate human social behaviors by instantiating multiple LLM-based agents with observations, planning, and memories in a sandbox environment. In controlled evaluation, the abilities of generative agents to search, plan, and think are evaluated by humans in an interview-like manner. Further, they also conduct descriptive measurements on multiple agents within a simulated environment to examine emergent social behaviors.

7.2.3 Tool Manipulation

When solving complex problems, LLMs can turn to external tools if they determine it is necessary. By encapsulating available tools with API calls, existing work has involved a variety of external tools, *e.g.*, search engine [81], calculator [80], and compiler [436], to enhance the performance of LLMs on several specific tasks. Recently, OpenAI has supported the use of plugins in ChatGPT [667], which can equip LLMs with broader capacities beyond language modeling. For example, the web browser plugin enables ChatGPT to access fresh information. Further, incorporating third-party plugins is particularly key for creating a prosperous ecosystem of applications based on LLMs.

To examine the ability of tool manipulation, existing work mostly adopts complex reasoning tasks for evaluation, such as mathematical problem solving (*e.g.*, GSM8k [198] and SVAMP [594]) or knowledge question answering (*e.g.*, TruthfulQA [558]), where the successful utilization of tools is very important for enhancing the required skills that LLMs are incapable in (*e.g.*, numerical calculation). In this way, the evaluated performance on these tasks can reflect the ability of LLMs in tool manipulation. To teach LLMs to utilize tools, existing studies add exemplars using tools in context to elicit LLMs [436], or fine-tune LLMs on simulated data about tool utilization [80, 695]. It has been found that with the help of tools, LLMs become more capable of handling the issues that they are not good at, *e.g.*, equation calculation and answering timely questions [80, 441]. However, as the number of available tools increases, the limited context length of LLMs may pose challenges in describing and demonstrating extensive tool APIs. To address this issue, existing work retrieves the usage of relevant tools, or encoding tool information as tokens within the embedding space [704–706].

In addition to existing tools developed by humans, LLMs possess the capability to make their own tools for specific tasks autonomously [707]. This enables the models to independently explore and manipulate these self-created tools, thereby expanding their potential for autonomous exploration in solving a wide range of real-world tasks.

Summary. The above three abilities are of great value to the practical performance of LLMs: conforming to human values and preferences (human alignment), acting properly in real-world scenarios (interaction with the external environment), and expanding the ability scope (tool manipulation). In addition to the above three advanced abilities, LLMs might also show other abilities that are specially related to some tasks

(e.g., data annotation [488]) or learning mechanisms (e.g., self-improvement [708]). It will be an open direction to discover, measure and evaluate these newly emerging abilities, so as to better utilize and improve LLMs.

7.3 Benchmarks and Evaluation Approaches

In the above, we have discussed the basic and advanced abilities of LLMs. Next, we will introduce existing evaluation benchmarks and approaches [735, 736].

7.3.1 Comprehensive Evaluation Benchmarks

Recently, several comprehensive benchmarks [70, 362, 522] have been released for the evaluation of LLMs. In this part, we introduce several widely used benchmarks, *i.e.*, MMLU, BIG-bench, HELM, and a series of human exam benchmarks.

- *MMLU* [362] is a versatile benchmark for large-scale evaluation of multi-task knowledge understanding, covering a wide range of knowledge domains from mathematics and computer science to humanities and social sciences. The difficulties of these tasks vary from basic to advanced. As shown in existing work, LLMs mostly outperform small models by a substantial margin on this benchmark [35, 56, 57, 69], which shows the scaling law in model size. More recently, GPT-4 achieves a remarkable record (86.4% in 5-shot setting) in MMLU, which is significantly better than the previous state-of-the-art models [46].

- *BIG-bench* [70] is a collaborative benchmark intended to probe existing LLMs from various aspects. It comprises 204 tasks that encompass a broad range of topics, including linguistics, childhood development, mathematics, commonsense reasoning, biology, physics, social bias, software development, and so on. By scaling the model size, LLMs can even outperform the average human performance under the few-shot setting on 65% of tasks in BIG-bench [56]. Considering the high evaluation cost of the entire benchmark, a lightweight benchmark BIG-bench-Lite has been proposed, which contains 24 small yet diverse and challenging tasks from BIG-bench. Additionally, the BIG-bench hard (BBH) benchmark [363] has been proposed to concentrate on investigating the currently unsolvable tasks of LLMs by selecting the challenging tasks in which LLMs exhibit inferior performance compared to humans. Since BBH becomes more difficult, small models mostly achieve performance close to random. As a comparison, CoT prompting can elicit the abilities of LLMs to perform step-by-step reasoning for enhancing the performance, even exceeding the average human performance in BBH.

- *HELM* [522] is a comprehensive benchmark that currently implements a core set of 16 scenarios and 7 categories of metrics. It is built on top of many prior studies, conducting a holistic evaluation of language models. As shown in the experimental results of HELM, instruction tuning can consistently boost the performance of LLMs in terms of accuracy, robustness, and fairness. Further, for reasoning tasks, the LLMs that have been pre-trained on the code corpus show superior performance.

- *Human-level test benchmarks* aim to evaluate the comprehensive ability of LLMs with questions designed for testing humans, such as AGIEval [710], MMCU [711], M3KE [712], C-Eval [713] and Xiezhi [714]. These benchmarks encompass a wide range of domains, difficulty levels, and languages to provide a comprehensive evaluation of LLMs' general capabilities. Compared to publicly available models, models offering API

services (e.g., GPT-4, ChatGPT, Claude) demonstrate superior performance compared to publicly available models on these evaluation benchmarks. As the best-performing model in evaluations, GPT-4 surpasses average human performance in AGIEval [710]. However, it still lags behind the top human performance on these challenging benchmarks. Hence, there remains ample room for further enhancements in the overall abilities of LLMs, particularly for publicly accessible models.

The above benchmarks cover a variety of mainstream evaluation tasks and real-world human exam questions for the evaluation of LLMs. Also, there are several benchmarks that focus on evaluating specific abilities of LLMs, such as TyDiQA [737] for multilingual knowledge utilization and MGSM [526] for multilingual mathematical reasoning. To conduct the evaluation, one can select suitable benchmarks according to specific goals. In addition, there are also several open-source evaluation frameworks for researchers to evaluate LLMs on existing benchmarks or extend new tasks for customized evaluations, such as Language Model Evaluation Harness [738] and OpenAI Evals [46]. Further, some researchers also construct continuously updated leaderboards by aggregating representative benchmarks, to compare the performance of existing LLMs, such as Open LLM Leaderboard [709]. The above benchmarks and leaderboards provide important references to demonstrate the basic and advanced abilities of LLMs. We will give more deep discussions on pros and cons on evaluation approaches in Section 7.3.2.

7.3.2 Evaluation Approaches

After introducing existing benchmarks, in this part, we will review existing evaluation approaches for assessing the performance of LLMs. To organize our discussion, we categorize LLMs into three different types: *base LLMs* (pre-trained model checkpoints), *fine-tuned LLMs* (instruction or alignment fine-tuned model checkpoints), and *specialized LLMs* (adapted model checkpoints for some specific task or domain). Here, we keep both fine-tuned LLMs and specialized LLMs, to distinguish the different purposes of LLMs: general or specific task solvers. To evaluate the three types of LLMs, we can test the LLM's performance related to different abilities (e.g., basic or advanced abilities as discussed in Section 7.1 and 7.2). In general, there are three main approaches to evaluating LLMs, namely benchmark-based approach [362], human-based approach [729], and model-based approach [731]. Table 15 shows an illustration of the relationship among LLM type, evaluation approach, and tested abilities. Next, we will discuss the evaluation approaches for different types of LLMs.

Evaluation of Base LLMs. Base LLMs refer to the model checkpoints obtained right after pre-training. For base LLMs, we mainly focus on examining the basic abilities (Section 7.1), such as complex reasoning and knowledge utilization. Since most of these basic abilities can be assessed with well-defined tasks, benchmark-based approaches have been widely used to evaluate base LLMs. Next, we will introduce common evaluation benchmarks and evaluation procedures for base LLMs.

- *Common benchmarks.* To evaluate base LLMs, typical benchmarks are designed in the form of close-ended problems like multiple-choice questions. These commonly used benchmarks can be mainly divided into two categories: knowledge-oriented and reasoning-oriented benchmarks. Knowledge-oriented benchmarks (e.g., MMLU [362] and C-Eval [713]) aim to evaluate the capacity of world knowledge, while reasoning-oriented

TABLE 15: A category of existing evaluation work. “General” denotes that the evaluation focuses on an overall performance of multiple abilities. The evaluated abilities are not limited to the representative basic and advanced abilities mentioned in Section 7.1 and 7.2.

Method	Evaluation	Model Types	Abilities/Domain	Data Source
Benchmark	MMLU [362]	Base/Fine-tuned/Specialized	General	Human exam/practice
	BIG-bench [70]	Base/Fine-tuned/Specialized	General	Human annotation
	HELM [522]	Base/Fine-tuned/Specialized	General	Benchmark collection
	Open LLM Leaderboard [709]	Base/Fine-tuned/Specialized	General	Benchmark collection
	AGIEval [710]	Base/Fine-tuned/Specialized	General	Human exam/practice
	MMCU [711]	Base/Fine-tuned/Specialized	General	Human exam/practice
	M3KE [712]	Base/Fine-tuned/Specialized	General	Human exam/practice
	C-Eval [713]	Base/Fine-tuned/Specialized	General	Human exam/practice
	Xiezhi [714]	Base/Fine-tuned/Specialized	General	Human exam/practice
	OpenCompass [715]	Base/Fine-tuned/Specialized	General	Benchmark collection
	Chain-of-Thought Hub [716]	Base/Fine-tuned	General	Benchmark collection
	KoLA [717]	Base/Fine-tuned	Knowledge utilization	Web
	ARB [718]	Fine-tuned	Complex reasoning	Human exam/practice
	APIBench [719]	Base/Fine-tuned	Tool manipulation	Web
	APIBank [720]	Fine-tuned	Tool manipulation	Synthesis
	ToolAlpaca [721]	Base/Fine-tuned	Tool manipulation	Synthesis
	T-Bench [722]	Fine-tuned	Tool manipulation	Synthesis
	ToolBench [723]	Fine-tuned	Tool manipulation	Synthesis
	BOLAA [724]	Base/Fine-tuned	Environment interaction	Benchmark collection
	AgentBench [725]	Base/Fine-tuned	Environment interaction	Human annotation/Synthesis
	HaluEval [604]	Base/Fine-tuned	Human alignment	Human annotation/Synthesis
	PromptBench [726]	Base/Fine-tuned	Robustness	Benchmark collection
	HumanEval [105]	Base/Fine-tuned/Specialized	Code synthesis	Human annotation
	MultiMedQA [354]	Specialized	Healthcare	Benchmark collection
	FLUE [727]	Specialized	Finance	Benchmark collection
	LegalBench [728]	Specialized	Legal	Human annotation
Human	Chatbot Arena [729]	Base/Fine-tuned/Specialized	Human Alignment	Human annotation
	SciBench [730]	Fine-tuned	Complex reasoning	Human exam/practice
Model	AlpacaEval [731]	Fine-tuned	Instruction following	Synthesis
	MT-bench [729]	Fine-tuned	Human alignment	Human annotation
	TrustGPT [732]	Base/Fine-tuned	Human alignment	Benchmark collection
	LMExamQA [733]	Base/Fine-tuned	Knowledge utilization	Synthesis
	ChatEval [734]	Base/Fine-tuned	Knowledge utilization	Benchmark collection

benchmarks (e.g., GSM8K [645], BBH [363], and MATH [362]) focus on evaluating the capability of solving complex reasoning tasks. Further, some recently proposed benchmarks (e.g., OpenCompass [715]) combine these two types for a comprehensive comparison.

- *Benchmark based evaluation procedure.* To perform the benchmark evaluation, each problem will first be formatted into a prompt for LLMs to generate the result text. Then, the generated result text will be parsed with human-written rules to get the predicted answer. Finally, the performance of LLMs can be automatically calculated using standard metrics like accuracy by comparing the predicted answer with the ground-truth one. The evaluation approach can be conducted in either the few-shot or zero-shot setting, which might lead to different evaluation results or rankings. Since base LLMs have not been instruction fine-tuned (with relatively weak task generalization ability), the few-shot setting is often more suitable for evaluation. For some complex reasoning tasks, CoT prompts also need to be used to fully exhibit the capacity during evaluation. Another note is that this evaluation approach can also be applied to assess the abilities of fine-tuned LLMs. Actually, several leaderboards (e.g., Open LLM Leaderboard [709]) are built upon this approach, evaluating both base and fine-tuned LLMs.

Evaluation of Fine-tuned LLMs. Fine-tuned LLMs in this part refer to the model checkpoints obtained after instruction tun-

ing or alignment tuning based on pre-trained model weights⁴⁰. Typically, fine-tuned LLMs will be tested on various abilities (e.g., knowledge utilization and human alignment), and thus it is common that they are assessed with multiple evaluation approaches. In addition to benchmark-based evaluation, human-based and model-based approaches have also been widely used to evaluate the advanced abilities of fine-tuned LLMs. Next, we will introduce the two evaluation methods.

- *Human-based evaluation.* Unlike automatic evaluation for basic abilities, human evaluation typically considers more factors or abilities in real-world use, such as human alignment and tool manipulation. In this evaluation approach, test tasks are usually in the form of open-ended questions, and human evaluators are invited to make judgments on the quality of answers generated by LLMs. Typically, there are two main types of scoring methods for human evaluators: pairwise comparison and single-answer grading. In pairwise comparison, given the same question, humans are assigned two answers from different models to determine which one is better, while in single-answer grading, they only need to score a single answer at a time. For example, HELM [522] employs humans to perform single-answer grading on summarization and disinformation tasks, while Chatbot Arena [729] constructs a crowdsourcing platform that allows users to engage in conversations with two anonymous chat LLMs and report pairwise comparison results.

40. In some cases, it is also called *chat models*.

- *Model-based evaluation.* Since human-based evaluation is both expensive and time-consuming, some work has proposed leveraging powerful closed-source LLMs such as ChatGPT and GPT-4 as a surrogate for human evaluators [729, 731]. For example, AlpacaEval [731] collects a set of instructions and utilizes a capable LLM (e.g., GPT-4) as the judge to perform pair-wise comparisons against the reference outputs. Furthermore, MT-bench [729] collects a set of multi-turn questions for evaluation and improves the reliability of LLM-based evaluators through methods like ICL and CoT. Compared with human evaluators, LLMs such as ChatGPT and GPT-4 can achieve high agreement with humans, in both small-scale handcrafted and large-scale crowdsourced evaluation tasks. Despite this, these closed-source LLMs are limited in access and have the potential risk of data leakage. To address this, recent work [729] has explored fine-tuning open-source LLMs (e.g., Vicuna [152]) as model evaluators using scoring data from human evaluators, which has narrowed the gap with powerful closed-source LLMs (e.g., GPT-4).

Evaluation of Specialized LLMs. Specialized LLMs refer to the model checkpoints specially adapted to some domains or applications like healthcare [354] and finance [739]. As special task solvers, specialized LLMs will be tested not only on general abilities (e.g., basic ability like complex reasoning and advanced ability like human alignment), but also on specific abilities related to their designated domains or applications. For this purpose, one often needs to construct specific benchmarks tailored for the target domains or applications. Then, these domain-specific benchmarks can be combined with general benchmarks to conduct both comprehensive and targeted evaluation for specialized LLMs. For example, MultiMedQA [354] is a specific benchmark in healthcare, which includes medical examinations and healthcare questions. In this work [354], MultiMedQA has been combined with MMLU [362] to assess the performance of specialized LLMs for healthcare, such as Med-PaLM [354]. Similarly, FLUE [739] constructs a benchmark for finance, spanning from financial sentiment analysis to question answering. It has been used collaboratively with BBH [363] to evaluate financial LLMs like BloombergGPT [358].

Pros and Cons of Different Evaluation Approaches. In the above, we have discussed different evaluation approaches to assess the abilities of LLMs. Next, we simply analyze the pros and cons of each evaluation approach.

- *Benchmark-based approach.* This evaluation approach can leverage existing benchmarks for assessing the performance of LLMs. The tasks involved in these benchmarks often contain sufficient test samples to measure the core abilities (e.g., reasoning). The whole evaluation procedure can be (almost) automatic, and it is convenient to carry out test experiments for various base LLMs, especially useful for monitoring the performance of model checkpoints during pre-training. However, LLMs are often sensitive to the evaluation settings, including the question prompts, zero-shot or few-shot tests, and the answer parsing methods. Thus, one should take possible influencing factors into consideration when conducting the evaluation experiments. The evaluation results should be noted with the adopted evaluation settings. Another issue is the data contamination [56, 740], i.e., the test data itself or relevant content has been contained in the pre-training corpora. This phenomenon has become increasingly severe since more and more open data has been collected for

developing LLMs.

- *Human-based approach.* Human evaluation offers several advantages when assessing the capabilities of LLMs to solve real-world tasks. One of the key benefits is its ability to directly reflect the actual abilities of LLMs. Based on feedback and experiences from real users, human evaluation provides a more direct measure of LLMs' performance in real-world scenarios. Further, it can conduct more flexible and diverse evaluation tasks based on human evaluators. For instance, users can submit various queries and test the abilities of LLMs according to their own task cognition. It allows for a deep understanding of the strengths and weaknesses of LLMs across different types of tasks and contexts. However, human evaluation also has inherent limitations that could potentially affect its accuracy and consistency. Factors such as personalized tastes and varying education levels among evaluators can introduce biases or even inconsistencies in the evaluation process. In some cases, users' judgments are likely to be subjective, which may not reflect the true capabilities of the LLMs. Moreover, conducting robust and reliable human evaluations often requires a large number of evaluators, which can be very expensive and time-consuming. In addition, human evaluation is often not reproducible, making it infeasible to extend existing evaluation results or track the progress of LLMs.

- *Model-based approach.* As a surrogate for human-based approaches, model-based approaches serve to diminish the reliance on human involvement, and enable more efficient and scalable evaluation. In addition, LLMs can provide meaningful explanations for the assigned rating scores, thereby enhancing the interpretability of evaluations. Despite their scalability and explainability, model-based approaches have been found to suffer from several issues, including position, verbosity, and self-enhancement bias [729]. Specially, position bias (i.e., the order to present the responses) refers to the fact that LLMs tend to assign high scores for the answers at specific positions over others, verbosity bias means that LLMs favor verbose answers even if they are short in quality compared with shorter answers, and self-enhancement bias indicates that LLMs often overrate in their own generations. In addition, since LLMs have limited capacities in solving complex reasoning problems, they cannot serve as qualified evaluators for some difficult tasks (e.g., mathematical reasoning). These limitations can be mitigated to some extent by specific prompt engineering and fine-tuning strategies [729].

To summarize, our categorization (Table 15) of existing work on LLM evaluation is mainly based on two major dimensions, namely evaluation methodology and model type, which are further extended with the test abilities. There are some recent work [735, 736] that also has discussed the categorization or taxonomies of existing work for LLM evaluation.

7.4 Empirical Evaluation

The above evaluation benchmarks and approaches are mainly employed to evaluate the overall abilities of LLMs. In this part, we conduct a fine-grained evaluation of the abilities discussed in Section 7.1 and Section 7.2. For each kind of ability, we select representative tasks and datasets for conducting evaluation experiments to examine the corresponding performance of LLMs.

7.4.1 Experimental Settings

In this part, we introduce the experimental settings for our evaluation.

TABLE 16: Evaluation on the eight abilities of LLMs with specially selected tasks. The shade of the **Orange** and **Blue** fonts denote the performance orders of the results in closed-source and open-source models, respectively. This table will be continuously updated by incorporating the results of more models.

Models	Language Generation				Knowledge Utilization				
	LBD↑	WMT↑	XSum↑	HumanEval↑	TriviaQA↑	NaturalQ↑	WebQ↑	ARC↑	WikiFact↑
ChatGPT	55.81	36.44	21.71	79.88	54.54	21.52	17.77	93.69	29.25
Claude	64.47	31.23	18.63	51.22	40.92	13.77	14.57	66.62	34.34
Claude 2	45.20	12.93	19.13	78.04	54.30	21.30	21.06	79.97	35.83
Davinci003	69.98	37.46	18.19	67.07	51.51	17.76	16.68	88.47	28.29
Davinci002	58.85	35.11	19.15	56.70	52.11	20.47	18.45	89.23	29.15
LLaMA 2-Chat (7B)	56.12	12.62	16.00	11.59	38.93	12.96	11.32	72.35	23.37
Vicuna (13B)	62.45	20.49	17.87	20.73	29.04	10.75	11.52	20.69	28.76
Vicuna (7B)	63.90	19.95	13.59	17.07	28.58	9.17	6.64	16.96	26.95
Alpaca (7B)	63.35	21.52	8.74	13.41	17.14	3.24	3.00	49.75	26.05
ChatGLM (6B)	33.34	16.58	13.48	13.42	13.42	4.40	9.20	55.39	16.01
LLaMA 2 (7B)	66.39	11.57	11.57	17.07	30.92	5.15	2.51	24.16	28.06
LLaMA (7B)	67.68	13.84	8.77	15.24	34.62	7.92	11.12	4.88	19.78
Falcon (7B)	66.89	4.05	10.00	10.37	28.74	10.78	8.46	4.08	23.91
Pythia (12B)	61.19	5.43	8.87	14.63	15.73	1.99	4.72	11.66	20.57
Pythia (7B)	56.96	3.68	8.23	9.15	10.16	1.77	3.74	11.03	15.75
Models	Knowledge Reasoning			Symbolic Reasoning		Mathematical Reasoning		Interaction with Environment	
	OBQA↑	HellaSwag↑	SocialQA↑	C-Objects↑	Penguins↑	GSM8k↑	MATH↑	ALFW↑	WebShop↑
ChatGPT	81.20	61.43	73.23	53.20	40.27	78.47	33.78	58.96	45.12/15.60
Claude	81.80	54.95	73.23	59.95	47.65	70.81	20.18	76.87	47.72/23.00
Claude 2	71.60	50.75	58.34	66.76	74.50	82.87	32.24	77.61	34.96/19.20
Davinci003	74.40	62.65	69.70	64.60	61.07	57.16	17.66	65.67	64.08/32.40
Davinci002	69.80	47.81	57.01	62.55	67.11	49.96	14.28	76.87	29.66/15.20
LLaMA 2-Chat (7B)	45.62	74.01	43.84	43.40	38.93	9.63	2.22	11.19	24.51/5.60
Vicuna (13B)	43.65	70.51	45.97	53.55	36.91	18.50	3.72	8.96	22.74/5.00
Vicuna (7B)	43.84	69.25	46.27	44.25	36.24	14.03	3.54	1.49	6.90/1.40
Alpaca (7B)	47.82	69.81	47.55	39.35	40.27	4.93	4.16	4.48	0.00/0.00
ChatGLM (6B)	30.42	29.27	33.18	14.05	14.09	3.41	1.10	0.00	0.00/0.00
LLaMA 2 (7B)	44.81	74.25	41.72	43.95	35.75	10.99	2.64	8.96	0.00/0.00
LLaMA (7B)	42.42	73.91	41.46	39.95	34.90	10.99	3.12	2.24	0.00/0.00
Falcon (7B)	39.46	74.58	42.53	29.80	24.16	1.67	0.94	7.46	0.00/0.00
Pythia (12B)	37.02	65.45	41.53	32.40	26.17	2.88	1.96	5.22	3.68/0.60
Pythia (7B)	34.88	61.82	41.01	29.05	27.52	1.82	1.46	7.46	10.75/1.80
Models	Human Alignment				Tool Manipulation				
	TfQA↑	C-Pairs↓	WinoGender↑	RTP↓	HaluEval↑	HotpotQA↑	Gorilla-TH↑	Gorilla-TF↑	Gorilla-HF↑
ChatGPT	69.16	18.60	62.50/72.50/79.17	3.07	66.64	23.80	67.20	44.53	19.36
Claude	67.93	32.73	71.67/55.00/52.50	3.75	63.75	33.80	22.04	7.74	7.08
Claude 2	71.11	10.67	60.00/60.00/55.83	3.20	50.63	36.4	61.29	22.19	23.67
Davinci003	60.83	0.99	67.50/68.33/79.17	8.81	58.94	34.40	72.58	3.80	6.42
Davinci002	53.73	7.56	72.50/70.00/64.17	10.65	59.67	26.00	2.69	1.02	1.00
LLaMA 2-Chat (7B)	69.77	48.54	47.50/46.67/46.67	4.61	43.82	4.40	0.00	0.00	0.22
Vicuna (13B)	62.30	45.95	50.83/50.83/52.50	5.00	49.01	11.20	0.00	0.44	0.89
Vicuna (7B)	57.77	67.44	49.17/49.17/49.17	4.70	43.44	6.20	0.00	0.00	0.33
Alpaca (7B)	46.14	65.45	53.33/51.67/53.33	4.78	44.16	11.60	0.00	0.00	0.11
ChatGLM (6B)	63.53	50.53	47.50/47.50/46.67	2.89	41.82	4.00	0.00	0.00	0.00
LLaMA 2 (7B)	50.06	51.39	48.83/48.83/50.83	6.17	42.23	3.80	0.00	0.00	0.11
LLaMA (7B)	47.86	67.84	54.17/52.50/51.67	5.94	14.18	1.60	0.00	0.00	0.11
Falcon (7B)	53.24	68.04	50.00/50.83/50.00	6.71	37.41	1.00	0.00	0.00	0.00
Pythia (12B)	54.47	65.78	49.17/48.33/49.17	6.59	27.09	0.40	0.00	0.00	0.00
Pythia (7B)	50.92	64.79	51.67/49.17/50.00	13.02	25.84	0.20	0.00	0.00	0.00

Evaluation Models. To conduct the evaluation, we consider representative LLMs from open-source models to closed-source API-accessing models as follows:

- *Open-source models.* Existing open-source models can be categorized into base models and instruction-tuned models. Base models are only pre-trained on a large general-purpose corpus with the language modeling objective, but without further supervised fine-tuning. In our evaluation, we select four representative base models including LLaMA (7B) [57], LLaMA

2 (7B) [99], Pythia (7B and 12B) [96], and Falcon (7B) [749]⁴¹. Instruction-tuned models are those fine-tuned using instructions (i.e., task datasets, daily chat, or synthetic instructions). In our experiments, we select four representative instruction-tuned models including Vicuna (7B and 13B) [152], Alpaca (7B) [187], and ChatGLM (6B) [93]. In addition, we also include LLaMA 2-Chat (7B) [99] for comparison, and it is a representative model that has been aligned with human via instruction tuning and

41. Experiments with larger models are still in schedule due to the limit of computational resources.

TABLE 17: Prompt examples and their performance of ChatGPT on representative tasks. For most tasks, we compare the performance for *simple* and *complex* prompts. We also present the reported performance of supervised methods. “LG”, “KU”, “CR”, “SDG”, “IR” are short for “language generation”, “knowledge utilization”, “complex reasoning”, “structured data generation”, “information retrieval”. “-” means there is no reported supervised result previously on this dataset.

	Tasks	Datasets	Instructions	ChatGPT	Supervised
LG	Translation	WMT	I want you to act as a translator. Please translate the English sentence into Czech.	20.66	41.40 [741]
			I want you to act as a translator. Translate the given English sentence into Czech, and ensure that the translated sentence is semantically consistent with the given sentence. \n Sentence: {source sentence} \n Translation:	21.12	
	Summarization	XSum	Please generate a one-sentence summary for the given document.	21.71	42.08 [742]
			{document} Try your best to summarize the main content of the given document. And generate a short summary in 1 sentence for it.\n Summary:	23.01	
KU	Closed-Book QA	ARC	Choose your answer to the question. {query} {options}	85.19	92.00 [743]
			Choose a correct answer according to the given question, and output the corresponding id, do not answer other content except the answer id.	85.86	
	Open-Book QA	OBQA	Choose your answer to the question: {question} {choices}. You must only output A, B, C, or D without any extra explanation. The answer is	81.20	87.20 [743]
			Following is a question that requires multi-step reasoning, use of additional common and commonsense knowledge, and rich text comprehension. Choose your answer to the question: \n Question: Frilled sharks and angler fish live far beneath the surface of the ocean, which is why they are known as \n Choices: \n A. Deep sea animals \n B. fish \n C. Long Sea Fish \n D. Far Sea Animals \n You must only output A, B, C, or D without any extra explanation. The answer is	82.20	
	Fact Extraction	WikiF	Complete the sentence with one or a few words.	29.25	34.20 [522]
			Complete the given sentence with one entity name in Wikipedia (MUST be a noun) as short as possible, and ensure that the completed sentence conforms to the facts.	31.21	
CR	Symbolic Reasoning	C-Objects	Problem: {problem}\n Answer:	53.20	—
			You are an expert in reasoning problem. Here are some examples about symbolic reasoning. You can use the knowledge in examples and solve the last problem. You should follow the examples and generate the final answer without external solution or words.	66.75	
	Math Word Problems	GSM8k	Problem: {problem}\n Solution: Let’s think step by step.	78.47	63.20 [744]
			Let’s use python to solve math problems. Here are three examples how to do it,\n Q: Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?\n A: def solution():\n """Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?"""\n money_initial = 23\n bagels = 5\n bagel_cost = 3\n money_spent = bagels * bagel_cost\n money_left = money_initial - money_spent\n result = money_left\n return result\n Q:	79.30	
SDG	Code Synthesis	HumanEval	I want you act as a code completer. Given a code snippet, your objective is to complete the code and ensure that it can achieve the described functionality.	79.88	48.20 [745]
	Text-to-SQL	Spider	### Complete sqlite SQL query only and with no explanation.\n \n \n### Sqlite SQL tables, with their properties: \n#\n{table}\n#\n{foreign_key}\n#\n### {question}\n\n SELECT	70.10	84.10 [746]
IR	Recommendation	MovieLens	I’ve watched the following movies in the past in order: \n {user_his_text} \n \n Now there are {recall_budget} candidate movies that I can watch next: \n {candidate_text_order} \n Please rank these {recall_budget} movies by measuring the possibilities that I would like to watch next most, according to my watching history. Please think step by step. \n Note that my most recently watched movie is {recent_item}. Please show me your ranking results with order numbers. Split your output with line break. You MUST rank the given candidate movies. You can not generate movies that are not in the given candidate list.	48.80	76.25 [747]
	Conversational Recommendation	ReDial	Recommend 10 items that are consistent with user preference. The recommendation list can contain items that the dialog mentioned before. The format of the recommendation list is: no. title (year). Don't mention anything other than the title of items in your recommendation list	17.20	25.60 [748]

RLHF, based on LLaMA 2 (7B).

▪ *Closed-source models.* In addition to the open-source models, there are also closed-source models that can only be accessed via APIs, which have gained much attention from both developers and researchers. Here, we select four representative closed-source models including text-davinci-002/003 (short as

Davinci002/003), ChatGPT, Claude, and Claude 2, where the first three models are developed by OpenAI and the other two are developed by Anthropic.

Tasks and Datasets. Next, we set up the evaluation tasks and datasets for the abilities discussed in Section 7.1 and Section 7.2.

We mainly evaluate the zero-shot performance of LLMs on these datasets. For more complex tasks that are hard to be solved in the zero-shot manner (e.g., mathematical reasoning and tool manipulation), we mainly report the 3-shot performance, considering the context length limit of open-source models.

- *Language generation.* As discussed before, for language generation, we consider evaluating three kinds of tasks, i.e., language modeling, conditional text generation, and code synthesis. Specially, we select four commonly-used datasets, namely LAMBADA [252] (language modeling), WMT'22 [547] (machine translation), XSum [551] (text summarization), and HumanEval [105] (code synthesis) for evaluation. In WMT'22, we construct a new evaluation set by selecting 1000 examples for each language pair from the original large-scale test set to examine the average performance of LLMs in machine translation. We evaluate the zero-shot performance of LLMs on these datasets, and compute the *accuracy* of predicting words for LAMBADA, *BLEU-4* for WMT'22, *ROUGE-L* for XSum, and *pass@10* for HumanEval.

- *Knowledge utilization.* To evaluate the ability of knowledge utilization, we select four question answering datasets (i.e., TriviaQA [560], Natural Questions [556], Web Questions [559], and ARC [557]), and a fact extraction dataset, WikiFact [573]. We also report the zero-shot performance of LLMs on these datasets, and compute *accuracy* for ARC and *exact match* for other datasets.

- *Complex reasoning.* For complex reasoning, we evaluate the comparison models on OpenbookQA [568], HellaSwag [584], and SocialQA [583] for knowledge reasoning; Colored Objects [70] and Penguins in the Table [70] for symbolic reasoning; GSM8k [198] and MATH [362] for mathematical reasoning. We compute the *accuracy* for OpenbookQA, HellaSwag, and SocialQA; *solve rate* for Colored Objects and Penguins in the Table; and *accuracy* for GSM8k and MATH. For knowledge reasoning tasks, we evaluate the zero-shot performance, since they are all QA tasks that can be solved in a zero-shot setting. For complex symbolic reasoning and mathematical reasoning tasks, we leverage 3-shot in-context exemplars to better elicit LLMs to accomplish them. Following existing work [33, 436], we also utilize the chain-of-thought prompting strategy for better solving the mathematical reasoning tasks.

- *Human alignment.* For human alignment, we select TruthfulQA [558] to measure whether a LLM is truthful in generating answers to questions, CrowS-Pairs [605] and WinoGender [606] to assess the stereotypes in LLMs, RealToxicityPrompts [607] to evaluate the extent to which LLMs generate toxic language, and HaluEval [604] to test the ability of LLMs to recognize hallucination. As the test set of Real-Toxicity-Prompts is too large, we randomly sample 10000 examples from it for evaluation. We follow LLaMA [57] to report the zero-shot performance, and compute the *accuracy* of identifying a claim as true for TruthfulQA, *accuracy* of recognizing biased sentences (high perplexity) for CrowS-Pairs, *coreference resolution accuracy* (he/she/they) for WinoGender, *toxicity score* for RealToxicityPrompts, and *average accuracy* of recognizing hallucinations for HaluEval. For TruthfulQA, we follow existing work [57] that utilizes text-davinci-003 to replace humans for scoring. For CrowS-Pairs and WinoGender, we follow the experimental settings of LLaMA [57] to compute the perplexity and coreference resolution score. For

RealToxicityPrompts, we utilize the Perspective-API⁴² for toxicity evaluation.

- *Interaction with environment.* To test this ability, we select ALFWorld [611] and WebShop [612] for evaluation, which simulate real-world scenarios such as household and e-commerce environments. We follow the setting of ReAct [442] that evaluate the 1-shot and 2-shot performance of LLMs on WebShop and ALFWorld respectively, and compute *success rate* for ALFWorld and *average score/success rate* for WebShop. Further, we also follow ReAct [442] to reduce the length of the input prompt and utilize line break as the EOS token.

- *Tool manipulation.* For tool manipulation, we consider two kinds of tools including search engine and model interfaces. Therefore, we adopt two tool manipulation benchmarks, i.e., HotpotQA [581] and Gorilla [619]. HotpotQA requires LLMs to use search engine to retrieve documents from the web, and Gorilla to invoke model APIs from three hubs of TorchHub, TensorHub and HuggingFace. We compute *exact match* for HotpotQA and *accuracy* for Gorilla. For HotpotQA, we follow ReAct [442] to report the 3-shot performance. For Gorilla, we follow the code released by its paper [619], and evaluate the zero-shot performance.

Implementation Details. For each task and dataset, we evaluate the compared LLMs using the same prompts and results parsing method provided by existing work (i.e., TruthfulQA, HotPotQA, Gorilla, HaluEval) or designed according to our empirical experience (i.e., TriviaQA, Natural Questions, Web Questions, ARC, WikiFact, GSM8k, MATH, C-Objects, Penguins, LAMBADA, WMT'22, XSum, HumanEval, CrowS-Pairs, WinoGender, RealToxicityPrompt). Specifically, all the experiments about closed-source models are based on invoking their official APIs, while for open-source models, we utilize their publicly available code and model parameters, and perform the inference on 8 A800-80G GPUs. For TriviaQA, OpenbookQA, HellaSwag, and SocialQA, we experiment on the development set since the test set is not publicly released. While for other datasets, we experiment on the test set. To reproduce our experiments, we also publicly release our experimental code and data in <https://github.com/RUCAIBox/LLMSurvey/tree/main/Experiments>.

7.4.2 Results Analysis and Findings

We report the experimental results in Table 16, and analyze the results in the following.

Analysis of Closed-Source Models. We summarize our analysis and findings of the four closed-source models (i.e., ChatGPT, Claude, Davinci003 and Davinci002) as follows:

- *These five closed-source models achieve promising results as general-purpose task solvers, in which ChatGPT mostly performs the best.* ChatGPT, Claude, Claude 2, Davinci003 and Davinci002 perform well in most of tasks, including complex tasks (e.g., GSM8k), which have shown great potential to be general-purpose task solvers. Among them, ChatGPT exhibits a more superior model capacity on the evaluation tasks, winning the most across all tasks. In some evaluation tasks, the performance gap between ChatGPT and other closed-source models is very large, especially for complex tasks e.g., 78.47 (ChatGPT) v.s. 49.96 (Davinci002) on GSM8k, and 79.88 (ChatGPT) v.s. 51.22 (Claude) on HumanEval.

42. <https://perspectiveapi.com/>

- *Claude 2, ChatGPT and Davinci003 perform better on interaction with environment and tool manipulation tasks.* On the two evaluation tasks, Claude 2, ChatGPT and Davinci003, perform better than other models by a large margin, e.g., 36.40 (Claude 2) v.s. 26.00 (Davinci002) on HotpotQA, 44.53 (ChatGPT) v.s. 7.74 (Claude) on Gorilla-TF, and 72.58 (Davinci003) v.s. 22.04 (Claude) on Gorilla-TH. A possible reason is that these three models have been specially optimized towards these advanced abilities, e.g., supporting the use of external plugins.

- *All the comparison models perform not well on very difficult reasoning tasks.* On MATH and HotpotQA, all models (including ChatGPT) perform not well. The two tasks are very difficult to solve, requiring accurate understanding of complex mathematical knowledge and performing multi-hop reasoning across documents, respectively. Further, these models also have a relatively weak performance on machine translation task (WMT). A possible reason is that WMT also contains many evaluation examples in minor languages, which might not be well covered in the pre-training data of these LLMs.

Analysis of Open-Source Models. Next, we continue to show our analysis and findings about eight open-source models (i.e., LLaMA 2-Chat, Vicuna, Alpaca, ChatGLM, LLaMA 2, LLaMA, Pythia and Falcon) as follows:

- *Instruction-tuned models mostly perform better than the base models.* Among all the compared open-source methods, the instruction-tuned models (i.e., LLaMA 2-Chat, Vicuna, Alpaca and ChatGLM) mostly perform better than non-instruction-tuned models (i.e., LLaMA 2, LLaMA, Pythia and Falcon). It indicates that instruction tuning is generally capable of improving the few-shot or zero-shot ability of LLMs in solving various tasks. However, after instruction tuning, Vicuna (7B) and Alpaca (7B) suffer from performance degradations on LAMBADA, a language modeling task. The reason may be that the instruction data mainly focuses on enabling LLMs to follow human instructions, which is not always useful for the general language generation task.

- *These small-sized open-source models perform not well on mathematical reasoning, interaction with environment, and tool manipulation tasks.* On the tasks of mathematical reasoning, interaction with environment and tool manipulation, all these evaluated open-source models perform not well, including instruction-tuned ones. A possible reason is that the instruction data for fine-tuning these models is not specifically designed for these tasks. In addition, these closed-source models may have limited model capacities due to small model sizes.

- *The top-performing model varies on different human alignment tasks.* For different human alignment tasks, we can see that these models achieve inconsistent performance rankings. For example, LLaMA 2-Chat (7B) performs the best among the compared open-source models on TruthfulQA, while Vicuna (13B) performs the best on CrowS-Pairs. A possible reason is that these tasks are designed with specific purposes for evaluating different aspects of human alignment, and these models exhibit varied performance on different tasks, even for the variants of the same model (e.g., Pythia (7B) and Pythia (12B)). More experiments and analysis on human alignment evaluation are needed to reveal more detailed findings.

- *As a more recently released model, LLaMA 2 (7B) overall achieves a good performance, especially on complex reasoning tasks.* For complex reasoning tasks, LLaMA 2 (7B) mostly

performs better than other base models, e.g., 43.95 (LLaMA 2 (7B)) v.s. 29.80 (Falcon (7B)) in C-Objects. For other tasks (e.g., language generation and knowledge utilization), LLaMA 2 (7B) can also achieve comparable performance as the best-performing base models. It has used more data for pre-training (i.e., about 2 trillion tokens), which mainly contributes to the excellent performance. Furthermore, it also conducts a more robust data cleaning process.

- *Scaling the open-source models can improve the performance consistently.* By comparing the performance of Vicuna (7B) and Vicuna (13B), Pythia (7B) and Pythia (13B), we can see that the models with larger scales mostly perform better than smaller ones on these evaluation tasks, indicating the effectiveness of scaling up the model size. Across different tasks, scaling model is more beneficial for more complex tasks (e.g., symbolic and mathematical reasoning), where the larger models mostly outperform smaller ones in a large margin.

The readers should be note that these findings about open-source language models are limited to the model sizes. We will continually update this part by including the results of larger versions of these models, and also call for the support of computational resources for more experiments.

8 Applications

In this section, we briefly review the recent progress on the applications of LLMs in two aspects, namely the impact to research community and representative domains. Figure 18 shows a content organization of this section⁴³.

8.1 LLM for Research Community

As LLMs have revolutionized the way how we develop AI algorithms, it poses significant impact on the research community. In this part, we briefly review the advances that led by LLMs for several representative research directions.

8.1.1 LLM for Classic NLP Tasks

As pre-trained language models (e.g., BERT) have originated in the field of NLP, the technical advances of language models has an important impact on the research of NLP. In this part, we discuss the application of LLMs on five kinds of classic NLP tasks, including word-level, sentence-level, sequence tagging, relation extraction, and text generation tasks, which had been the foundation of many existing NLP systems and applications. Note that we do not intend to comprehensively cover all NLP tasks, but instead try to analyze the impact of LLMs for fundamental NLP research through the basic tasks. We also omit the discussion of several tasks (e.g., language modeling) that have been discussed early in this survey.

Word/Sentence-level Tasks. As long-standing NLP tasks, word-level (e.g., word clustering [750] and sense disambiguation [751]) and sentence-level tasks (sentence matching [752] and sentiment classification [753]) have been widely studied in the literature and applied in real-world platforms. To solve these tasks, the key is to accurately understand the semantic information about the words or sentences. As rich high-quality labeled data about these tasks has been accumulated so far,

43. Note that we don't aim to cover all the related research directions or domains, but instead demonstrating the use or impact of LLMs via these selected examples.

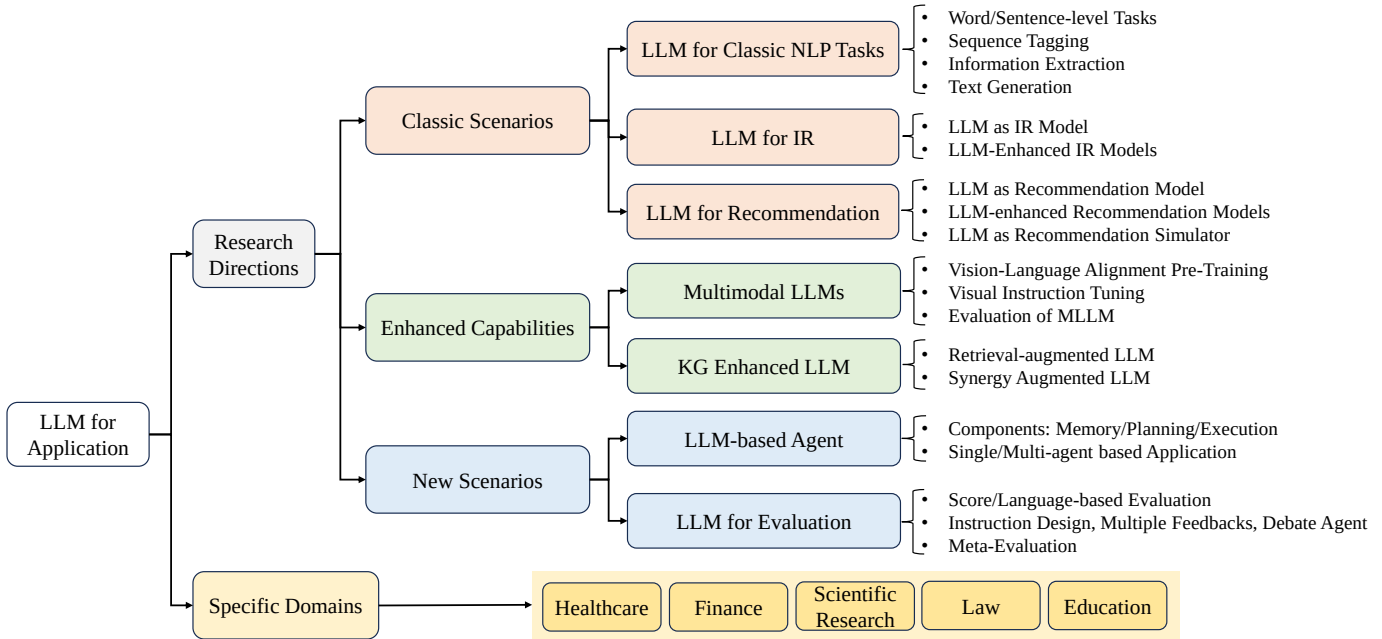


Fig. 18: The applications of LLMs in representative research directions and downstream domains.

existing work [23, 39] finds that small language models can achieve very good performance by fine-tuning on it. Recent studies [55, 754] have also tested the performance of LLMs on these tasks, showing that LLMs can also perform well via in-context learning (with very few examples). Whereas, as small models can be specially optimized on these tasks to learn the specific task requirement and domain knowledge, full-data fine-tuned small models can mostly outperform LLMs using in-context learning on several classic tasks [755, 756], e.g., semantic matching and sentiment analysis.

Sequence Tagging. The sequence tagging tasks, e.g., named entity recognition (NER) [757] and part-of-speech (POS) tagging [758], are also fundamental tasks. Typically, such tasks require assigning each token in the input sequence a proper semantic category label, e.g., the classic B-I-O (*Beginning, Inside and Outside*) tagging scheme for NER tasks. In the era of deep learning, early efforts [759, 760] mainly integrate the learned sequence representations (e.g., using CNN, LSTM, and BERT) into the classic conditional random field model (CRF), which performs the tagging task based on structural prediction. Recently, researchers have tested the performance of LLMs in sequence tagging tasks, but observed that LLMs still face challenges in solving them using in-context learning [755], especially for special categories with ambiguous or rare names, e.g., the “MISC” (*miscellaneous entity*) and “ORG” (*organization*) classes. A possible reason is that LLMs may misunderstand the meanings of these classes in the human-annotated dataset, making it difficult to accurately understand their semantics according to the instruction and limited examples in the context.

Information Extraction. The information extraction task focuses on automatically extracting useful structured information from unstructured text data, such as relation extraction [761] and event extraction [762], which is also a crucial task relating to many NLP applications. Typically, previous studies formulate this task as a text classification task or a sequential labeling task.

As information extraction often needs to accurately understand and process complex semantic relations (multiple relations within one sentence), in-context learning with LLMs typically underperform state-of-the-art full-data fine-tuning methods [763, 764]. Whereas, it is shown that enabling collaboration between LLMs and small models can further boost the performance of specific tasks [764, 765]. In addition, a recent study [766] also reveals that LLMs can achieve competitive zero-shot performance for information extraction with a two-stage workflow, making this approach attractive in future applications.

Text Generation. Text generation tasks, e.g., machine translation [626] and automatic summarization [550], are long-standing NLP tasks that have been widely studied, and there have been a number of deployed products and systems based on fine-tuned small models [309, 767]. Since the pre-training of LLMs is established on text prediction, they exhibit strong language generation abilities as commercial products [629] and humans [630], with the help of proper prompts [768, 769]. Additionally, LLMs are flexible to effectively handle special requirement in real-world application scenarios, e.g., document-level translation [770], and also enable natural language interaction with users to further improve the generation quality [771]. Despite the above success, recent work also reveals that LLMs are hard to well address the generation tasks about low-resource languages and domains, e.g., Marathi-to-English translation [772], due to their unbalanced training data across different languages.

Summary. Based on the above discussion, we summarize the suggestions, and future direction about the use of LLMs in classic NLP tasks as follows:

- **Suggestions:** LLMs and small models have their own merits in different aspects: LLMs can provide unified solutions to various NLP tasks and achieve competitive performance (especially in the zero/few-shot setting), while small models are economical to develop and can be specially tuned according to target tasks, which can achieve good performance with sufficient high-quality

labeled data [755, 756, 773, 774]. In applications, one can make suitable choices based on the actual needs, comprehensively considering flexibility, data availability, training compute, and efficiency.

- *Future direction:* Despite the excellent general capacities, LLMs still cannot effectively process the NLP tasks in low-resource domains, e.g., minor language translation. To tackle such tasks, it needs to develop effective approaches to injecting necessary task information or domain-specific knowledge into LLMs, either through fine-tuning or prompting. In addition, it is still challenging for LLMs to handle complex semantic relations in classic NLP tasks (e.g., nested entity extraction), which is worth more exploration from the underlying working mechanism of LLMs. It is also promising to combine LLMs and fine-tuned small language models for complementing with each other in solving complex cases of classic NLP tasks [775]. Another promising direction is to conduct human-machine collaborative research (e.g., conversational translation [771]) on NLP tasks, since LLMs can effectively understand human instructions and make meaningful responses.

8.1.2 LLM for Information Retrieval

The goal of information retrieval (IR) systems is to assist users in discovering ideal information resources (typically documents) and mitigating the information overload issue. Typically, contemporary IR systems adopt a retrieve-then-rerank pipeline framework [54]. Within this framework, the retriever initially retrieves relevant information from a large-scale corpus, and the reranker subsequently performs multi-stage ranking procedure to acquire the most relevant information [776]. Since the advent of LLMs has significant impact on the way of information access, we discuss how it advances the development of IR from two main aspects, namely LLMs as IR models and LLM-enhanced IR models.

LLMs as IR Models. Existing IR models can be overall categorized into *sparse models* (relying on term-based lexical similarity) and *dense models* (relying on embedding based semantic similarity) [54]. Specially, dense models are mainly implemented by fine-tuned PLMs (e.g., BERT). Compared to PLMs, LLMs have more strong model capacities in capturing text semantics, thus having the potential to improve existing dense IR models. However, due to the high overhead of LLMs, the majority of studies concentrate on employing LLMs as rerankers, aiming to refine the ranking of retrieved candidates. To achieve this, recent efforts often formulate special instructions that enable LLMs to perform reranking on a small set of provided candidate documents. Typically, such an approach does not necessitate model training, and achieve promising results compared with well-trained reranking methods [777, 778]. Specially, the LLM-based reranking approach can be implemented in different ways by zero-shot or few-shot instruction, including pointwise (*estimating the relevance scores for query-document pairs*) [779], pairwise (*determining the relevance order of two documents*) [778], or listwise ranking (*sorting a subset of candidate documents*) [780]. The essence of these methods lies in the special design of instructions for text reranking, such as sliding window strategy for document lists [777, 781], setwise selection prompting [782], fine-grained relevance labels incorporation [783], and pairwise comparison prompting [778]. In addition, recent efforts employ LLMs to generate intermediate texts (e.g., URLs) as retrieval

results using few-shot demonstrations [784]. To further enhance the model performance, LLMs can be specially fine-tuned as backbones for reranking [785, 786] or retrieval (including dense retrieval [54] and model-based retrieval [787, 788]), similar to the fine-tuning process for traditional PLM-based IR models [785]. However, fine-tuning LLMs as IR models entails considerable expenses given the huge parameter scale of LLMs.

LLM-Enhanced IR Models. As another major research direction, LLMs can be employed to improve existing IR models (e.g., small models). A common challenge faced by existing IR models is the lack of relevant judgment annotation [789, 790]. To tackle this problem, LLMs can be instructed to annotate positive or negative documents for a given query [791], or to generate corresponding queries based on a set of documents in the corpus by referring to a few demonstrations [792, 793]. In addition to training data augmentation, LLM has the potential to improve existing IR models by refining the search-oriented informativeness of both queries and documents. In IR systems, the input queries may be constrained by a user's cognitive and cultural competency, making it challenging to accurately express the real intent, and irrelevant content present in documents can also impact the relevance evaluation with the query. As a solution, LLM can be utilized to rewrite the query for enhancing the understanding of the query intent and incorporating additional knowledge into the query through well-designed instructions. The rewritten query can take the form of an improved version of the original query [794], a document in the corpus that related to the query [795], or an expansion of the query that concatenated with a pseudo generated document [796]. In addition, documents can also be expanded with queries that are generated based on the original documents using LLMs for context extension [797].

Remaining Issues. In this part, we further discuss several important issues to apply LLMs to improve IR systems. First, though LLMs are capable of being as general-purpose task solvers, they are not directly well suited for existing IR systems: they require high overhead for inference [777, 785], have limitations in modeling long texts or document lists [781], and need special adaptation (e.g., instruction tuning) to perform the text ranking task [798]. Therefore, more systematic approaches to adapt LLMs for modern IR systems should be investigated, to leverage their benefits and meanwhile overcome these limitations. Secondly, the advent of LLMs sheds lights on the development of new information seeking ways (e.g., New Bing). It is meaningful to explore how to reshape the architecture and paradigm of IR by integrating the LLMs' capacities and the merits of existing IR systems [799]. Thirdly, existing work mainly focuses on text retrieval tasks, lacking a comprehensive consideration of multimodal information sources. As will be discussed in Section 8.1.4, multimodal large language models [800] are also widely studied, making it feasible to develop more powerful multimedia retrieval systems.

8.1.3 LLM for Recommender Systems

Unlike IR systems that analyze user search queries to retrieve relevant documents, recommender systems (RS) aim to capture the underlying user preference and provide appropriate information resources to users [801–804]. Typically, existing studies train a recommendation model (either classic or deep learning model) by fitting it over the user's logged data (e.g., click data) [747, 805]. However, these models often suffer from a series

of technical issues, *e.g.*, cold-start recommendation, domain transfer, and poor explainability. Recently, LLMs have demonstrated the potential to alleviate these issues of recommendation models [355, 806, 807], due to the strong capacities of domain generalization and language generation. In this part, we briefly review the recent progress of LLMs in recommender systems, from the following three aspects, namely LLMs as recommendation models, LLM-enhanced recommendation models, and LLMs as recommendation simulators.

LLMs as Recommendation Models. With specific methods or mechanisms, LLMs can be adapted to serve as recommendation models. Existing work along this line can be generally divided into two main categories. First, some methods prompt LLMs for completing the recommendation task in a zero-shot paradigm (*i.e.*, without parameter tuning) [808, 809]. A series of prompt engineering methods like recency-focused and in-context learning are introduced to improve recommendation performance as well as alleviate the potential model biases [810, 811]. Second, another category of studies aim to specialize LLMs for personalized recommendation through instruction tuning [355, 812]. Specially, high-quality instruction data is key to adapt LLMs to the recommendation tasks, which can be constructed based on user-item interactions with heuristic templates. To further improve the instruction diversity, InstructRec [355] employs self-instruct technique to simulate large amounts of potential user instructions in various scenarios like product search and personalized recommendations. In addition to representing each item by its text description, there is also growing attention on extending LLM's vocabulary with semantic identifiers in recommender systems [813, 814], to incorporate collaborative semantics into LLMs.

LLM-enhanced Recommendation Models. In addition to instructing LLMs to directly provide recommendations, researchers also propose leveraging the universal knowledge encoded in LLMs to improve traditional recommender systems. Existing approaches in this line can be divided into three main categories. The first category employs LLMs to infer users' potential intention from their historical interaction data. Furthermore, traditional recommendation/search models employ the inferred intentions to improve the retrieval of relevant items [815, 816]. Additionally, several studies explore the use of LLMs as feature encoders. They employ LLMs to encode the side information of items and users (*e.g.*, item's descriptions and user's reviews), thus deriving more informative representations of users and items. These representations are then fed into traditional recommender systems as augmented input [817, 818]. As another alternative approach, several studies [819, 820] adopt a distillation-like way to transfer LLM's capacities (*e.g.*, semantic encoding) to improve traditional recommenders (*i.e.*, small models). Specially, they align the hidden states of LLMs and traditional recommendation models via joint training. After training, since only the enhanced small model will be deployed online, it can avoid the huge overhead of LLMs in online service.

LLM as Recommendation Simulator. Inspired by the recent success of autonomous AI agents [821], LLMs have been also utilized to develop recommendation simulators [822, 823] (exemplified by RecAgent [822]), showing great potential to simulate user real behaviors in recommender systems [822, 824, 825]. Specifically, to make personalized simulation, an agent will be

equipped with a profiling module that encompasses relevant identity information. Then, a memory module is introduced to store agents' past interaction experiences. During the process of simulation, agents are further prompted to conduct self-reflection based on their past experiences, to capture their underlying user preference. Most of existing recommendation simulators are conducted in a user-oriented way, without explicitly modeling the items in the interaction process. To address this, AgentCF [824] models both users and items as agents, and further facilitates collaborative reflections to simulate user-item interactions, so as to capturing the two-sided relations between users and items.

Remaining Issues. Despite these efforts, there are still several challenges to address when applying LLMs in recommender systems. First, existing studies have shown that LLM-based recommendation models in zero/few-shot settings tend to perform worse than traditional ID-based recommenders [809, 810]. This indicates that LLMs might lack an understanding of personalized user behaviors and domain-specific collaborative semantics. Although instruction tuning alleviates this issue to some extent [355, 812], it can't fully reduce the semantic gap between LLMs and recommender systems, and also suffers from high tuning costs. Furthermore, recommender systems prioritize minimizing inference latency to enhance users' experience in low-resourced environments (*e.g.*, phones), which poses a challenge to LLMs' inference speed as well as memory overhead. Therefore, it is important to explore improvement techniques, such as efficient tuning and quantization methods, to deploy LLMs efficiently and effectively in real-world recommender systems. In addition, existing LLMs have limited capacities in long context modeling, make it difficult to process the huge amount of user-item interaction data. Improved context length extension and context information utilization approaches should be developed to improve the modeling capacities of LLMs in long interaction sequences.

8.1.4 Multimodal Large Language Model

In existing literature [826, 827], multimodal models mainly refer to the models that can process and integrate information of various modalities (*e.g.*, text, image, and audio) from input, and further produce corresponding output in certain modalities. In this part, we mainly focus on the multimodal extension of LLMs by enabling the information modeling of non-textual modalities, especially the vision modality, called *multimodal large language models (MLLMs)* [800]⁴⁴. To start our discussion, we specify the input to be text-image pairs and the output to be text responses. Similar discussions can be made for other modalities, *e.g.*, language-audio models [828], which is beyond our scope here. In essence, MLLMs are developed by adapting the information from other modalities to the text modality, so as to leverage the excellent model capacities of LLMs that are learned based on world text. Typically, a MLLM comprises an image encoder for image encoding and a LLM for text generation, associated by a connection module that aligns vision and language representations. During generation, the image is first split into patches, and then transformed into patch embeddings by the image encoder and the connection module, to derive a visual representation

44. In existing work, large vision language models (LVLMs) [664] are also used to term such bimodal models that are developed based on LLMs. We use the naming of MLLMs in this part due to its wide use in existing literature.

that can be understood by the LLM. Subsequently, the patch embeddings and text embeddings are concatenated, and fed into the MLLM, allowing the language model to generate the response autoregressively. In the following, we will discuss the training, evaluation, and key points to develop capable MLLMs.

Training Process. The training process of the MLLM includes two major stages: vision-language alignment pre-training and visual instruction tuning.

- *Vision-language alignment pre-training.* To develop MLLMs, existing work mostly initializes the vision encoder and the LLM with pre-trained models [154, 155, 829]. These models retain excellent vision and language capacities, but span different semantic spaces. Thus, the goal of vision-language alignment pre-training (*i.e.*, the first-stage training) is to align the vision encoder and the LLM through end-to-end training on large-scale image-text pairs [830, 831]. However, directly tuning these two models on image-text pairs may cause the degradation of the original representation capacities. To improve the alignment performance, it is crucial to design effective training strategies and select appropriate pre-training data [832, 833]. Existing work mainly employs the following strategies for cross-modality alignment: (1) if the number of image-text pairs is not sufficiently large (*e.g.*, less than 1M), it is often suggested to only update the connection module [834]; (2) if the training data includes high-quality text corpora [835] or image-text pairs with fine-grained annotations [836], fine-tuning the LLM can be conducted to boost the performance; (3) if the number of image-text pairs is very large (*e.g.*, about 1B), fine-tuning the vision encoder is also plausible [832, 833], but the benefit remains further verification.

- *Visual instruction tuning.* After vision-language pre-training, the second-stage training, *i.e.*, visual instruction tuning, aims to improve the instruction-following and task-solving abilities of MLLMs. Generally, the input of visual instruction tuning consists of an image and a task description, and the task is to generate a corresponding text output. To boost the performance, high-quality visual instruction data is key to eliciting and enhancing the abilities of MLLMs. Therefore, most studies are dedicated to constructing various visual instruction datasets. As the basic approaches, early studies construct visual instructions by distilling from GPT-4 [154] or reformulating vision-language task datasets [156]. To enhance the quality of instruction data, recent work further proposes improved strategies by increasing the instruction diversity [837], incorporating fine-grained information (*e.g.*, coordinate of objects) into the instruction [836], or synthesizing complex visual reasoning instructions [838].

Evaluation of MLLM. After introducing the approaches to developing MLLMs, we further discuss how to effectively assess the multimodal capabilities of MLLMs from the following three aspects.

- *Evaluation perspectives.* The evaluation tasks for MLLMs can be categorized into two main types: *perception* and *cognition* tasks. Specifically, *perception* tasks aim to assess the model's abilities in understanding the basic semantics of the image content, while *cognition* tasks evaluate models with more complex tasks that require reasoning based on perception results. The perception ability is typically evaluated through classification tasks about attributes of image (*e.g.*, topic and style) and object (*e.g.*, existence and color) or OCR-related tasks, based on existing datasets or new datasets derived from existing images with annotations by humans or LLMs [839–842]. A notable

perception issue is hallucination [843], where the model's responses contain inconsistent content with the image. Among existing studies about hallucination in MLLMs [837, 844, 845], object hallucination [846] has received much research attention. To conduct a stable, robust evaluation of object hallucination, POPE [847] proposes a polling-based object probing approach for converting object recognition into a series of binary questions, and the results indicate that current MLLMs often struggle with object hallucination. *Cognition* tasks, on the other hand, require MLLMs to perform reasoning based on image perception. A common reasoning task is visual question answering (VQA), where models answer questions about images that demand reasoning about spatial relationships [848], general knowledge [849], or scene text [850]. To fully explore the capabilities of MLLMs, HallusionBench [851] collects 200 sophisticated visual dependent or supplement questions, on which even the most advanced MLLMs like LLaVA-1.5 [834] and GPT-4V [133] fail to achieve good performance.

- *Evaluation paradigms.* The responses of MLLMs can be evaluated either in a closed-ended or an open-ended manner. Traditional multimodal tasks often rely on a closed-ended evaluation framework, where the assessment is based on the exact match between the model's response and the ground-truth answer. Examples include the VQA score [852] for visual question answering tasks and the CIDEr [853] score for captioning tasks. However, MLLMs generate responses in an open-ended way, which may contain the correct answer but not exactly match the ground-truth perfectly. This discrepancy can lead to the underestimation of the model's performance in previous evaluation paradigms. To address this issue, recent approaches have incorporated humans or LLMs as evaluators [832]. For instance, MMBench [841] employs ChatGPT to align the model responses with the most relevant option in a set of multiple-choice questions. Similarly, LLaVA [854] utilizes GPT-4 for evaluating MLLMs' output, where GPT-4 takes the generated image captions and object bounding boxes as visual inputs for assessment. Such open-ended evaluation methods can improve assessment accuracy while incurring higher costs due to the involvement of humans or LLMs.

- *Evaluation benchmarks.* To facilitate a more thorough evaluation of MLLMs, various benchmarks have been developed. Part of them collect existing vision-language tasks for comprehensive evaluation. For instance, LVLM-eHub [855] aggregates 47 existing text-related visual tasks to assess six distinct capabilities of MLLMs, and Reform-Eval [856] takes this a step further by standardizing questions from existing benchmarks into a uniform format and discusses how the backbone models influence MLLMs' performance. In addition to incorporating existing tasks, several work also derives new questions annotated by humans or with the help of LLMs. MME [842] creates a dataset by pairing images from public sources with manually-collected text instructions for perception and cognition evaluations. MMBench [841] transforms these instructions into multiple-choice questions and introduces CircularEval to ensure evaluation consistency. SEED-Bench [857] further considers temporal understanding tasks and enlarges the evaluation scale to 19K multiple-choice questions with the assistance of LLMs. MM-Vet [858] presents more complex tasks to assess the integrated multimodal capabilities of MLLMs. It starts by defining six essential multimodal abilities and then creates intricate questions by combining multiple abilities. In summary, the above benchmarks collectively contribute to the

comprehensive evaluation and improved development of MLLMs.

Key Points for Improving MLLMs. To develop capable MLLMs, we continue to discuss three key points to improve the model capacities, from the perspectives of instruction data, training strategy, and safety and alignment.

- *Visual instruction data.* Extensive work [834, 859] has empirically found that both quantity and quality of visual instructions have an important impact on model performance of MLLMs. One basic way to construct visual instructions is to leverage the exceptional capability of LLMs to synthesize instructions based on text descriptions of images [854]. To further enhance the quality of instructions, one can construct fine-grained visual instructions with the help of human annotation [836, 860] or synthesize more complex data through carefully-designed prompts [838]. Despite the effectiveness of the above LLM-based approaches, one primary question emerges as to whether a LLM (*i.e.*, text generation model without training on any images) possesses the ability to generate sufficiently good visual instructions solely based on verbalized visual information (*e.g.*, captions and coordinates). Specially, existing work has also revealed that visual instructions generated by LLMs sometimes contain misinterpretations about the visual information, *e.g.*, object hallucination [847]. Therefore, it is crucial to design effective verification methods to control the quality of instruction data generated by LLMs [838]. Furthermore, it still needs more investigation about what makes good visual instructions and how visual instructions elicit specific multimodal abilities in MLLMs.

- *Model training.* Different from LLMs, MLLMs are not trained from scratch, but instead developed based on pre-trained language and vision models. Existing work employs a typical two-stage approach for training MLLMs, *i.e.*, vision-language alignment pre-training and visual instruction tuning. In essence, existing MLLMs aim to (1) preserve the inherent capabilities and parametric knowledge of LLMs as possible, and meanwhile (2) effectively adapt to multimodal tasks by leveraging the pre-trained LLMs and visual encoders. To achieve the above two goals, two typical training strategies are often employed for visual instruction tuning, either only optimizing the connection module [156] or fine-tuning both the connector module and LLM component [854]. As we can see, the former can reserve the original capacities of LLMs but likely have a weak adaptation performance, while the latter can fully adapt to multimodal tasks but suffer from the loss of original capacities of LLMs. More efforts should be made to investigate how to effectively balance the two aspects, so as to achieving improved multimodal capacities. In addition, existing MLLMs are still overly dependent on the capacities of LLMs, which pose the limits on many multimodal tasks (*e.g.*, space positioning). It will be meaningful to explore improved training approaches of language models, so that multimodal information can be also utilized in this process.

- *Safety and alignment.* Safety and alignment has been widely discussed in LLMs, which aim to regulate the behaviors of models by technical approaches [66]. This topic is also important to MLLMs. Even a highly advanced MLLM (*e.g.*, GPT-4V [133]) can be susceptible to safety issues. For example, GPT-4V might occasionally exhibit factual inaccuracies and baseless inferences about images. In some cases, it may even generate harmful content targeting specific individuals or groups [133]. Furthermore, open-sourced MLLMs are also prone to generate hallucinated response [847] and can be easily manipulated to produce

harmful content [861]. To address the aforementioned issues, some studies collect specialized visual instructions to mitigate the problem of hallucination [837]. Another alternative approach is to train a revision model to rectify hallucinated response generated by MLLMs in a post-hoc way [862]. Additionally, aligning MLLMs with RLHF can also assist MLLMs in generating responses with improved factuality [863]. Despite these efforts, existing alignment techniques for MLLMs mainly concentrate on several specific aspects (*e.g.*, hallucination), lacking a comprehensive consideration of alignment criteria. More efforts should be made to promote the research of safety and alignment for MLLMs. As a promising solution, knowledge graphs (KGs), which store enormous knowledge in the triple format, *i.e.*, $\langle head_entity, relation, tail_entity \rangle$, can be utilized to enhance the task performance of LLMs by providing precise and necessary knowledge. Generally, knowledge enhanced approaches can be expanded into other forms of structured data (*e.g.*, tables and databases) [864], while we limit our discussion to the integration of KG for improving LLMs, which are detailed in two aspects, namely retrieval-augmented LLM and synergy-augmented LLM.

8.1.5 KG-Enhanced LLM

Despite the excellent capacities, LLMs often suffer from challenges on knowledge-intensive tasks, such as the potential to generate hallucinated content [604] and the lack of domain-specific knowledge [865]. As a promising solution, knowledge graphs (KGs), which store enormous knowledge in the triple format, *i.e.*, $\langle head_entity, relation, tail_entity \rangle$, can be utilized to enhance the task performance of LLMs by providing precise and necessary knowledge. Generally, knowledge enhanced approaches can be expanded into other forms of structured data (*e.g.*, tables and databases) [864], while we limit our discussion to the integration of KG for improving LLMs, which are detailed in two aspects, namely retrieval-augmented LLM and synergy-augmented LLM.

Retrieval-Augmented LLM. Due to the huge amount of fact records in a KG, existing work typically adopts a retrieval model to first obtain a relatively small subgraph from KG, and then leverages it to enhance LLMs by enriching the relevant knowledge. Before the advent of LLMs, the retrieved subgraphs are often supplemented into training data, injecting knowledge information into PLMs via parameter learning [866–868]. In contrast, to leverage the retrieved knowledge, LLMs mainly incorporate it as part of the prompt, without parameter update. To implement this approach, there are two main technical problems, *i.e.*, how to retrieve relevant knowledge from KGs and how to make better use of the structured data by LLMs. For the first issue (*i.e.*, retrieving relevant knowledge), a typical approach is to train a small language model (*e.g.*, RoBERTa) to identify question-related fact triples [869]. To further improve the retrieval performance, several studies also propose an iterative reading-then-reasoning framework, enabling the LLM to interact with the KG multiple times and acquire the required knowledge in a more accurate way [451]. For the second issue (*i.e.*, utilizing retrieved knowledge), a straightforward approach is to serialize the retrieved subgraph and craft specific prompts to include it as the input of LLMs [468, 653]. However, due to the loss of structured information in knowledge serialization, LLMs cannot fully capture the structural semantics conveyed by original KGs. To address this issue, several model-based approaches train a specialized language model (*e.g.*, T5) to transform the subgraph into

the natural language text [870]. To guarantee the transformation accuracy, it relies on sufficient training pairs (often unsupervised constructed) [871] and excellent model capability [872].

Synergy-Augmented LLM. To solve complex tasks (e.g., multi-hop question answering [658]), it often requires LLMs to query a KG multiple times, following a systematic solution plan. We call such a multi-turn interaction approach to enhancing LLM *synergy-augmented LLM*. To better synergize the LLM and KG in a complementary manner, recent studies propose to decompose the complex task into multiple sub-goals and iteratively solve each one by leveraging the necessary knowledge from KG [451, 873, 874]. In this process, the LLM can be regarded as an autonomous agent (detailed in Section 9.2), which automatically generates the plan and executes it through interaction with the KG environment [873]. Specially, the mainstream approaches typically start by enumerating the candidates using the available knowledge information at the current step, and then retrieve the most appropriate candidates for the next step according to the question [873, 874]. By iterating the above two steps, LLMs can gradually collect relevant evidence [873, 874], and finally approach the correct solution. Despite the effectiveness, enumeration of the candidates over the KG would lead to a vast search space [875]. To address it, StructGPT [451] proposes a more efficient way to access knowledge information using the specialized interfaces for KGs. Specifically, it carefully designs the specialized interfaces according to the common data operations on KG (e.g., relation extraction and triple extraction), to ensure efficient and accurate data extraction. In this way, LLMs can be instructed to better manipulate and process the structural information of KGs, thus achieving improved task performance.

Future Directions. Besides the above approaches, there are several promising directions for KG-enhanced LLM remaining underexplored. First, due to the variety of structured data, it is still difficult for LLMs to directly leverage various kinds of knowledge sources, e.g., domain-specific KGs. Therefore, it is essential to explore the unified way to manipulate and utilize different knowledge sources by LLMs. As a potential solution, it is promising to develop effective approaches to help LLMs comprehend and make use of the access interfaces provided by specific knowledge sources to acquire precise knowledge [451], while more efforts should be made to investigate how to adapt to the data variety in a cost-effective way. Second, with the evolution of real-world information, the knowledge stored in LLMs may become outdated or incorrect. It is necessary to explore how to synchronize the updated knowledge into LLMs through a cost-effective manner [876, 877]. Third, it is promising to investigate the use of factual information from KG to align LLMs in generating more faithful content [878, 879], which can help reduce the hallucination of LLMs.

In addition to exploring KG-enhanced LLMs, it is also meaningful to leverage LLMs to improve the tasks on the KG side (i.e., LLM4KG) [865, 880]. A typical example is that LLMs can help supplement or construct the KG. We omit the discussion of this part, since it is beyond our scope.

8.1.6 LLM for Evaluation

While human evaluation can generally offer reliable quality assessment, it is also often hindered by high annotation costs, significant time requirements, and annotation inconsistencies [881].

In contrast, automatic evaluation can be employed as a scalable alternative to human evaluation. Traditional automatic evaluations have relied on reference-based metrics (e.g., BLEU and ROUGE). Recently, with the emergence of LLMs as general task solvers highlights their potential as automatic evaluators [649, 729], making it promising to conduct LLM based evaluation. In the following part, we will introduce the recent progress on LLM for evaluation, including evaluation formats, methods, meta-evaluation, and the remaining issues.

Evaluation Formats. Depending on the type of evaluation outcome, the evaluation format can be categorized into *score-based evaluation* and *language-based evaluation*. Score-based evaluation employs measurable metrics to assign quality scores (e.g., ratings or rankings) for evaluated texts. A prevalent way is to conduct pairwise comparison, where LLMs are used to determine the partial order relation of candidate texts following specific guidelines [352, 649, 729], which greatly simplifies the evaluation task. However, it may face the inefficiency issue when scaling up the number of candidates [729]. When high-quality reference texts are available during evaluation, LLMs can be instructed to score texts under the guidance provided by references [718, 729, 730]. On the other hand, language-based evaluation focuses on generating critiques and suggestions, offering qualitative explanation beyond simple quantitative scoring [369, 377, 882, 883]. It is particularly useful for gathering language feedback signals for human alignment tuning [369, 882]. Furthermore, it can evolve into a multi-turn interaction framework, where LLM-based evaluators provide natural language feedback to existing solutions from task solvers [884]. This framework evaluates the ability of LLMs to leverage language feedback for refining self-generated solutions.

Evaluation Methods. A common method for LLM-based evaluation involves prompting LLMs with specific instructions. To further improve the quality of LLM-based evaluation, recent work proposes to prompt LLMs with varied contexts to generate diverse evaluation feedback. These contexts vary in aspects such as the candidate order [649, 729], evaluation perspectives [885, 886] (e.g., relevance, clarity, originality), and evaluation explanation [649]. The generated multiple evaluation feedbacks are then aggregated to produce a final evaluation result, which makes the evaluation process less prone to biases from individual feedback and allows for a more thorough evaluation by covering a wider range of evaluation aspects. To further improve the quality of the single-model evaluation, recent studies also develop multi-agent collaboration frameworks [886–888] or fine-tune LLMs as specified evaluators [369, 377, 882, 883, 889]. In a multi-model collaboration mode, different LLMs evaluate the candidates by engaging in discussions to align preferences and reach a consensus [887, 888]. This method helps reduce the potential biases in individual models through the consensus reached by multiple agents. Another approach to improving single-model evaluation is to specialize LLMs as scorers or critics through fine-tuning [369, 377, 882, 883, 889]. This process involves creating datasets annotated with preferences and feedback from humans or proficient LLMs. These datasets are then used to train evaluation-oriented models, enabling them to generate pairwise preference or language feedback. The specialized LLM evaluators demonstrate competitive performance with fewer parameters [377, 883, 889].

Meta-Evaluation. To effectively assess the quality of LLM-based evaluators, meta-evaluation benchmarks have been introduced, for gauging the agreement with human preferences and the fairness of the evaluations made by LLMs [649, 729, 886, 890, 891]. As a representative benchmark, MT-Bench [729] evaluates the agreement between LLMs and human judgments, demonstrating that GPT-4 aligns closely with human preferences in no-tie comparisons on 80 multi-turn questions. In addition, to address potential biases arising from subjective human evaluations, LLMBar [890] manually designs outputs that are objectively worse but superficially appealing, which could mislead evaluators. The evaluation results reveal that even the most advanced LLMs still fall short of human-level evaluation in the challenging setting.

Remaining Issues. As discussed in Section 7.1.1, recent studies demonstrate that LLM-based evaluators expose multiple types of bias, such as order bias, self-preference bias, and length bias [649, 729]. Although some biases can be mitigated through methods like multi-path ensemble or multi-agent collaboration, they remain inherent to LLM-based evaluators. Consequently, addressing these biases intrinsically within the models continues to be a challenging issue. In addition, recent work has revealed that LLMs may be incapable of understanding the self-generated content, exhibiting a weaker understanding capacity compared to their generation capabilities [892]. Even the most advanced LLMs still struggle identifying their reasoning or factual errors without external feedback [893, 894]. Consequently, current LLM-based evaluators might not be adequate for evaluating top-tier LLMs or complex tasks. This underscores the importance of improvement approaches for LLM-based evaluators, especially for evaluating capable LLMs and complex tasks demanding sophisticated reasoning, planning, and domain-specific knowledge.

8.2 LLM for Specific Domains

In this part, we discuss the applications of LLMs on several representative domains, including healthcare, education, law, finance, and scientific research assistance.

Healthcare is a vital application field closely related to human life. Ever since the advent of ChatGPT, a number of studies have applied ChatGPT or other LLMs to the medical domain. It has been shown that LLMs are capable of handling a variety of healthcare tasks, e.g., biology information extraction [765], medical advice consultation [895], mental health analysis [896], and report simplification [897]. As the major technical approach, researchers typically design specific prompts or instructions to guide LLMs to perform a wide range of medical tasks. To further harness the power of LLMs in the healthcare domain, researchers propose to develop healthcare-related LLMs [354, 898, 899]. Specifically, the Med-PaLM models [354, 898] achieves expert-level performance on the United States Medical Licensing Examination (USMLE), and earns greater approval from physicians in answering consumer’s medical questions. However, LLMs may fabricate medical misinformation [897, 900], e.g., misinterpreting medical terms and suggesting advice inconsistent with medical guidelines. In addition, it would also raise privacy concerns to upload the health information of patients [765] into a commercial server that support the LLM.

Education is also an important application domain where LLMs potentially exert significant influence. Existing work has found

that LLMs can achieve student-level performance on standardized tests [46] in a variety of subjects of mathematics (e.g., physics, computer science) on both multiple-choice and free-response problems. In addition, empirical studies have shown that LLMs can serve as writing or reading assistant for education [901, 902]. A recent study [902] reveals that ChatGPT is capable of generating logically consistent answers across disciplines, balancing both depth and breadth. Another quantitative analysis [901] shows that students utilizing ChatGPT (either keeping or refining the results from LLMs as their own answers) perform better than average students in some courses from the computer security field. Recently, several perspective papers [903, 904] also explore various application scenarios of LLMs in classroom teaching, such as teacher-student collaboration, personalized learning, and assessment automation. However, the application of LLMs in education may lead to a series of practical issues, e.g., plagiarism, potential bias in AI-generated content, overreliance on LLMs, and inequitable access for non-English speaking individuals [905].

Law is a specialized domain that is built on professional domain knowledge. Recently, a number of studies have applied LLMs to solve various legal tasks, e.g., legal document analysis [906], legal judgment prediction [907], and legal document writing [908]. A recent study [909] has found that LLMs exhibit powerful abilities of legal interpretation and reasoning. Moreover, the latest GPT-4 model achieves a top 10% score in a simulated bar exam compared with human test-takers [46]. To further improve the performance of LLMs in the law domain, specially designed legal prompt engineering are employed to yield advanced performance in long legal document comprehension and complex legal reasoning [910, 911]. To summarize the progress, LLMs can act as helpful assistants to legal profession. Despite the progress, the use of LLMs in law raises concerns about legal challenges, including copyright issues [912], personal information leakage [913], or bias and discrimination [914].

Finance is an important field where LLMs have promising application prospects. LLMs have been employed on various finance related tasks, such as numerical claim detection [915], financial sentiment analysis [916], financial named entity recognition [917], and financial reasoning [918]. Despite the competitive zero-shot performance exhibited by general-purpose LLMs in the finance tasks, they still underperform domain-specific PLMs containing million-scale parameters [915]. To leverage the scaling effect of LLMs, researchers collect large-scale finance corpora for continually pre-training LLMs (e.g., BloombergGPT [358], XuanYuan 2.0 [919], and FinGPT [920]). BloombergGPT has demonstrated remarkable performance across a diverse range of financial tasks while maintaining competitive performance in general-purpose tasks [358]. Nevertheless, it is imperative to consider the potential risks in the application of LLMs in finance, as the generation of inaccurate or harmful content by LLMs could have significant adverse implications for financial markets [358]. Therefore, it needs more strict reviewing and monitoring on the use of LLMs in the financial field.

Scientific research is another promising field that LLMs can empower the development progress. Prior research demonstrates the effectiveness of LLMs in handling knowledge-intensive scientific tasks (e.g., PubMedQA [921], BioASQ [922]), especially for LLMs that are trained on scientific-related corpora [35, 218, 923].

Given the excellent general abilities and broad scientific knowledge, LLMs hold significant potential as helpful assistants across various stages of the scientific research pipeline [924]. First, during the literature survey stage, LLMs can help conduct a comprehensive overview of the progress in a specific research field [925, 926]. Second, during the research idea generation stage, LLMs demonstrate the ability to generate intriguing scientific hypotheses [927]. Third, during the data analysis stage, LLMs can be employed to conduct automatic approaches to analyzing the data characteristics, including data exploration, visualization, and deriving analytical conclusions [928, 929]. Fourth, during the paper writing stage, researchers can also benefit from the assistance of LLMs in scientific writing [930, 931], in which LLMs can offer valuable support for scientific writing through diverse means, such as summarizing the existing content and polishing the writing [932]. In addition, LLMs can aid in the automated paper review process, encompassing tasks such as error detection, checklist verification, and candidate ranking [933]. Despite these advances, there is much room for improving the capacities of LLMs to serve as helpful, trustworthy scientific assistants, to both increase the quality of the generated scientific content and reduce the harmful hallucinations.

Summary. In addition to the aforementioned work, the applications of LLMs have been also discussed in several other domains. For instance, in the psychology domain, some recent work has studied the human-like characteristics of LLMs, such as self-awareness, theory of mind (ToM), and affective computing [934, 935]. In particular, an empirical evaluation of ToM conducted on two classic false-belief tasks speculates that LLMs may have ToM-like abilities since the model in the GPT-3.5 series achieves comparable performance with nine-year-old children in ToM task [934]. In addition, another line of work has investigated applying LLMs into the software development domain, e.g., code suggestion [936], code summarization [937], and automated program repair [938]. To summarize, to assist humans by LLMs in real-world tasks has become a significant area of research. However, it also presents challenges. Ensuring the accuracy of LLM-generated content, addressing biases, and maintaining user privacy and data security are crucial considerations when applying LLMs to real-world scenarios.

9 Advanced Topics

In this section, we focus on discussing several advanced topics that have attracted extensive attention in the research community, and these topics are related to challenging technical issues that largely limit LLM’s capacity. Next, we will introduce these issues and discuss how to address them with feasible approaches.

9.1 Long Context Modeling

In real-world application scenarios, there are increasing demands for long context modeling capacities of LLMs, especially for text file processing (e.g., information parsing, extraction, and summarization). Many mainstream LLMs have provided support for long context window. To enhance the long context modeling abilities, there are generally two widely used approaches, namely scaling position embeddings and adapting context window. Next, we introduce the two approaches in detail.

9.1.1 Scaling Position Embeddings

Transformer-based LLMs can learn effective position embeddings within the maximum training length. When adapting LLMs to language tasks beyond the maximum training length, it is necessary to scale to larger position indices. Specially, some position embedding methods have been shown to possess a certain degree of ability to generalize to text beyond the training length, which is termed as *extrapolation capability*, including T5 bias [82], ALiBi [283], xPos [296] and even NoPE [939]. However, as one of the mainstream position embedding methods, RoPE exhibits limited extrapolation ability in empirical studies [259]. In the following, we discuss several methods that adapt RoPE to longer texts.

- **Direct model fine-tuning.** To adapt LLMs to a long context window, a straightforward approach is to directly fine-tune the models on long texts with the target length. The context extension can be scheduled with gradually increased lengths in a multi-stage manner (e.g., 2K $\rightarrow$ 8K $\rightarrow$ 32K). To conduct effective extension, it often requires specially prepared long text data for training (Section 9.1.3), and data quality plays a critical role in improving LLM’s long context capacities [940]. However, such a direct fine-tuning approach tends to be inherently slow when adapting LLMs for long texts [259].

- **Position interpolation.** This method downscales the position indices within the original context window, to avoid out-of-distribution rotation angles during pre-training [259, 941]. Specifically, this approach multiplies all position indices by a scaling coefficient L/L' ($L < L'$), where L and L' denote the original and target context window length, respectively. Experimental results [259] have shown that this method can extend the context window effectively and efficiently, compared to the above approach of direct model fine-tuning. However, it is worth noting that this technique may have an adverse impact on the model’s performance when handling normal texts within the original context window [259, 942].

- **Position truncation.** To mitigate the challenges posed by out-of-distribution rotation angles, another practical approach is to truncate longer relative positions to satisfy the requirement of the maximum training length. ReRoPE and LeakyReRoPE [943] introduce a pre-defined window length for truncation, which is smaller than the maximum training length. Specifically, position indices within this pre-defined window would be retained, while those indices beyond the window are either truncated to the pre-defined window length or interpolated to align with the maximum training length. This strategy can preserve the attention mechanism with the neighbor tokens (within the window length), and further enhance the extrapolation capacity. However, this approach needs to compute the attention matrices twice, accommodating additional computational costs.

- **Base modification.** Since LLMs are usually trained with a pre-set maximum training length, wavelengths in certain dimensions of RoPE may exceed the training length for longer text [295], on which language models may not be sufficiently trained, i.e., training data can’t cover a complete rotation cycle. Thus, when processing long text, some rotation angles for certain dimensions would never be seen in the training phase [351]. Formally, given a fixed rotation angle $t \cdot \theta_i$, a smaller basis θ_i allows for a greater distance t , i.e., enabling the modeling of longer texts [254, 295, 940]. According to the formula $\theta_i = b^{-2(i-1)/d}$ in Equation 4, decreasing the basis can be achieved by increasing the value of the base. In addition, decreasing the base can also

help re-scale the wavelengths of all dimensions below the training length, while it often needs continual pre-training to adapt the LLMs to long context windows [351]. A recent study [351] has empirically compared these two base modification methods, and shown that decreasing the base demonstrates better extrapolation performance, while increasing the base performs better within the training length.

- *Basis truncation.* Similar to the base modification, the truncation of the basis also concentrates on dealing with the singular dimensions with wavelengths exceeding the training length [944]. According to the definition $\lambda_i = 2\pi/\theta_i$ in Equation 5, the dimension with a large wavelength λ_i has a small basis θ_i accordingly. Based on this observation, this approach first defines a basis range $[a, c]$. Given the basis range, the value of basis is modified according to the following ways: (1) when $\theta_i \geq c$, the value is retained, (2) when $\theta_i \leq a$, the value is set to zero, and (3) when $a < \theta_i < c$, the value is truncated to a fixed small value. Via basis truncation, the out-of-distribution rotation angles can be avoided at larger position indices. However, this approach does not perform very well at long context tasks [944].

9.1.2 Adapting Context Window

Since Transformer-based LLMs have limited context windows, they can not directly integrate or utilize the entire information of the long sequences exceeding the context window. To alleviate the limitation, several methods have been proposed to adapt LLMs to long context, as discussed below.

- *Parallel context window.* Inspired by fusion-in-decoder [945], parallel context window methods [424, 946] adopt a divide-and-conquer strategy to process input text. Specially, it divides the input text into multiple segments, each independently encoded with shared position embeddings. At the generation stage, the attention masks are modified to make that subsequent tokens can access to previous tokens in each segment. Nevertheless, this method cannot distinguish the order of different segments, resulting in a limited model capacity on certain tasks.

- *Λ -shaped context window.* Some prior work has revealed that LLMs tend to allocate greater attention weights to the starting and nearest tokens among all previous tokens [947, 948], and it potentially results in the “lost in the middle” phenomenon [949]. Based on this observation, LM-Infinite [950] and StreamingLLM [948] propose to employ a “ Λ -shaped” attention mask, which selectively preserves the initial tokens and the nearest tokens that each query can attend to and then discards any tokens beyond this scope. Experiments demonstrate that this method can facilitate extra-long text generation with a fixed memory [948]. However, it may struggle to model the long-range dependency in the context window, since it cannot effectively utilize the information from the discarded tokens [948].

- *Token selection.* It has been shown that a relatively small subset of tokens can effectively capture the majority of attention patterns in a Transformer [951], e.g., the top- k attention scores can well approximate the original full attention. Therefore, a number of studies propose different methods to select the most relevant tokens from token-level or block-level memory units for generation. Token-level selection methods store the past keys in external memory and utilize a k -NN search method to retrieve the k most relevant tokens for generation [257, 951, 952]. For a decoder model, it typically employs one certain layer to access these top- k external tokens, while still adopting the normal

context window in the rest layers [257, 952]. Block-level selection methods [953, 954] first segment the long sequence into blocks with the same length and represent each block into several key vectors for retrieval. Then, the most relevant blocks to the query as well as the neighbor and initial blocks will be selected for attention computations. Unlike token-level selection methods, block-level selection methods typically retrieve different tokens with specific heads.

9.1.3 Long Text Data

To further enhance the long context modeling capacity, it typically requires continual pre-training with specially curated long text data. Next, we discuss how to prepare the long text data from the two aspects of quantity and quality.

- *Quantity effect.* Different from the pre-training phase that requires vast amounts of data, a small amount of long-text data for continual pre-training is sufficient for context window extension [259]. Several studies show that LLMs have obtained the capability of utilizing distant information via large-scale pre-training data, and thus it only needs to adapt for extended context windows during continual pre-training [955]. Typically, it has shown that LLaMA-2-7B or LLaMA-2-13B can achieve a context window length of over 100K tokens and effective context utilization [955] with the training on several billion tokens. However, the ability to handle short text of LLMs may be affected to some extent [259].

- *Quality effect.* In addition to the quantity, the quality of long text data is essential to long context modeling for LLMs. For instance, LongWanjuan [956] categorize long texts into holistic, aggregated, and chaotic long texts based on three metrics, i.e., coherence, cohesion, and complexity, and they show that removing chaotic data and keeping coherent and cohesive data are useful to enhance the long text modeling capacities of LLMs. Further, up-sampling cohesive data can lead to further improvement. In addition, when preparing long text data, data mixture should be carefully adjusted for avoiding large distribution drift with the original pre-training data.

In addition to the studies based on vanilla Transformer, there are a surge of Transformer variants with efficient attentions and other efficient architectures, aiming to alleviate the high computational costs for modeling long texts. These studies are discussed in Section 4.2.1 and Section 4.2.2. Furthermore, context compression and prompting techniques (e.g., iterative reasoning [957]) have also been proven to be a viable strategy for handling long text tasks [957–960], without the need of model adaption.

9.2 LLM-empowered Agent

The research on agents in AI aims to develop entities that can perceive the environment, make decisions, and take actions to achieve specific goals [961]. However, traditional agents are often limited to heuristic rules or specific environments, which constrain their generalization to open-domain scenarios [962]. Given that LLMs possess excellent capacities in solving complex tasks, they have rapidly emerged as promising solutions for serving as the core computation unit of agents [821]. In this part, we will first introduce the framework for LLM-based agents, then explore their applications, and finally discuss the future directions.

9.2.1 Overall Framework.

Next, we first detail the key components of an LLM-based agent and then present the typical workflow.

Components. Typically, there are three main components in an LLM-based agent: *memory*, *planning*⁴⁵, and *execution*. Specifically, the *memory* component aims to store the information perceived from the environment and can be utilized to support decision-making. In particular, LLM-based agents usually maintain information in both short-term memory and long-term memory with the operations of reading and writing. Short-term memory usually refers to the internal context window of LLMs (i.e., input), where LLMs can read and write through actions like reasoning [963]. While long-term memory can be mapped to the external storage like vector databases [539], where LLMs can read through retrieval and write with reflection [688]. Specially, profiles are usually implemented with long-term memory, which is an important feature for an agent that specifies its role and function [821]. The *planning* component is responsible for generating the action plan based on the information from the memory component. In data format, the plan usually takes the form of text-based instructions [434] or code-based programs [436]. To generate it, LLM-based agents will first propose several candidates and then select a more suitable one among them [429]. The initial plan can be further refined with execution feedback from the environment [530]. The *execution* component is in charge of carrying out the plan from the planning component, which can be fulfilled by the internal LLM [434] or external tools [963].

Workflow. With the three components mentioned above, a typical workflow of an LLM-based agent is as follows. First, it receives information from the environment and writes it into short-term memory. Then, the agent processes the newly received information in the short-term memory. Such a process can be enhanced with information retrieved from long-term memory. Subsequently, the planning component utilizes the processed information from short-term memory to generate the next plan. Finally, the execution component carries out the plan generated from the planning component, which can be further assisted with external tools. By repeating the aforementioned process, the LLM-based agent can autonomously adjust its behavior in response to feedback from the environment and ultimately achieve its goal. Once LLM-based agents receive user requests or are assigned goals, they follow the above workflow to accomplish tasks through multi-turn interactions with the environment.

To summarize, in an LLM-based agent, the LLM serves as the core computation unit and is equipped with components including *memory*, *planning*, and *execution*. These components are integrated in a systematic way under the control of the LLM during interactions with the environment. For more details, the readers might refer to the comprehensive survey for LLM-based AI agents [821].

9.2.2 Applications

Recently, LLM-based agents have shown great potential in autonomously solving complex tasks, making it feasible to rapidly develop capable applications for specific domains or tasks. In this

section, we will discuss the applications in single-agent and multi-agent scenarios.

Single-agent based Applications. Applications based on a single-agent mode mainly aim to develop capable task solvers that can autonomously complete user requests. A large number of single-agent projects have been developed, which focus on general-purpose task solving. As a representative project, AutoGPT [536] empowers LLMs with long/short-term memory management and external tools like search engines. In order to autonomously address a user request, AutoGPT understands the request with knowledge from its memory and actions like reasoning, decomposes it into a detailed plan, executes the plan step-by-step with the assistance of tools, and refines the rest plan based on feedback from the environment. Such an iterative process continues until the user request is successfully resolved. Other similar projects include GPT-Engineer [964] and XAgent [965]. In addition, there is also some work that aims to develop autonomous agents for specific domains, such as WebGPT [81] for the web-browsing environment, ProgPrompt [532] for the real-life environment, and Voyager [699] for the Minecraft environment.

Multi-agent based Applications. Different from single-agent systems where agents work independently, multi-agent systems work in collaboration to unleash collective intelligence. Typically, multiple agents can be instantiated from the same or different LLMs, each with their respective roles and functions. According to the coordinating strategies among these agents, multi-agent systems can be divided into two categories: cooperation-based and competition-based. In the cooperation-based mode, to share information and seek collaborative actions among agents, various communication protocols have been proposed, including free-form dialogue [966], structured document [967], and data embedding [968]. Based on the communication protocol, agents can be effectively organized for downstream applications, such as software engineering [967], user behavior analysis [822, 824], and society simulation [535]. As a representative project, LangChain⁴⁶ is a framework for developing multi-agent based applications powered by LLMs. It enables users to deploy different roles of LLM-based agents and utilize them to solve tasks via working in collaboration. In addition, other similar frameworks, such as AgentVerse [969] and AutoGen [970], can also be utilized for developing multi-agent collaborative systems. In the competition-based mode, debate serves as one of the popular communication protocols to foster divergent thinking and elicit valuable external feedback among agents. Such a way is beneficial for domains that demand precise decision-making and accurate responses, such as mathematical reasoning [971] and evaluation [734].

9.2.3 Discussion

Despite the huge success, there still remain several technical challenges that limit the development and application of LLM-based agents. In this part, we discuss the remaining challenges from the perspective of computational burden, human alignment, complex capability extension, and robustness.

Computational Costs. With the ever-increasing capabilities of LLMs [821], their performance on agent applications demonstrate promising performance. However, it also introduces significant issues in terms of efficiency due to the high computational

45. Section 6.4 introduces planning as a utilization approach for LLMs, while in this section, we describe its utilization as a functional component in LLM-based agents.

46. <https://www.langchain.com/>

demands and intricate interaction mechanisms involved. Furthermore, in multi-agent systems with numerous LLM instances, as the number of agents increases, this issue would be more severe, since the communication network within multi-agent systems also becomes increasingly complex. Therefore, more effective and efficient communication protocols and architectures are essential to support the heightened coordination demands among agents.

Alignment with Human Sociality. LLM-based agents can be conceptualized as individual entities, with the emergence of sociability resulting from the interaction among these agents. Autonomous agents often assume specific roles such as coders or researchers, making role-playing a vital capability for agents to solve downstream tasks [972]. However, LLMs, typically trained on web corpora, face difficulties in accurately mimicking roles that are infrequently discussed online or are emergent. They also lack self-awareness in conversational scenarios due to inadequate modeling of human cognitive psychology. Thus, it is imperative to develop improved agent technique, including both training methods and architectures, to better align LLMs with human preferences and enhance their role-playing abilities.

Capability Extension. LLM-based agents, similar to humans, require advanced capabilities (e.g., tool learning) to fulfill complex functions or tasks, which might be beyond their capacity scope. To address this issue, tool use has become a widely-used approach to enhancing LLMs' capacities in various complex tasks. For example, when answering informative user questions, they use search engines to retrieve information from the internet. However, the quality and quantity of existing available tools impose limitations on their accessibility and comprehensiveness. And it would become more difficult for LLM-based agents to use such limited tools when interacting with dynamic and changing environments. In addition, as the scale of tools expands, the compatibility and extensibility between the agents and tools must be further improved to facilitate complex task resolution.

Robustness and Trustworthiness. The deployment of LLM-based agent systems necessitates robustness and trustworthiness [973]. The system should be resilient against adversarial inputs from various modalities such as text, image, or audio. Incorporating existing techniques like adversarial training, data augmentation, and sample detection to increase sensitivity to aggressive information in the input can fortify the system's security. Concurrently, it is challenging to ensure the credibility of LLM-based agents given the severe hallucination issues inherently rooted in LLMs. While existing methods such as constrained decoding during inference and external knowledge integration can mitigate these issues to some extent [974], further exploration of efficient and effective alignment methods is necessary to develop reliable agent systems.

9.3 Analysis and Optimization for Model Training

In Section 4.3, we have introduced basic techniques for training LLMs. As the scale of model parameters and data continues to expand, efficiently training larger models with limited computational resources has become a critical technical challenge in the development of LLMs. This challenge primarily encompasses two technical issues: firstly, how to optimize memory usage when loading and processing models across GPU clusters, and secondly, how to maintain or improve training efficiency as models scale.

Next, we will conduct quantitative analyses and introduce advanced training techniques addressing the two aforementioned issues.

9.3.1 Estimation of Training Memory Consumption

In this part, we will first estimate the GPU memory consumption for training LLMs.

Model States Cost. Model states often occupy the majority of memory during training, typically consisting of model parameters, gradients, and optimizer states. As introduced in Section 4.3.2, mixed precision training has been widely utilized in LLM training. For a model containing P parameters, both the model parameters and their gradients are typically stored as 16-bit floating-point numbers, requiring a total storage of $4P$ bytes ($2P$ for the parameters and $2P$ for the gradients). When using optimizers such as Adam [318] or AdamW [975], an additional set of 32-bit floating-point numbers are needed to store the optimizer states, including the copy of model parameters, gradient momenta, and gradient variances, which leads to a total storage of $12P$ bytes ($4P$ each for each of these components). Consequently, the total memory required for storing the model states during training is $16P$ bytes. For instance, training LLaMA-7B ($P \approx 6.7 \times 10^9$) requires around 100GB memory to store the model states alone.

Activations Cost. Activations are the intermediate states that require to be stored in the forward pass for gradient computation during backpropagation. For example, for a binary operation $\mathbf{Y} = \mathbf{W}\mathbf{X}$, calculating the gradient $\frac{\partial \mathbf{Y}}{\partial \mathbf{W}}$ necessitates the input $\mathbf{X}$, which should be preserved during the forward pass. In Table 18, we list the estimation of the activation memory consumption for different components within the Transformer model. Take LLaMA-7B ($V = 32,000$, $L = 32$, $H = 4,096$, $H' = 11,008$, $N = 32$) as an example, it would take 16GB memory to store activations per device under the setting $B = 1$, $T = 2,048$.

TABLE 18: The activation memory consumption of each computation within the LLaMA model based on research work [976]. We denote batch size by B , sequence length by T , the vocabulary size by V , the number of head in the attention module by N , the dimension of each head by D , the hidden size by H ($H = ND$), and the intermediate size inside FFN by H' . Equations ①-⑩ are layer-wise and need to be multiplied by the number of the layers L when computing the total consumption.

Equations	Activation consumption
① $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}\mathbf{W}^{Q,K,V}$	store $\mathbf{X}$ with size $2BTH$
② $\mathbf{Q}, \mathbf{K} = \text{RoPE}(\mathbf{Q}, \mathbf{K})$	store $\mathbf{Q}$ and $\mathbf{K}$ with size $4BTH$
③ $\mathbf{O} = \text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$	store $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ with size $6TH$ and results of softmax with size $2BT^2N$
④ $\mathbf{X} = \mathbf{O}\mathbf{W}^O$	store $\mathbf{O}$ with size $2BTH$
⑤ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	store $\mathbf{X}$ with size $2BTH$
⑥ $\mathbf{G}, \mathbf{U} = \mathbf{X}[\mathbf{W}^G, \mathbf{W}^U]$	store $\mathbf{X}$ with size $2BTH$
⑦ $\mathbf{D} = \text{Swish}(\mathbf{G}) \cdot \mathbf{U}$	store $\mathbf{G}$ and $\mathbf{U}$ with size $4BTH'$
⑧ $\mathbf{X} = \mathbf{D}\mathbf{W}^D$	store $\mathbf{D}$ with size $2BTH'$
⑨ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	store $\mathbf{X}$ with size $2BTH$
⑩ $\text{CE}(\text{softmax}(\mathbf{X}\mathbf{W}^L))$	store $\mathbf{X}$ with size $2BTH$ and results of softmax with size $4BTV$

Other Memory Cost. In addition to the main factors affecting GPU memory consumption discussed above, the memory usage also includes the following aspects:

- *Deep learning frameworks.* The PyTorch framework requires approximately 1GB of GPU memory when loading its core functions. This is the essential overhead for the framework to operate.
- *Distributed frameworks.* When utilizing distributed training frameworks (e.g., DeepSpeed), its GPU memory usage can fluctuate between 1GB and 4GB. The exact amount depends on the level of optimization and the hyper-parameter settings. This portion of the memory is primarily used to optimize memory management and communication efficiency during the training process.
- *Intermediate results and memory fragmentation.* Besides the activations, there also exist intermediate results that will affect the peak memory consumption. Take the computation of the softmax function in Equation ⑩ as an example, the implementation of the Transformers library requires an additional 8BTB bytes of memory, as it needs to store two additional copies of the 32-bit input (4BTB bytes each). Moreover, during the training process, memory fragmentation occurs due to the non-contiguous allocation and release of memory, typically leading to an additional 0.5GB to 1GB of memory consumption.

9.3.2 Memory Optimization Methods

Based on the aforementioned analysis, we will next introduce several typical methods for optimizing the memory usage for training LLMs.

Gradient Checkpointing. Gradient checkpointing [329], also known as activation recomputation, is a technique used to optimize memory usage during backpropagation. Specifically, the activations need to be retained during the forward pass. However, storing all activation values for each layer requires a significant amount of memory resources (detailed in Table 18). To reduce the memory cost, gradient checkpointing retains only a subset of the activations during the forward pass and recomputes these values during the backward pass to save memory, albeit with additional computational overhead. In implementation, gradient checkpointing typically involves storing the input of each Transformer layer and recomputing the corresponding activation values during backpropagation.

ZeRO. Zero redundancy optimizer (ZeRO) [977] technique, proposed by the DeepSpeed library, focuses on alleviating the issue of memory redundancy in data parallelism. As mentioned in Section 4.3.2, data parallelism requires each GPU to store the same copy of the model states, resulting in a memory consumption of $16P$ bytes per GPU. A direct side effect of data parallelism is that it memory redundancy issues, since not all of the above data is necessary to be retained on each GPU. To resolve it, the ZeRO technique aims to retain only a fraction of data on each GPU, while the rest data can be obtained from other GPUs when required. Specifically, ZeRO provides three strategies, depending on how the three parts of the data are stored, namely optimizer state partitioning (ZeRO-1), gradient partitioning (ZeRO-2), and parameter partitioning (ZeRO-3). Empirical results indicate that the first two strategies do not increase the communication overhead, and the third solution increases about 50% communication overhead but saves memory proportional to the number of GPUs. PyTorch has implemented

a similar technique as ZeRO, called fully sharded data parallel (FSDP) [330].

Offload. In GPU-limited environments, DeepSpeed has proposed the offload technique [978], which can significantly reduce the GPU memory required for training by offloading part of the model states and computational overhead to CPU memory. Specifically, gradients and optimizer states would be offloaded to CPU memory, with only the model parameters kept on GPU. The computationally intensive forward and backward propagation still need to be performed on GPU to ensure efficiency, while parameter update, which requires relatively fewer computations, are executed on CPU to reduce GPU memory overhead. Furthermore, Infinity [979] allows training models that exceed the GPU memory limits by utilizing high-speed disk storage (e.g., NVMe).

9.3.3 Efficiency Optimization Methods

In addition to memory-saving techniques, it is also crucial to maintain computational throughput as the model scales. In what follows, we will describe two representative efficiency optimization methods.

FlashAttention. FlashAttention [303, 980] is an optimization method for the attention mechanism that significantly reduces the memory transfer during attention computation. The core idea is to minimize the storage of intermediate results and directly obtain the final result. According to the attention computation equation $\text{softmax}(\frac{QK^T}{\sqrt{D}})V$, multiple intermediate results, such as QK^T and the attention score matrix, need to be explicitly retained, leading to numerous memory read-write operations. FlashAttention uses specially designed methods, such as matrix partition and operator fusion, to keep intermediate results in the cache until the final result is obtained, thus reducing the amount of memory read and write operations. Additionally, FlashAttention can effectively reduce the peak memory usage and activation memory consumption (Section 9.3) during the LLM training and inference. By using FlashAttention, LLaMA-2 (7B) with a sequence length of 2,048 and a batch size of 8 requires only one-tenth of the computation time compared to the standard method.

Sequence Parallelism. Compared with the 3D parallelism introduced in Section 4.3, sequence parallelism can be considered a fourth parallelism dimension in pre-training, particularly when handling long data sequences. The core idea is to partition the sequence across multiple devices for parallel computation. The primary challenge lies in minimizing communication across the devices during attention computation. DeepSpeed-Ulysses [981] partitions the sequence along the hidden dimension, allowing each device to receive a subset of the attention heads and compute attention for different heads in parallel. In comparison, Ring Attention [982] partitions the sequence along the length dimension, where the query matrices on each device are in turn computed with the key and value matrices on other devices. Furthermore, Ring Attention is also compatible with FlashAttention and can be considered as its distributed extension.

9.4 Analysis and Optimization for Model Inference

In Section 4.2.4, we have introduced the basic decoding strategies for using LLMs. As inference efficiency is critically important for the application of LLMs, we next will quantitatively analyze

TABLE 19: The computation, data transfer, and arithmetic intensity during the prefill stage. We use the asymptotic notation O to denote the complexity of data transfer amount, where the constant factor of the complexity is related to the specific implementation method. Table source: [983].

Equations	Computation	Data transfer	Arithmetic intensity
① $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{XW}^{Q,K,V}$	$6BTH^2$	$O(BTH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{BT}}\right)$
② $\mathbf{Q}, \mathbf{K} = \text{RoPE}(\mathbf{Q}, \mathbf{K})$	$6BTH$	$O(BTH)$	$O(1)$
③ $\mathbf{O} = \text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$	$4BT^2ND + 4BT^2N$	$O(BT^2N + BTND)$	$O\left(\frac{1+\frac{1}{D}}{\frac{1}{D} + \frac{1}{T}}\right)$
④ $\mathbf{X} = \mathbf{OW}^O$	$2BTH^2$	$O(BTH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{BT}}\right)$
⑤ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BTH$	$O(BTH + H)$	$O\left(\frac{1}{1 + \frac{1}{BT}}\right)$
⑥ $\mathbf{G}, \mathbf{U} = \mathbf{X}[\mathbf{W}^G, \mathbf{W}^U]$	$4BTHH'$	$O(BTH + BTH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{BT}}\right)$
⑦ $\mathbf{D} = \text{Swish}(\mathbf{G}) \cdot \mathbf{U}$	$2BTH'$	$O(BTH')$	$O(1)$
⑧ $\mathbf{X} = \mathbf{DW}^D$	$2BTHH'$	$O(BTH + BTH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{BT}}\right)$
⑨ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BTH$	$O(BTH + H)$	$O\left(\frac{1}{1 + \frac{1}{BT}}\right)$

TABLE 20: The computation, data transfer, and arithmetic intensity during the decoding stage. Table source: [983].

Equations	Computation	Data transfer	Arithmetic intensity
① $\mathbf{q}, \mathbf{k}, \mathbf{v} = \mathbf{XW}^{QKV}$	$6BH^2$	$O(BH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{B}}\right)$
② $\mathbf{q}, \mathbf{k} = \text{RoPE}(\mathbf{q}, \mathbf{k})$	$6BH$	$O(BH)$	$O(1)$
③ $\mathbf{K}, \mathbf{V} = \text{Cache}(\mathbf{k}, \mathbf{v})$	-	$O(BTND)$ or $O(BND)$	-
④ $\mathbf{o} = \text{Attn}(\mathbf{q}, \mathbf{K}, \mathbf{V})$	$4BTND + 4BTN$	$O(BTN + BTND + BND)$	$O\left(\frac{1+\frac{1}{D}}{1 + \frac{1}{D} + \frac{1}{T}}\right)$
⑤ $\mathbf{X} = \mathbf{oW}^O$	$2BH^2$	$O(BH + H^2)$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{B}}\right)$
⑥ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BH$	$O(BH + H)$	$O\left(\frac{1}{1 + \frac{1}{B}}\right)$
⑦ $\mathbf{g}, \mathbf{u} = \mathbf{X}[\mathbf{W}^G, \mathbf{W}^U]$	$4BHH'$	$O(BH + BH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{B}}\right)$
⑧ $\mathbf{d} = \text{Swish}(\mathbf{g}) \cdot \mathbf{u}$	$2BH'$	$O(BH')$	$O(1)$
⑨ $\mathbf{X} = \mathbf{dW}^D$	$2BHH'$	$O(BH + BH' + HH')$	$O\left(\frac{1}{\frac{1}{H} + \frac{1}{H'} + \frac{1}{B}}\right)$
⑩ $\mathbf{X} = \text{Add\&Norm}(\mathbf{X})$	$5BH$	$O(BH + H)$	$O\left(\frac{1}{1 + \frac{1}{B}}\right)$

the efficiency of the inference process and also present corresponding optimization methods.

9.4.1 Analysis of Inference Efficiency

Overall, the inference process of LLMs can be divided into two stages for overhead analysis: (1) the *prefill* stage, which computes the states and caches the key-value tensors for the input sequence; and (2) the *decoding* stage, which computes the states of the newly generated tokens, updates the key-value cache (KV cache, and continuously generate tokens in an auto-regressive way until the generation process is complete [984].

Inference Efficiency Measurement. To quantitatively analyze the inference efficiency, we next will introduce two widely-used metrics for measuring inference efficiency.

- **GPU performance metrics.** First, we introduce the *compute capability* and *memory bandwidth* to evaluate the efficiency of a certain GPU. The compute capability of a GPU refers to the number of floating-point operations (FLOP) that it can perform per second, measured in FLOP/s. The bandwidth of a GPU refers to the amount of memory read and write operations it can perform per second, measured in byte/s. The ratio of compute to bandwidth is known as the *maximum arithmetic intensity* of the GPU, denoted as $I_{\max}$, which is measured in FLOP/byte⁴⁷. For example, the half-precision compute and bandwidth of the A100 GPU are 312 TFLOP/s and 2039GB/s, respectively. Correspondingly, its maximum arithmetic intensity is 142.51 FLOP/byte⁴⁷.

- **Model efficiency metrics.** Similarly, each operation (e.g., matrix multiplication) of the model can be measured by two

47. <https://www.nvidia.com/en-us/data-center/a100/>

corresponding metrics: the *computation amount* and the *data transfer amount*. The former refers to the total number of floating-point operations, measured in FLOPs. The latter refers to the total amount of GPU memory read and write operations, measured in bytes. Analogous to the arithmetic intensity of a GPU, the *arithmetic intensity* I of a model operation (e.g., matrix multiplication) can be defined as the ratio of computation to data transfer, with units of FLOP/byte.

When the model's arithmetic intensity I is less than the GPU's maximum arithmetic intensity $I_{\max}$, it indicates that the maximum memory bandwidth of the GPU is lower than the speed required. Consequently, the model's efficiency will primarily be limited by memory bandwidth, and the operation is called *memory-bound*. Conversely, when I exceeds $I_{\max}$, it suggests that the GPU's maximum floating-point operation speed is lower than the speed required. In this case, the model's efficiency will mainly be constrained by the GPU's compute capability, and the operation is called *compute-bound*.

Bottleneck Analysis. Based on the above analysis, we can obtain the arithmetic intensity for each operation during both the prefill and decoding stages, as shown in Tables 19 and 20, thereby better identifying the bottleneck operations in the inference process.

- *Prefill stage.* In the following analysis, we will still take the LLaMA (7B) model in Table 18 as an example ($N = 32, D = 128, H = 4096$) and assume a batch size of 8 and a sequence length of 1024 (i.e., $B = 8, T = 1024$). Substituting these values into Table 19, we can find that the arithmetic intensity for linear transformations (Equations ①④⑥⑧) is approximately 2730.67, for multi-head attention (Equation ③) it is approximately 114.67, while the intensity for other operations (Equations ②⑤⑦⑩) is around 1. When using an A100 (80G) GPU with $I_{\max} = 142.51$, the arithmetic intensities of the linear transformations and multi-head attention operations are all above or close to the maximum value. Given that these operations occupy the majority of the computations during the prefill stage, we can conclude that prefill stage is actually compute-bound.

- *Decoding stage.* Similarly, substituting these values into the arithmetic intensity formulas in Table 20 for the decoding stage reveals that the arithmetic intensities of the linear transformations and multi-head attention are all below 8, which is much lower than the A100 GPU's maximum intensity 142.51. This indicates that the decoding stage is constrained by the GPU's data transfer speed (i.e., memory-bound), a problem commonly referred to as the *memory wall*. The analysis indicates that inefficiencies in LLM inference primarily occur during the decoding stage.

9.4.2 System-level Optimization

To mitigate the memory wall issue, an intuitive idea is to reduce the data transfer operations as possible, thereby enhancing the arithmetic intensity. In this part, we will introduce several system-level optimization methods to achieve the reduction in data transfer.

FlashAttention and Flash-Decoding. The FlashAttention method discussed in Section 9.3.3 can also be applied at the prefill stage, as it reduces data transfer operations and effectively increases arithmetic intensity. However, this optimization technique is not directly applicable during the decoding stage, where only the current query vector needs to be computed with the KV

cache matrices. To further optimize the decoding process, Flash-Decoding [985] has been proposed based on FlashAttention, particularly for long sequences, which shares a similar idea with sequence parallelism. Specifically, Flash-Decoding splits the KV cache into smaller chunks, allowing the computation of the query vector with these chunks in parallel, thereby improving the decoding efficiency.

PagedAttention. PagedAttention [304] focuses on optimizing KV cache and attention computation, significantly reducing data transfer operations in these two aspects. In KV cache concatenation, traditional methods often need to allocate new GPU memory for each concatenation, copying the original KV cache and the new hidden states into the newly allocated memory. This process leads to repeated memory read-write operations and substantial memory fragmentation. PagedAttention addresses this issue by introducing a memory paging management method, preallocating several blocks of memory for future KV caches, which can largely reduce the memory allocation operations during concatenation. Additionally, PagedAttention optimizes the attention computation by increasing the parallelism. It uses operator fusion to parallelize the computation of the query vector with multiple KV cache chunk, thereby enhancing the computational efficiency.

Batch Management Optimization. Batch management optimization aims to increase the batch size during the decoding stage to enhance arithmetic intensity. A representative method is continuous batching, proposed by vLLM [304]. Unlike traditional fixed-length batch processing, this technique breaks down each request into a prefill iteration and several single-step decoding iterations, and continuous batching further employ heuristic algorithms to select requests for prefill or single-step decoding iteration. This fine-grained batching mechanism allows for handling more requests simultaneously, which has the same effect as increasing the batch size. Furthermore, DeepSpeed-MII [986] introduces Dynamic SplitFuse, which splits the prefill stage into multiple iterations and allows simultaneous prefill and decoding in one computation, resulting in larger batches and higher inference throughput.

9.4.3 Algorithm-level Optimization

In addition to system-level optimization methods, existing research work has proposed a series of improvements for autoregressive inference algorithms aimed at enhancing inference efficiency. This part introduces four typical inference optimization algorithms.

Speculative Decoding. Intuitively, the generation steps in language modeling have varied difficulty levels. For example, predicting the next word of "*The founder of Microsoft is*" may be more challenging than predicting the next word of "*The founder of Microsoft is Bill*". Even a small model may successfully predict the answer in this case. Based on this idea, *speculative decoding* [987, 988] has been proposed to accelerate the inference speed. Specifically, it employs a relatively smaller yet more efficient model (such as an n -gram statistical model or a small pre-trained model) to autoregressively generate several tokens. Then, a larger model then verifies these tokens, determining whether each token is the top-ranked prediction at the each generation step. The small and large models iteratively repeat this process until decoding is complete. Speculative decoding can

lead to a notable $2\times$ to $3\times$ speedup without compromising the generation quality. Researchers further suggest several variants to improve the efficiency of this approach, such as a learning-based method to combine several small models [989] and a stage-wise acceleration which employs a more smaller model to accelerate the small model first [990].

Cascade Inference. *Cascade inference* optimizes the inference efficiency by addressing requests of varying difficulty with models of different scales. FrugalGPT [991] introduces a series of models arranged by efficiency from high to low, sequentially processing a request through these models. A specially trained binary classification model then evaluates whether the generated result meets the task requirements. If the result is deemed reliable, subsequent models would be bypassed, thus improving the inference speed. This strategy can be applied to various open-source models and commercial APIs, allowing for the flexible adjustment the classification threshold to balance inference efficiency and generation quality according to specific needs. For reasoning tasks, researchers [992] further propose to utilize the self-consistency [429] of generated answers to evaluate the quality of the small model: the large model is employed for generation only when the small model’s answers exhibit a low consistency.

Non-autoregressive Decoding. Existing decoding methods predominantly adopt the autoregressive mechanism, generating tokens one by one, which is a primary reason for lower inference efficiency. Therefore, *non-autoregressive decoding* [993] has been proposed by generating all tokens based on the input at once. However, the generation quality of this method still largely lags behind autoregressive methods. To improve the quality of the generated text, several studies attempt to combine both decoding methods, proposing semi-autoregressive decoding methods [994] that generate a group of tokens (e.g., 3 to 10 tokens) at each step and use these tokens as input to generate the next group. However, existing mainstream LLMs are pre-trained to predict the next token, making direct non- or semi-autoregressive generation infeasible. To address this, Medusa [995] trains two additional prediction heads on the Vicuna model to predict the second and third tokens respectively, thereby achieving the generation of three tokens simultaneously. However, due to the decreased generation quality, these methods have been rarely used directly in practice, but are more often combined with other methods (e.g., speculative decoding) to accelerate the inference process of LLMs. For instance, after Medusa generates three tokens in parallel, the original Vicuna model would still be employed to verify the generation quality.

Early Exit. It has been found that in multi-layer Transformer models, it may not be necessary to perform the computation through all layers to reliably predict the next token [996]. Based on this idea, several studies [996, 997] have proposed improved generation methods based on *early exit*. During model decoding, when the conditions for early exit are satisfied, the model can directly use intermediate computation results from certain layers to generate tokens, thereby improving the inference efficiency. To determine the exit condition, prediction confidence [997] or the entropy [996] of the next token’s generation probability distribution can be used as reference measures. More recently, mixture-of-depths [998] has proposed to dynamically adjust the computation load of each layer. Similar to MoE networks, the

mixture-of-depths method calculates a score for each layer’s input via a routing network. If the score exceeds a preset threshold, the layer would be computed; otherwise, the layer would be skipped. Unlike traditional early exit mechanisms that skip all subsequent layers, the mixture-of-depths method selectively skips certain layers, which can adaptively utilize the characteristics of different layers during generation.

9.5 Model Compression

Due to the huge number of model parameters, LLMs take a significant memory footprint for inference, making it very costly to be deployed in real-world applications [999]. In this section, we focus on how to reduce the memory footprint of LLMs via technical approaches. In particular, we will primarily introduce the model quantization approach, and also briefly discuss other model compression methods, e.g., model pruning and distillation.

9.5.1 Quantization Methods

There are generally two major model quantization approaches, namely *quantization-aware training (QAT)* (requiring additional full model retraining) and *post-training quantization (PTQ)* (requires no model retraining). Compared with small-sized language models, two major differences need to be considered when designing or selecting quantization methods for LLMs. Firstly, LLMs consist of a huge number of parameters, and thus PTQ methods are more preferred due to a much lower computational cost than QAT methods. Secondly, LLMs exhibit very different activation patterns (i.e., large outlier features), and it becomes more difficult to quantize LLMs, especially hidden activations. Next, we will briefly review several representative PTQ methods⁴⁸ for LLMs.

Background for Quantization. In this part, we present a general introduction of quantization techniques for neural networks. In neural network compression, quantization often refers to the mapping process from floating-point numbers to integers [1000], especially the 8-bit integer quantization (i.e., *INT8 quantization*). For neural network models, there are typically two kinds of data to be quantized, namely *weights* (model parameters) and *activations* (hidden activations), which are originally represented in floating-point numbers. To illustrate the essential idea of model quantization, we introduce a simple yet popular quantization function: $x_q = R(x/S) - Z$, which transforms a floating number x into a quantized value x_q . In this function, S and Z denote the scaling factor (involving two parameters α and β that determine the clipping range) and zero-point factor (determining symmetric or asymmetric quantization), respectively, and $R(\cdot)$ denotes the rounding operation that maps a scaled floating value to an approximate integer. As the reverse process, *dequantization* recovers the original value from the quantized value accordingly: $\tilde{x} = S \cdot (x_q + Z)$. The quantization error is calculated as the numerical difference between the original value x and the recovered value $\tilde{x}$. The range parameters α and β have a large impact on the quantization performance, which often need to be *calibrated* according to real data distributions, in either a *static* (offline) or *dynamic* way (runtime). For more details, we refer to the readers to the excellent survey [1000] about quantization methods on neural networks.

48. Since we mainly focus on discussing quantization methods in the context of LLMs, the line of quantization work on small-sized language models (e.g., BERT) has not been included in this survey.

Post-Training Quantization (PTQ). We first introduce the PTQ methods for LLMs.

- *Mixed-precision decomposition.* As found in [1001], extremely large values would occur in hidden activations (called *the emergence of outliers*) when the model size reaches 6.7B parameters or above. These outliers significantly influence the data distribution ranges of the hidden activations, making it challenging to conduct effective model quantization. To reduce the quantization error, a straightforward method is to separately process the outliers and the rest weight values. Specifically, LLM.int8() [1001] has observed that these outliers are mainly distributed in certain feature dimensions at Transformer layers. Based on this finding, a vector-wise quantization approach is proposed to separate the outliers and the rest in matrix multiplication.

- *Salient weights protection.* For Transformer based language models, there often exists a subset of weight values that are more sensitive to quantization, which are also referred to as *salient weights* [1002]. Unlike activation outliers, which occur dynamically during inference and may require complex runtime handling, weight outliers are static and can be pre-processed before model deployment. By identifying and preserving these salient weights, the error associated with model quantization can be effectively reduced. In existing literature, various methods have been proposed to detect these salient weights. For instance, PB-LLM [1003] utilizes the magnitude of weights for finding critical weights, SpQR [1004] categorizes the outliers in weights into small groups by investigating the structural patterns, APTQ [1005] employs the Hessian trace as a sensitivity metric, and OWQ [1006] selects the top sensitive columns based on both the Hessian matrix and weight perturbations.

- *Fine-grained quantization.* For Transformer models, weights and activations are usually represented in the form of tensors. A straightforward approach is to use coarse-grained quantization parameters for the whole tensor (*i.e.*, per-tensor quantization) [1007]. However, it usually leads to inaccurate reconstruction results. Thus, fine-grained methods are proposed to reduce the quantization error. ZeroQuant [1008] adopts a token-wise quantization approach with dynamic calibration for compressing activations. Whereas for weights (easier to be quantized), it uses a group-wise quantization. In practice, a group size of 128 [1002, 1008] is commonly used for model quantization.

- *Balancing the quantization difficulty.* Considering that weights are easier to be quantized than activations, SmoothQuant [1007] proposes to migrate the difficulty from activations to weights. Specially, they incorporate a scaling transformation to balance the difficulty between weights and activations in a linear layer: $\mathbf{Y} = (\mathbf{X}\text{diag}(\mathbf{s})^{-1}) \cdot (\text{diag}(\mathbf{s})\mathbf{W})$. By introducing an mathematically equivalent transformation, this formula controls the quantization difficulty through the scaling factor $\mathbf{s}$. To set $\mathbf{s}$, it incorporates a migration strength parameter α to balance the difficulties, where each entry $s_j = \max(\mathbf{x}_j)^\alpha / \max(\mathbf{w}_j)^{(1-\alpha)}$ is determined by the migration strength.

- *Layerwise quantization.* This approach finds optimal quantized weights that minimize a layerwise reconstruction loss: $\arg \min_{\hat{\mathbf{W}}} \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|_2^2$. To efficiently optimize this objective, GPTQ [1009] improves the original optimal brain quantization (OBQ) [1010] method by fixing the quantization order of weights for all rows. Further, with specially designed methods (*i.e.*, lazy batch-updates and Cholesky reformulation), GPTQ is feasible to quantize very large models (*e.g.*, 175B OPT) in 3

or 4 bit precision. More recently, AWQ [1002] further simplifies the optimization form by incorporating activation-aware scaling for weights, which resembles the idea of SmoothQuant [1007]: weights corresponding to outlier activations are more important to be precisely quantized. It does not directly optimize the reconstruction loss, but instead performs simple hyper-parameter search to achieve the minimal loss on calibration data.

These strategies in the above methods can be jointly used to improve the quantization performance. In order to achieve high-efficiency implementation, quantization methods also rely on hardware- or system-level support (*e.g.*, efficient GPU kernels or hardware-friendly group partition).

Other Quantization Methods. In the above, we mainly focus on PTQ methods, and next introduce two recent studies that explore efficient fine-tuning methods or QAT methods for quantizing LLMs.

- *Efficient fine-tuning enhanced quantization.* For post-training quantization, direct low-bit quantization (*e.g.*, INT4 quantization) often results in large performance degradation. To overcome this challenge, QLoRA [1011] incorporates additional small tunable adapters (16-bit precision) into the quantized models, to achieve an efficient, high-precision model fine-tuning. It combines the merits of LoRA (See Section 5.3.1) and quantization methods. The experiment results show that 4-bit quantized models can achieve the full 16-bit fine-tuning performance by QLoRA.

- *Quantization-aware training (QAT) for LLMs.* A recent study [1012] explores the effect of QAT methods by applying a data-free distillation method to compress the weights, activations as well as key-value cache. By conducting extensive experiments based on LLaMA, they show promising results with 4-bit quantization on both weights and key-value cache, but not on 4-bit activation quantization, which still needs more exploration.

Empirical Analysis and Findings. Quantization has currently become a common technique to reduce the memory footprint and latency of LLMs in deployment. In particular, it is important to understand what level of precision (*e.g.*, INT8 or INT4) can be applied to quantize different parts of LLMs (*e.g.*, weights or activations), while retaining a high accuracy. In this part, we first summarize the major findings about the quantization of LLMs in existing literature, and then present some empirical analysis with quantization experiments.

- *INT8 weight quantization can often yield very good results on LLMs, while the performance of lower precision weight quantization depends on specific methods* [1002, 1007, 1009, 1013]. In most cases, INT8 weight quantization can be effectively applied to reduce the memory footprint without performance degradation. While for INT4 (or INT3) weight quantization, existing methods rely on specific strategies to reduce the performance degradation, *e.g.*, layerwise method [1008, 1009], activation-aware scaling [1002] and low-rank adapter tuning [1011]. Interestingly, LLMs seem to be less sensitive to low-bit weight quantization than small-sized language models [1013]. In practice, with the same memory cost, it is suggested to use a larger language model with a lower quantization precision rather than a smaller language model with a higher quantization precision. For example, a 4-bit 60B LLM is demonstrated to have better performance than an 8-bit 30B LLM [1014]. Moreover, focusing on emergent capabilities, the study [1015] finds that in-context learning, step-by-step reasoning, and instruction following all

seem to be seldom affected with 4-bit weight quantization. This result suggests that INT4 quantization exhibits a favorable trade-off in terms of both total bits and performance of emergent abilities.

- *Activations are more difficult to be quantized than weights* [1001, 1007, 1013]. It has been found that large outliers would occur for Transformer language models having a size of 6.7B or above [1001]. This issue has been one of the most fundamental difficulties to quantize LLMs. To overcome this issue, various methods, e.g., mixed-precision decomposition [1001], fine-grained quantization [766, 1001] and difficulty migration [1007], can be applied to alleviate the influence of outlier values. Since large outliers mainly exist in the activations of LLMs, small language models are more resistant to activation quantization [1013, 1015]. In practice, high-quality INT8 activation quantization is still a difficult task, though several methods can attain satisfying results. Further, lower precision activation quantization has still not been successfully explored, even for QAT methods [1012].

- *Efficient fine-tuning enhanced quantization is a good option to enhance the performance of quantized LLMs* [149, 1011]. The benefits of efficient fine-tuning methods in quantization can be twofold. Firstly, it can directly compensate for the performance degradation suffered from low-bit quantization. This can be achieved either by increasing the fitting capacity via updating high precision adapters [1013, 1015, 1016], or by finding a proper low-rank initialization for LoRA fine-tuning [1017]. Secondly, it is flexible to support task-specific or goal-specific fine-tuning of LLMs in a lightweight way [1011], e.g., instruction tuning or chat-oriented tuning, by only tuning the small adapters. Overall, it makes a good trade-off between the effectiveness and training cost, which provides a promising approach to enhancing the performance of quantized LLMs.

Empirical Analysis on Quantization Experiments. To further help readers understand the impact of quantization on LLMs, we also conduct a group of experiments to investigate the inference performance of quantized models here. Specifically, we focus on the fine-tuned LLaMA models (i.e., 7B and 13B) using popular SFT datasets, including FLAN-v2 [69], Alpaca-52K [187] and ShareGPT [153]. For evaluation, we utilize the same tasks in Table 10, and follow the quantization settings in the study [1015] examining the performance of quantized language models at three precision levels: 4-bit, 8-bit and 16-bit. The results are summarized in Table 21. As can be observed from Table 21, the results obtained with 8-bit and 4-bit weight quantization are close to the performance of 16-bit models while significantly reducing memory consumption. In practice, it is recommended to first examine the performance of 4-bit weight quantization for LLMs if reducing memory usage is a critical consideration for deployment.

9.5.2 Other Model Compression Methods

In addition to model quantization, we next introduce two other model compression methods for LLMs, namely model distillation and model pruning. Unlike model quantization, model distillation and pruning aim to simplify the model architecture, thereby reducing the total number of parameters.

Distillation for LLMs. In general, *model distillation* aims to transfer the capabilities from a capable model (referred to as the *teacher model*) to a less capable model (referred to as

the *student model*), thereby achieving the compression of the capable model. Based on whether the weights of teacher models are accessible, one can employ either the white-box approach or the black-box approach for LLM distillation. The white-box approach often employs the traditional knowledge distillation technique, which incorporates additional loss functions (i.e., distillation loss) for aligning the outputs or intermediate states of the student model to those of the teacher model. Based on this approach, MINILLM [1018] effectively distills the 13B LLaMA model down to a 7B model. The black-box approach [1019], on the other hand, can only make use of the textual response of the teacher model for training the student model. These studies mainly focus on utilizing the generated responses for enhancing the key capabilities from the teacher model [146, 384], such as in-context learning and chain-of-thought prompting.

Pruning for LLMs. The goal of model pruning is to minimize the number of parameters in a model while preserving its performance as much as possible. In general, model pruning methods can be categorized into two lines: structured pruning and unstructured pruning. Structured pruning aims to remove certain less important model components (e.g., neurons, channels, layers) that have minimal impact on performance. On the other hand, unstructured pruning mainly focuses on removing individual weights or connections within a neural network model without changing the model’s main structure. As for LLMs, unstructured pruning can generally lead to higher compression rates. For instance, SparseGPT [1020] achieves 60% unstructured sparsity for OPT-175B using unstructured pruning (i.e., 60% of the elements in the weights are masked), and the pruned LLM still retains a relatively low perplexity. With suitable strategies, structured pruning for LLMs can also achieve promising model compression rate. For instance, LLM-pruner [1021] selectively removes 20% of the non-essential parameters from LLaMA (7B) based on gradient information, while maintaining 93.6% performance of the original model. Furthermore, Sheared LLaMA [1022] introduces two techniques: targeted structured pruning and dynamic batch loading, which effectively prunes LLaMA-2 (7B) to a parameter size of 2.7B, while preserving 87.8% of the original model’s performance.

9.5.3 Open-source Libraries

In this part, we briefly introduce the available open-source libraries for memory-efficient deployment.

Quantization Libraries. Next, we introduce three popular quantization libraries for LLMs, including:

- *Bitsandbytes*⁴⁹ is developed based on the methods introduced in the papers of LLM.int8() [1001] and 8-bit optimizers [1023]. It focuses on the quantization of both activations and weights for LLMs, including the support on 8-bit and 4-bit (NF4,FP4) matrix multiplication for efficient inference, as well as an 8-bit optimizer for efficient training.

- *GPTQ-for-LLaMA*⁵⁰ is developed specially for quantizing LLaMA models. It enables 4-bit quantization of LLaMA models of varied sizes based on the GPTQ algorithm [1009]. Also, it provides a comparison with bitsandbytes in both memory and performance (PPL) on the project website.

49. <https://github.com/TimDettmers/bitsandbytes>

50. <https://github.com/qwopqwop200/GPTQ-for-LLaMa>

TABLE 21: Evaluation results for quantized LLaMA models (7B and 13B). We employ existing model checkpoints provided by [350] for quantization experiments, which have been fine-tuned on FLAN-v2, Alpaca-52K, and ShareGPT, respectively. Specifically, we report the performance with AlpacaFarm, MMLU, and BBH, as well as the memory usage of the loaded model (Mem.). For quantization, we employ *bitsandbytes* to quantize the 16-bit models to 8/4 bits by specifying the commands `load_in_8bit` and `load_in_4bit` when loading the weights. It is worth noting that we select *text-davinci-003* as the baseline model for the AlpacaFarm dataset.

Models	SFT Dataset	16-bit				8-bit				4-bit			
		AlpacaFarm	MMLU	BBH	Mem. (GiB)	AlpacaFarm	MMLU	BBH	Mem. (GiB)	AlpacaFarm	MMLU	BBH	Mem. (GiB)
LLaMA (7B)	FLAN-v2	6.65	47.34	35.05	12.58	6.15	47.02	35.17	6.65	7.83	46.23	34.77	3.94
	Alpaca-52K	32.55	40.87	33.66	12.58	33.60	39.98	34.38	6.65	29.57	39.24	32.80	3.94
	ShareGPT	72.05	41.30	32.90	12.58	72.86	39.34	32.71	6.65	70.31	40.08	32.11	3.94
LLaMA (13B)	FLAN-v2	8.14	51.67	41.46	24.40	7.64	51.02	41.25	12.53	7.52	50.48	40.68	7.34
	Alpaca-52K	33.60	47.63	36.10	24.40	31.43	47.04	35.98	12.53	30.87	46.20	36.16	7.34
	ShareGPT	75.59	47.58	38.00	24.40	73.79	47.71	38.31	12.53	71.99	45.77	36.97	7.34

- *AutoGPTQ*⁵¹ is a quantization package developed based on the GPTQ algorithm [1009], which supports INT4 quantization for LLMs. It includes a number of quantized models in the library, and supports LoRA by integrating with HuggingFace PEFT library.

- *llama.cpp*⁵² makes it feasible to run quantized LLaMA models on a MacBook device. It supports INT4, INT5 and INT8 quantization, which is developed in efficient C/C++ implementation. It also supports a number of LLaMA based models, such as Alpaca and Vicuna.

Other Libraries. In addition, there are also libraries for supporting other model compression methods.

- *Torch-Pruning*⁵³ is a toolkit developed for general-purpose structural pruning, including the pruning for vision models, diffusion models and large language models. It employs dependency graph for automatic structural pruning and supports several high-level pruners (e.g., MetaPruner and BNScalePruner).

- *LLM-Pruner*⁵⁴ is designed specifically for the pruning of LLMs. It enables efficient gradient-based structural pruning for LLMs with minimal training samples and training time. Currently, it supports a number of LLMs, such as Baichuan, BLOOM, and LLaMA3.

9.6 Retrieval-Augmented Generation

When dealing with real-time information or specialized domain knowledge, LLMs may struggle to generate accurate outputs solely based on their internal knowledge. To address this issue, *retrieval-augmented generation* (RAG) technique [1024, 1025] has been proposed by incorporating external knowledge source for improving the model response. This technique aims to retrieve relevant information from external sources (e.g., the internet or domain-specific knowledge bases) using an information retrieval system, thereby providing LLMs with timely or domain-relevant context to reduce the factual errors in generated content. In the format, RAG can also be considered as a specific prompting strategy that integrates auxiliary information from external sources into the original prompt. Next, we will introduce the basic workflow of the retrieval-augmented generation technique and related optimization strategies.

Basic Workflow. Typically, the standard RAG procedure consists of three steps, including context retrieval, prompt construction, and response generation.

- *Context Retrieval.* The retrieval step primarily focuses on finding relevant context information from existing information sources that are helpful for addressing the current information need. To achieve efficient retrieval, it is often necessary to build a search index over the collection of candidate documents and then use appropriate methodologies for text retrieval. There are two commonly used retrieval approaches: lexical-based retrieval [1026] using sparse vector representations and semantic retrieval methods using dense vector representations [54]. The former tokenizes the documents and building an inverted index based on a vocabulary, followed by retrieving relevant documents using lexical matching. The latter maps documents to low-dimensional dense vectors and then constructs an efficient index of document vectors using approximate nearest neighbor search algorithms, ranking candidate documents based on the similarity of embeddings. Both methods can often perform well for large-scale document collection, which are widely used in existing RAG systems.

- *Prompt Construction.* After the retrieval stage returns the relevant documents, these documents need to be incorporated into the input prompt of the LLM along with the task description. The prompt should guide the model to utilize the retrieved information to complete the corresponding task. For example, a prompt could be, “Please refer to the information contained in the following documents to complete the task”. Since the retrieved documents are typically lengthy, simply concatenating them into the prompt might lead to a poor utilization of the provided context due to the biased attention (e.g., *lost in the middle* [949]). To address this issue, existing approaches often introduce reranking models to select the most relevant documents from the retrieval results [1027]. Alternatively, information extraction or text compression techniques can be used to retain only the highly relevant information from the documents, thereby reducing the input context length [1028, 1029].

- *Response Generation.* In this step, the constructed prompt is input into the LLM, enabling it to utilize the retrieved content to better accomplish the corresponding task. However, the retrieved documents may contain irrelevant information or even contradictory information to the true answer, which might affect the output generated by the LLM. To address this, the LLM can be further prompted to self-check the quality of the generated output and decide whether to re-perform the retrieval based on the new outputs [1030], or it can perform a confidence assess-

51. <https://github.com/PanQiWei/AutoGPTQ>

52. <https://github.com/gganganov/llama.cpp>

53. <https://github.com/VainF/Torch-Pruning>

54. <https://github.com/horseee/LLM-Pruner>

ment to determine whether the current task requires retrieval or the use of retrieved content [662].

Improvement Strategies. In practice, factors such as the quality of retrieved documents, prompt design, and the generation method of LLMs might impact the final performance of RAG. Next, we discuss how to enhance the RAG performance by summarizing existing improvement strategies.

- *Retrieval method improvement.* The incorporation of retrieval supplements the LLM with relevant contextual information, and the retrieval performance directly affects the quality of the final generated response [454]. To design effective retrieval strategy, an important factor to consider is the text granularity. Intuitively, a coarser granularity (e.g., document-level) may result in efficient retrieval but tend to incorporate substantial irrelevant information, while a finer granularity (e.g., sentence-level) increases the proportion of relevant content in the retrieval results but can lead to higher retrieval latency. To balance relevance and latency, existing research work proposes using “*propositions*” as the retrieval unit [1031], corresponding to semantically complete and relatively independent text fragments, which can effectively reduce the recall of irrelevant information. In particular, they mainly use GPT-4 to synthesize instruction data for the extraction of proposition text, training a smaller model specifically to construct proposition text data [1031]. Furthermore, to improve retrieval performance, methods such as query expansion and query rewriting can be utilized to optimize query formulation. Query expansion focuses on adding supplementary information to the original query, such as incorporating related entity information or providing detailed explanations of key information in the query [796], which helps strengthen relevance matching. However, traditional query expansion methods may disrupt the original semantics for complex queries. To address this issue, we can employ LLMs to decompose complex queries into several sub-queries, which are subsequently expanded individually, allowing for multi-path recall of related information [1032]. As another query enhancement technique, query rewriting focuses on modifying the query content to highlight key information and eliminate potential ambiguities, facilitating the retrieval of related documents [1033]. LLMs can be applied directly to query rewriting, transforming the original query into a more suitable form through well-designed prompts [1034]. To reduce inference overhead, the query optimization capabilities of LLMs can also be transferred to smaller models through knowledge distillation [1035].

- *Retrieval results refinement.* In addition to the initial retrieval methods, the refinement of retrieval results also plays an important role in RAG systems, since the retrieved documents may be not best suited for RAG systems, e.g., LLMs might have difficulty in utilizing long contexts or be affected by irrelevant information in the retrieved documents. As a solution, the documents returned during the retrieval stage can be reranked according to their relevance to the input [1036], filtering out low-quality or irrelevant documents or placing less relevant documents in non-optimal positions within the prompt. Furthermore, both generation and reranking tasks [1027] can be jointly optimized to facilitate better utilize of context documents. Additionally, LLMs can be directly used for document re-ranking by designing specific prompts or using context examples to accomplish this task [777]. In addition to document filtering or reranking, information extraction or automatic summarization techniques can be employed to refine the retrieved content by

extracting more concise and query-relevant content from the retrieved documents. Furthermore, existing research has proposed token-level compression strategies [1037], which select important tokens and remove unimportant parts from the candidate documents.

- *Iterative retrieval enhancement.* In some complex application scenarios, a single retrieval procedure may not suffice for RAG systems. To address this issue, we can further use iterative retrieval augmentation and adaptive retrieval augmentation. Iterative retrieval augmentation aims to iteratively refine the initial query based on the model’s generated results to achieve a comprehensive coverage of the required information. As it involves accumulating multiple rounds of retrieval information, the performance of RAG systems may be affected by redundant or conflicting information. To address this issue, stop mechanism has been introduced for retrieval iteration, using the LLM to evaluate the confidence of the current generation results to determine whether to continue the iteration process [662]. Additionally, for more complex scenarios, iterative retrieval can be combined with the LLM’s own CoT reasoning capability. For example, intermediate results from the chain of thought can be used as the query input for the next round of retrieval, and after completing the retrieval process, the returned results can be integrated into the chain of thought. Building on the iterative retrieval augmentation method, adaptive retrieval augmentation further enhances the LLM’s autonomous use of the retrieval mechanism [1038], thereby improving the overall framework’s efficacy in using the retrieval systems. In practical implementation, for the above two types of augmentation methods, LLM first need to determine when to use the retriever and then utilize pre-set prompts to initiate query generation and retrieval result processing [1039].

- *RAG-enhanced training.* In addition to the improvement strategies mentioned above, specialized training tasks can be designed to further enhance the LLM’s ability to utilize the retrieved content, including both instruction tuning and pre-training tasks. By constructing instruction data focused on retrieval context utilization [1040], instruction tuning can improve the LLM’s ability to utilize relevant retrieval information. When curating the instruction data, it is essential to consider two important issues: positional bias and irrelevant information within the input context. Specifically, relevant documents can be placed at different positions within the prompt, which can enhance the model’s attention to relevant content in various positions and prevent the model from neglecting certain positions [949]. Additionally, irrelevant information can be added to the instructions data, so as to improving the model’s ability to resist interference from such information [1041]. In addition, special training tasks can be introduced during the pre-training stage to further enhance the LLM’s retrieval and generation capabilities [657, 1042]. Existing work mainly constructs unsupervised pre-training data aimed at retrieval augmentation. A common data construction method uses portions of the original document as queries and then trains the model to reconstruct the remaining content of the original document based on the retrieval results [1043].

9.7 Hallucination

Hallucination, which refers to the phenomenon that LLMs generate content inconsistent with factual information, has become a significant issue that greatly affects the task performance of LLMs [1044]. In this section, we focus on discussing the topic of

LLM hallucination, first introducing the definition and source of hallucination and then summarizing the detection and mitigation methods.

9.7.1 Definition of Hallucination

Early research typically defines hallucinations based on the relationship between a model's output and the given input [1045]. In this manner, hallucinations are categorized into *intrinsic hallucinations* where the model's output does not match the input text and *extrinsic hallucinations* where the model's output cannot be verified against the input. However, in real-world scenarios, user inputs often do not contain reference documents, and thus existing work mainly focuses on open-domain factual hallucinations, where the model-generated content does not align with or cannot be verified by existing world knowledge [1044, 1046]. According to a recent study [1044], factual hallucinations can be further categorized into the following types:

- *Entity-error hallucination*. This type of hallucination refers to LLMs generating text containing incorrect entities, such as names of people, dates, locations, or objects that contradict world knowledge.
- *Relation-error hallucination*. This type of hallucination involves LLMs generating incorrect relationships between entities, such as inaccurate quantitative or chronological connections.
- *Incompleteness hallucination*. LLMs may produce incomplete outputs, especially when generating lengthy or list-based responses. This hallucination arises when LLMs are asked about aggregated facts and they fail to reserve the factual completeness.
- *Outdatedness hallucination*. This type of hallucination occurs when LLMs generate information that was accurate at a past time but is no longer correct at present. This issue typically arises due to that most LLMs were trained on time-limited corpora.
- *Overclaim hallucination*. This type of hallucination refers to cases where the statement expressed in the generated text of LLMs is beyond the scale of factual knowledge.
- *Unverifiability hallucination*. This hallucination refers to cases where the information produced by LLMs cannot be verified against existing information sources, making it difficult to assess its accuracy.

9.7.2 Source of Hallucination

In this part, we will discuss the potential factors that might lead to hallucination for LLMs.

Training Data. The quality of training data significantly impacts the model's output and is a primary source of hallucinations. Further, the distribution of training data also plays a key role in shaping the behaviors of LLMs. We next introduce the effect of training data on hallucinations from these two aspects.

- *Data quality*. In practice, the pre-training dataset is typically constructed by collecting diverse data from various sources. While increasing pre-training data can lead to improved model performance, low-quality data can severely damage the generation performance of large models. On the one hand, pre-training data may contain erroneous information, and the goal of training large models is to imitate and memorize the training data as possible. If inaccurate information frequently appears in the training data, the model may memorize and directly copy this content during generation, leading to the phenomenon known

as "*imitative falsehoods*" [558]. On the other hand, pre-training data may contain biased content and the subjective views of its creators. Such biased content can severely affect the model's learning of world knowledge, possibly leading to inappropriate representations.

- *Data distribution*. The distribution of pre-training data also significantly affects the model's behavior. Firstly, regarding the recency factor, LLMs are typically trained on data from a limited period. As world knowledge continuously evolves, the model's stored knowledge can become outdated, thereby likely leading to fabrications or outdated information when addressing questions beyond its knowledge scope. In terms of data composition, pre-training data may lack domain-specific knowledge, which would affect the model performance on tasks requiring specialized knowledge, such as medical or legal issues, and it will also result in significant hallucinations. Additionally, recent studies show that when addressing questions involving long-tail knowledge that appears infrequently in the training corpus, models are more likely to generate inaccurate content [1044].

Training Methods. The training process of LLMs typically includes two major stages: pre-training and post-training. Inappropriate training methods across the two stages are also likely to result in the hallucination behaviors of LLMs.

- *Pre-training*. Currently, the pre-training stage primarily employs the next token prediction method for model training. Recent studies [949] indicate that under the autoregressive training method, the model's attention distribution tends to decay as the sequence length increases. This would prevent LLMs from effectively modeling long-range dependencies, potentially resulting in inference errors or hallucinations. Additionally, the teacher-forcing strategy is commonly used during the training of large models. In this approach, the correct tokens from the previous steps are used to predict the next token instead of the model output. However, during model inference, the model can only use its own generated content for subsequent predictions. This discrepancy between the training and generation phases leads to "*exposure bias*" [1047], which may in turn cause hallucination issues.

- *Post-training*. During the instruction-tuning process, existing works typically employ knowledge distillation to improve the model's instruction-following ability. This involves using high-performance models (such as GPT-4) to generate large-scale instruction data and then fine-tuning weaker models with this data. However, these synthesized data may contain hallucinated content, which might lead to more hallucinations for the trained model. Additionally, during the human alignment process, existing training methods may also cause hallucination issues. Some research work has revealed that LLMs may cater to human responses for earning higher rewards, likely resulting in answers that do not align with factual knowledge [1048].

Response Generation. Given the input prompt, LLMs employ decoding strategies (e.g., top-*k* sampling in Section 4.2.4) for generating the response. In this process, the prompt formulation and the decoding strategies potentially affect the generation behaviors of LLMs.

- *Prompt design*. Prompting has become the primary way for using LLMs to solve downstream tasks. However, inappropriate prompt design can cause the model to overlook or misunderstand important information, leading to incorrect or irrelevant content [1044]. Recent studies have shown that the readability,

format, and concreteness of user instructions would impact the model’s output [1049]. For instance, the use of complex words or long phrases in the prompt reduces the readability, which makes LLMs more difficult to understand the real intentions of user instruction, thereby increasing the chance of hallucination. Additionally, non-standard expressions or abstract concepts can also exacerbate hallucinations.

- **Decoding strategy.** To improve the diversity of the generated content, multiple random sampling strategies are introduced (e.g., beam search, top- p sampling). However, increasing diversity also brings a higher likelihood of generating hallucinated content. For example, increasing the temperature t (Equation 10) will result in a more uniform token probability distribution, which potentially leads to more hallucinations, since low-frequency yet irrelevant words would be assigned a higher probability for generation in this setting.

9.7.3 Hallucination Detection

To effectively detect the hallucinated content, existing work mainly adopts three approaches, namely model-based, uncertainty-based and tool-based methods.

Model-based Methods. Due to the powerful language capabilities and rich world knowledge, existing work extensively adopts powerful LLMs to detect hallucinations from the model-generated text. In this approach, hallucination detection can be considered as a normal text task that requires prompt formulation. To facilitate the research in this line, HaluEval [604] introduces a comprehensive dataset of model-generated and human-annotated hallucinated samples to evaluate how well LLMs can identify such instances, and they empirically show specific prompting strategies such as CoT can effectively improve the model’s accuracy in detecting hallucinations. Furthermore, research work proposes to decompose the hallucination detection into two subtasks: first, extract factual statements, and then assess whether each statement is hallucinated or not [1044, 1050].

Uncertainty-based Methods. Recent studies suggest that the occurrence of hallucinations in LLMs may be related to the uncertainty of their outputs [1051]. Based on such assumptions, a series of works propose detecting hallucinations by assessing the uncertainty of model-generated content. Some research work focuses on the internal features of LLMs, such as token probability and logits. For key concepts in the generated text, a lower token probability indicates a higher uncertainty, which represents an increased likelihood of hallucination [1052]. Other research efforts evaluate the uncertainty by examining the consistency of the models’ responses. For instance, SelfCheckGPT [1051] lets LLMs answer the same questions multiple times to judge whether the generated answers are consistent or not. Another alternative way requires LLMs to reconstruct the input questions based on the responses and then check the consistency between the generated and original questions [1053].

Tool-based Methods. LLMs can detect hallucinations by calling external tools to verify the model-generated content. Typically, the model’s output contains various segments of factual knowledge, which can be broken down into fine-grained factual statements. FActScore [1054] refers to knowledge sources like search engines to verify these statements. FacTool [1055] further proposes to use a series of external verification tools such as calculators and code interpreters to check different types of text.

In addition, HaluAgent [1056] proposes an agent framework to employ smaller open-source models for hallucination detection. With the assistance of tools like search engines and calculators, HaluAgent enables 7B-size models to achieve comparable performance as GPT-4 in hallucination detection.

9.7.4 Hallucination Mitigation

In practice, it is essential to effectively mitigate the hallucination behaviors of LLMs, to provide accurate and helpful responses. In this part, we will introduce several widely-used approaches for alleviating the hallucination, including human alignment, retrieval-augmented generation and improved decoding strategy.

Human Alignment. Hallucination mitigation is closely related to the *honest* criterion in “3H” standards for human alignment, and various alignment methods like RLHF can be adopted to mitigate the model hallucination. HaluEval 2.0 [1044] proposes to first collect hallucinated and non-hallucinated responses to train a reward model, and then fine-tune the LLM with the reward model’s feedback using RL algorithms. However, recent research shows that human preference data may lead LLMs to exhibit sycophantic behavior [1057], where models prioritize catering to human demands over maintaining truthfulness. Some work proposes to refine the annotation process of preference data, such as by aggregating multiple human preferences to improve feedback quality [1057] or fine-tuning LLMs on prompts where the truthfulness of a claim is independent of the user’s opinion [1058].

Retrieval-Augmented Generation. Providing LLMs with highly reliable external knowledge as context can help reduce hallucinations. RARR [1059] first generates multiple questions about the generated text, then retrieves web pages from Google Search as evidence, and finally, an editing model is employed if any disagreement is detected between the evidence and the generated text. LLM-Augmenter [661] further expands the knowledge source to local databases, devising an agent framework to retrieve, consolidate, and generate feedback to the LLM for the final answer. Other research explores placing the retrieval process at different positions relative to the generation process. Verify-and-Edit [1060] proposes to perform the retrieval procedure after the generation process, allowing the original answer to be edited based on the retrieved documents. Furthermore, to help LLMs better handle complex tasks, IRCot [1061] interleaves the knowledge retrieval process with CoT generation, where the retrieved documents guide the LLM in generating additional reasoning steps and CoT sentences assist in retrieving more relevant and diverse documents.

Improved Decoding Strategy. In addition to the above methods, hallucinations can also be mitigated by using improved decoding strategies. Typically, the internal states or knowledge of LLMs themselves can be exploited to reduce the hallucinations. DoLa [317] proposes that the lower layers of LLMs tend to assign higher probabilities to syntactically plausible words, while higher layers encode more factual knowledge. Therefore, DoLa devises a contrastive decoding strategy by subtracting the lower logits from the last layer’s logits and using the results for next-token prediction. ITI [1062] finds that specific attention heads show high linear probing accuracy and regards their activation as truth-correlated directions. During inference, certain heads’ activations would be shifted along these pivot directions. Some

other work introduces external knowledge sources to aid the decoding process. CAD [1063] provides LLMs with extra context about the query, and then contrasts the output probabilities by those without using context, thereby adjusting the influence of the model’s prior knowledge. KCTS [878] applies an auxiliary knowledge classifier on top of the LLM to detect hallucinations, and uses its knowledge faithfulness score to reweight the token distribution.

9.8 Complex Reasoning

In this section, we introduce a new reasoning paradigm for LLMs aimed at solving complex tasks by allocating more time to *thinking* before responding to a problem, *i.e.*, conducting complex reasoning. Specially, we focus on long chain-of-thought (CoT) reasoning⁵⁵, which is the mainstream approach taken by recent large reasoning models, such as OpenAI’s o-series models. We will begin by providing an overview of long CoT reasoning, then introduce the construction of long CoT data and the corresponding training methods, and finally discuss more general test-time scaling methods.

9.8.1 Overview and Analysis

Generally, long CoT reasoning is a method to search for solutions within the natural language space, as reflected in the output responses of LLMs. This approach is akin to the *slow thinking mode* of the human brain [1064], which takes significantly more time to think through difficult problems compared to the *fast thinking mode* used for simpler ones. This subsection will first qualitatively analyze the reasoning patterns and then briefly discuss the main advantages of this reasoning mode.

User: How many positive whole-number divisors does 196 have?

DeepSeek-R1: Okay, so I need to figure out how many positive whole-number ...
 Alright, let me try to **factorize 196** ...
 Now, the formula for the number of divisors is to **take the exponents** ...
 Let me **double-check** that by **listing them out** to make sure I didn’t make a mistake...
 Alternatively, **backtracking** could be used to generate all combinations ...
Wait, let me **list them all systematically**: 1, 2, 4, 7, 14, 28, 49, 98, 196 ...
Therefore, I think the answer is 9. But let me just **confirm once more**...
 Answer: 196 has 9 positive whole-number divisors.

User: Which city in China has the largest population?

DeepSeek-R1: Okay, the user is asking which Chinese city has the largest ...
 First, **confirm the latest data**. As of 2023 or 2024, Shanghai’s urban population ...
 The user might not be aware of the distinction between city proper and administrative area. So, in my answer, I should **clarify** both scenarios to avoid confusion...
Make sure to **reiterate the numbers**: Shanghai 24.9 million (city proper), Chongqing 32 million (administrative)...
Avoid using technical jargon, keep it simple...
 Answer: Shanghai has the largest urban population, while Chongqing is larger administratively (but not purely urban).

Fig. 19: Examples of long CoT reasoning from DeepSeek-R1 (accessed on January 25, 2025). Grey fonts denote the thought part of the model output, and italic fonts denote the final answer.

55. The phrase “*long CoT*” may not be conceptually precise since the model’s thought process could be tree- or graph-structured rather than strictly linear. We use this terminology in line with OpenAI’s introduction of the o1 model, which generally refers to extended thought processes for complex reasoning.

Reasoning Patterns Analysis. As demonstrated in Example 19, existing long CoT reasoning models typically generate an *extended thought process* (in grey) before arriving at the *final answer* (in italic). It is crucial to understand how this thought process is conducted and the types of reasoning patterns generated by LLMs during problem-solving. To provide an intuitive understanding of this reasoning process, we present two examples from the DeepSeek-R1 model.

In the first example, we present a mathematical problem to the model, and the corresponding long CoT can be observed in the reasoning portion of the response. The thought process here is informal and flexible, while showcasing a systematic exploration of the solution within the natural language space. Concretely, the model follows a complete reasoning process consisting of action steps like “*factorize 196*” and “*take the exponents*”. Notably, the thought process naturally includes trigger keywords like “*double check*” and “*wait*”, which invoke the corresponding verification or reflection actions. In the second example, we ask the model which Chinese city has the largest population. Interestingly, it exhibits similar thought patterns, even though the question could be addressed in a more compact and straightforward manner. The model generates a comprehensive reasoning process with actions such as “*confirm the latest data*” and “*clarify*”, with trigger keywords like “*make sure*” and “*avoid*”.

To gain a more comprehensive understanding of this complex reasoning mode, some research has further analyzed the reasoning patterns exhibited in the o1 model [1065]. These studies, based on empirical investigation, have identified several key reasoning patterns, including systematic analysis, method reuse, divide-and-conquer, self-refinement, context identification, and constraint emphasis. Additionally, the use of these reasoning patterns varies across different tasks, significantly enhancing cognitive processes compared to standard CoT reasoning.

Reasoning Advantages. Unlike standard CoT reasoning, this approach does not enforce a linear reasoning chain. Instead, it integrates various reasoning actions and strategies, such as reflection and backtracking, into a single response. Overall, it has two major advantages compared to the standard CoT method or direct prompting methods.

Firstly, due to the autoregressive nature, the standard generation paradigm of LLMs is a “*one-time*” reasoning process. This means that if the generated solution contains obvious mistakes, or even if LLMs are aware of other promising solutions, there are no opportunities for refinement or verification. This issue becomes more pronounced in complex reasoning tasks, where the search space is much larger, preventing LLMs from fully exploring it [1066]. In contrast, long CoT reasoning mitigates this problem by allowing the model to autonomously check and revise its attempts, thus enabling more effective reasoning.

Secondly, this text-based reasoning process can, in principle, emulate various search algorithms that rely on more complex search structures. For example, to represent a tree-structured search space, one might employ a textual process that combines forward exploration with backward revisits, incorporating necessary reflection and verification steps along the way. Consequently, long CoT reasoning can replicate the effects of previously introduced methods like tree-of-thought (ToT) and graph-of-thought (GoT). However, this capability is not inherently present in the LLM; it emerges in a manner similar to the standard CoT ability, developing through appropriate training (see Section 9.8.3).

Overall, long CoT represents a significant different reasoning mode compared to the standard CoT method, facilitating search algorithms within the natural language space of LLMs. It emphasizes how to navigate correct paths through a trial-and-error approach, typically incorporating critical reasoning actions such as planning, evaluation, reflection, and exploration. In contrast, short CoT data typically presents a direct solution process in which all reasoning steps are expected to be correct.

9.8.2 Construction of Long CoT Data

To guide LLMs in producing long-form reasoning followed by solutions, it is crucial to curate high-quality long CoT data for warming up or training the models. While human annotators can construct extended CoT data, this process is costly and requires professional expertise for challenging problems, making it difficult to scale. Consequently, existing studies often develop various methods for automatically constructing long CoT data, such as distillation from more advanced models, search based data synthesis, and multi-agent collaboration, which are detailed below.

Long CoT Data Distillation. Benefiting from the openness of o1-like LLMs endowed with powerful reasoning capabilities, the leading approach to curating high-quality long CoT data involves using open models or APIs for data synthesis. The basic idea is to first construct a set of prompts (*i.e.*, problems) and then feed them into the teacher model to collect the corresponding long CoT response data. Specifically, STILL-2 [1067] utilizes two slow-thinking systems, *i.e.*, DeepSeek-R1-Lite-Preview [1068] and QwQ-32B-preview [1069] for distillation to construct a dataset of long-form thought processes. A key finding is that length distribution is a critical factor in determining the quality of long CoT data. They suggest that length directly reflects the difficulty of prompt problems, with mathematical problems being particularly important to collect, as they often involve extensive thought processes in their solutions. The research shows that even a small amount of carefully curated long CoT data can effectively activate the slow-thinking mode in LLMs. Furthermore, this effect is corroborated by the work on DeepSeek-R1 [1070], which demonstrates that training with distilled data from DeepSeek-R1 consistently enhances the performance of multiple Qwen and Llama models.

Search based Data Synthesis. Search algorithms like Monte Carlo Tree Search (MCTS) [382] have been widely applied to synthesizing long-form reasoning data. As a representative technique, MCTS integrates the principles of tree exploration and random simulation to estimate potential outcomes of actions, making it particularly effective for decision-making tasks with large action spaces. In complex problem-solving, MCTS decomposes the process into multi-step generation, with each node at a specific tree layer representing a step in the solution [1071]. At each step, a LLM, serving as the policy model, samples several candidate nodes, each generating a one-step CoT. MCTS extensively uses rollouts to automatically assign a Q-value to each intermediate step based on its contribution: steps potentially leading to more trajectories that correctly solve the problem receive higher Q-values. After iterating through multiple steps to successfully address the problem, the complete reasoning trajectories from the root node to the terminal node can be viewed as long-form CoT data, where intermediate nodes represent either correct reasoning steps or trial-and-error attempts.

Multi-Agent Collaboration. Beyond relying on a single model, an alternative approach to generating long CoT data is to construct a multi-agent framework [1072] in which several models collaborate or debate to produce long-form reasoning data.

The multi-agent framework for synthesizing long-form CoT data typically involves the coordination of multiple autonomous agents, each specializing in distinct roles or functions. These agents work together using iterative reflection and strategic debate to enhance the reasoning process. Within this framework, one agent might initiate a chain of thought by presenting an initial hypothesis or argument, while others critique and challenge these ideas through logical examination and counter-arguments. This process encourages deep reflection by prompting agents to reconsider assumptions, address potential biases, and refine conclusions through continuous discourse. In this context, reflection involves not only reconsidering past decisions but also assessing whether each agent’s contribution is grounded in logical consistency. Additionally, the debate mechanism incorporates alternative perspectives and counterarguments into the reasoning process, resulting in more robust and nuanced outcomes for complex decision-making tasks. By combining these cognitive processes, the framework fosters an environment where complex problems can be tackled collaboratively, with diverse viewpoints contributing to more comprehensive solutions.

9.8.3 Training Methods

To elicit and enhance long CoT reasoning capabilities, the existing literature extensively explore two methods: long CoT instruction tuning and scaling reinforcement learning (RL) training. We will describe each approach in detail below.

Long CoT Instruction Tuning. As discussed in Section 9.8.1, long-form thought processes require models to engage in extended reasoning before responding. To develop this reasoning capability, we can instruction-tune LLMs using carefully curated long CoT data. The core concept is to train LLMs to “*imitate*” the demonstrated behaviors presented in the long CoT data.

In general, this fine-tuning method aims to achieve two key objectives: *format adherence* (*i.e.*, following a long CoT format) and *ability elicitation* (*i.e.*, activating the complex reasoning mode). Specifically, format adherence requires the model to produce outputs consisting of two sequential parts—thought and solution—while ability elicitation activates the model’s inherent capacities for executing appropriate long-form thought processes. It has been shown that both objectives can be effectively achieved through supervised fine-tuning: a small amount of high-quality long CoT data can suffice to elicit the long CoT reasoning capabilities of LLMs. For instance, by fine-tuning Qwen2.5 (32B) on just 3.9K distilled long CoT data, STILL-2 [1067] achieved performance comparable to industry counterparts such as o1-preview and QwQ in mathematical problem-solving. This effectiveness is largely because strong LLMs inherently possess various specific reasoning abilities (*e.g.*, reflection and backtracking). Instruction tuning with long CoT data further enhances these innate abilities, comprehensively integrating and extending their utilization, which enables the model to manage more complex reasoning processes.

An interesting finding is that this reasoning capability appears to generalize well across different domains. For example, when trained exclusively on mathematical data, it can lead to significant improvements in other disciplines, such as physics and

chemistry [1067]. This is primarily because long CoT reasoning is inherently a reasoning mode rather than a specific ability tied to any particular domain. This can be seen in the example shown in Example 19, where the query, “Which city in China has the largest population?”, is answered through a complex thought process, despite being solvable in a more straightforward manner. Moreover, this capability can be naturally extended to multimodal LLMs, as these models are typically built on the backbone of language models [1073].

Furthermore, this training approach can be naturally enhanced by other supervised training strategies [1067], such as rejection sampling and directional preference optimization. In general, one can begin by warming up a LLM through instruction tuning with long CoT instruction data and then use the model itself to generate rollout samples as training data. These enhancements can have a certain effect, particularly when the amount of warmup instruction data is limited. However, their impact tends to diminish when sufficient long CoT instruction data is available, especially if the quality of self-generated samples is not superior to that of the demonstration data [1067]. These findings suggest that this advanced capability of a model may quickly reach a performance ceiling when trained through supervised fine-tuning, due to the inherent limitations typical of imitation learning (for further discussion, see Section 9.8.4).

Another downside of this fine-tuning method is its tendency to default to long CoT reasoning mode even for simpler problems (See Example 19). To better manage reasoning behavior, it is essential to explore systematic approaches that integrate both long CoT reasoning mode and standard response mode.

Scaling RL Training. Although OpenAI has not disclosed technical details about the o-series models, training methods have been published through initiatives that implement long chain-of-thought (CoT) reasoning systems, such as DeepSeek-R1 [1070] and Kimi-K1.5 [1074], which have demonstrated performance comparable to o1. The technical methods employed converge on the approach of scaling RL training to enhance the complex reasoning capabilities of LLMs. In the following part, we introduce the detailed RL method through three components: the policy model, the reward model, and the RL training algorithm.

- **Policy model.** The policy model refers to the LLM that needs to be enhanced by the complex reasoning capacities. Typically, it should be warmed up through supervised fine-tuning with long CoT data, as outlined in the aforementioned method. The main purpose of this warm-up is to activate the long CoT reasoning mode, enabling the policy model to conduct appropriate explorations using a long-form thought process. It is also recommended that the policy model possesses strong foundational capabilities, as this is crucial for eliciting high-reward actions in a more efficient way. An interesting attempt taken by DeepSeek-R1-Zero is to omit the supervised fine-tuning step. Instead, it leverages its strong instruction-following capacity to adhere to the response format and reasoning mode, guiding the model to generate formatted responses comprising two parts: thought and answer. This method uses a format reward to reinforce the correct reasoning mode.

- **Reward model.** To effectively guide the policy model, it is necessary to set an appropriate reward model in RL algorithms. As discussed in Section 5.2, RLHF employs a specially trained reward model to instruct the learning of the policy model. However, this approach has become less effective for long CoT

reasoning, given the difficulty of training reliable reward models to assess the quality of long CoT reasoning processes. Consequently, existing approaches typically employ a verifiable reward model primarily built on reference answers (e.g., mathematical problems) or test samples (e.g., coding problems). Typically, the mathematical domain serves as the major source of training data, where problems with specific answers are selected. The ground-truth answer is used to derive the reward scores, such as 1 for a correct solution and 0 for an incorrect solution. This might seem counterintuitive: how can a complex reasoning system be effectively developed using such a simple reward model? The explanation lies in the essence of RL: unlike supervised fine-tuning, it encourages the autonomous explorations of models through simple yet appropriate incentives. In this way, the complex reasoning capability can be well internalized within the model. In addition to the accuracy reward, other simple rewards can be considered, including completeness, avoidance of excessively long texts, and other formatting issues like repetition. OpenAI has proposed the *reinforcement fine-tuning (ReFT)* [1075] approach for tuning the o-series models to build domain-specific models, which also uses a simple accuracy reward to guide the training. One limitation of this reward model is that it can only utilize problems with definite and concise answers for training. More general task data, such as summarization, cannot be directly used for training. In such cases, incorporating a trainable reward model becomes necessary. However, as we have discussed, once this reasoning mode is elicited in specific domains, it can naturally generalize well across different domains.

- **RL Training.** After configuring the policy and reward models, suitable RL algorithms are selected to train the policy model [1076, 1077]. In Section 5.2, we provide a detailed implementation of the PPO algorithm, which can be applied directly for training such models. Nonetheless, PPO requires the maintenance and updating of an additional value model, which leads to high training costs, especially when scaling RL training. As a result, existing approaches [1070] often prefer more simplified RL algorithms, such as GRPO [1078] and RLOO [1079], which use heuristic methods to eliminate the need of a value model. These algorithms typically exhibit higher efficiency and demonstrate strong training performance, especially in long CoT reasoning. A critical factor to monitor during RL training is the response length of the reasoning models, as a longer average response length often corresponds to enhanced reasoning capabilities. Therefore, it is important to track the trends in average response lengths. With appropriate training, the model should show progressively longer response lengths, accompanied by simultaneous performance improvements. In fact, response length is directly connected to the *test-time scaling law* demonstrated by OpenAI⁵⁶. This law suggests that as more output tokens are generated, a model’s reasoning performance can improve substantially. Nonetheless, achieving stable and effective RL training is challenging and necessitates consideration of various factors, such as the selection of query problems (e.g., choosing problems that are challenging yet solvable by the model), the updating of the reference model (e.g., continually updating it as training progresses), and the enhancement of exploration strategies (e.g., sampling more responses with higher temperature settings).

56. <https://openai.com/index/learning-to-reason-with-llms/>

9.8.4 Extended Discussion

In the preceding discussions, we have introduced the long CoT reasoning in technical detail. Actually, it can be considered a specific approach to achieve *test-time scaling* (a.k.a., inference-time scaling), which is the focus of this subsection.

From a broader perspective, test-time scaling encompasses various approaches that enhance model performance by increasing the outputs or computations from LLMs. In this way, many methods can be considered test-time scaling techniques. For example, Self-Consistency [429] generates multiple responses and then aggregates the solutions using majority vote, resulting in higher inference costs due to the increased number of rollouts. Additionally, planning techniques (Section 6.4) and their agentic instantiations (Section 9.2) can also be considered test-time scaling approaches, as they involve prompting LLMs multiple times and utilizing tools or memory components. Therefore, the essence of test-time scaling is to trade additional inference costs for performance gains. Unlike previous approaches, long CoT reasoning directly searches for solutions within the natural language space, notably within a single response.

When comparing different test-time scaling methods, two critical factors require careful examination: *token efficiency* (the performance improvement per token cost) and *performance ceiling* (the maximum attainable performance). Research has shown that scaling test-time computation can effectively enhance model performance [1070, 1080] through the use of simple aggregation methods or specially trained models, though token efficiency may vary. Overall, scaling RL training tends to exhibit higher token efficiency compared to existing test-time scaling methods [1070]. Additionally, both heuristic methods and supervised fine-tuning often exhibit a relatively limited performance ceiling that cannot be substantially elevated once scaling reaches a certain level [1067, 1080]. In contrast, scaling RL training can lead to continuous performance improvements in reasoning models as training time increases. For example, DeepSeek-R1-Zero demonstrates a consistent upward trend in performance even after more than 8,000 training steps [1070].

These scaling effects are crucial for solving complex tasks. Notably, a potential advantage of long CoT reasoning models is that they make it feasible to develop expert-level models in specialized domains or for specific tasks, which could significantly impact the advancement of scientific research challenges. Moreover, as inference methods and hardware techniques improve, the deployment and use cost of these models will be significantly reduced, enhancing the contribution of these highly intelligent models to real-world applications. Additionally, addressing security issues in long CoT reasoning models is crucial. Given their unique reasoning mode, specialized alignment strategies should be developed to ensure safer use of these models.

10 Conclusion and Future Directions

In this survey, we have reviewed the recent progress of large language models (LLMs), and introduced the key concepts, findings, and techniques for understanding and utilizing LLMs. We focus on the large-sized models (i.e., having a size larger than 10B) while excluding the contents of early pre-trained language models (e.g., BERT and GPT-2) that have been well covered in the existing literature. In particular, our survey has discussed four important aspects of LLMs, i.e., pre-training, adaptation, utilization, and evaluation. For each aspect, we highlight the techniques

or findings that are key to the success of LLMs. Furthermore, we also summarize the available resources for developing LLMs and discuss important implementation guidelines for reproducing LLMs. This survey tries to cover the most recent literature about LLMs and provides a good reference resource on this topic for both researchers and engineers.

Next, we summarize the discussions of this survey, and introduce the challenges and future directions for LLMs, in the following aspects.

Basics and Principles. Instead of training on specific task goals, LLMs learn from unsupervised pre-training on large-scale text data. This is quite different from previous multi-task learning approaches, which aim to extend the training tasks as possible to achieve sufficient generalization. Thus, it is essential to reveal the basic principles or elements that establish the foundation of the abilities of LLMs. Although the basic idea of language models is intuitive, it is still challenging to formally explain why LLMs trained by simple language modeling objectives (e.g., next token prediction) can become capable of solving various real-world tasks. To investigate this problem, a promising approach is to study the capacity learning (or selection) mechanism based on unsupervised pre-training, since the model capacity of LLMs strongly depends on pre-training data. In addition, *scaling* plays an important role in improving the capacity of LLMs [31, 55, 64], and it is very useful to conduct more theoretical analysis about how the behaviors of large models relate to those of small models, e.g., what behaviors of large models can be inferred from small models and what can't be predicted indeed. Another research direction is to explore more deep analysis on model generalization for LLMs, since increasing concerns have been raised about whether LLMs can generalize beyond the knowledge encoded by pre-training data. Furthermore, data contamination has become a severe issue for fairly assessing the performance of LLMs [740], and thus setting appropriate evaluation protocol will be the basis to investigate and analyze the model capacity of LLMs.

Model Architecture. Due to the scalability and effectiveness, Transformer has become the de facto architecture for building LLMs. Various strategies have been proposed to improve the performance of this architecture, such as neural network configuration and scalable parallel training (see discussions in Section 4.2.2). However, Transformer still suffers from high training costs and slow inference rates. More efforts [270, 271] are still in need to develop improved model architectures for large-scale pre-training. Specially, system-level or hardware-level optimization (e.g., FlashAttention [303]) is worth more exploration to improve the efficiency of Transformer architectures. In addition, as an important basic capacity, existing LLMs typically maintain a long context window. For example, the most recent GPT-4 Turbo enables a long context of 128K tokens, and Claude 2.1 also supports the input up to 200K tokens. Although many efforts have been made to enhance the long context modeling ability of LLMs [283, 943], the resulting models still can't well process the information in the context window [949]. To address this issue, specific architecture adaptations or algorithms might be needed to enhance the modeling and utilization of long context information. Another worrying concern is that existing work mostly focuses on training LLMs with decoder-only Transformers. Despite the effectiveness, it severely limits the more wide, diverse explorations on alternative model architectures.

Model Training. For pre-training, it is essential to establish a data-centric infrastructure and training procedure for LLM optimization, which can effectively support a systematic process of data collection, data cleaning, data mixture, and data curriculum. Furthermore, it also calls for more flexible mechanisms of hardware support or resource schedule, so as to better organize and utilize the resources in a computing cluster. In practice, it is very challenging to pre-train capable LLMs, due to the huge compute consumption and the sensitivity to data quality and training tricks [78, 93]. Thus, it becomes particularly important to develop systemic, economical pre-training approaches for optimizing LLMs, e.g., predictable scaling [46] and proxy model training [59]. More training recipes or principles should be investigated and shared to reduce the potential risk of degradation or failure in large-scale model optimization. Although increasingly more model checkpoints and cleaned datasets have been released, there still lacks reproducible work on pre-training data preparation (e.g., detailed cleaning strategies) and data scheduling (e.g., data mixture and curriculum). Since it is very costly to pre-train a LLM from scratch, it is important to design suitable mechanisms for continually pre-training or fine-tuning the LLM based on publicly available model checkpoints (e.g., LLaMA [57] and Flan-T5 [69]). For this purpose, a number of technical issues have to be resolved, e.g., catastrophic forgetting and task specialization. Furthermore, it is also useful to develop effective tuning strategies that effectively inject or edit specific knowledge [674], e.g., correcting the outdated facts.

Model Utilization. Based on the natural language interface, *prompting* has become the prominent approach for using LLMs to solving various tasks. By combining task descriptions and demonstration examples into prompts, in-context learning (ICL) endows LLMs with the ability to perform well on new tasks, even outperforming full-data fine-tuned models in some cases. To enhance the ability of complex reasoning, advanced prompting techniques have been proposed, exemplified by the chain-of-thought (CoT) strategy, which includes the intermediate reasoning steps into prompts. Furthermore, planning is a promising approach for solving complex tasks, which iteratively invokes LLMs by leveraging tool use capacities. Despite these efforts, several basic problems related to prompting are still under-explored: why a good prompt can elicit the correct answer but a bad prompt cannot, how to reveal the working principles of advanced prompting methods (e.g., ICL and CoT) and further improve these existing approaches, and how to efficiently find the effective prompts for LLMs on specific tasks. Furthermore, from a practical perspective, it has become a fundamental challenge to reduce the inference cost of LLMs, especially in large-scale deployment. Another popular research direction is retrieval-augmented generation, where retrieved contexts from supporting sources are included into prompts for task solving. It has been shown that retrieval augmentation can extend the knowledge boundary and improve the question answering capacity [454], but may suffer from the effectiveness of long context utilization by LLMs [949].

Safety and Alignment. Despite the capacities, LLMs are faced with great safety challenges in practical use. As a fundamental issue of probabilistic modeling nature, LLMs exhibit a tendency to generate hallucinations [640], referring to texts that seem plausible but may be factually incorrect [46]. What is worse,

LLMs might be elicited by intentional instructions to produce harmful, biased, or toxic texts for malicious systems, leading to the potential risks of misuse [55, 66]. To have a detailed discussion of the safety issues of LLMs (e.g., privacy, overreliance, disinformation, and influence operations), the readers can refer to the GPT-3/4 technical reports [46, 55]. As the major technical approach to averting these issues, alignment methods (e.g., RLHF) [66, 116] have been widely used by leveraging human feedback for developing well-aligned LLMs. However, RLHF heavily relies on high-quality human feedback data from professional labelers, which is costly and time-consuming to recruit qualified human annotators. Therefore, it is necessary to improve the RLHF framework for reducing the efforts of human labelers and seek a more efficient annotation approach with guaranteed data quality, e.g., LLMs can be employed to assist the labeling work. Furthermore, it is also suggested to develop simplified optimization algorithms for alignment [388, 391], to reduce the training difficulty and instability of RLHF. As another practical approach, red teaming [132, 367] has been adopted for improving the model safety of LLMs, which utilizes the collected adversarial prompts to refine the LLMs (*i.e.*, avoiding the attacks from red teaming). In addition, privacy concerns are also important to consider when fine-tuning LLMs with domain-specific data, and thus federated based learning [1081] can be useful in privacy-restricted scenarios.

Application and Ecosystem. As LLMs have shown strong capacities in solving various tasks, they can be applied in a broad range of real-world applications (*i.e.*, following task-specific natural language instructions). As a remarkable progress, ChatGPT has potentially changed the way how humans access information, which has been additionally integrated in the release of *New Bing*. Generally, in the near future, it can be foreseen that LLMs would have a significant impact on information-seeking techniques, including both search engines and recommender systems. Furthermore, LLMs make it possible to develop more intelligent systems (e.g., autonomous AI agents) to tackle various complex tasks in real-world scenarios. Specially, Assistants API has been launched by OpenAI (featured by instructions, knowledge and tool use), enabling rapid development of agent-like assistants within the applications. This wave of technical innovation would lead to an ecosystem of LLM-empowered applications (e.g., OpenAI’s GPT Store), which has a close connection with human life. Lastly, the rise of LLMs sheds light on the exploration of artificial general intelligence (AGI). It is promising to develop more smart AI systems than ever. However, in this development process, AI safety should be one of the primary concerns, *i.e.*, making AI lead to good for humanity but not bad [40].

Coda

It is not an easy job to write this long survey and update its content with timely work. First of all, we would like to sincerely thank the support from the readers and our team members. We work very hard on this survey, and hope that it can present a comprehensive, timely reference for LLMs.

Survey Writing. This survey was planned during a discussion meeting held by our research team, and we aimed to summarize the recent advances of large language models as a highly readable report for our team members. The first draft was finished on March 13, 2023, in which our team members tried their best

to include the related studies about LLMs in a relatively objective, comprehensive way. Then, we have extensively revised the writing and contents in several passes. Due to the space limit, we can only include a fraction of existing LLMs in Figure 3 and Table 1, by setting the selection criterion. However, we set a more relaxed criterion for model selection on our GitHub page (<https://github.com/RUCAIBox/LLMSurvey>), which will be regularly maintained. We release the initial version on March 31, 2023, the major revision on June 29, 2023, and second version on September 10, 2023, and this latest version (major revision) on November 23, 2023.

Seeking for Advice. Despite all our efforts, this survey is still far from perfect: we are likely to miss important references or topics, and might also have non-rigorous expressions or discussions. We will continuously update this survey, and improve the quality as much as we can. For us, survey writing is also a learning process for LLMs by ourselves. For readers with constructive suggestions to improve this survey, you are welcome to leave comments on the GitHub page of our survey or directly email our authors. We will make revisions following the received comments or suggestions in a future version, and acknowledge the readers who have contributed constructive suggestions in our survey.

Update log. In this part, we regularly maintain an update log for the submissions of this survey to arXiv:

- First release on March 31, 2023: the initial version.
- Update on April 9, 2023: add the affiliation information, revise Figure 3 and Table 1 and clarify the corresponding selection criterion for LLMs, improve the writing, and correct some minor errors.
- Update on April 11, 2023: correct the errors for library resources.
- Update on April 12, 2023: revise Figure 3 and Table 1, and clarify the release date of LLMs.
- Update on April 16, 2023: add a new Section 2.2 about the technical evolution of GPT-series models.
- Update on April 24, 2023: add the discussion about scaling laws and add some explanations about the model sizes for emergent abilities (Section 2.1); add an illustrative figure for the attention patterns for different architectures in Figure 9, and add the detailed formulas in Table 7.
- Update on April 25, 2023: revise some copy errors in figures and tables.
- Update on April 27, 2023: add efficient tuning in Section 5.3.
- Update on April 28, 2023: revise Section 5.3.
- Update on May 7, 2023: revise Table 1, Table 2, and some minor points.
- Update on June 29, 2023 (major revision):
 - Section 1: add Figure 1 for the trends of published LLM papers in arXiv;
 - Section 2: add Figure 4 for GPT’s evolution and the corresponding discussion;
 - Section 3: add Figure 5 for LLaMA family and the corresponding discussion;
 - Section 5: add latest discussion about the synthetic data formatting of instruction tuning in Section 5.1.1, the empirical analysis for instruction tuning in Section 5.1.4, parameter-efficient model adaptation in Section 5.3 and memory-efficient adaptation in Section 5.3;
 - Section 6: add latest discussion about the underlying mechanism of ICL 6.2.3, planning for complex task solving in Section 6.4;
- Section 7: update Table 14 for representative datasets for evaluating advanced abilities of LLMs, and empirical ability evaluation in Section 7.4;
- Section 6.1.1: add prompt design;
- Section 8: add the discussions on applications of LLMs in finance and scientific research domains;
- Update on September 10, 2023 (major revision):
 - Claim the copyrights of the figures and tables in this paper.
 - Add latest LLMs, techniques and their descriptions in Section 3, Section 4, Section 5, Section 6 and Section 7;
 - Section 4: add latest discussion about the decoding strategy in Section 4.2.4;
 - Section 5: add latest discussion about the practical tricks for instruction tuning in Section 5.1.2, the empirical analysis on LLaMA (13B) for instruction tuning in Section 5.1.4, practical strategies for RLHF in Section 5.2.3, alignment without RLHF in Section 5.2.4 and remarks on SFT and RLHF in Section 5.2.5;
 - Section 6: update the content about the planning for complex task solving in Section 6.4;
 - Section 7: add discussions about evaluation approaches in Section 7.3.2, Table 15 for the category of existing evaluation work, and update empirical ability evaluation in Section 7.4 and the results on Table 16;
 - Section 6.1.1: add new prompt examples in Table 12;
- Update on November 23, 2023 (major revision):
 - Section 1: add Figure 2 for the evolution process of four generations of language models;
 - Section 2: add more discussion about scaling laws and how emergent abilities relate to scaling laws;
 - Section 3: add latest LLMs in Figure 3 and Table 1, latest APIs in Section 3.1, commonly used datasets for instruction tuning and alignment tuning in Section 3.3, and several libraries in Section 3.4;
 - Section 4: add latest discussion about the data scheduling, including data mixtures and data curriculum in Section 4.1.3; add summary of data preparation in Section 4.1.4; add discussion about modeling long context in Section 9.1; add discussion about decoding efficiency issues and add latest decoding strategies in Section 4.2.4;
 - Section 5: add latest discussion about instance construction and tuning strategies in Section 5.1; add latest discussion about process-supervised RLHF in Section 5.2.3, and the empirical study on quantized LLaMA models (7B and 13B) in Section 9.5.1;
 - Section 6: add latest discussion about prompt optimization in Section 6.1.2, and update the content about chain-of-thought prompting in Section 6.3;
 - Section 8: add latest discussion about LLM for research directions in Section 8.1;
 - Section 10: revise the content in the several aspects.
- Update on September 25, 2024:
 - Section 3: reorganize the content of “public available model checkpoints” into multiple series; add the latest LLMs in Figure 3.
 - Section 4: add LLM-based data filtering and selection

methods in Section 4.1.2; update Section 4.2.1, “Emergent Architectures” to include more discussions about SSM-based architectures; add Table 6 to compare parallelism and complexity of different architectures.

- Section 5: add latest discussion about instruction quality improvement and instruction selection in Section 5.1.1; add latest discussion about practical strategies for RLHF and process-supervised RLHF in Section 5.2.3; update the content about supervised alignment tuning in Section 5.2.4.
- Section 6: add latest papers about discrete prompt optimization in Section 6.1.2.
- Section 9: add latest discussion about advanced topics, including long context modeling, LLM-based agent, analysis and optimization for training and inference, model inference, model compression, retrieval-augmented generation, and hallucination.
- Update on October 12, 2024:
 - Section 8.1.5: correct the errors.
- Update on March 11, 2025:
 - Section 9.8: add latest papers about long CoT reasoning, including the analysis of reasoning patterns and advantages, construction of long CoT data (*i.e.*, distillation, search-based, and multi-agent collaboration), and training methods (*i.e.*, instruction tuning and reinforcement learning).
- Update on February 27, 2026:
 - Figure 3: update with new LLMs.

Clarifications on Experiments. In this version, we have included a number experiments on instruction-tuning (Table 10), overall ability evaluation (Table 16), and prompt engineering (Table 17). Due to the limit of computational resources, our experiments are not complete, limited to small-sized models or a few comparisons. Despite that, we feel that it might be meaningful to share the partial results to the public. We will try to include the missing results of larger models or more comparisons in the future versions. **We also call for support of computing power for conducting more comprehensive experiments.**

Chinese Book. We also released a Chinese book based on this survey article, at the link: <https://llmbook-zh.github.io>. This book is in the publication process.

Acknowledgments

The authors would like to thank Yankai Lin and Yutao Zhu for proofreading this paper. Since the first release of this paper, we have received a number of valuable comments from the readers. We sincerely thank the readers who have written to us with constructive suggestions and comments: Tyler Suard, Damai Dai, Liang Ding, Stella Biderman, Kevin Gray, Jay Alammar, Yubo Feng, Mark Holmstrom, Xingdong Liu, Il-Seok Oh, Yiting Liu, Shaojun Wang, Gaoyan Ou, Todd Morrill, Hao Liu, Zhenyu Zhang, and Xinlin Zhuang.

Since the v11 version (June 29, 2023), we have been adding a large number of experiments and prompt practices. These new contents are completed by a number of volunteers in our team. Here, we add a special part to thank all the students who have

worked very hard on this part (also including the ones on our author list).

Contribution on Experiments. We would like to sincerely thank the following people for their hard work involved in experiments shown in Table 16.

- Xiaoxue Cheng: implement the experiments for evaluation on Language Generation and HaluEval tasks.
- Yuhao Wang: implement the experiments for evaluation on interaction with environment tasks.
- Bowen Zheng: implement the experiments for evaluation on tool manipulation tasks.

Contribution on Tips. We list the following guys for their contributions on the corresponding numbers of provided tips for designing prompts in Table 12.

- Xiaolei Wang: T3, O3
- Beichen Zhang: D2, D5
- Zhipeng Chen: D3, D4
- Junjie Zhang: D6
- Bowen Zheng: D7
- Zican Dong: D8
- Xinyu Tang: C2
- Yifan Du: T4
- Tianyi Tang: O6, O7, D9
- Yupeng Hou: O8, C3
- Salvatore Raieli: C4

References

- [1] Y. Bengio, R. Ducharme, P. Vincent, and C. Janvin, “A neural probabilistic language model,” *J. Mach. Learn. Res.*, vol. 3, pp. 1137–1155, 2003.
- [2] R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu, and P. P. Kuksa, “Natural language processing (almost) from scratch,” *J. Mach. Learn. Res.*, vol. 12, pp. 2493–2537, 2011.
- [3] S. Pinker, *The Language Instinct: How the Mind Creates Language*. Brilliance Audio; Unabridged edition, 2014.
- [4] M. D. Hauser, N. Chomsky, and W. T. Fitch, “The faculty of language: what is it, who has it, and how did it evolve?” *science*, vol. 298, no. 5598, pp. 1569–1579, 2002.
- [5] A. M. Turing, “Computing machinery and intelligence,” *Mind*, vol. LIX, no. 236, pp. 433–460, 1950.
- [6] F. Jelinek, *Statistical Methods for Speech Recognition*. MIT Press, 1998.
- [7] J. Gao and C. Lin, “Introduction to the special issue on statistical language modeling,” *ACM Trans. Asian Lang. Inf. Process.*, vol. 3, no. 2, pp. 87–93, 2004.
- [8] R. Rosenfeld, “Two decades of statistical language modeling: Where do we go from here?” *Proceedings of the IEEE*, vol. 88, no. 8, pp. 1270–1278, 2000.
- [9] A. Stolcke, “Srlm—an extensible language modeling toolkit,” in *Seventh international conference on spoken language processing*, 2002.
- [10] X. Liu and W. B. Croft, “Statistical language modeling for information retrieval,” *Annu. Rev. Inf. Sci. Technol.*, vol. 39, no. 1, pp. 1–31, 2005.
- [11] C. Zhai, *Statistical Language Models for Information Retrieval*, ser. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers, 2008.

- [12] S. M. Thede and M. P. Harper, "A second-order hidden markov model for part-of-speech tagging," in *27th Annual Meeting of the Association for Computational Linguistics, University of Maryland, College Park, Maryland, USA, 20-26 June 1999*, R. Dale and K. W. Church, Eds. ACL, 1999, pp. 175–182.
- [13] L. R. Bahl, P. F. Brown, P. V. de Souza, and R. L. Mercer, "A tree-based statistical language model for natural language speech recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 37, no. 7, pp. 1001–1008, 1989.
- [14] T. Brants, A. C. Popat, P. Xu, F. J. Och, and J. Dean, "Large language models in machine translation," in *EMNLP-CoNLL 2007, Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning, June 28-30, 2007, Prague, Czech Republic*, J. Eisner, Ed. ACL, 2007, pp. 858–867.
- [15] S. M. Katz, "Estimation of probabilities from sparse data for the language model component of a speech recognizer," *IEEE Trans. Acoust. Speech Signal Process.*, vol. 35, no. 3, pp. 400–401, 1987.
- [16] W. A. Gale and G. Sampson, "Good-turing frequency estimation without tears," *J. Quant. Linguistics*, vol. 2, no. 3, pp. 217–237, 1995.
- [17] T. Mikolov, M. Karafiát, L. Burget, J. Cernocký, and S. Khudanpur, "Recurrent neural network based language model," in *INTERSPEECH 2010, 11th Annual Conference of the International Speech Communication Association, Makuhari, Chiba, Japan, September 26-30, 2010*, T. Kobayashi, K. Hirose, and S. Nakamura, Eds. ISCA, 2010, pp. 1045–1048.
- [18] S. Kombrink, T. Mikolov, M. Karafiát, and L. Burget, "Recurrent neural network based language modeling in meeting recognition," in *INTERSPEECH 2011, 12th Annual Conference of the International Speech Communication Association, Florence, Italy, August 27-31, 2011*. ISCA, 2011, pp. 2877–2880.
- [19] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in Neural Information Processing Systems 26: 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States*, C. J. C. Burges, L. Bottou, Z. Ghahramani, and K. Q. Weinberger, Eds., 2013, pp. 3111–3119.
- [20] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2013.
- [21] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer, "Deep contextualized word representations," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, M. A. Walker, H. Ji, and A. Stent, Eds. Association for Computational Linguistics, 2018, pp. 2227–2237.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, 2017, pp. 5998–6008.
- [23] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186.
- [24] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, 2020, pp. 7871–7880.
- [25] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, pp. 1–40, 2021.
- [26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever et al., "Language models are unsupervised multitask learners," *OpenAI blog*, p. 9, 2019.
- [27] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019.
- [28] V. Sanh, A. Webson, C. Raffel, S. H. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M. S. Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. V. Nayak, D. Datta, J. Chang, M. T. Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Févry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, and A. M. Rush, "Multitask prompted training enables zero-shot task generalization," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [29] T. Wang, A. Roberts, D. Hesslow, T. L. Scao, H. W. Chung, I. Beltagy, J. Launay, and C. Raffel, "What language model architecture and pretraining objective works best for zero-shot generalization?" in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 22 964–22 984.
- [30] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling laws for neural language models," *CoRR*, vol. abs/2001.08361, 2020.
- [31] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent abilities of large

- language models," *CoRR*, vol. abs/2206.07682, 2022.
- [32] M. Shanahan, "Talking about large language models," *CoRR*, vol. abs/2212.03551, 2022.
- [33] J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. H. Chi, Q. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models," *CoRR*, vol. abs/2201.11903, 2022.
- [34] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, "Training compute-optimal large language models," vol. abs/2203.15556, 2022.
- [35] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, and R. Stojnic, "Galactica: A large language model for science," *CoRR*, vol. abs/2211.09085, 2022.
- [36] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," *ACM Comput. Surv.*, pp. 195:1–195:35, 2023.
- [37] C. Zhou, Q. Li, C. Li, J. Yu, Y. Liu, G. Wang, K. Zhang, C. Ji, Q. Yan, L. He, H. Peng, J. Li, J. Wu, Z. Liu, P. Xie, C. Xiong, J. Pei, P. S. Yu, and L. Sun, "A comprehensive survey on pretrained foundation models: A history from BERT to chatgpt," *CoRR*, vol. abs/2302.09419, 2023.
- [38] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang, W. Han, M. Huang, Q. Jin, Y. Lan, Y. Liu, Z. Liu, Z. Lu, X. Qiu, R. Song, J. Tang, J. Wen, J. Yuan, W. X. Zhao, and J. Zhu, "Pre-trained models: Past, present and future," *AI Open*, vol. 2, pp. 225–250, 2021.
- [39] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, and X. Huang, "Pre-trained models for natural language processing: A survey," *CoRR*, vol. abs/2003.08271, 2020.
- [40] S. Altman, "Planning for agi and beyond," *OpenAI Blog*, February 2023.
- [41] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang, "Sparks of artificial general intelligence: Early experiments with gpt-4," vol. abs/2303.12712, 2023.
- [42] S. Huang, L. Dong, W. Wang, Y. Hao, S. Singhal, S. Ma, T. Lv, L. Cui, O. K. Mohammed, B. Patra, Q. Liu, K. Aggarwal, Z. Chi, J. Bjorck, V. Chaudhary, S. Som, X. Song, and F. Wei, "Language is not all you need: Aligning perception with language models," *CoRR*, vol. abs/2302.14045, 2023.
- [43] Y. Cao, S. Li, Y. Liu, Z. Yan, Y. Dai, P. S. Yu, and L. Sun, "A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt," *arXiv preprint arXiv:2303.04226*, 2023.
- [44] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu *et al.*, "Palm-e: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023.
- [45] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan, "Visual chatgpt: Talking, drawing and editing with visual foundation models," *arXiv preprint arXiv:2303.04671*, 2023.
- [46] OpenAI, "Gpt-4 technical report," *OpenAI*, 2023.
- [47] Y. Fu, H. Peng, and T. Khot, "How does gpt obtain its ability? tracing emergent abilities of language models to their sources," *Yao Fu's Notion*, Dec 2022.
- [48] J. Li, T. Tang, W. X. Zhao, and J. Wen, "Pretrained language model for text generation: A survey," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021*, Z. Zhou, Ed. ijcai.org, 2021, pp. 4492–4499.
- [49] P. Lu, L. Qiu, W. Yu, S. Welleck, and K. Chang, "A survey of deep learning for mathematical reasoning," *CoRR*, vol. abs/2212.10535, 2022.
- [50] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, L. Li, and Z. Sui, "A survey for in-context learning," *CoRR*, vol. abs/2301.00234, 2023.
- [51] J. Huang and K. C. Chang, "Towards reasoning in large language models: A survey," *CoRR*, vol. abs/2212.10403, 2022.
- [52] S. Qiao, Y. Ou, N. Zhang, X. Chen, Y. Yao, S. Deng, C. Tan, F. Huang, and H. Chen, "Reasoning with language model prompting: A survey," *CoRR*, vol. abs/2212.09597, 2022.
- [53] J. Zhou, P. Ke, X. Qiu, M. Huang, and J. Zhang, "Chatgpt: potential, prospects, and limitations," in *Frontiers of Information Technology & Electronic Engineering*, 2023, pp. 1–6.
- [54] W. X. Zhao, J. Liu, R. Ren, and J.-R. Wen, "Dense text retrieval based on pretrained language models: A survey," *ACM Transactions on Information Systems*, vol. 42, no. 4, pp. 1–60, 2024.
- [55] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., 2020.
- [56] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," *CoRR*, vol. abs/2204.02311, 2022.
- [57] H. Touvron, T. Lavril, G. Izacard, X. Martinet,

- M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *CoRR*, 2023.
- [58] T. Henighan, J. Kaplan, M. Katz, M. Chen, C. Hesse, J. Jackson, H. Jun, T. B. Brown, P. Dhariwal, S. Gray *et al.*, "Scaling laws for autoregressive generative modeling," *arXiv preprint arXiv:2010.14701*, 2020.
- [59] S. M. Xie, H. Pham, X. Dong, N. Du, H. Liu, Y. Lu, P. Liang, Q. V. Le, T. Ma, and A. W. Yu, "Doremi: Optimizing data mixtures speeds up language model pretraining," *arXiv preprint arXiv:2305.10429*, 2023.
- [60] P. Villalobos, J. Sevilla, L. Heim, T. Besiroglu, M. Hobbhahn, and A. Ho, "Will we run out of data? an analysis of the limits of scaling datasets in machine learning," *CoRR*, vol. abs/2211.04325, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.04325>
- [61] N. Muennighoff, A. M. Rush, B. Barak, T. L. Scao, A. Piktus, N. Tazi, S. Pyysalo, T. Wolf, and C. Raffel, "Scaling data-constrained language models," *arXiv preprint arXiv:2305.16264*, 2023.
- [62] I. McKenzie, A. Lyzhov, A. Parrish, A. Prabhu, A. Mueller, N. Kim, S. Bowman, and E. Perez, "The inverse scaling prize," 2022. [Online]. Available: <https://github.com/inverse-scaling/prize>
- [63] B. A. Huberman and T. Hogg, "Phase transitions in artificial intelligence systems," *Artificial Intelligence*, vol. 33, no. 2, pp. 155–171, 1987.
- [64] J. W. Rae, S. Borgeaud, T. Cai, K. Millican, J. Hoffmann, H. F. Song, J. Aslanides, S. Henderson, R. Ring, S. Young, E. Rutherford, T. Hennigan, J. Menick, A. Cassirer, R. Powell, G. van den Driessche, L. A. Hendricks, M. Rauh, P. Huang, A. Glaese, J. Welbl, S. Dathathri, S. Huang, J. Uesato, J. Mellor, I. Higgins, A. Creswell, N. McAleese, A. Wu, E. Elsen, S. M. Jayakumar, E. Buchatskaya, D. Budden, E. Sutherland, K. Simonyan, M. Paganini, L. Sifre, L. Martens, X. L. Li, A. Kuncoro, A. Nematzadeh, E. Gribovskaya, D. Donato, A. Lazaridou, A. Mensch, J. Lespiau, M. Tsimpoukelli, N. Grigorev, D. Fritz, T. Sottiaux, M. Pajarskas, T. Pohlen, Z. Gong, D. Toyama, C. de Masson d'Autume, Y. Li, T. Terzi, V. Mikulik, I. Babuschkin, A. Clark, D. de Las Casas, A. Guy, C. Jones, J. Bradbury, M. J. Johnson, B. A. Hechtman, L. Weidinger, I. Gabriel, W. S. Isaac, E. Lockhart, S. Osindero, L. Rimell, C. Dyer, O. Vinyals, K. Ayoub, J. Stanway, L. Bennett, D. Hassabis, K. Kavukcuoglu, and G. Irving, "Scaling language models: Methods, analysis & insights from training gopher," *CoRR*, vol. abs/2112.11446, 2021.
- [65] D. Dai, Y. Sun, L. Dong, Y. Hao, Z. Sui, and F. Wei, "Why can GPT learn in-context? language models secretly perform gradient descent as meta-optimizers," *CoRR*, vol. abs/2212.10559, 2022.
- [66] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," *CoRR*, vol. abs/2203.02155, 2022.
- [67] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [68] R. Thoppilan, D. D. Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, Y. Li, H. Lee, H. S. Zheng, A. Ghafouri, M. Menegali, Y. Huang, M. Krikun, D. Lepikhin, J. Qin, D. Chen, Y. Xu, Z. Chen, A. Roberts, M. Bosma, Y. Zhou, C. Chang, I. Krivokon, W. Rusch, M. Pickett, K. S. Meier-Hellstern, M. R. Morris, T. Doshi, R. D. Santos, T. Duke, J. Soraker, B. Zevenbergen, V. Prabhakaran, M. Diaz, B. Hutchinson, K. Olson, A. Molina, E. Hoffman-John, J. Lee, L. Aroyo, R. Rajakumar, A. Butryna, M. Lamm, V. Kuzmina, J. Fenton, A. Cohen, R. Bernstein, R. Kurzweil, B. Aguera-Arcas, C. Cui, M. Croak, E. H. Chi, and Q. Le, "Lamda: Language models for dialog applications," *CoRR*, vol. abs/2201.08239, 2022.
- [69] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, S. Narang, G. Mishra, A. Yu, V. Y. Zhao, Y. Huang, A. M. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, "Scaling instruction-finetuned language models," *CoRR*, vol. abs/2210.11416, 2022.
- [70] A. Srivastava, A. Rastogi, A. Rao, A. A. M. Shueb, A. Abid, A. Fisch, A. R. Brown, A. Santoro, A. Gupta, A. Garriga-Alonso, A. Kluska, A. Lewkowycz, A. Agarwal, A. Power, A. Ray, A. Warstadt, A. W. Kocurek, A. Safaya, A. Tazarv, A. Xiang, A. Parrish, A. Nie, A. Hussain, A. Askell, A. Dsouza, A. Rahane, A. S. Iyer, A. Andreassen, A. Santilli, A. Stuhlmüller, A. M. Dai, A. La, A. K. Lampinen, A. Zou, A. Jiang, A. Chen, A. Vuong, A. Gupta, A. Gottardi, A. Norelli, A. Venkatesh, A. Gholamidavoodi, A. Tabassum, A. Menezes, A. Kirubakaran, A. Mullokandov, A. Sabharwal, A. Herrick, A. Efrat, A. Erdem, A. Karakas, and et al., "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," *CoRR*, vol. abs/2206.04615, 2022.
- [71] R. Schaeffer, B. Miranda, and S. Koyejo, "Are emergent abilities of large language models a mirage?" *arXiv preprint arXiv:2304.15004*, 2023.
- [72] S. Hu, X. Liu, X. Han, X. Zhang, C. He, W. Zhao, Y. Lin, N. Ding, Z. Ou, G. Zeng, Z. Liu, and M. Sun, "Unlock predictable scaling from emergent abilities," 2023.
- [73] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177*, 2022.
- [74] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deep-speed: System optimizations enable training deep learning models with over 100 billion parameters," in *KDD*, 2020, pp. 3505–3506.
- [75] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," *CoRR*, vol. abs/1909.08053, 2019.
- [76] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand,

- P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using megatron-lm," in *International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2021, St. Louis, Missouri, USA, November 14-19, 2021*. ACM, 2021, p. 58.
- [77] V. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoenybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," *CoRR*, vol. abs/2205.05198, 2022.
- [78] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilıc, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, J. Tow, A. M. Rush, S. Biderman, A. Webson, P. S. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. V. del Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P. O. Suarez, V. Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell, C. Raffel, A. Gokaslan, A. Simhi, A. Soroa, A. F. Aji, A. Alfassy, A. Rogers, A. K. Nitzav, C. Xu, C. Mou, C. Emezue, C. Klamm, C. Leong, D. van Strien, D. I. Adelani, and et al., "BLOOM: A 176b-parameter open-access multilingual language model," *CoRR*, vol. abs/2211.05100, 2022.
- [79] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V. N. Vishwanathan, and R. Garnett, Eds., 2017, pp. 4299–4307.
- [80] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *CoRR*, vol. abs/2302.04761, 2023.
- [81] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "Webgpt: Browser-assisted question-answering with human feedback," *CoRR*, vol. abs/2112.09332, 2021.
- [82] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, pp. 140:1–140:67, 2020.
- [83] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, "mt5: A massively multilingual pre-trained text-to-text transformer," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, 2021, pp. 483–498.
- [84] W. Zeng, X. Ren, T. Su, H. Wang, Y. Liao, Z. Wang, X. Jiang, Z. Yang, K. Wang, X. Zhang, C. Li, Z. Gong, Y. Yao, X. Huang, J. Wang, J. Yu, Q. Guo, Y. Yu, Y. Zhang, J. Wang, H. Tao, D. Yan, Z. Yi, F. Peng, F. Jiang, H. Zhang, L. Deng, Y. Zhang, Z. Lin, C. Zhang, S. Zhang, M. Guo, S. Gu, G. Fan, Y. Wang, X. Jin, Q. Liu, and Y. Tian, "Pangu- α : Large-scale autoregressive pre-trained chinese language models with auto-parallel computation," *CoRR*, vol. abs/2104.12369, 2021.
- [85] Z. Zhang, Y. Gu, X. Han, S. Chen, C. Xiao, Z. Sun, Y. Yao, F. Qi, J. Guan, P. Ke, Y. Cai, G. Zeng, Z. Tan, Z. Liu, M. Huang, W. Han, Y. Liu, X. Zhu, and M. Sun, "CPM-2: large-scale cost-effective pre-trained language models," *CoRR*, vol. abs/2106.10715, 2021.
- [86] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," *arXiv preprint arXiv:2203.13474*, 2022.
- [87] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonnell, J. Phang, M. Pieler, U. S. Prashanth, S. Purohit, L. Reynolds, J. Tow, B. Wang, and S. Weinbach, "Gpt-neox-20b: An open-source autoregressive language model," *CoRR*, vol. abs/2204.06745, 2022.
- [88] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Naik, A. Ashok, A. S. Dhanasekaran, A. Arunkumar, D. Stap, E. Pathak, G. Karamanolakis, H. G. Lai, I. Purohit, I. Mondal, J. Anderson, K. Kuznia, K. Doshi, K. K. Pal, M. Patel, M. Moradshahi, M. Parmar, M. Purohit, N. Varshney, P. R. Kaza, P. Verma, R. S. Puri, R. Karia, S. Doshi, S. K. Sampat, S. Mishra, S. R. A. S. Patro, T. Dixit, and X. Shen, "Super-naturalinstructions: Generalization via declarative instructions on 1600+ NLP tasks," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, 2022, pp. 5085–5109.
- [89] Y. Tay, M. Dehghani, V. Q. Tran, X. García, J. Wei, X. Wang, H. W. Chung, D. Bahri, T. Schuster, H. Zheng, D. Zhou, N. Houlsby, and D. Metzler, "U12: Unifying language learning paradigms," 2022.
- [90] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. T. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "OPT: open pre-trained transformer language models," *CoRR*, vol. abs/2205.01068, 2022.
- [91] M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard, A. Sun, S. Wang, G. Wenzek, A. Youngblood, B. Akula, L. Barrault, G. M. Gonzalez, P. Hansanti, J. Hoffman, S. Jarrett, K. R. Sadagopan, D. Rowe, S. Spruit, C. Tran, P. Andrews, N. F. Ayan, S. Bhosale, S. Edunov, A. Fan, C. Gao, V. Goswami, F. Guzmán, P. Koehn, A. Mourachko, C. Ropers, S. Saleem, H. Schwenk, and J. Wang, "No language left behind: Scaling human-centered machine translation," *CoRR*, vol. abs/2207.04672, 2022.
- [92] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Li et al., "Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x," *arXiv preprint arXiv:2303.17568*, 2023.
- [93] A. Zeng, X. Liu, Z. Du, Z. Wang, H. Lai, M. Ding, Z. Yang, Y. Xu, W. Zheng, X. Xia, W. L. Tam, Z. Ma, Y. Xue, J. Zhai, W. Chen, P. Zhang, Y. Dong, and

- J. Tang, "GLM-130B: an open bilingual pre-trained model," vol. abs/2210.02414, 2022.
- [94] N. Muennighoff, T. Wang, L. Sutawika, A. Roberts, S. Biderman, T. L. Scao, M. S. Bari, S. Shen, Z. X. Yong, H. Schoelkopf, X. Tang, D. Radev, A. F. Aji, K. Al-mubarak, S. Albanie, Z. Alyafeai, A. Webson, E. Raff, and C. Raffel, "Crosslingual generalization through multitask finetuning," *CoRR*, vol. abs/2211.01786, 2022.
- [95] S. Iyer, X. V. Lin, R. Pasunuru, T. Mihaylov, D. Simig, P. Yu, K. Shuster, T. Wang, Q. Liu, P. S. Koura, X. Li, B. O'Horo, G. Pereyra, J. Wang, C. Dewan, A. Celikyilmaz, L. Zettlemoyer, and V. Stoyanov, "OPT-IML: scaling language model instruction meta learning through the lens of generalization," *CoRR*, vol. abs/2212.12017, 2022.
- [96] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, "Pythia: A suite for analyzing large language models across training and scaling," *arXiv preprint arXiv:2304.01373*, 2023.
- [97] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "Codegen2: Lessons for training llms on programming and natural languages," *CoRR*, vol. abs/2305.02309, 2023.
- [98] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, M. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. M. V. J. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, N. Fahmy, U. Bhattacharyya, W. Yu, S. Singh, S. Luccioni, P. Villegas, M. Kunakov, F. Zhdanov, M. Romero, T. Lee, N. Timor, J. Ding, C. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, J. Robinson, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "Starcode: may the source be with you!" *CoRR*, vol. abs/2305.06161, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.06161>
- [99] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [100] A. Yang, B. Xiao, B. Wang, B. Zhang, C. Yin, C. Lv, D. Pan, D. Wang, D. Yan, F. Yang *et al.*, "Baichuan 2: Open large-scale language models," *arXiv preprint arXiv:2309.10305*, 2023.
- [101] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.
- [102] X. Li, Y. Yao, X. Jiang, X. Fang, X. Meng, S. Fan, P. Han, J. Li, L. Du, B. Qin *et al.*, "Flan-101b: An open llm and how to train it with \$100 k budget," *arXiv preprint arXiv:2309.03852*, 2023.
- [103] T. Wei, L. Zhao, L. Zhang, B. Zhu, L. Wang, H. Yang, B. Li, C. Cheng, W. Lü, R. Hu *et al.*, "Skywork: A more open bilingual foundation model," *arXiv preprint arXiv:2310.19341*, 2023.
- [104] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, "Gshard: Scaling giant models with conditional computation and automatic sharding," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*, 2021.
- [105] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *CoRR*, vol. abs/2107.03374, 2021.
- [106] Y. Sun, S. Wang, S. Feng, S. Ding, C. Pang, J. Shang, J. Liu, X. Chen, Y. Zhao, Y. Lu, W. Liu, Z. Wu, W. Gong, J. Liang, Z. Shang, P. Sun, W. Liu, X. Ouyang, D. Yu, H. Tian, H. Wu, and H. Wang, "ERNIE 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation," *CoRR*, vol. abs/2107.02137, 2021.
- [107] O. Lieber, O. Sharir, B. Lenz, and Y. Shoham, "Jurassic-1: Technical details and evaluation," *White Paper. AI21 Labs*, vol. 1, 2021.
- [108] B. Kim, H. Kim, S. Lee, G. Lee, D. Kwak, D. H. Jeon, S. Park, S. Kim, S. Kim, D. Seo, H. Lee, M. Jeong, S. Lee, M. Kim, S. Ko, S. Kim, T. Park, J. Kim, S. Kang, N. Ryu, K. M. Yoo, M. Chang, S. Suh, S. In, J. Park, K. Kim, H. Kim, J. Jeong, Y. G. Yeo, D. Ham, D. Park, M. Y. Lee, J. Kang, I. Kang, J. Ha, W. Park, and N. Sung, "What changes can large-scale language models bring? intensive study on hyperclova: Billions-scale korean generative pretrained transformers," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*. Association for Computational Linguistics, 2021.
- [109] S. Wu, X. Zhao, T. Yu, R. Zhang, C. Shen, H. Liu, F. Li, H. Zhu, J. Luo, L. Xu *et al.*, "Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning," *arXiv preprint arXiv:2110.04725*, 2021.
- [110] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. Das-Sarma, N. Elhage, Z. Hatfield-Dodds, D. Hernandez, J. Kernion, K. Ndousse, C. Olsson, D. Amodei, T. B. Brown, J. Clark, S. McCandlish, C. Olah, and J. Kaplan, "A general language assistant as a laboratory for alignment," *CoRR*, vol. abs/2112.00861, 2021.
- [111] S. Wang, Y. Sun, Y. Xiang, Z. Wu, S. Ding, W. Gong, S. Feng, J. Shang, Y. Zhao, C. Pang, J. Liu, X. Chen, Y. Lu, W. Liu, X. Wang, Y. Bai, Q. Chen, L. Zhao, S. Li, P. Sun, D. Yu, Y. Ma, H. Tian, H. Wu, T. Wu, W. Zeng, G. Li, W. Gao, and H. Wang, "ERNIE 3.0 titan: Exploring larger-scale knowledge enhanced pre-training

- for language understanding and generation," *CoRR*, vol. abs/2112.12731, 2021.
- [112] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, B. Zoph, L. Fedus, M. P. Bosma, Z. Zhou, T. Wang, Y. E. Wang, K. Webster, M. Pellat, K. Robinson, K. S. Meier-Hellstern, T. Duke, L. Dixon, K. Zhang, Q. V. Le, Y. Wu, Z. Chen, and C. Cui, "Glam: Efficient scaling of language models with mixture-of-experts," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, 2022, pp. 5547–5569.
- [113] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V. Korthikanti, E. Zheng, R. Child, R. Y. Aminabadi, J. Bernauer, X. Song, M. Shoeybi, Y. He, M. Houston, S. Tiwary, and B. Catanzaro, "Using deepspeed and megatron to train megatron-turing NLG 530b, A large-scale generative language model," *CoRR*, vol. abs/2201.11990, 2022.
- [114] Y. Li, D. H. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P. Huang, J. Welbl, S. Goyal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with alphacode," *Science*, 2022.
- [115] S. Soltan, S. Ananthakrishnan, J. FitzGerald, R. Gupta, W. Hamza, H. Khan, C. Peris, S. Rawls, A. Rosenbaum, A. Rumshisky, C. S. Prakash, M. Sridhar, F. Triefenbach, A. Verma, G. Tr, and P. Natarajan, "Alexatm 20b: Few-shot learning using a large-scale multilingual seq2seq model," *CoRR*, vol. abs/2208.01448, 2022.
- [116] A. Glaese, N. McAleese, M. Trebacz, J. Aslanides, V. Firoiu, T. Ewalds, M. Rauh, L. Weidinger, M. Chadwick, P. Thacker, L. Campbell-Gillingham, J. Uesato, P. Huang, R. Comanescu, F. Yang, A. See, S. Dathathri, R. Greig, C. Chen, D. Fritz, J. S. Elias, R. Green, S. Mokr, N. Fernando, B. Wu, R. Foley, S. Young, I. Gabriel, W. Isaac, J. Mellor, D. Hassabis, K. Kavukcuoglu, L. A. Hendricks, and G. Irving, "Improving alignment of dialogue agents via targeted human judgements," *CoRR*, vol. abs/2209.14375, 2022.
- [117] H. Su, X. Zhou, H. Yu, Y. Chen, Z. Zhu, Y. Yu, and J. Zhou, "Welm: A well-read pre-trained language model for chinese," *CoRR*, vol. abs/2209.10372, 2022.
- [118] Y. Tay, J. Wei, H. W. Chung, V. Q. Tran, D. R. So, S. Shakeri, X. Garcia, H. S. Zheng, J. Rao, A. Chowdhery, D. Zhou, D. Metzler, S. Petrov, N. Houlsby, Q. V. Le, and M. Dehghani, "Transcending scaling laws with 0.1% extra compute," *CoRR*, vol. abs/2210.11399, 2022.
- [119] X. Ren, P. Zhou, X. Meng, X. Huang, Y. Wang, W. Wang, P. Li, X. Zhang, A. Podolskiy, G. Arshinov, A. Bout, I. Piontkovskaya, J. Wei, X. Jiang, T. Su, Q. Liu, and J. Yao, "Pangu- Σ : Towards trillion parameter language model with sparse heterogeneous computing," *CoRR*, vol. abs/2303.10845, 2023.
- [120] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen *et al.*, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
- [121] A. Radford, R. Jzefowicz, and I. Sutskever, "Learning to generate reviews and discovering sentiment," *CoRR*, vol. abs/1704.01444, 2017.
- [122] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.
- [123] B. McCann, N. S. Keskar, C. Xiong, and R. Socher, "The natural language decathlon: Multitask learning as question answering," *CoRR*, vol. abs/1806.08730, 2018.
- [124] Y. Zhang, S. Sun, M. Galley, Y. Chen, C. Brockett, X. Gao, J. Gao, J. Liu, and B. Dolan, "DIALOGPT : Large-scale generative pre-training for conversational response generation," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, ACL 2020, Online, July 5-10, 2020*, A. Celikyilmaz and T. Wen, Eds. Association for Computational Linguistics, 2020, pp. 270–278.
- [125] D. Ham, J. Lee, Y. Jang, and K. Kim, "End-to-end neural pipeline for goal-oriented dialogue systems using GPT-2," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*. Association for Computational Linguistics, 2020, pp. 583–592.
- [126] I. Drori, S. Tran, R. Wang, N. Cheng, K. Liu, L. Tang, E. Ke, N. Singh, T. L. Patti, J. Lynch, A. Shporer, N. Verma, E. Wu, and G. Strang, "A neural network solves and generates mathematics problems by program synthesis: Calculus, differential equations, linear algebra, and more," *CoRR*, vol. abs/2112.15594, 2021.
- [127] A. Neelakantan, T. Xu, R. Puri, A. Radford, J. M. Han, J. Tworek, Q. Yuan, N. Tezak, J. W. Kim, C. Hallacy, J. Heidecke, P. Shyam, B. Power, T. E. Nekoul, G. Sastry, G. Krueger, D. Schnurr, F. P. Such, K. Hsu, M. Thompson, T. Khan, T. Sherbakov, J. Jang, P. Welinder, and L. Weng, "Text and code embeddings by contrastive pre-training," *CoRR*, vol. abs/2201.10005, 2022.
- [128] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [129] N. Stiennon, L. Ouyang, J. Wu, D. M. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, "Learning to summarize from human feedback," *CoRR*, vol. abs/2009.01325, 2020.
- [130] OpenAI, "Our approach to alignment research," *OpenAI Blog*, August 2022.
- [131] —, "Introducing chatgpt," *OpenAI Blog*, November 2022.
- [132] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse, A. Jones, S. Bowman, A. Chen, T. Conerly, N. DasSarma, D. Drain, N. Elhage, S. E. Showk, S. Fort, Z. Hatfield-Dodds, T. Henighan, D. Hernandez, T. Hume, J. Jacobson, S. Johnston, S. Kravec, C. Olsson, S. Ringer, E. Tran-Johnson, D. Amodei, T. Brown, N. Joseph, S. McCandlish, C. Olah, J. Kaplan, and J. Clark, "Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned," *CoRR*, vol. abs/2209.07858, 2022.
- [133] OpenAI, "Gpt-4v(ision) system card," *OpenAI*, 2023.
- [134] —, "Lessons learned on language model safety and misuse," *OpenAI blog*, 2022.

- [135] Meta, "Introducing meta llama 3: The most capable openly available llm to date," <https://ai.meta.com/blog/meta-llama-3/>, 2024.
- [136] "Introducing Llama 3.1: Our most capable models to date," <https://ai.meta.com/blog/meta-llama-3-1/>, 2023.
- [137] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023.
- [138] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de Las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," *CoRR*, vol. abs/2401.04088, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.04088>
- [139] T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, and et al., "Gemma: Open models based on gemini research and technology," *CoRR*, vol. abs/2403.08295, 2024.
- [140] M. Rivière, S. Pathak, P. G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ramé, J. Ferret, P. Liu, P. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P. Stanczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J. Grill, B. Neyshabur, O. Bachem, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson, B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. Welty, C. A. Choquette-Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozinska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshv, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Plucinska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. P. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonnell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjöstrand, L. Usui, L. Sifre, L. Heuermann, L. Lago, and L. McNealus, "Gemma 2: Improving open language models at a practical size," *CoRR*, vol. abs/2408.00118, 2024.
- [141] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, and Z. Fan, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.
- [142] Q. Team, "Qwen2.5: A party of foundation models," September 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [143] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Rojas, G. Feng, H. Zhao, H. Lai, H. Yu, H. Wang, J. Sun, J. Zhang, J. Cheng, J. Gui, J. Tang, J. Zhang, J. Li, L. Zhao, L. Wu, L. Zhong, M. Liu, M. Huang, P. Zhang, Q. Zheng, R. Lu, S. Duan, S. Zhang, S. Cao, S. Yang, W. L. Tam, W. Zhao, X. Liu, X. Xia, X. Zhang, X. Gu, X. Lv, X. Liu, X. Liu, X. Yang, X. Song, X. Zhang, Y. An, Y. Xu, Y. Niu, Y. Yang, Y. Li, Y. Bai, Y. Dong, Z. Qi, Z. Wang, Z. Yang, Z. Du, Z. Hou, and Z. Wang, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," 2024.
- [144] H. Zhong, C. Xiao, C. Tu, T. Zhang, Z. Liu, and M. Sun, "JEC-QA: A legal-domain question answering dataset," in *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 2020, pp. 9701–9708.
- [145] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, "What disease does this patient have? a large-scale open domain question answering dataset from medical exams," *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021.
- [146] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [147] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, "Self-instruct: Aligning language model with self generated instructions," *CoRR*, vol. abs/2212.10560, 2022.
- [148] Alpaca-LoRA, "Instruct-tune llama on consumer hardware," <https://github.com/tloen/alpaca-lora>, 2023.
- [149] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [150] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, and D. Song, "Koala: A dialogue model for academic research," Blog post, April 2023.
- [151] Y. Ji, Y. Deng, Y. Gong, Y. Peng, Q. Niu, B. Ma, and X. Li, "Belle: Be everyone's large language model engine," <https://github.com/LianjiaTech/BELLE>, 2023.
- [152] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing, "Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality," 2023. [Online]. Available: <https://vicuna.lmsys.org>
- [153] D. Eccleston, "Sharegpt," <https://sharegpt.com/>, 2023.

- [154] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *CoRR*, vol. abs/2304.08485, 2023.
- [155] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, "Minigt-4: Enhancing vision-language understanding with advanced large language models," *CoRR*, vol. abs/2304.10592, 2023.
- [156] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, and S. C. H. Hoi, "Instructblip: Towards general-purpose vision-language models with instruction tuning," *CoRR*, vol. abs/2305.06500, 2023.
- [157] Y. Su, T. Lan, H. Li, J. Xu, Y. Wang, and D. Cai, "Pandagpt: One model to instruction-follow them all," 2023.
- [158] Y. Zhu, R. Kiros, R. S. Zemel, R. Salakhutdinov, R. Ur-
tasun, A. Torralba, and S. Fidler, "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books," in *2015 IEEE International Conference on Computer Vision, ICCV 2015, Santiago, Chile, December 7-13, 2015*. IEEE Computer Society, 2015, pp. 19–27.
- [159] "Project gutenber." [Online]. Available: <https://www.gutenberg.org/>
- [160] T. H. Trinh and Q. V. Le, "A simple method for common-sense reasoning," *CoRR*, vol. abs/1806.02847, 2018.
- [161] R. Zellers, A. Holtzman, H. Rashkin, Y. Bisk, A. Farhadi, F. Roesner, and Y. Choi, "Defending against neural fake news," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 9051–9062.
- [162] A. Gokaslan, V. C. E. Pavlick, and S. Tellex, "Openwebtext corpus," <http://Skylion007.github.io/OpenWebTextCorpus>, 2019.
- [163] J. Baumgartner, S. Zannettou, B. Keegan, M. Squire, and J. Blackburn, "The pushshift reddit dataset," in *Proceedings of the Fourteenth International AAAI Conference on Web and Social Media, ICWSM 2020, Held Virtually, Original Venue: Atlanta, Georgia, USA, June 8-11, 2020*. AAAI Press, 2020, pp. 830–839.
- [164] "Wikipedia." [Online]. Available: https://en.wikipedia.org/wiki/Main_Page
- [165] "Bigquery dataset." [Online]. Available: <https://cloud.google.com/bigquery?hl=zh-cn>
- [166] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800gb dataset of diverse text for language modeling," *CoRR*, vol. abs/2101.00027, 2021.
- [167] H. Laurençon, L. Saulnier, T. Wang, C. Akiki, A. V. del Moral, T. Le Scao, L. Von Werra, C. Mou, E. G. Ponferada, H. Nguyen *et al.*, "The bigscience roots corpus: A 1.6 tb composite multilingual dataset," in *Thirty-sixth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2022.
- [168] "Common crawl." [Online]. Available: <https://commoncrawl.org/>
- [169] G. Wenzek, M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin, and É. Grave, "Ccnets: Extracting high quality monolingual datasets from web crawl data," in *Proceedings of The 12th Language Resources and Evaluation Conference*, 2020, pp. 4003–4012.
- [170] T. Computer, "Redpajama: an open dataset for training large language models," <https://github.com/togethercomputer/RedPajama-Data>, 2023.
- [171] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay, "The RefinedWeb dataset for Falcon LLM: outperforming curated corpora with web data, and web data only," *arXiv preprint arXiv:2306.01116*, 2023.
- [172] C. B. Clement, M. Bierbaum, K. P. O'Keeffe, and A. A. Alemi, "On the use of arxiv as a dataset," *arXiv preprint arXiv:1905.00075*, 2019.
- [173] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. Weld, "S2ORC: The semantic scholar open research corpus," in *ACL*, 2020.
- [174] L. Soldaini and K. Lo, "peS2o (Pretraining Efficiently on S2ORC) Dataset," ODC-By, <https://github.com/allenai/pes2o>, 2023.
- [175] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf *et al.*, "The stack: 3 tb of permissively licensed source code," *arXiv preprint arXiv:2211.15533*, 2022.
- [176] B. Wang and A. Komatsuzaki, "GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model," <https://github.com/kingoflolz/mesh-transformer-jax>, 2021.
- [177] L. Soldaini, R. Kinney, A. Bhagia, D. Schwenk, D. Atkinson, R. Authur, B. Bogin, K. Chandu, J. Dumas, Y. Elazar, V. Hofmann, A. H. Jha, S. Kumar, L. Lucy, X. Lyu, N. Lambert, I. Magnusson, J. Morrison, N. Muenighoff, A. Naik, C. Nam, M. E. Peters, A. Ravichander, K. Richardson, Z. Shen, E. Strubell, N. Subramani, O. Tafjord, P. Walsh, L. Zettlemoyer, N. A. Smith, H. Hajishirzi, I. Beltagy, D. Groeneveld, J. Dodge, and K. Lo, "Dolma: an open corpus of three trillion tokens for language model pretraining research," *arXiv preprint arXiv:2402.00159*, 2024.
- [178] D. Groeneveld, I. Beltagy, P. Walsh, A. Bhagia, R. Kinney, O. Tafjord, A. H. Jha, H. Ivison, I. Magnusson, Y. Wang *et al.*, "Olmo: Accelerating the science of language models," *arXiv preprint arXiv:2402.00838*, 2024.
- [179] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi, "Cross-task generalization via natural language crowdsourcing instructions," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., 2022, pp. 3470–3487.
- [180] S. H. Bach, V. Sanh, Z. X. Yong, A. Webson, C. Raffel, N. V. Nayak, A. Sharma, T. Kim, M. S. Bari, T. Févry, Z. Alyafeai, M. Dey, A. Santilli, Z. Sun, S. Ben-David, C. Xu, G. Chhablani, H. Wang, J. A. Fries, M. S. AlShaibani, S. Sharma, U. Thakker, K. Almubarak, X. Tang, D. R. Radev, M. T. Jiang, and A. M. Rush, "Promptsources: An integrated development environment and repository for natural language prompts," in *ACL (demo)*. Association for Computational Linguistics, 2022, pp. 93–104.
- [181] T. Tang, J. Li, W. X. Zhao, and J. Wen, "MVP: multi-task supervised pre-training for natural language generation," *CoRR*, vol. abs/2206.12131, 2022.

- [182] H. Nguyen, S. Suri, K. Tsui, Shahules786, T. team, and C. Schuhmann, "The oig dataset," <https://laion.ai/blog/oig-dataset/>, 2023.
- [183] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, N. Joseph, S. Kadavath, J. Kernion, T. Conerly, S. E. Showk, N. Elhage, Z. Hatfield-Dodds, D. Hernandez, T. Hume, S. Johnston, S. Kravec, L. Lovitt, N. Nanda, C. Olsson, D. Amodei, T. B. Brown, J. Clark, S. McCandlish, C. Olah, B. Mann, and J. Kaplan, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *CoRR*, vol. abs/2204.05862, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2204.05862>
- [184] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How close is chatgpt to human experts? comparison corpus, evaluation, and detection," *arXiv preprint arXiv:2301.07597*, 2023.
- [185] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin. (2023) Free dolly: Introducing the world's first truly open instruction-tuned llm.
- [186] A. Köpf, Y. Kilcher, D. von Rütte, S. Anagnostidis, Z.-R. Tam, K. Stevens, A. Barhoum, N. M. Duc, O. Stanley, R. Nagyfi *et al.*, "Openassistant conversations—democratizing large language model alignment," *arXiv preprint arXiv:2304.07327*, 2023.
- [187] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [188] J. Cheung, "Guanaco - generative universal assistant for natural-language adaptive context-aware omnilingual outputs," <https://guanaco-model.github.io/>, 2023.
- [189] C. Xu, D. Guo, N. Duan, and J. McAuley, "Baize: An open-source chat model with parameter-efficient tuning on self-chat data," *arXiv preprint arXiv:2304.01196*, 2023.
- [190] Y. Ji, Y. Gong, Y. Deng, Y. Peng, Q. Niu, B. Ma, and X. Li, "Towards better instruction following language models for chinese: Investigating the impact of training data and evaluation," *arXiv preprint arXiv:2304.07854*, 2023.
- [191] K. Ethayarajh, Y. Choi, and S. Swayamdipta, "Understanding dataset difficulty with $\mathcal{V}$ -usable information," in *Proceedings of the 39th International Conference on Machine Learning*, 2022, pp. 5988–6008.
- [192] N. Lambert, L. Tunstall, N. Rajani, and T. Thrush. (2023) Huggingface h4 stack exchange preference dataset. [Online]. Available: <https://huggingface.co/datasets/HuggingFaceH4/stack-exchange-preferences>
- [193] R. Liu, R. Yang, C. Jia, G. Zhang, D. Zhou, A. M. Dai, D. Yang, and S. Vosoughi, "Training socially aligned language models in simulated human society," *CoRR*, vol. abs/2305.16960, 2023.
- [194] G. Xu, J. Liu, M. Yan, H. Xu, J. Si, Z. Zhou, P. Yi, X. Gao, J. Sang, R. Zhang, J. Zhang, C. Peng, F. Huang, and J. Zhou, "Cvalues: Measuring the values of chinese large language models from safety to responsibility," 2023.
- [195] J. Dai, X. Pan, R. Sun, J. Ji, X. Xu, M. Liu, Y. Wang, and Y. Yang, "Safe rlhf: Safe reinforcement learning from human feedback," *arXiv preprint arXiv:2310.12773*, 2023.
- [196] V. Sanh, A. Webson, C. Raffel, S. H. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, A. Raja, M. Dey, M. S. Bari, C. Xu, U. Thakker, S. S. Sharma, E. Szczechla, T. Kim, G. Chhablani, N. V. Nayak, D. Datta, J. Chang, M. T. Jiang, H. Wang, M. Manica, S. Shen, Z. X. Yong, H. Pandey, R. Bawden, T. Wang, T. Neeraj, J. Rozen, A. Sharma, A. Santilli, T. Févry, J. A. Fries, R. Teehan, T. L. Scao, S. Biderman, L. Gao, T. Wolf, and A. M. Rush, "Multitask prompted training enables zero-shot task generalization," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [197] S. Longpre, L. Hou, T. Vu, A. Webson, H. W. Chung, Y. Tay, D. Zhou, Q. V. Le, B. Zoph, J. Wei *et al.*, "The flan collection: Designing data and methods for effective instruction tuning," *arXiv preprint arXiv:2301.13688*, 2023.
- [198] K. Cobbe, V. Kosaraju, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *CoRR*, vol. abs/2110.14168, 2021.
- [199] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? A question answering benchmark with implicit reasoning strategies," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 346–361, 2021.
- [200] O. Camburu, B. Shillingford, P. Minervini, T. Lukasiewicz, and P. Blunsom, "Make up your mind! adversarial generation of inconsistent natural language explanations," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, D. Jurafsky, J. Chai, N. Schluter, and J. R. Tetraault, Eds. Association for Computational Linguistics, 2020, pp. 4157–4165.
- [201] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, EMNLP 2020 - Demos, Online, November 16-20, 2020*. Association for Computational Linguistics, 2020, pp. 38–45.
- [202] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: <http://github.com/google/jax>
- [203] Z. Bian, H. Liu, B. Wang, H. Huang, Y. Li, C. Wang, F. Cui, and Y. You, "Colossal-ai: A unified deep learning system for large-scale parallel training," *CoRR*, vol. abs/2110.14883, 2021.
- [204] J. Fang, Y. Yu, S. Li, Y. You, and J. Zhou, "Patrickstar: Parallel training of pre-trained models via a chunk-based memory management," *CoRR*, vol. abs/2108.05818, 2021.
- [205] Y. You, "Colossalchat: An open-source

- solution for cloning chatgpt with a complete rlhf pipeline," 2023. [Online]. Available: https://medium.com/@yangyou_berkeley/colossalchat-an-open-source-solution-for-cloning-chatgpt-with-a-complete-rlhf-pipeline-5edf08fb538b
- [206] "Bmtrain: Effient training for big models." [Online]. Available: <https://github.com/OpenBMB/BMTrain>
- [207] J. He, J. Qiu, A. Zeng, Z. Yang, J. Zhai, and J. Tang, "Fastmoe: A fast mixture-of-expert training system," *CoRR*, vol. abs/2103.13262, 2021.
- [208] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [209] (2023) Deepspeed-mii. [Online]. Available: <https://github.com/microsoft/DeepSpeed-MII>
- [210] Z. Yao, R. Y. Aminabadi, O. Ruwase, S. Rajbhandari, X. Wu, A. A. Awan, J. Rasley, M. Zhang, C. Li, C. Holmes, Z. Zhou, M. Wyatt, M. Smith, L. Kurilenko, H. Qin, M. Tanaka, S. Che, S. L. Song, and Y. He, "DeepSpeed-Chat: Easy, Fast and Affordable RLHF Training of ChatGPT-like Models at All Scales," *arXiv preprint arXiv:2308.01320*, 2023.
- [211] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 8024–8035.
- [212] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. A. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: A system for large-scale machine learning," in *12th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2016, Savannah, GA, USA, November 2-4, 2016*, K. Keeton and T. Roscoe, Eds. USENIX Association, 2016, pp. 265–283.
- [213] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, and Z. Zhang, "Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems," *CoRR*, vol. abs/1512.01274, 2015.
- [214] Y. Ma, D. Yu, T. Wu, and H. Wang, "Paddlepaddle: An open-source deep learning platform from industrial practice," *Frontiers of Data and Domputing*, vol. 1, no. 1, p. 105, 2019.
- [215] L. Huawei Technologies Co., "Huawei mindspore ai development framework," in *Artificial Intelligence Technology*. Springer, 2022, pp. 137–162.
- [216] J. Yuan, X. Li, C. Cheng, J. Liu, R. Guo, S. Cai, C. Yao, F. Yang, X. Yi, C. Wu, H. Zhang, and J. Zhao, "Oneflow: Redesign the distributed deep learning framework from scratch," *CoRR*, vol. abs/2110.15032, 2021.
- [217] S. Roller, E. Dinan, N. Goyal, D. Ju, M. Williamson, Y. Liu, J. Xu, M. Ott, E. M. Smith, Y. Boureau, and J. Weston, "Recipes for building an open-domain chatbot," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL 2021, Online, April 19 - 23, 2021*, 2021, pp. 300–325.
- [218] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, Y. Wu, B. Neyshabur, G. Gur-Ari, and V. Misra, "Solving quantitative reasoning problems with language models," *CoRR*, vol. abs/2206.14858, 2022.
- [219] T. Saier, J. Krause, and M. Färber, "unaxrvice 2022: All arxiv publications pre-processed for nlp, including structured full-text and citation network," *arXiv preprint arXiv:2303.14957*, 2023.
- [220] H. A. Simon, "Experiments with a heuristic compiler," *J. ACM*, vol. 10, no. 4, pp. 493–506, 1963.
- [221] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, no. 3, pp. 151–165, 1971.
- [222] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.
- [223] J. Austin, A. Odena, M. I. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. V. Le, and C. Sutton, "Program synthesis with large language models," *CoRR*, vol. abs/2108.07732, 2021.
- [224] S. Black, L. Gao, P. Wang, C. Leahy, and S. Biderman, "GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow," 2021.
- [225] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *MAPS@PLDI*, 2022.
- [226] A. Madaan, S. Zhou, U. Alon, Y. Yang, and G. Neubig, "Language models of code are few-shot commonsense learners," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Association for Computational Linguistics, 2022, pp. 1384–1403.
- [227] S. Longpre, G. Yauney, E. Reif, K. Lee, A. Roberts, B. Zoph, D. Zhou, J. Wei, K. Robinson, D. Mimno *et al.*, "A pretrainer's guide to training data: Measuring the effects of data age, domain coverage, quality, & toxicity," *arXiv preprint arXiv:2305.13169*, 2023.
- [228] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao, Y. Li, B. Ding, and J. Zhou, "Data-juicer: A one-stop data processing system for large language models," 2023.
- [229] M. Abidin, S. A. Jacobs, A. A. Awan, J. Aneja, A. Awadallah, H. Awadalla, N. Bach, A. Bahree, A. Bakhtiari, H. Behl *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
- [230] G. Penedo, H. Kydlíček, A. Lozhkov, M. Mitchell, C. Raffel, L. Von Werra, T. Wolf *et al.*, "The fineweb datasets:

- Decanting the web for the finest text data at scale," *arXiv preprint arXiv:2406.17557*, 2024.
- [231] P. Maini, S. Seto, H. Bai, D. Grangier, Y. Zhang, and N. Jaitly, "Rephrasing the web: A recipe for compute and data-efficient language modeling," in *ICLR 2024 Workshop on Navigating and Addressing Data Problems for Foundation Models*, 2024.
- [232] M. Marion, A. Üstün, L. Pozzobon, A. Wang, M. Fadaee, and S. Hooker, "When less is more: Investigating data pruning for pretraining llms at scale," *arXiv preprint arXiv:2309.04564*, 2023.
- [233] N. Sachdeva, B. Coleman, W.-C. Kang, J. Ni, L. Hong, E. H. Chi, J. Caverlee, J. McAuley, and D. Z. Cheng, "How to train data-efficient llms," *arXiv preprint arXiv:2402.09668*, 2024.
- [234] D. Hernandez, T. B. Brown, T. Conerly, N. DasSarma, D. Drain, S. E. Showk, N. Elhage, Z. Hatfield-Dodds, T. Henighan, T. Hume, S. Johnston, B. Mann, C. Olah, C. Olsson, D. Amodei, N. Joseph, J. Kaplan, and S. McCandlish, "Scaling laws and interpretability of learning from repeated data," *CoRR*, vol. abs/2205.10487, 2022.
- [235] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.
- [236] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, pp. 8424–8445.
- [237] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, "Quantifying memorization across neural language models," *CoRR*, 2022.
- [238] N. Kandpal, E. Wallace, and C. Raffel, "Deduplicating training data mitigates privacy risks in language models," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*. PMLR, 2022, pp. 10 697–10 707.
- [239] J. D. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proceedings of the Eighteenth International Conference on Machine Learning (ICML 2001), Williams College, Williamstown, MA, USA, June 28 - July 1, 2001*, C. E. Brodley and A. P. Danyluk, Eds. Morgan Kaufmann, 2001, pp. 282–289.
- [240] P. Gage, "A new algorithm for data compression," *C Users Journal*, vol. 12, no. 2, pp. 23–38, 1994.
- [241] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics, 2016.
- [242] M. Schuster and K. Nakajima, "Japanese and korean voice search," in *2012 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2012, pp. 5149–5152.
- [243] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, L. Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, "Google's neural machine translation system: Bridging the gap between human and machine translation," *CoRR*, vol. abs/1609.08144, 2016.
- [244] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 1: Long Papers*, I. Gurevych and Y. Miyao, Eds. Association for Computational Linguistics, 2018, pp. 66–75.
- [245] T. Kudo and J. Richardson, "Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018: System Demonstrations, Brussels, Belgium, October 31 - November 4, 2018*, E. Blanco and W. Lu, Eds. Association for Computational Linguistics, 2018.
- [246] M. Davis and M. Dürst, "Unicode normalization forms," 2001.
- [247] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever, "Deep double descent: Where bigger models and more data hurt," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.
- [248] K. Tirumala, D. Simig, A. Aghajanyan, and A. S. Morcos, "D4: Improving llm pretraining via document de-duplication and diversification," *arXiv preprint arXiv:2308.12284*, 2023.
- [249] Z. Shen, T. Tao, L. Ma, W. Neiswanger, J. Hestness, N. Vassilieva, D. Soboleva, and E. Xing, "Slimpajama-dc: Understanding data combinations for llm training," *arXiv preprint arXiv:2309.10818*, 2023.
- [250] S. M. Xie, S. Santurkar, T. Ma, and P. Liang, "Data selection for language models via importance resampling," *arXiv preprint arXiv:2302.03169*, 2023.
- [251] X. Wang, W. Zhou, Q. Zhang, J. Zhou, S. Gao, J. Wang, M. Zhang, X. Gao, Y. Chen, and T. Gui, "Farewell to aimless large-scale pretraining: Influential subset selection for language model," *arXiv preprint arXiv:2305.12816*, 2023.
- [252] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, "The LAMBADA dataset: Word prediction requiring a broad discourse context," in *ACL (1)*. The Association for Computer Linguistics, 2016.
- [253] M. F. Chen, N. Roberts, K. Bhatia, J. Wang, C. Zhang, F. Sala, and C. Ré, "Skill-it! a data-driven skills framework for understanding and training language models," *arXiv preprint arXiv:2307.14430*, 2023.
- [254] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. Canton-Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for

- code," *CoRR*, vol. abs/2308.12950, 2023.
- [255] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *ICML*, 2009, pp. 41–48.
- [256] C. Xu, C. Rosset, L. Del Corro, S. Mahajan, J. McAuley, J. Neville, A. H. Awadallah, and N. Rao, "Contrastive post-training large language models on data curriculum," *arXiv preprint arXiv:2310.02263*, 2023.
- [257] S. Tworkowski, K. Staniszewski, M. Pacek, Y. Wu, H. Michalewski, and P. Milos, "Focused transformer: Contrastive training for context scaling," *CoRR*, vol. abs/2307.03170, 2023.
- [258] Z. Azerbayev, H. Schoelkopf, K. Paster, M. D. Santos, S. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck, "Llemma: An open language model for mathematics," *arXiv preprint arXiv:2310.10631*, 2023.
- [259] S. Chen, S. Wong, L. Chen, and Y. Tian, "Extending context window of large language models via positional interpolation," *CoRR*, vol. abs/2306.15595, 2023.
- [260] G. Wenzek, M.-A. Lachaux, A. Conneau, V. Chaudhary, F. Guzmán, A. Joulin, and É. Grave, "Ccnets: Extracting high quality monolingual datasets from web crawl data," in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2020, pp. 4003–4012.
- [261] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *EACL*, 2017, pp. 427–431.
- [262] D. Chen, Y. Huang, Z. Ma, H. Chen, X. Pan, C. Ge, D. Gao, Y. Xie, Z. Liu, J. Gao *et al.*, "Data-juicer: A one-stop data processing system for large language models," *arXiv preprint arXiv:2309.02033*, 2023.
- [263] B. Zhang, B. Ghorbani, A. Bapna, Y. Cheng, X. Garcia, J. Shen, and O. Firat, "Examining scaling and transfer of language model architectures for machine translation," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, 2022, pp. 26 176–26 192.
- [264] L. Dong, N. Yang, W. Wang, F. Wei, X. Liu, Y. Wang, J. Gao, M. Zhou, and H. Hon, "Unified language model pre-training for natural language understanding and generation," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 2019, pp. 13 042–13 054.
- [265] A. Clark, D. de Las Casas, A. Guy, A. Mensch, M. Paganini, J. Hoffmann, B. Damoc, B. A. Hechtman, T. Cai, S. Borgeaud, G. van den Driessche, E. Rutherford, T. Hennigan, M. J. Johnson, A. Casirer, C. Jones, E. Buchatskaya, D. Budden, L. Sifre, S. Osindero, O. Vinyals, M. Ranzato, J. W. Rae, E. Elsen, K. Kavukcuoglu, and K. Simonyan, "Unified scaling laws for routed language models," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, 2022, pp. 4057–4086.
- [266] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. [Online]. Available: <https://openreview.net/forum?id=uYLFoz1vIAC>
- [267] J. T. Smith, A. Warrington, and S. Linderman, "Simplified state space layers for sequence modeling," in *ICLR*, 2023.
- [268] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu, and S. De, "Resurrecting recurrent neural networks for long sequences," in *ICML*, 2023.
- [269] M. Poli, S. Massaroli, E. Nguyen, D. Y. Fu, T. Dao, S. Baccus, Y. Bengio, S. Ermon, and C. Ré, "Hyena hierarchy: Towards larger convolutional language models," in *ICML*, 2023.
- [270] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, H. Cao, X. Cheng, M. Chung, M. Grella, K. K. G. V., X. He, H. Hou, P. Kazienko, J. Kocon, J. Kong, B. Koptyra, H. Lau, K. S. I. Mantri, F. Mom, A. Saito, X. Tang, B. Wang, J. S. Wind, S. Wozniak, R. Zhang, Z. Zhang, Q. Zhao, P. Zhou, J. Zhu, and R. Zhu, "RWKV: reinventing rnns for the transformer era," *CoRR*, vol. abs/2305.13048, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.13048>
- [271] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, "Retentive network: A successor to transformer for large language models," *arXiv preprint arXiv:2307.08621*, 2023.
- [272] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *CoRR*, vol. abs/2312.00752, 2023.
- [273] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, H. Cao, X. Cheng, M. Chung, M. Grella, K. K. GV *et al.*, "Rwkv: Reinventing rnns for the transformer era," *arXiv preprint arXiv:2305.13048*, 2023.
- [274] M. Ding, Z. Yang, W. Hong, W. Zheng, C. Zhou, D. Yin, J. Lin, X. Zou, Z. Shao, H. Yang, and J. Tang, "Cogview: Mastering text-to-image generation via transformers," in *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, 2021, pp. 19 822–19 835.
- [275] L. J. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," vol. abs/1607.06450, 2016.
- [276] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, 2019, pp. 12 360–12 371.
- [277] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, and F. Wei, "Deepnet: Scaling transformers to 1, 000 layers," vol. abs/2203.00555, 2022.
- [278] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.
- [279] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the Workshop: Analyzing and Interpreting Neural Networks for NLP, BlackboxNLP@EMNLP 2018, Brussels, Belgium, November 1, 2018*, T. Linzen, G. Chrupala, and A. Alishahi, Eds. Association for Computational Linguistics, 2018, pp. 353–355.
- [280] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

- [281] N. Shazeer, "GLU variants improve transformer," vol. abs/2002.05202, 2020.
- [282] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," vol. abs/2104.09864, 2021.
- [283] O. Press, N. A. Smith, and M. Lewis, "Train short, test long: Attention with linear biases enables input length extrapolation," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*, 2022.
- [284] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, ser. JMLR Workshop and Conference Proceedings, F. R. Bach and D. M. Blei, Eds., vol. 37. JMLR.org, 2015, pp. 448–456. [Online]. Available: <http://proceedings.mlr.press/v37/ioffe15.html>
- [285] S. Narang, H. W. Chung, Y. Tay, L. Fedus, T. F  vry, M. Matena, K. Malkan, N. Fiedel, N. Shazeer, Z. Lan, Y. Zhou, W. Li, N. Ding, J. Marcus, A. Roberts, and C. Raffel, "Do transformer modifications transfer across implementations and applications?" in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, 2021, pp. 5758–5773.
- [286] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu, "On layer normalization in the transformer architecture," in *ICML*, 2020.
- [287] A. Baevski and M. Auli, "Adaptive input representations for neural language modeling," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
- [288] L. Liu, X. Liu, J. Gao, W. Chen, and J. Han, "Understanding the difficulty of training transformers," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. Association for Computational Linguistics, 2020, pp. 5747–5763.
- [289] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.
- [290] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," in *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*, 2017, pp. 933–941.
- [291] T. L. Scao, T. Wang, D. Hesslow, S. Bekman, M. S. Bari, S. Biderman, H. Elsahar, N. Muennighoff, J. Phang, O. Press, C. Raffel, V. Sanh, S. Shen, L. Sutawika, J. Tae, Z. X. Yong, J. Launay, and I. Beltagy, "What language model to train if you have one million GPU hours?" in *Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, 2022, pp. 765–782.
- [292] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)*, M. A. Walker, H. Ji, and A. Stent, Eds. Association for Computational Linguistics, 2018, pp. 464–468. [Online]. Available: <https://doi.org/10.18653/v1/n18-2074>
- [293] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. V. Le, and R. Salakhutdinov, "Transformer-xl: Attentive language models beyond a fixed-length context," in *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, A. Korhonen, D. R. Traum, and L. M  rquez, Eds. Association for Computational Linguistics, 2019, pp. 2978–2988. [Online]. Available: <https://doi.org/10.18653/v1/p19-1285>
- [294] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. R. Salakhutdinov, and Q. V. Le, "Xlnet: Generalized autoregressive pretraining for language understanding," *Advances in neural information processing systems*, vol. 32, 2019.
- [295] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "Yarn: Efficient context window extension of large language models," *CoRR*, vol. abs/2309.00071, 2023.
- [296] Y. Sun, L. Dong, B. Patra, S. Ma, S. Huang, A. Benhaim, V. Chaudhary, X. Song, and F. Wei, "A length-extrapolatable transformer," *CoRR*, vol. abs/2212.10554, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2212.10554>
- [297] H. Peng, N. Pappas, D. Yogatama, R. Schwartz, N. A. Smith, and L. Kong, "Random feature attention," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.
- [298] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Onta  n, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [299] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *CoRR*, vol. abs/1904.10509, 2019.
- [300] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *CoRR*, vol. abs/1911.02150, 2019. [Online]. Available: <http://arxiv.org/abs/1911.02150>
- [301] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebr  n, and S. Shanghai, "Gqa: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.
- [302] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "Flashattention: Fast and memory-efficient exact attention with IO-awareness," in *NeurIPS*, 2022.
- [303] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.
- [304] "vllm: Easy, fast, and cheap llm serving with pagedattention." [Online]. Available: <https://vllm.ai/>
- [305] K. Murray and D. Chiang, "Correcting length bias in neural machine translation," in *WMT*. Association for Computational Linguistics, 2018, pp. 212–223.
- [306] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi,

- "The curious case of neural text degeneration," in *ICLR*, 2020.
- [307] C.-M. U. P. P. D. O. C. SCIENCE, *Speech Understanding Systems. Summary of Results of the Five-Year Research Effort at Carnegie-Mellon University*, 1977.
- [308] P. Koehn and R. Knowles, "Six challenges for neural machine translation," in *NMT@ACL*. Association for Computational Linguistics, 2017, pp. 28–39.
- [309] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, L. Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, "Google's neural machine translation system: Bridging the gap between human and machine translation," *CoRR*, vol. abs/1609.08144, 2016.
- [310] R. Paulus, C. Xiong, and R. Socher, "A deep reinforced model for abstractive summarization," in *ICLR (Poster)*. OpenReview.net, 2018.
- [311] A. K. Vijayakumar, M. Cogswell, R. R. Selvaraju, Q. Sun, S. Lee, D. J. Crandall, and D. Batra, "Diverse beam search: Decoding diverse solutions from neural sequence models," *CoRR*, vol. abs/1610.02424, 2016.
- [312] A. Fan, M. Lewis, and Y. N. Dauphin, "Hierarchical neural story generation," in *ACL (1)*. Association for Computational Linguistics, 2018, pp. 889–898.
- [313] J. Hewitt, C. D. Manning, and P. Liang, "Truncation sampling as language model desmoothing," in *EMNLP (Findings)*. Association for Computational Linguistics, 2022, pp. 3414–3427.
- [314] Y. Su, T. Lan, Y. Wang, D. Yogatama, L. Kong, and N. Collier, "A contrastive framework for neural text generation," in *NeurIPS*, 2022.
- [315] C. Meister, T. Pimentel, G. Wiher, and R. Cotterell, "Locally typical sampling," *Trans. Assoc. Comput. Linguistics*, 2023.
- [316] X. L. Li, A. Holtzman, D. Fried, P. Liang, J. Eisner, T. Hashimoto, L. Zettlemoyer, and M. Lewis, "Contrastive decoding: Open-ended text generation as optimization," in *ACL (1)*. Association for Computational Linguistics, 2023, pp. 12 286–12 312.
- [317] Y. Chuang, Y. Xie, H. Luo, Y. Kim, J. R. Glass, and P. He, "Dola: Decoding by contrasting layers improves factuality in large language models," *CoRR*, vol. abs/2309.03883, 2023.
- [318] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2015.
- [319] I. Loshchilov and F. Hutter, "Fixing weight decay regularization in adam," *CoRR*, vol. abs/1711.05101, 2017.
- [320] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," in *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018*, ser. Proceedings of Machine Learning Research, J. G. Dy and A. Krause, Eds., vol. 80. PMLR, 2018, pp. 4603–4611.
- [321] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. X. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, Eds., 2019, pp. 103–112.
- [322] A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, and P. B. Gibbons, "Pipedream: Fast and efficient pipeline parallel DNN training," *CoRR*, vol. abs/1806.03377, 2018.
- [323] P. Micikevicius, S. Narang, J. Alben, G. F. Diamos, E. Elsen, D. García, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," *CoRR*, vol. abs/1710.03740, 2017.
- [324] Q. Xu, S. Li, C. Gong, and Y. You, "An efficient 2d method for training super-large deep learning models," *CoRR*, vol. abs/2104.05343, 2021.
- [325] B. Wang, Q. Xu, Z. Bian, and Y. You, "Tesseract: Parallelize the tensor parallelism efficiently," in *Proceedings of the 51st International Conference on Parallel Processing, ICPP 2022, Bordeaux, France, 29 August 2022 - 1 September 2022*. ACM, 2022.
- [326] Z. Bian, Q. Xu, B. Wang, and Y. You, "Maximizing parallelism in distributed training for huge neural networks," *CoRR*, vol. abs/2105.14450, 2021.
- [327] S. Li, F. Xue, C. Baranwal, Y. Li, and Y. You, "Sequence parallelism: Long sequence training from system perspective," *arXiv e-prints*, pp. arXiv–2105, 2021.
- [328] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing et al., "Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning," in *OSDI*, 2022, pp. 559–578.
- [329] T. Chen, B. Xu, C. Zhang, and C. Guestrin, "Training deep nets with sublinear memory cost," *CoRR*, vol. abs/1604.06174, 2016.
- [330] FairScale authors, "Fairscale: A general purpose modular pytorch library for high performance and large scale training," <https://github.com/facebookresearch/fairscale>, 2021.
- [331] R. Lou, K. Zhang, and W. Yin, "Is prompt all you need? no. A comprehensive and broader view of instruction learning," *CoRR*, vol. abs/2303.10475, 2023.
- [332] X. Liu, P. He, W. Chen, and J. Gao, "Multi-task deep neural networks for natural language understanding," in *ACL (1)*. Association for Computational Linguistics, 2019, pp. 4487–4496.
- [333] A. Aghajanyan, A. Gupta, A. Shrivastava, X. Chen, L. Zettlemoyer, and S. Gupta, "Muppet: Massive multi-task representations with pre-finetuning," in *EMNLP (1)*. Association for Computational Linguistics, 2021, pp. 5799–5811.
- [334] S. Longpre, L. Hou, T. Vu, A. Webson, H. W. Chung, Y. Tay, D. Zhou, Q. V. Le, B. Zoph, J. Wei, and A. Roberts, "The flan collection: Designing data and methods for effective instruction tuning," *CoRR*, vol. abs/2301.13688, 2023.
- [335] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang, "Wizardlm: Empowering large language models to follow complex instructions,"

- CoRR, vol. abs/2304.12244, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.12244>
- [336] Z. Sun, Y. Shen, Q. Zhou, H. Zhang, Z. Chen, D. Cox, Y. Yang, and C. Gan, "Principle-driven self-alignment of language models from scratch with minimal human supervision," *arXiv preprint arXiv:2305.03047*, 2023.
- [337] X. Li, P. Yu, C. Zhou, T. Schick, L. Zettlemoyer, O. Levy, J. Weston, and M. Lewis, "Self-alignment with instruction backtranslation," *CoRR*, vol. abs/2308.06259, 2023.
- [338] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu et al., "Lima: Less is more for alignment," *arXiv preprint arXiv:2305.11206*, 2023.
- [339] L. Chen, S. Li, J. Yan, H. Wang, K. Gunaratna, V. Yadav, Z. Tang, V. Srinivasan, T. Zhou, H. Huang, and H. Jin, "Alpagasus: Training A better alpaca with fewer data," *CoRR*, vol. abs/2307.08701, 2023.
- [340] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. H. Awadallah, "Orca: Progressive learning from complex explanation traces of GPT-4," *CoRR*, vol. abs/2306.02707, 2023.
- [341] YuLan-Chat-Team, "Yulan-chat: An open-source bilingual chatbot," <https://github.com/RUC-GSAI/YuLan-Chat>, 2023.
- [342] Y. Huang, X. Liu, Y. Gong, Z. Gou, Y. Shen, N. Duan, and W. Chen, "Key-point-driven data synthesis with its enhancement on mathematical reasoning," *CoRR*, vol. abs/2403.02333, 2024.
- [343] N. Ding, Y. Chen, B. Xu, Y. Qin, S. Hu, Z. Liu, M. Sun, and B. Zhou, "Enhancing chat language models by scaling high-quality instructional conversations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Association for Computational Linguistics, 2023, pp. 3029–3051.
- [344] K. Zhou, B. Zhang, J. Wang, Z. Chen, W. X. Zhao, J. Sha, Z. Sheng, S. Wang, and J. Wen, "Jiuzhang3.0: Efficiently improving mathematical reasoning by training small data synthesis models," *CoRR*, vol. abs/2405.14365, 2024.
- [345] Y. Cao, Y. Kang, and L. Sun, "Instruction mining: High-quality instruction data selection for large language models," *CoRR*, vol. abs/2307.06290, 2023.
- [346] M. Li, Y. Zhang, Z. Li, J. Chen, L. Chen, N. Cheng, J. Wang, T. Zhou, and J. Xiao, "From quantity to quality: Boosting LLM performance with self-guided data selection for instruction tuning," *CoRR*, vol. abs/2308.12032, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2308.12032>
- [347] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," in *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. [Online]. Available: <https://openreview.net/forum?id=H1aluk-RW>
- [348] M. Xia, S. Malladi, S. Gururangan, S. Arora, and D. Chen, "LESS: selecting influential data for targeted instruction tuning," *CoRR*, vol. abs/2402.04333, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2402.04333>
- [349] P. W. Koh and P. Liang, "Understanding black-box pre-dictions via influence functions," in *International conference on machine learning*. PMLR, 2017, pp. 1885–1894.
- [350] Y. Wang, H. Ivison, P. Dasigi, J. Hessel, T. Khot, K. R. Chandu, D. Wadden, K. MacMillan, N. A. Smith, I. Beltagy, and H. Hajishirzi, "How far can camels go? exploring the state of instruction tuning on open resources," *CoRR*, vol. abs/2306.04751, 2023.
- [351] X. Liu, H. Yan, S. Zhang, C. An, X. Qiu, and D. Lin, "Scaling laws of rope-based extrapolation," *CoRR*, vol. abs/2310.05209, 2023.
- [352] B. Peng, C. Li, P. He, M. Galley, and J. Gao, "Instruction tuning with GPT-4," *CoRR*, vol. abs/2304.03277, 2023.
- [353] M. M. Krell, M. Kosec, S. P. Perez, and A. Fitzgibbon, "Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance," *arXiv preprint arXiv:2107.02027*, 2021.
- [354] K. Singhal, S. Azizi, T. Tu, S. S. Mahdavi, J. Wei, H. W. Chung, N. Scales, A. Tanwani, H. Cole-Lewis, S. Pfohl et al., "Large language models encode clinical knowledge," *arXiv preprint arXiv:2212.13138*, 2022.
- [355] J. Zhang, R. Xie, Y. Hou, W. X. Zhao, L. Lin, and J. Wen, "Recommendation as instruction following: A large language model empowered recommendation approach," *CoRR*, vol. abs/2305.07001, 2023.
- [356] H. Wang, C. Liu, N. Xi, Z. Qiang, S. Zhao, B. Qin, and T. Liu, "Huatuo: Tuning llama model with chinese medical knowledge," *arXiv preprint arXiv:2304.06975*, 2023.
- [357] Q. Huang, M. Tao, Z. An, C. Zhang, C. Jiang, Z. Chen, Z. Wu, and Y. Feng, "Lawyer llama technical report," *arXiv preprint arXiv:2305.15062*, 2023.
- [358] S. Wu, O. Irsoy, S. Lu, V. Dabrovolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [359] T. Liu and B. K. H. Low, "Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks," *arXiv preprint arXiv:2305.14201*, 2023.
- [360] T. Sun, X. Zhang, Z. He, P. Li, Q. Cheng, H. Yan, X. Liu, Y. Shao, Q. Tang, X. Zhao, K. Chen, Y. Zheng, Z. Zhou, R. Li, J. Zhan, Y. Zhou, L. Li, X. Yang, L. Wu, Z. Yin, X. Huang, and X. Qiu, "Moss: Training conversational language models from synthetic data," 2023.
- [361] Y. Dubois, X. Li, R. Taori, T. Zhang, I. Gulrajani, J. Ba, C. Guestrin, P. Liang, and T. B. Hashimoto, "Alpacafarm: A simulation framework for methods that learn from human feedback," *CoRR*, vol. abs/2305.14387, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.14387>
- [362] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *ICLR*. OpenReview.net, 2021.
- [363] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, and J. Wei, "Challenging big-bench tasks and whether chain-of-thought can solve them," *CoRR*, vol. abs/2210.09261, 2022.
- [364] Z. Kenton, T. Everitt, L. Weidinger, I. Gabriel, V. Mikulik, and G. Irving, "Alignment of language agents," *CoRR*, vol. abs/2103.14659, 2021.

- [365] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. F. Christiano, and G. Irving, "Fine-tuning language models from human preferences," *CoRR*, vol. abs/1909.08593, 2019.
- [366] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma, N. Elhage, Z. Hatfield-Dodds, D. Hernandez, J. Kernion, K. Ndousse, C. Olsson, D. Amodei, T. B. Brown, J. Clark, S. McCandlish, C. Olah, and J. Kaplan, "A general language assistant as a laboratory for alignment," *CoRR*, vol. abs/2112.00861, 2021.
- [367] E. Perez, S. Huang, H. F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, "Red teaming language models with language models," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Association for Computational Linguistics, 2022, pp. 3419–3448.
- [368] J. Menick, M. Trebacz, V. Mikulik, J. Aslanides, H. F. Song, M. Chadwick, M. Glaese, S. Young, L. Campbell-Gillingham, G. Irving, and N. McAleese, "Teaching language models to support answers with verified quotes," *CoRR*, vol. abs/2203.11147, 2022.
- [369] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosiute, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan, "Constitutional AI: harmlessness from AI feedback," *CoRR*, vol. abs/2212.08073, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2212.08073>
- [370] H. Lee, S. Phatale, H. Mansoor, K. Lu, T. Mesnard, C. Bishop, V. Carbune, and A. Rastogi, "RLAIF: scaling reinforcement learning from human feedback with AI feedback," *CoRR*, vol. abs/2309.00267, 2023.
- [371] H. Dong, W. Xiong, D. Goyal, R. Pan, S. Diao, J. Zhang, K. Shum, and T. Zhang, "RAFT: reward ranked finetuning for generative foundation model alignment," *CoRR*, vol. abs/2304.06767, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.06767>
- [372] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma *et al.*, "A general language assistant as a laboratory for alignment," *arXiv preprint arXiv:2112.00861*, 2021.
- [373] R. Zheng, S. Dou, S. Gao, W. Shen, B. Wang, Y. Liu, S. Jin, Q. Liu, L. Xiong, L. Chen *et al.*, "Secrets of rlhf in large language models part i: Ppo," *arXiv preprint arXiv:2307.04964*, 2023.
- [374] J. Uesato, N. Kushman, R. Kumar, H. F. Song, N. Y. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins, "Solving math word problems with process- and outcome-based feedback," *CoRR*, vol. abs/2211.14275, 2022.
- [375] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," *CoRR*, vol. abs/2305.20050, 2023.
- [376] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt, "Measuring coding challenge competence with APPS," in *NeurIPS Datasets and Benchmarks*, 2021.
- [377] T. Wang, P. Yu, X. E. Tan, S. O'Brien, R. Pasunuru, J. Dwivedi-Yu, O. Golovneva, L. Zettlemoyer, M. Fazel-Zarandi, and A. Celikyilmaz, "Shepherd: A critic for language model generation," *CoRR*, vol. abs/2308.04592, 2023.
- [378] G. Chen, M. Liao, C. Li, and K. Fan, "Alphamath almost zero: process supervision without process," *CoRR*, vol. abs/2405.03553, 2024.
- [379] Q. Ma, H. Zhou, T. Liu, J. Yuan, P. Liu, Y. You, and H. Yang, "Let's reward step by step: Step-level reward model as the navigators for reasoning," *CoRR*, vol. abs/2310.10080, 2023.
- [380] Z. Chen, K. Zhou, W. X. Zhao, J. Wan, F. Zhang, D. Zhang, and J. Wen, "Improving large language models via fine-grained reinforcement learning with minimum editing constraint," *CoRR*, vol. abs/2401.06081, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2401.06081>
- [381] Z. Xi, W. Chen, B. Hong, S. Jin, R. Zheng, W. He, Y. Ding, S. Liu, X. Guo, J. Wang, H. Guo, W. Shen, X. Fan, Y. Zhou, S. Dou, X. Wang, X. Zhang, P. Sun, T. Gui, Q. Zhang, and X. Huang, "Training large language models for reasoning through reverse curriculum reinforcement learning," *CoRR*, vol. abs/2402.05808, 2024.
- [382] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. P. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nat.*, pp. 354–359, 2017.
- [383] T. Anthony, Z. Tian, and D. Barber, "Thinking fast and slow with deep learning and tree search," in *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, 2017*, pp. 5360–5370.
- [384] H. Luo, Q. Sun, C. Xu, P. Zhao, J. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, and D. Zhang, "Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct," *CoRR*, vol. abs/2308.09583, 2023.
- [385] R. Liu, C. Jia, G. Zhang, Z. Zhuang, T. X. Liu, and S. Vosoughi, "Second thoughts are best: Learning to re-align with human values from text edits," in *NeurIPS*, 2022.
- [386] X. Lu, S. Welleck, J. Hessel, L. Jiang, L. Qin, P. West, P. Ammanabrolu, and Y. Choi, "QUARK: controllable text generation with reinforced unlearning," in *NeurIPS*, 2022.
- [387] J. Scheurer, J. A. Campos, T. Korbak, J. S. Chan, A. Chen, K. Cho, and E. Perez, "Training language

- models with language feedback at scale," *CoRR*, vol. abs/2303.16755, 2023.
- [388] G. Guo, R. Zhao, T. Tang, W. X. Zhao, and J.-R. Wen, "Beyond imitation: Leveraging fine-grained quality signals for alignment," *arXiv preprint arXiv:2311.04072*, 2023.
- [389] R. Krishna, D. Lee, L. Fei-Fei, and M. S. Bernstein, "Socially situated artificial intelligence enables learning from human interaction," *Proceedings of the National Academy of Sciences of the United States of America*, vol. 119, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:252381954>
- [390] H. Liu, C. Sferrazza, and P. Abbeel, "Chain of hindsight aligns language models with feedback," *CoRR*, vol. abs/2302.02676, 2023.
- [391] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *CoRR*, vol. abs/2305.18290, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.18290>
- [392] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, "KTO: model alignment as prospect theoretic optimization," *CoRR*, vol. abs/2402.01306, 2024.
- [393] Y. Meng, M. Xia, and D. Chen, "SimpO: Simple preference optimization with a reference-free reward," *CoRR*, vol. abs/2405.14734, 2024.
- [394] D. Feng, B. Qin, C. Huang, Z. Zhang, and W. Lei, "Towards analyzing and understanding the limitations of DPO: A theoretical perspective," *CoRR*, vol. abs/2404.04626, 2024.
- [395] A. Gorbатовski, B. Shaposhnikov, A. Malakhov, N. Surnachev, Y. Aksenov, I. Maksimov, N. Balagansky, and D. Gavrilov, "Learn your reference model for real good alignment," *CoRR*, vol. abs/2404.09656, 2024.
- [396] D. Kim, Y. Kim, W. Song, H. Kim, Y. Kim, S. Kim, and C. Park, "sdpo: Don't use your data all at once," *CoRR*, vol. abs/2403.19270, 2024.
- [397] Z. Yuan, H. Yuan, C. Tan, W. Wang, S. Huang, and F. Huang, "RRHF: rank responses to align language models with human feedback without tears," *CoRR*, vol. abs/2304.05302, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2304.05302>
- [398] Y. Zhao, R. Joshi, T. Liu, M. Khalman, M. Saleh, and P. J. Liu, "Slic-hf: Sequence likelihood calibration with human feedback," *CoRR*, vol. abs/2305.10425, 2023.
- [399] A. Fisch, J. Eisenstein, V. Zayats, A. Agarwal, A. Beirami, C. Nagpal, P. Shaw, and J. Berant, "Robust preference optimization through reward model distillation," *CoRR*, vol. abs/2405.19316, 2024.
- [400] T. Zhang, F. Liu, J. Wong, P. Abbeel, and J. E. Gonzalez, "The wisdom of hindsight makes language models better instruction followers," *CoRR*, vol. abs/2302.05206, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.05206>
- [401] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Comput. Surv.*, vol. 50, no. 2, apr 2017. [Online]. Available: <https://doi.org/10.1145/3054912>
- [402] S. Levine, "Should i imitate or reinforce," 2022. [Online]. Available: <https://www.youtube.com/watch?v=sVPm7zOrBxM>
- [403] J. Schulman, "Reinforcement learning from human feedback: Progress and challenges," 2023. [Online]. Available: https://www.youtube.com/watch?v=hhiLw5Q_UFg
- [404] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Association for Computational Linguistics, 2021, pp. 4582–4597.
- [405] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 3045–3059.
- [406] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA, 2019*, pp. 2790–2799.
- [407] Z. Hu, Y. Lan, L. Wang, W. Xu, E. Lim, R. K. Lee, L. Bing, and S. Poria, "Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models," *CoRR*, vol. abs/2304.01933, 2023.
- [408] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, and G. Neubig, "Towards a unified view of parameter-efficient transfer learning," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [409] X. Liu, K. Ji, Y. Fu, Z. Du, Z. Yang, and J. Tang, "P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks," *CoRR*, vol. abs/2110.07602, 2021.
- [410] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, and J. Tang, "GPT understands, too," *CoRR*, vol. abs/2103.10385, 2021.
- [411] Y. Gu, X. Han, Z. Liu, and M. Huang, "Ppt: Pre-trained prompt tuning for few-shot learning," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 8410–8423.
- [412] Z. Jiang, F. F. Xu, J. Araki, and G. Neubig, "How can we know what language models know?" *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 423–438, 2020.
- [413] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, "Autoprompt: Eliciting knowledge from language models with automatically generated prompts," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 4222–4235.
- [414] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," *CoRR*, vol. abs/2303.10512, 2023. [Online]. Available:

- <https://doi.org/10.48550/arXiv.2303.10512>
- [415] M. Valipour, M. Rezagholizadeh, I. Kobyzev, and A. Ghodsi, “Dylora: Parameter efficient tuning of pre-trained models using dynamic search-free low-rank adaptation,” *CoRR*, vol. abs/2210.07558, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.07558>
 - [416] N. Ding, Y. Qin, G. Yang, F. Wei, Y. Zonghan, Y. Su, S. Hu, Y. Chen, C.-M. Chan, W. Chen, J. Yi, W. Zhao, X. Wang, Z. Liu, H.-T. Zheng, J. Chen, Y. Liu, J. Tang, J. Li, and M. Sun, “Parameter-efficient fine-tuning of large-scale pre-trained language models,” *Nature Machine Intelligence*, vol. 5, pp. 1–16, 03 2023.
 - [417] R. Zhang, J. Han, A. Zhou, X. Hu, S. Yan, P. Lu, H. Li, P. Gao, and Y. Qiao, “Llama-adapter: Efficient fine-tuning of language models with zero-init attention,” *CoRR*, vol. abs/2303.16199, 2023.
 - [418] J. Pfeiffer, I. Vulic, I. Gurevych, and S. Ruder, “MAD-X: an adapter-based framework for multi-task cross-lingual transfer,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Association for Computational Linguistics, 2020, pp. 7654–7673.
 - [419] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, and S. Paul, “Peft: State-of-the-art parameter-efficient fine-tuning methods,” <https://github.com/huggingface/peft>, 2022.
 - [420] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, “What makes good in-context examples for gpt-3?” in *Proceedings of Deep Learning Inside Out: The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures, DeeLIO@ACL 2022, Dublin, Ireland and Online, May 27, 2022*, 2022, pp. 100–114.
 - [421] O. Rubin, J. Herzig, and J. Berant, “Learning to retrieve prompts for in-context learning,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, 2022, pp. 2655–2671.
 - [422] H. J. Kim, H. Cho, J. Kim, T. Kim, K. M. Yoo, and S. Lee, “Self-generated in-context learning: Leveraging auto-regressive language models as a demonstration generator,” *CoRR*, vol. abs/2206.08082, 2022.
 - [423] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, “Large language models are human-level prompt engineers,” in *Proc. of ICLR*, 2023.
 - [424] Y. Hao, Y. Sun, L. Dong, Z. Han, Y. Gu, and F. Wei, “Structured prompting: Scaling in-context learning to 1, 000 examples,” *CoRR*, 2022.
 - [425] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, S. Muresanu, P. Nakov, and A. Villavicencio, Eds., 2022, pp. 8086–8098.
 - [426] Y. Fu, H. Peng, A. Sabharwal, P. Clark, and T. Khot, “Complexity-based prompting for multi-step reasoning,” *CoRR*, vol. abs/2210.00720, 2022.
 - [427] Z. Zhang, A. Zhang, M. Li, and A. Smola, “Automatic chain of thought prompting in large language models,” *CoRR*, vol. abs/2210.03493, 2022.
 - [428] A. Creswell, M. Shanahan, and I. Higgins, “Selection-inference: Exploiting large language models for interpretable logical reasoning,” *CoRR*, vol. abs/2205.09712, 2022.
 - [429] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *CoRR*, vol. abs/2203.11171, 2022.
 - [430] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J. Lou, and W. Chen, “On the advance of making language models better reasoners,” *CoRR*, vol. abs/2206.02336, 2022.
 - [431] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, and D. Zhou, “Rationale-augmented ensembles in language models,” *CoRR*, 2022.
 - [432] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, O. Bousquet, Q. Le, and E. H. Chi, “Least-to-most prompting enables complex reasoning in large language models,” *CoRR*, vol. abs/2205.10625, 2022.
 - [433] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, “Decomposed prompting: A modular approach for solving complex tasks,” *CoRR*, vol. abs/2210.02406, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.02406>
 - [434] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K. Lee, and E. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *CoRR*, vol. abs/2305.04091, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.04091>
 - [435] Q. Lyu, S. Havaladar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, “Faithful chain-of-thought reasoning,” *CoRR*, vol. abs/2301.13379, 2023.
 - [436] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “PAL: program-aided language models,” *CoRR*, vol. abs/2211.10435, 2022.
 - [437] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface,” *arXiv preprint arXiv:2303.17580*, 2023.
 - [438] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, “Ada-planner: Adaptive planning from feedback with language models,” *arXiv preprint arXiv:2305.16653*, 2023.
 - [439] Y. Lu, P. Lu, Z. Chen, W. Zhu, X. E. Wang, and W. Y. Wang, “Multimodal procedural planning via dual text-image prompting,” *CoRR*, vol. abs/2305.01795, 2023.
 - [440] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, “Reasoning with language model is planning with world model,” *CoRR*, vol. abs/2305.14992, 2023.
 - [441] Z. Chen, K. Zhou, B. Zhang, Z. Gong, W. X. Zhao, and J. Wen, “Chatcot: Tool-augmented chain-of-thought reasoning on chat-based large language models,” *CoRR*, vol. abs/2305.14323, 2023.
 - [442] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *CoRR*, vol. abs/2210.03629, 2022.
 - [443] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” 2023.

- [444] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," *CoRR*, vol. abs/2305.10601, 2023.
- [445] V. Liu and L. B. Chilton, "Design guidelines for prompt engineering text-to-image generative models," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022, pp. 1–23.
- [446] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with chatgpt," *arXiv preprint arXiv:2302.11382*, 2023.
- [447] S. K. K. Santu and D. Feng, "Teler: A general taxonomy of LLM prompts for benchmarking complex tasks," *CoRR*, vol. abs/2305.11430, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.11430>
- [448] OpenAI, "Gpt best practices," *OpenAI*, 2023. [Online]. Available: <https://platform.openai.com/docs/guides/gpt-best-practices>
- [449] Contributors, "Ai short," 2023. [Online]. Available: <https://www.aishort.top/>
- [450] —, "Awesome chatgpt prompts," *Github*, 2023. [Online]. Available: <https://github.com/f/awesome-chatgpt-prompts/>
- [451] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J. Wen, "Structgpt: A general framework for large language model to reason over structured data," *CoRR*, vol. abs/2305.09645, 2023.
- [452] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: A query language for large language models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.
- [453] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao, "Chameleon: Plug-and-play compositional reasoning with large language models," *arXiv preprint arXiv:2304.09842*, 2023.
- [454] R. Ren, Y. Wang, Y. Qu, W. X. Zhao, J. Liu, H. Tian, H. Wu, J.-R. Wen, and H. Wang, "Investigating the factual knowledge boundary of large language models with retrieval augmentation," *arXiv preprint arXiv:2307.11019*, 2023.
- [455] X. Amatriain, "Prompt design and engineering: Introduction and advanced methods," *CoRR*, vol. abs/2401.14423, 2024.
- [456] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. J. McAuley, and W. X. Zhao, "Large language models are zero-shot rankers for recommender systems," *CoRR*, vol. abs/2305.08845, 2023.
- [457] S. Chang and E. Fosler-Lussier, "How to prompt llms for text-to-sql: A study in zero-shot, single-domain, and cross-domain settings," *CoRR*, vol. abs/2305.11853, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.11853>
- [458] Y. Wen, N. Jain, J. Kirchenbauer, M. Goldblum, J. Geiping, and T. Goldstein, "Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery," *CoRR*, vol. abs/2302.03668, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.03668>
- [459] T. Gao, A. Fisch, and D. Chen, "Making pre-trained language models better few-shot learners," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Association for Computational Linguistics, 2021, pp. 3816–3830.
- [460] L. Chen, J. Chen, T. Goldstein, H. Huang, and T. Zhou, "Instructzero: Efficient instruction optimization for black-box large language models," *CoRR*, vol. abs/2306.03082, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2306.03082>
- [461] X. Lin, Z. Wu, Z. Dai, W. Hu, Y. Shu, S. Ng, P. Jaillet, and B. K. H. Low, "Use your INSTINCT: instruction optimization using neural bandits coupled with transformers," *CoRR*, vol. abs/2310.02905, 2023.
- [462] M. Deng, J. Wang, C. Hsieh, Y. Wang, H. Guo, T. Shu, M. Song, E. P. Xing, and Z. Hu, "Rlprompt: Optimizing discrete text prompts with reinforcement learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Association for Computational Linguistics, 2022, pp. 3369–3391.
- [463] T. Zhang, X. Wang, D. Zhou, D. Schuurmans, and J. E. Gonzalez, "TEMPERA: test-time prompt editing via reinforcement learning," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [464] Y. Jafari, D. Mekala, R. Yu, and T. Berg-Kirkpatrick, "Morl-prompt: An empirical analysis of multi-objective reinforcement learning for discrete prompt optimization," *CoRR*, vol. abs/2402.11711, 2024.
- [465] W. Kong, S. A. Hombaiah, M. Zhang, Q. Mei, and M. Bendersky, "Prewrite: Prompt rewriting with reinforcement learning," *CoRR*, vol. abs/2401.08189, 2024.
- [466] H. Xu, Y. Chen, Y. Du, N. Shao, Y. Wang, H. Li, and Z. Yang, "GPS: genetic prompt search for efficient few-shot learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Association for Computational Linguistics, 2022, pp. 8162–8171.
- [467] A. Prasad, P. Hase, X. Zhou, and M. Bansal, "Grips: Gradient-free, edit-based instruction search for prompting large language models," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik, Croatia, May 2-6, 2023*, A. Vlachos and I. Augenstein, Eds. Association for Computational Linguistics, 2023, pp. 3827–3846.
- [468] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [469] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," *CoRR*, vol.

- abs/2305.03495, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.03495>
- [470] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," *CoRR*, vol. abs/2309.03409, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2309.03409>
- [471] Q. Ye, M. Axmed, R. Pryzant, and F. Khani, "Prompt engineering a prompt engineer," *CoRR*, vol. abs/2311.05661, 2023.
- [472] X. Tang, X. Wang, W. X. Zhao, S. Lu, Y. Li, and J. Wen, "Unleashing the potential of large language models as prompt optimizers: An analogical analysis with gradient-based model optimizers," *CoRR*, vol. abs/2402.17564, 2024.
- [473] H. Yang and K. Li, "Instoptima: Evolutionary multi-objective instruction optimization via large language model-based instruction operators," in *EMNLP (Findings)*. Association for Computational Linguistics, 2023, pp. 13 593–13 602.
- [474] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "Connecting large language models with evolutionary algorithms yields powerful prompt optimizers," *CoRR*, vol. abs/2309.08532, 2023.
- [475] X. L. Do, Y. Zhao, H. Brown, Y. Xie, J. X. Zhao, N. F. Chen, K. Kawaguchi, M. Q. Xie, and J. He, "Prompt optimization via adversarial in-context learning," *CoRR*, vol. abs/2312.02614, 2023.
- [476] X. Wang, C. Li, Z. Wang, F. Bai, H. Luo, J. Zhang, N. Jojic, E. P. Xing, and Z. Hu, "Promptagent: Strategic planning with language models enables expert-level prompt optimization," *CoRR*, vol. abs/2310.16427, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.16427>
- [477] T. Tang, J. Li, W. X. Zhao, and J. Wen, "Context-tuning: Learning contextualized prompts for natural language generation," in *Proceedings of the 29th International Conference on Computational Linguistics, COLING 2022, Gyeongju, Republic of Korea, October 12-17, 2022*, N. Calzolari, C. Huang, H. Kim, J. Pustejovsky, L. Wanner, K. Choi, P. Ryu, H. Chen, L. Donatelli, H. Ji, S. Kurohashi, P. Paggio, N. Xue, S. Kim, Y. Hahm, Z. He, T. K. Lee, E. Santus, F. Bond, and S. Na, Eds. International Committee on Computational Linguistics, 2022, pp. 6340–6354.
- [478] T. Vu, B. Lester, N. Constant, R. Al-Rfou', and D. Cer, "Spot: Better frozen model adaptation through soft prompt transfer," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Association for Computational Linguistics, 2022, pp. 5039–5059.
- [479] J. Li, T. Tang, J. Nie, J. Wen, and X. Zhao, "Learning to transfer prompts for text generation," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, M. Carpuat, M. de Marneffe, and I. V. M. Ruiz, Eds. Association for Computational Linguistics, 2022, pp. 3506–3518.
- [480] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?" in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*. Association for Computational Linguistics, 2022, pp. 11 048–11 064.
- [481] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, M. Meila and T. Zhang, Eds., 2021, pp. 12 697–12 706.
- [482] Y. Lee, C. Lim, and H. Choi, "Does GPT-3 generate empathetic dialogues? A novel in-context example selection method and automatic evaluation metric for empathetic dialogue generation," in *Proceedings of the 29th International Conference on Computational Linguistics, COLING 2022, Gyeongju, Republic of Korea, October 12-17, 2022*, N. Calzolari, C. Huang, H. Kim, J. Pustejovsky, L. Wanner, K. Choi, P. Ryu, H. Chen, L. Donatelli, H. Ji, S. Kurohashi, P. Paggio, N. Xue, S. Kim, Y. Hahm, Z. He, T. K. Lee, E. Santus, F. Bond, and S. Na, Eds. International Committee on Computational Linguistics, 2022, pp. 669–683.
- [483] I. Levy, B. Bogin, and J. Berant, "Diverse demonstrations improve in-context compositional generalization," *CoRR*, vol. abs/2212.06800, 2022.
- [484] H. Su, J. Kasai, C. H. Wu, W. Shi, T. Wang, J. Xin, R. Zhang, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and T. Yu, "Selective annotation makes language models better few-shot learners," *CoRR*, 2022.
- [485] X. Ye, S. Iyer, A. Celikeyilmaz, V. Stoyanov, G. Durrett, and R. Pasunuru, "Complementary explanations for effective in-context learning," *CoRR*, 2022.
- [486] X. Li and X. Qiu, "Finding supporting examples for in-context learning," *CoRR*, 2023.
- [487] Y. Zhang, S. Feng, and C. Tan, "Active example selection for in-context learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, 2022, pp. 9134–9148.
- [488] F. Gilardi, M. Alizadeh, and M. Kubli, "Chatgpt outperforms crowd-workers for text-annotation tasks," 2023.
- [489] H. J. Kim, H. Cho, J. Kim, T. Kim, K. M. Yoo, and S. Lee, "Self-generated in-context learning: Leveraging auto-regressive language models as a demonstration generator," *CoRR*, vol. abs/2206.08082, 2022.
- [490] S. M. Xie, A. Raghunathan, P. Liang, and T. Ma, "An explanation of in-context learning as implicit bayesian inference," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*, 2022.
- [491] Z. Wu, Y. Wang, J. Ye, and L. Kong, "Self-adaptive in-context learning," *CoRR*, vol. abs/2212.10375, 2022.
- [492] Y. Gu, L. Dong, F. Wei, and M. Huang, "Pre-training to learn in context," *CoRR*, vol. abs/2305.09137, 2023.
- [493] S. Min, M. Lewis, L. Zettlemoyer, and H. Hajishirzi, "Metaicl: Learning to learn in context," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*

- tics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, M. Carpuat, M. de Marneffe, and I. V. M. Ruíz, Eds., 2022, pp. 2791–2809.
- [494] M. Hahn and N. Goyal, “A theory of emergent in-context learning as implicit structure induction,” *CoRR*, vol. abs/2303.07971, 2023.
- [495] J. Pan, T. Gao, H. Chen, and D. Chen, “What in-context learning ‘learns’ in-context: Disentangling task recognition and task learning,” *CoRR*, vol. abs/2305.09731, 2023.
- [496] N. Wies, Y. Levine, and A. Shashua, “The learnability of in-context learning,” *CoRR*, vol. abs/2303.07895, 2023.
- [497] A. Webson and E. Pavlick, “Do prompt-based models really understand the meaning of their prompts?” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, 2022, pp. 2300–2344.
- [498] J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, “Transformers learn in-context by gradient descent,” *CoRR*, vol. abs/2212.07677, 2022.
- [499] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. Das-Sarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, S. Johnston, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, “In-context learning and induction heads,” *CoRR*, vol. abs/2209.11895, 2022.
- [500] E. Akyürek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou, “What learning algorithm is in-context learning? investigations with linear models,” *CoRR*, vol. abs/2211.15661, 2022.
- [501] J. Wei, J. Wei, Y. Tay, D. Tran, A. Webson, Y. Lu, X. Chen, H. Liu, D. Huang, D. Zhou *et al.*, “Larger language models do in-context learning differently,” *arXiv preprint arXiv:2303.03846*, 2023.
- [502] J. Coda-Forno, M. Binz, Z. Akata, M. M. Botvinick, J. X. Wang, and E. Schulz, “Meta-in-context learning in large language models,” *CoRR*, vol. abs/2305.12907, 2023.
- [503] J. W. Wei, L. Hou, A. K. Lampinen, X. Chen, D. Huang, Y. Tay, X. Chen, Y. Lu, D. Zhou, T. Ma, and Q. V. Le, “Symbol tuning improves in-context learning in language models,” *CoRR*, vol. abs/2305.08298, 2023.
- [504] Z. Chu, J. Chen, Q. Chen, W. Yu, T. He, H. Wang, W. Peng, M. Liu, B. Qin, and T. Liu, “A survey of chain of thought reasoning: Advances, frontiers and future,” *CoRR*, vol. abs/2309.15402, 2023.
- [505] S. Miao, C. Liang, and K. Su, “A diverse corpus for evaluating and developing english math word problem solvers,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, D. Jurafsky, J. Chai, N. Schluter, and J. R. Tetreault, Eds. Association for Computational Linguistics, 2020, pp. 975–984.
- [506] A. Talmor, J. Herzig, N. Lourie, and J. Berant, “Commonsenseqa: A question answering challenge targeting commonsense knowledge,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4149–4158.
- [507] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *CoRR*, vol. abs/2205.11916, 2022.
- [508] W. Chen, X. Ma, X. Wang, and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,” *CoRR*, vol. abs/2211.12588, 2022.
- [509] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, “PAL: program-aided language models,” in *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., 2023.
- [510] X. Zhao, Y. Xie, K. Kawaguchi, J. He, and Q. Xie, “Automatic model selection with large language models for reasoning,” *CoRR*, vol. abs/2305.14333, 2023.
- [511] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J.-G. Lou, and W. Chen, “Making large language models better reasoners with step-aware verifier,” 2023.
- [512] O. Yoran, T. Wolfson, B. Bogin, U. Katz, D. Deutch, and J. Berant, “Answering questions by meta-reasoning over multiple chains of thought,” *CoRR*, vol. abs/2304.13007, 2023.
- [513] Z. Ling, Y. Fang, X. Li, Z. Huang, M. Lee, R. Memisevic, and H. Su, “Deductive verification of chain-of-thought reasoning,” *CoRR*, vol. abs/2306.03872, 2023.
- [514] T. Xue, Z. Wang, Z. Wang, C. Han, P. Yu, and H. Ji, “RCOT: detecting and rectifying factual inconsistency in reasoning by reversing chain-of-thought,” *CoRR*, vol. abs/2305.11499, 2023.
- [515] Y. Weng, M. Zhu, F. Xia, B. Li, S. He, K. Liu, and J. Zhao, “Large language models are better reasoners with self-verification,” *CoRR*, abs/2212.09561, 2023.
- [516] W. Jiang, H. Shi, L. Yu, Z. Liu, Y. Zhang, Z. Li, and J. T. Kwok, “Forward-backward reasoning in large language models for mathematical verification,” 2023.
- [517] J. Long, “Large language model guided tree-of-thought,” *CoRR*, vol. abs/2305.08291, 2023.
- [518] S. Mo and M. Xin, “Tree of uncertain thoughts reasoning for large language models,” *CoRR*, vol. abs/2309.07694, 2023.
- [519] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, L. Gianinazzi, J. Gajda, T. Lehmann, M. Podstawski, H. Niewiadomski, P. Nyczyk, and T. Hoefler, “Graph of thoughts: Solving elaborate problems with large language models,” *CoRR*, vol. abs/2308.09687, 2023.
- [520] B. Lei, P. Lin, C. Liao, and C. Ding, “Boosting logical reasoning in large language models through a new framework: The graph of thought,” *CoRR*, vol. abs/2308.08614, 2023.
- [521] R. Ding, C. Zhang, L. Wang, Y. Xu, M. Ma, W. Zhang, S. Qin, S. Rajmohan, Q. Lin, and D. Zhang, “Everything of thoughts: Defying the law of penrose triangle for thought generation,” *arXiv preprint arXiv:2311.04254*,

- jar, F. Bougares, R. Chatterjee, M. R. Costa-jussà, C. Federmann, M. Fishel, A. Fraser, Y. Graham, P. Guzman, B. Haddow, M. Huck, A. Jimeno-Yepes, P. Koehn, A. Martins, M. Morishita, C. Monz, M. Nagata, T. Nakazawa, and M. Negri, Eds. Association for Computational Linguistics, 2020, pp. 1–55.
- [546] F. Akhbardeh, A. Arkhangorodsky, M. Biesalska, O. Bojar, R. Chatterjee, V. Chaudhary, M. R. Costa-jussà, C. España-Bonet, A. Fan, C. Federmann, M. Freitag, Y. Graham, R. Grundkiewicz, B. Haddow, L. Harter, K. Heafield, C. Homan, M. Huck, K. Amponsah-Kaakyire, J. Kasai, D. Khashabi, K. Knight, T. Kocmi, P. Koehn, N. Lourie, C. Monz, M. Morishita, M. Nagata, A. Nagesh, T. Nakazawa, M. Negri, S. Pal, A. A. Tapo, M. Turchi, V. Vydrin, and M. Zampieri, “Findings of the 2021 conference on machine translation (WMT21),” in *Proceedings of the Sixth Conference on Machine Translation, WMT@EMNLP 2021, Online Event, November 10-11, 2021*, L. Barrault, O. Bojar, F. Bougares, R. Chatterjee, M. R. Costa-jussà, C. Federmann, M. Fishel, A. Fraser, M. Freitag, Y. Graham, R. Grundkiewicz, P. Guzman, B. Haddow, M. Huck, A. Jimeno-Yepes, P. Koehn, T. Kocmi, A. Martins, M. Morishita, and C. Monz, Eds. Association for Computational Linguistics, 2021, pp. 1–88.
- [547] T. Kocmi, R. Bawden, O. Bojar, A. Dvorkovich, C. Federmann, M. Fishel, T. Gowda, Y. Graham, R. Grundkiewicz, B. Haddow, R. Knowles, P. Koehn, C. Monz, M. Morishita, M. Nagata, T. Nakazawa, M. Novák, M. Popel, and M. Popovic, “Findings of the 2022 conference on machine translation (WMT22),” in *Proceedings of the Seventh Conference on Machine Translation, WMT 2022, Abu Dhabi, United Arab Emirates (Hybrid), December 7-8, 2022*, P. Koehn, L. Barrault, O. Bojar, F. Bougares, R. Chatterjee, M. R. Costa-jussà, C. Federmann, M. Fishel, A. Fraser, M. Freitag, Y. Graham, R. Grundkiewicz, P. Guzman, B. Haddow, M. Huck, A. Jimeno-Yepes, T. Kocmi, A. Martins, M. Morishita, C. Monz, M. Nagata, T. Nakazawa, M. Negri, A. Névél, M. Neves, M. Popel, M. Turchi, and M. Zampieri, Eds. Association for Computational Linguistics, 2022, pp. 1–45.
- [548] N. Goyal, C. Gao, V. Chaudhary, P. Chen, G. Wenzek, D. Ju, S. Krishnan, M. Ranzato, F. Guzmán, and A. Fan, “The flores-101 evaluation benchmark for low-resource and multilingual machine translation,” *Trans. Assoc. Comput. Linguistics*, vol. 10, pp. 522–538, 2022.
- [549] R. Bawden, E. Bilinski, T. Lavergne, and S. Rosset, “Diabla: a corpus of bilingual spontaneous written dialogues for machine translation,” *Lang. Resour. Evaluation*, vol. 55, no. 3, pp. 635–660, 2021.
- [550] R. Nallapati, B. Zhou, C. N. dos Santos, Ç. Gülçehre, and B. Xiang, “Abstractive text summarization using sequence-to-sequence rnns and beyond,” in *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning, CoNLL 2016, Berlin, Germany, August 11-12, 2016*, Y. Goldberg and S. Riezler, Eds. ACL, 2016, pp. 280–290.
- [551] S. Narayan, S. B. Cohen, and M. Lapata, “Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization,” in *EMNLP*. Association for Computational Linguistics, 2018, pp. 1797–1807.
- [552] F. Ladhak, E. Durmus, C. Cardie, and K. Mckeown, “Wikilingua: A new benchmark dataset for cross-lingual abstractive summarization,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4034–4048.
- [553] S. Moon, P. Shah, A. Kumar, and R. Subba, “Opendialkg: Explainable conversational reasoning with attention-based walks over knowledge graphs,” in *ACL (1)*. Association for Computational Linguistics, 2019, pp. 845–854.
- [554] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, S. W. Yih, D. Fried, S. I. Wang, and T. Yu, “DS-1000: A natural and reliable benchmark for data science code generation,” *CoRR*, vol. abs/2211.11501, 2022.
- [555] Z. Wang, S. Zhou, D. Fried, and G. Neubig, “Execution-based evaluation for open-domain code generation,” *CoRR*, vol. abs/2212.10481, 2022.
- [556] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, “Natural questions: a benchmark for question answering research,” *Trans. Assoc. Comput. Linguistics*, pp. 452–466, 2019.
- [557] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the AI2 reasoning challenge,” *CoRR*, vol. abs/1803.05457, 2018.
- [558] S. Lin, J. Hilton, and O. Evans, “Truthfulqa: Measuring how models mimic human falsehoods,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, 2022, pp. 3214–3252.
- [559] J. Berant, A. Chou, R. Frostig, and P. Liang, “Semantic parsing on freebase from question-answer pairs,” in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, EMNLP 2013, 18-21 October 2013, Grand Hyatt Seattle, Seattle, Washington, USA, A meeting of SIGDAT, a Special Interest Group of the ACL*, 2013, pp. 1533–1544.
- [560] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, 2017, pp. 1601–1611.
- [561] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, “PIQA: reasoning about physical commonsense in natural language,” in *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, 2020, pp. 7432–7439.
- [562] M. Dubey, D. Banerjee, A. Abdelkawi, and J. Lehmann, “Lc-quad 2.0: A large dataset for complex question answering over wikidata and dbpedia,” in *The Semantic Web - ISWC 2019 - 18th International Semantic Web Conference, Auckland, New Zealand, October 26-30, 2019, Proceedings, Part II*, 2019, pp. 69–78.

- [563] Y. Gu, S. Kase, M. Vanni, B. M. Sadler, P. Liang, X. Yan, and Y. Su, "Beyond I.I.D.: three levels of generalization for question answering on knowledge bases," in *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, 2021, pp. 3477–3488.
- [564] S. Cao, J. Shi, L. Pan, L. Nie, Y. Xiang, L. Hou, J. Li, B. He, and H. Zhang, "KQA pro: A dataset with explicit compositional programs for complex question answering over knowledge base," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, 2022, pp. 6101–6119.
- [565] X. Hu, X. Wu, Y. Shu, and Y. Qu, "Logical form generation via multi-task learning for complex question answering over knowledge bases," in *Proceedings of the 29th International Conference on Computational Linguistics, COLING 2022, Gyeongju, Republic of Korea, October 12-17, 2022*, 2022, pp. 1687–1696.
- [566] S. Longpre, Y. Lu, and J. Daiber, "MKQA: A linguistically diverse benchmark for multilingual open domain question answering," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 1389–1406, 2021.
- [567] T. Saikh, T. Ghosal, A. Mittal, A. Ekbal, and P. Bhattacharyya, "Scienceqa: a novel resource for question answering on scholarly articles," *Int. J. Digit. Libr.*, vol. 23, no. 3, pp. 289–301, 2022.
- [568] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a suit of armor conduct electricity? A new dataset for open book question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, 2018, pp. 2381–2391.
- [569] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng, "MS MARCO: A human generated machine reading comprehension dataset," in *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016*, 2016.
- [570] T. Khot, P. Clark, M. Guerquin, P. Jansen, and A. Sabharwal, "QASC: A dataset for question answering via sentence composition," in *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, 2020, pp. 8082–8090.
- [571] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100, 000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, 2016, pp. 2383–2392.
- [572] A. H. Miller, A. Fisch, J. Dodge, A. Karimi, A. Bordes, and J. Weston, "Key-value memory networks for directly reading documents," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, EMNLP 2016, Austin, Texas, USA, November 1-4, 2016*, 2016, pp. 1400–1409.
- [573] B. Goodrich, V. Rao, P. J. Liu, and M. Saleh, "Assessing the factual accuracy of generated text," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, 2019, pp. 166–175.
- [574] K. Toutanova and D. Chen, "Observed versus latent features for knowledge base and text inference," in *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality, CVSC 2015, Beijing, China, July 26-31, 2015*, 2015, pp. 57–66.
- [575] K. D. Bollacker, C. Evans, P. K. Paritosh, T. Sturge, and J. Taylor, "Freebase: a collaboratively created graph database for structuring human knowledge," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2008, Vancouver, BC, Canada, June 10-12, 2008*, 2008, pp. 1247–1250.
- [576] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, "Convolutional 2d knowledge graph embeddings," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, 2018, pp. 1811–1818.
- [577] G. A. Miller, "Wordnet: A lexical database for english," *Commun. ACM*, pp. 39–41, 1995.
- [578] F. Petroni, T. Rocktäschel, S. Riedel, P. S. H. Lewis, A. Bakhtin, Y. Wu, and A. H. Miller, "Language models as knowledge bases?" in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, 2019, pp. 2463–2473.
- [579] F. Mahdisoltani, J. Biega, and F. M. Suchanek, "YAGO3: A knowledge base from multilingual wikipedias," in *Seventh Biennial Conference on Innovative Data Systems Research, CIDR 2015, Asilomar, CA, USA, January 4-7, 2015, Online Proceedings*, 2015.
- [580] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: a core of semantic knowledge," in *Proceedings of the 16th International Conference on World Wide Web, WWW 2007, Banff, Alberta, Canada, May 8-12, 2007*, 2007, pp. 697–706.
- [581] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*. Association for Computational Linguistics, 2018, pp. 2369–2380.
- [582] C. Clark, K. Lee, M. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "Boolq: Exploring the surprising difficulty of natural yes/no questions," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 2924–2936.

- [583] M. Sap, H. Rashkin, D. Chen, R. L. Bras, and Y. Choi, "Socialiqa: Commonsense reasoning about social interactions," *CoRR*, vol. abs/1904.09728, 2019.
- [584] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "Hellaswag: Can a machine really finish your sentence?" in *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, A. Korhonen, D. R. Traum, and L. Màrquez, Eds. Association for Computational Linguistics, 2019, pp. 4791–4800.
- [585] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "Winogrande: An adversarial winograd schema challenge at scale," in *AAAI*. AAAI Press, 2020, pp. 8732–8740.
- [586] M. Roemmele, C. A. Bejan, and A. S. Gordon, "Choice of plausible alternatives: An evaluation of commonsense causal reasoning," in *Logical Formalizations of Commonsense Reasoning, Papers from the 2011 AAAI Spring Symposium, Technical Report SS-11-06, Stanford, California, USA, March 21-23, 2011*. AAAI, 2011.
- [587] K. Sakaguchi, C. Bhagavatula, R. L. Bras, N. Tandon, P. Clark, and Y. Choi, "proscript: Partially ordered scripts generation," in *Findings of the Association for Computational Linguistics: EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 16-20 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 2138–2149.
- [588] B. Dalvi, L. Huang, N. Tandon, W. Yih, and P. Clark, "Tracking state changes in procedural text: a challenge dataset and models for process paragraph comprehension," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, M. A. Walker, H. Ji, and A. Stent, Eds. Association for Computational Linguistics, 2018, pp. 1595–1604.
- [589] S. Saha, P. Yadav, L. Bauer, and M. Bansal, "Explanographs: An explanation graph generation task for structured commonsense reasoning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 7716–7740.
- [590] O. Tafjord, B. Dalvi, and P. Clark, "Proofwriter: Generating implications, proofs, and abductive statements over natural language," in *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, ser. Findings of ACL, C. Zong, F. Xia, W. Li, and R. Navigli, Eds., vol. ACL/IJCNLP 2021. Association for Computational Linguistics, 2021, pp. 3621–3634.
- [591] B. Dalvi, P. Jansen, O. Tafjord, Z. Xie, H. Smith, L. Pipatanangkura, and P. Clark, "Explaining answers with entailment trees," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 7358–7370.
- [592] A. Saparov and H. He, "Language models are greedy reasoners: A systematic formal analysis of chain-of-thought," *CoRR*, vol. abs/2210.01240, 2022.
- [593] C. Anil, Y. Wu, A. Andreassen, A. Lewkowycz, V. Misra, V. V. Ramasesh, A. Slone, G. Gur-Ari, E. Dyer, and B. Neyshabur, "Exploring length generalization in large language models," *CoRR*, vol. abs/2207.04901, 2022.
- [594] A. Patel, S. Bhattamishra, and N. Goyal, "Are NLP models really able to solve simple math word problems?" in *NAACL-HLT*. Association for Computational Linguistics, 2021, pp. 2080–2094.
- [595] S. Roy and D. Roth, "Solving general arithmetic word problems," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP 2015, Lisbon, Portugal, September 17-21, 2015*, L. Màrquez, C. Callison-Burch, J. Su, D. Pighin, and Y. Marton, Eds. The Association for Computational Linguistics, 2015, pp. 1743–1752.
- [596] A. Amini, S. Gabriel, S. Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi, "Mathqa: Towards interpretable math word problem solving with operation-based formalisms," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 2357–2367.
- [597] W. Ling, D. Yogatama, C. Dyer, and P. Blunsom, "Program induction by rationale generation: Learning to solve and explain algebraic word problems," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers*, R. Barzilay and M. Kan, Eds. Association for Computational Linguistics, 2017, pp. 158–167.
- [598] R. Koncel-Kedziorski, S. Roy, A. Amini, N. Kushman, and H. Hajishirzi, "Mawps: A math word problem repository," in *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, 2016, pp. 1152–1157.
- [599] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner, "DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, 2019, pp. 2368–2378.
- [600] S. Welleck, J. Liu, R. L. Bras, H. Hajishirzi, Y. Choi, and K. Cho, "Naturalproofs: Mathematical theorem proving in natural language," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, J. Vanschoren and S. Yeung, Eds., 2021.
- [601] A. Q. Jiang, W. Li, J. M. Han, and Y. Wu, "Lisa: Language models of isabelle proofs," in *6th Conference on Artificial Intelligence and Theorem Proving*, 2021, pp. 378–392.

- [602] K. Zheng, J. M. Han, and S. Polu, "minif2f: a cross-system benchmark for formal olympiad-level mathematics," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [603] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad, "Proofnet: Autoformalizing and formally proving undergraduate-level mathematics," *CoRR*, vol. abs/2302.12433, 2023.
- [604] J. Li, X. Cheng, W. X. Zhao, J. Nie, and J. Wen, "Halueval: A large-scale hallucination evaluation benchmark for large language models," *CoRR*, vol. abs/2305.11747, 2023.
- [605] N. Nangia, C. Vania, R. Bhalerao, and S. R. Bowman, "Crows-pairs: A challenge dataset for measuring social biases in masked language models," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, 2020, pp. 1953–1967.
- [606] R. Rudinger, J. Naradowsky, B. Leonard, and B. V. Durme, "Gender bias in coreference resolution," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 2 (Short Papers)*, 2018, pp. 8–14.
- [607] S. Gehman, S. Gururangan, M. Sap, Y. Choi, and N. A. Smith, "Realtoxicityprompts: Evaluating neural toxic degeneration in language models," in *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, ser. Findings of ACL, T. Cohn, Y. He, and Y. Liu, Eds., vol. EMNLP 2020. Association for Computational Linguistics, 2020, pp. 3356–3369.
- [608] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, "Virtualhome: Simulating household activities via programs," in *CVPR*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 8494–8502.
- [609] S. Srivastava, C. Li, M. Lingelbach, R. Martín-Martín, F. Xia, K. E. Vainio, Z. Lian, C. Gokmen, S. Buch, C. K. Liu, S. Savarese, H. Gweon, J. Wu, and L. Fei-Fei, "BEHAVIOR: benchmark for everyday household activities in virtual, interactive, and ecological environments," in *CoRL*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2021, pp. 477–490.
- [610] M. Shridhar, J. Thomason, D. Gordon, Y. Bisk, W. Han, R. Mottaghi, L. Zettlemoyer, and D. Fox, "ALFRED: A benchmark for interpreting grounded instructions for everyday tasks," in *CVPR*. Computer Vision Foundation / IEEE, 2020, pp. 10 737–10 746.
- [611] M. Shridhar, X. Yuan, M. Côté, Y. Bisk, A. Trischler, and M. J. Hausknecht, "Alfworld: Aligning text and embodied environments for interactive learning," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.
- [612] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "Webshop: Towards scalable real-world web interaction with grounded language agents," in *NeurIPS*, 2022.
- [613] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, "Mind2web: Towards a generalist agent for the web," *CoRR*, vol. abs/2306.06070, 2023.
- [614] W. H. Guss, B. Houghton, N. Topin, P. Wang, C. Codel, M. Veloso, and R. Salakhutdinov, "Minerl: A large-scale dataset of minecraft demonstrations," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, S. Kraus, Ed. ijcai.org, 2019, pp. 2442–2448.
- [615] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D. Huang, Y. Zhu, and A. Anandkumar, "Minedojo: Building open-ended embodied agents with internet-scale knowledge," in *NeurIPS*, 2022.
- [616] P. Lu, L. Qiu, K. Chang, Y. N. Wu, S. Zhu, T. Rajpurohit, P. Clark, and A. Kalyan, "Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning," *CoRR*, vol. abs/2209.14610, 2022.
- [617] B. Zhang, K. Zhou, X. Wei, W. X. Zhao, J. Sha, S. Wang, and J. rong Wen, "Evaluating and improving tool-augmented computation-intensive math reasoning," *CoRR*, vol. abs/2306.02408, 2023.
- [618] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, "Gpt4tools: Teaching large language model to use tools via self-instruction," *CoRR*, vol. abs/2305.18752, 2023.
- [619] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," *CoRR*, vol. abs/2305.15334, 2023.
- [620] W. Yih, M. Richardson, C. Meek, M. Chang, and J. Suh, "The value of semantic parse labeling for knowledge base question answering," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 2: Short Papers*. The Association for Computer Linguistics, 2016.
- [621] H. Puerto, G. G. Sahin, and I. Gurevych, "Metaqa: Combining expert agents for multi-skill question answering," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2023, Dubrovnik, Croatia, May 2-6, 2023*, A. Vlachos and I. Augenstein, Eds. Association for Computational Linguistics, 2023, pp. 3548–3562.
- [622] P. Pasupat and P. Liang, "Compositional semantic parsing on semi-structured tables," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July 26-31, 2015, Beijing, China, Volume 1: Long Papers*. The Association for Computer Linguistics, 2015, pp. 1470–1480.
- [623] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *CoRR*, vol. abs/1709.00103, 2017.
- [624] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang, "Tabfact: A large-scale dataset for table-based fact verification," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.
- [625] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li,

- J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. R. Radev, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Association for Computational Linguistics, 2018, pp. 3911–3921.
- [626] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *ICLR*, 2015.
- [627] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, July 6-12, 2002, Philadelphia, PA, USA*. ACL, 2002, pp. 311–318.
- [628] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Association for Computational Linguistics, Jul. 2004, pp. 74–81.
- [629] W. Jiao, W. Wang, J.-t. Huang, X. Wang, and Z. Tu, "Is chatgpt a good translator? a preliminary study," *arXiv preprint arXiv:2301.08745*, 2023.
- [630] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. R. McKeown, and T. B. Hashimoto, "Benchmarking large language models for news summarization," *CoRR*, vol. abs/2301.13848, 2023.
- [631] T. Goyal, J. J. Li, and G. Durrett, "News summarization and evaluation in the era of GPT-3," *CoRR*, vol. abs/2209.12356, 2022.
- [632] S. Gehrmann, E. Clark, and T. Sellam, "Repairing the cracked foundation: A survey of obstacles in evaluation practices for generated text," *CoRR*, vol. abs/2202.06935, 2022.
- [633] J. Wang, Y. Liang, F. Meng, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou, "Is chatgpt a good NLG evaluator? A preliminary study," *CoRR*, vol. abs/2303.04048, 2023.
- [634] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-eval: NLG evaluation using GPT-4 with better human alignment," *CoRR*, vol. abs/2303.16634, 2023.
- [635] K. Yang, Y. Tian, N. Peng, and D. Klein, "Re3: Generating longer stories with recursive reprompting and revision," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, Y. Goldberg, Z. Kozareva, and Y. Zhang, Eds. Association for Computational Linguistics, 2022, pp. 4393–4479.
- [636] W. Zhou, Y. E. Jiang, P. Cui, T. Wang, Z. Xiao, Y. Hou, R. Cotterell, and M. Sachan, "Recurrentgpt: Interactive generation of (arbitrarily) long text," *CoRR*, vol. abs/2305.13304, 2023.
- [637] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Found. Trends Program. Lang.*, vol. 4, no. 1-2, pp. 1–119, 2017.
- [638] S. Zhang, Z. Chen, Y. Shen, M. Ding, J. B. Tenenbaum, and C. Gan, "Planning with large language models for code generation," 2023.
- [639] M. Welsh, "The end of programming," *Commun. ACM*, vol. 66, no. 1, pp. 34–35, 2023.
- [640] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung, Q. V. Do, Y. Xu, and P. Fung, "A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity," *CoRR*, vol. abs/2302.04023, 2023.
- [641] Y. Liu, A. R. Fabbri, P. Liu, Y. Zhao, L. Nan, R. Han, S. Han, S. R. Joty, C. Wu, C. Xiong, and D. Radev, "Revisiting the gold standard: Grounding summarization evaluation with robust human evaluation," *CoRR*, vol. abs/2212.07981, 2022.
- [642] A. R. Fabbri, W. Kryscinski, B. McCann, C. Xiong, R. Socher, and D. R. Radev, "Summeval: Re-evaluating summarization evaluation," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 391–409, 2021.
- [643] T. Tang, H. Lu, Y. E. Jiang, H. Huang, D. Zhang, W. X. Zhao, and F. Wei, "Not all metrics are guilty: Improving NLG evaluation with LLM paraphrasing," *CoRR*, vol. abs/2305.15067, 2023.
- [644] X. Wang, X. Tang, W. X. Zhao, J. Wang, and J. Wen, "Rethinking the evaluation for conversational recommendation in the era of large language models," *CoRR*, vol. abs/2305.13112, 2023.
- [645] M. Gao, J. Ruan, R. Sun, X. Yin, S. Yang, and X. Wan, "Human-like summarization evaluation with chatgpt," *CoRR*, vol. abs/2304.02554, 2023.
- [646] Y. Ji, Y. Gong, Y. Peng, C. Ni, P. Sun, D. Pan, B. Ma, and X. Li, "Exploring chatgpt's ability to rank content: A preliminary study on consistency with human preferences," *CoRR*, vol. abs/2303.07610, 2023.
- [647] Y. Bai, J. Ying, Y. Cao, X. Lv, Y. He, X. Wang, J. Yu, K. Zeng, Y. Xiao, H. Lyu, J. Zhang, J. Li, and L. Hou, "Benchmarking foundation models with language-model-as-an-examiner," *CoRR*, vol. abs/2306.04181, 2023.
- [648] Y. Liu, S. Feng, D. Wang, Y. Zhang, and H. Schütze, "Evaluate what you can't evaluate: Unassessable generated responses quality," *CoRR*, vol. abs/2305.14658, 2023.
- [649] P. Wang, L. Li, L. Chen, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," *CoRR*, vol. abs/2305.17926, 2023.
- [650] J. Ye, X. Chen, N. Xu, C. Zu, Z. Shao, S. Liu, Y. Cui, Z. Zhou, C. Gong, Y. Shen, J. Zhou, S. Chen, T. Gui, Q. Zhang, and X. Huang, "A comprehensive capability analysis of gpt-3 and gpt-3.5 series models," *arXiv preprint arXiv:2303.10420*, 2023.
- [651] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of learning and motivation*, 1989, pp. 109–165.
- [652] R. Kemker, M. McClure, A. Abitino, T. L. Hayes, and C. Kanan, "Measuring catastrophic forgetting in neural networks," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, 2018, pp. 3390–3398.
- [653] T. Xie, C. H. Wu, P. Shi, R. Zhong, T. Scholak, M. Yasunaga, C. Wu, M. Zhong, P. Yin, S. I. Wang, V. Zhong, B. Wang, C. Li, C. Boyle, A. Ni, Z. Yao,

- D. Radev, C. Xiong, L. Kong, R. Zhang, N. A. Smith, L. Zettlemoyer, and T. Yu, "Unifiedskg: Unifying and multi-tasking structured knowledge grounding with text-to-text language models," in *EMNLP*. Association for Computational Linguistics, 2022, pp. 602–631.
- [654] A. Roberts, C. Raffel, and N. Shazeer, "How much knowledge can you pack into the parameters of a language model?" in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, 2020, pp. 5418–5426.
- [655] G. Izacard, P. S. H. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, and E. Grave, "Few-shot learning with retrieval augmented language models," *CoRR*, vol. abs/2208.03299, 2022.
- [656] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, 2020, pp. 3929–3938.
- [657] P. S. H. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [658] Y. Lan, G. He, J. Jiang, J. Jiang, W. X. Zhao, and J. Wen, "Complex knowledge base question answering: A survey," *CoRR*, vol. abs/2108.06688, 2021.
- [659] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. van den Driessche, J. Lespiau, B. Damoc, A. Clark, D. de Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. W. Rae, E. Elsen, and L. Sifre, "Improving language models by retrieving from trillions of tokens," in *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 2022, pp. 2206–2240.
- [660] S. Xu, L. Pang, H. Shen, X. Cheng, and T.-S. Chua, "Search-in-the-chain: Towards accurate, credible and traceable large language models for knowledge-intensive tasks," *CoRR*, vol. abs/2304.14732, 2023.
- [661] B. Peng, M. Galley, P. He, H. Cheng, Y. Xie, Y. Hu, Q. Huang, L. Liden, Z. Yu, W. Chen, and J. Gao, "Check your facts and try again: Improving large language models with external knowledge and automated feedback," *CoRR*, vol. abs/2302.12813, 2023.
- [662] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," *CoRR*, vol. abs/2305.06983, 2023.
- [663] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *CoRR*, vol. abs/2311.05232, 2023.
- [664] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J. Wen, "Evaluating object hallucination in large vision-language models," *CoRR*, vol. abs/2305.10355, 2023.
- [665] S. Kadavath, T. Conerly, A. Askell, T. J. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Dodds, N. Das-Sarma, E. Tran-Johnson, S. Johnston, S. El-Showk, A. Jones, N. Elhage, T. Hume, A. Chen, Y. Bai, S. Bowman, S. Fort, D. Ganguli, D. Hernandez, J. Jacobson, J. Kernion, S. Kravec, L. Lovitt, K. Ndousse, C. Olsson, S. Ringer, D. Amodei, T. B. Brown, J. Clark, N. Joseph, B. Mann, S. McCandlish, C. Olah, and J. Kaplan, "Language models (mostly) know what they know," *CoRR*, vol. abs/2207.05221, 2022.
- [666] P. Manakul, A. Liusie, and M. J. F. Gales, "Self-checkgpt: Zero-resource black-box hallucination detection for generative large language models," *ArXiv*, vol. abs/2305.06983, 2023.
- [667] S. Agarwal, I. Akkaya, V. Balcom, M. Bavarian, G. Bernadett-Shapiro, G. Brockman, M. Brundage, J. Chan, F. Chantzis, N. Deutsch, B. Eastman, A. Eleti, N. Felix, S. P. Fishman, I. Fulford, C. Gibson, J. Gross, M. Heaton, J. Hilton, X. Hu, S. Jain, H. Jin, L. Kilpatrick, C. Kim, M. Kolhede, A. Mayne, P. McMillan, D. Medina, J. Menick, A. Mishchenko, A. Nair, R. Nayak, A. Nee-lakantan, R. Nuttall, J. Parish, A. T. Passos, A. Perelman, F. de Avila Belbute Peres, V. Pong, J. Schulman, E. Sigler, N. Staudacher, N. Turley, J. Tworek, R. Greene, A. Vijayvergiya, C. Voss, J. Weng, M. Wiethoff, S. Yoo, K. Yu, W. Zaremba, S. Zhao, W. Zhuk, and B. Zoph, "Chatgpt plugins," *OpenAI Blog*, March 2023.
- [668] A. Lazaridou, E. Gribovskaya, W. Stokowiec, and N. Grigorev, "Internet-augmented language models through few-shot prompting for open-domain question answering," *CoRR*, vol. abs/2203.05115, 2022.
- [669] H. Qian, Y. Zhu, Z. Dou, H. Gu, X. Zhang, Z. Liu, R. Lai, Z. Cao, J. Nie, and J. Wen, "Webbrain: Learning to generate factually correct articles for queries by grounding on large web corpus," *CoRR*, vol. abs/2304.04358, 2023.
- [670] J. Liu, J. Jin, Z. Wang, J. Cheng, Z. Dou, and J. Wen, "RETA-LLM: A retrieval-augmented large language model toolkit," *CoRR*, vol. abs/2306.05212, 2023.
- [671] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei, "Knowledge neurons in pretrained transformers," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Association for Computational Linguistics, 2022, pp. 8493–8502.
- [672] K. Meng, D. Bau, A. J. Andonian, and Y. Belinkov, "Locating and editing factual associations in gpt," in *Advances in Neural Information Processing Systems*, 2022.
- [673] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 5484–5495.
- [674] Y. Yao, P. Wang, B. Tian, S. Cheng, Z. Li, S. Deng,

- H. Chen, and N. Zhang, "Editing large language models: Problems, methods, and opportunities," *CoRR*, vol. abs/2305.13172, 2023.
- [675] P. Wang, N. Zhang, X. Xie, Y. Yao, B. Tian, M. Wang, Z. Xi, S. Cheng, K. Liu, G. Zheng, and H. Chen, "Easyedit: An easy-to-use knowledge editing framework for large language models," *CoRR*, vol. abs/2308.07269, 2023.
- [676] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Synthetic prompting: Generating chain-of-thought demonstrations for large language models," *CoRR*, vol. abs/2302.00618, 2023.
- [677] Sifatkaur, M. Singh, V. S. B, and N. Malviya, "Mind meets machine: Unravelling gpt-4's cognitive psychology," *CoRR*, vol. abs/2303.11436, 2023.
- [678] M. I. Nye, A. J. Andreassen, G. Gur-Ari, H. Michalewski, J. Austin, D. Bieber, D. Dohan, A. Lewkowycz, M. Bosma, D. Luan, C. Sutton, and A. Odena, "Show your work: Scratchpads for intermediate computation with language models," *CoRR*, vol. abs/2112.00114, 2021.
- [679] J. Qian, H. Wang, Z. Li, S. Li, and X. Yan, "Limitations of language models in arithmetic and symbolic induction," *CoRR*, vol. abs/2208.05051, 2022.
- [680] W. X. Zhao, K. Zhou, Z. Gong, B. Zhang, Y. Zhou, J. Sha, Z. Chen, S. Wang, C. Liu, and J. Wen, "Jiuzhang: A chinese pre-trained language model for mathematical problem understanding," in *KDD '22: The 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, August 14 - 18, 2022*, A. Zhang and H. Rangwala, Eds. ACM, 2022, pp. 4571–4581.
- [681] Q. Wang, C. Kaliszyk, and J. Urban, "First experiments with neural translation of informal to formal mathematics," in *Intelligent Computer Mathematics - 11th International Conference, CICM 2018, Hagenberg, Austria, August 13-17, 2018, Proceedings*, ser. Lecture Notes in Computer Science, F. Rabe, W. M. Farmer, G. O. Passmore, and A. Youssef, Eds., vol. 11006. Springer, 2018, pp. 255–270.
- [682] S. Polu and I. Sutskever, "Generative language modeling for automated theorem proving," *CoRR*, vol. abs/2009.03393, 2020.
- [683] A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygóźdź, P. Miłoś, Y. Wu, and M. Jamnik, "Thor: Wielding hammers to integrate language models and automated theorem provers," *CoRR*, vol. abs/2205.10893, 2022.
- [684] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever, "Formal mathematics statement curriculum learning," *CoRR*, vol. abs/2202.01344, 2022.
- [685] Y. Wu, A. Q. Jiang, W. Li, M. N. Rabe, C. Staats, M. Jamnik, and C. Szegedy, "Autoformalization with large language models," *CoRR*, vol. abs/2205.12615, 2022.
- [686] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample, "Draft, sketch, and prove: Guiding formal theorem provers with informal proofs," *CoRR*, vol. abs/2210.12283, 2022.
- [687] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," *CoRR*, vol. abs/2303.17651, 2023.
- [688] N. Shinn, B. Labash, and A. Gopinath, "Reflexion: an autonomous agent with dynamic memory and self-reflection," *CoRR*, vol. abs/2303.11366, 2023.
- [689] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: large language models can self-correct with tool-interactive critiquing," *CoRR*, vol. abs/2305.11738, 2023.
- [690] J. Uesato, N. Kushman, R. Kumar, H. F. Song, N. Y. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins, "Solving math word problems with process- and outcome-based feedback," *CoRR*, vol. abs/2211.14275, 2022.
- [691] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," *CoRR*, vol. abs/2305.20050, 2023.
- [692] Z. Yuan, H. Yuan, C. Tan, W. Wang, and S. Huang, "How well do large language models perform in arithmetic tasks?" *CoRR*, vol. abs/2304.02015, 2023.
- [693] X. Pi, Q. Liu, B. Chen, M. Ziyadi, Z. Lin, Q. Fu, Y. Gao, J. Lou, and W. Chen, "Reasoning like program executors," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022*, 2022, pp. 761–779.
- [694] H. Zhou, A. Nova, H. Larochelle, A. C. Courville, B. Neyshabur, and H. Sedghi, "Teaching algorithmic reasoning via in-context learning," *CoRR*, vol. abs/2211.09066, 2022.
- [695] A. Parisi, Y. Zhao, and N. Fiedel, "TALM: tool augmented language models," *CoRR*, vol. abs/2205.12255, 2022.
- [696] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9118–9147.
- [697] T. Carta, C. Rómac, T. Wolf, S. Lamprier, O. Sigaud, and P. Oudeyer, "Grounding large language models in interactive environments with online reinforcement learning," *CoRR*, vol. abs/2302.02662, 2023.
- [698] X. Zhu, Y. Chen, H. Tian, C. Tao, W. Su, C. Yang, G. Huang, B. Li, L. Lu, X. Wang, Y. Qiao, Z. Zhang, and J. Dai, "Ghost in the minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory," *CoRR*, vol. abs/2305.17144, 2023.
- [699] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *CoRR*, vol. abs/2305.16291, 2023.
- [700] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao,

- P. Xu, S. Xu, and M. Yan, "Do as I can, not as I say: Grounding language in robotic affordances," *CoRR*, vol. abs/2204.01691, 2022.
- [701] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," *CoRR*, vol. abs/2209.07753, 2022.
- [702] Y. Fu, H. Peng, T. Khot, and M. Lapata, "Improving language model negotiation with self-play and in-context learning from AI feedback," *CoRR*, vol. abs/2305.10142, 2023.
- [703] N. Mehta, M. Teruel, P. F. Sanz, X. Deng, A. H. Awadallah, and J. Kiseleva, "Improving grounded language understanding in a collaborative environment by interacting with agents through help feedback," *CoRR*, vol. abs/2304.10750, 2023.
- [704] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," *CoRR*, vol. abs/2305.15334, 2023.
- [705] S. Hao, T. Liu, Z. Wang, and Z. Hu, "Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings," *CoRR*, vol. abs/2305.11554, 2023.
- [706] Y. Liang, C. Wu, T. Song, W. Wu, Y. Xia, Y. Liu, Y. Ou, S. Lu, L. Ji, S. Mao, Y. Wang, L. Shou, M. Gong, and N. Duan, "Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis," *CoRR*, vol. abs/2303.16434, 2023.
- [707] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," *CoRR*, vol. abs/2305.17126, 2023.
- [708] J. Huang, S. S. Gu, L. Hou, Y. Wu, X. Wang, H. Yu, and J. Han, "Large language models can self-improve," *CoRR*, vol. abs/2210.11610, 2022.
- [709] E. Beeching, C. Fourrier, N. Habib, S. Han, N. Lambert, N. Rajani, O. Sanseviero, L. Tunstall, and T. Wolf, "Open llm leaderboard," https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard, 2023.
- [710] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan, "Agieval: A human-centric benchmark for evaluating foundation models," *CoRR*, vol. abs/2304.06364, 2023.
- [711] H. Zeng, "Measuring massive multitask chinese understanding," *CoRR*, vol. abs/2304.12986, 2023.
- [712] C. Liu, R. Jin, Y. Ren, L. Yu, T. Dong, X. Peng, S. Zhang, J. Peng, P. Zhang, Q. Lyu, X. Su, Q. Liu, and D. Xiong, "M3KE: A massive multi-level multi-subject knowledge evaluation benchmark for chinese large language models," *CoRR*, vol. abs/2305.10263, 2023.
- [713] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, Y. Fu, M. Sun, and J. He, "C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models," *CoRR*, vol. abs/2305.08322, 2023.
- [714] Z. Gu, X. Zhu, H. Ye, L. Zhang, J. Wang, S. Jiang, Z. Xiong, Z. Li, Q. He, R. Xu, W. Huang, W. Zheng, H. Feng, and Y. Xiao, "Xiezhi: An ever-updating benchmark for holistic domain knowledge evaluation," *CoRR*, vol. abs/2306.05783, 2023.
- [715] O. Contributors, "Opencompass: A universal evaluation platform for foundation models," <https://github.com/InternLM/OpenCompass>, 2023.
- [716] Y. Fu, L. Ou, M. Chen, Y. Wan, H. Peng, and T. Khot, "Chain-of-thought hub: A continuous effort to measure large language models' reasoning performance," *CoRR*, vol. abs/2305.17306, 2023.
- [717] J. Yu, X. Wang, S. Tu, S. Cao, D. Zhang-li, X. Lv, H. Peng, Z. Yao, X. Zhang, H. Li, C. Li, Z. Zhang, Y. Bai, Y. Liu, A. Xin, N. Lin, K. Yun, L. Gong, J. Chen, Z. Wu, Y. Qi, W. Li, Y. Guan, K. Zeng, J. Qi, H. Jin, J. Liu, Y. Gu, Y. Yao, N. Ding, L. Hou, Z. Liu, B. Xu, J. Tang, and J. Li, "Kola: Carefully benchmarking world knowledge of large language models," *CoRR*, vol. abs/2306.09296, 2023.
- [718] T. Sawada, D. Paleka, A. Havrilla, P. Tadepalli, P. Vidas, A. Kranias, J. J. Nay, K. Gupta, and A. Komatsuzaki, "ARB: advanced reasoning benchmark for large language models," *CoRR*, vol. abs/2307.13692, 2023.
- [719] Y. Peng, S. Li, W. Gu, Y. Li, W. Wang, C. Gao, and M. R. Lyu, "Revisiting, benchmarking and exploring API recommendation: How far are we?" *IEEE Trans. Software Eng.*, vol. 49, no. 4, pp. 1876–1897, 2023.
- [720] M. Li, F. Song, B. Yu, H. Yu, Z. Li, F. Huang, and Y. Li, "Api-bank: A benchmark for tool-augmented llms," *CoRR*, vol. abs/2304.08244, 2023.
- [721] Q. Tang, Z. Deng, H. Lin, X. Han, Q. Liang, and L. Sun, "Toolalpaca: Generalized tool learning for language models with 3000 simulated cases," *CoRR*, vol. abs/2306.05301, 2023.
- [722] Q. Xu, F. Hong, B. Li, C. Hu, Z. Chen, and J. Zhang, "On the tool manipulation capability of open-source large language models," *CoRR*, vol. abs/2305.16504, 2023.
- [723] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "Toolllm: Facilitating large language models to master 16000+ real-world apis," *CoRR*, vol. abs/2307.16789, 2023.
- [724] Z. Liu, W. Yao, J. Zhang, L. Xue, S. Heinecke, R. Murthy, Y. Feng, Z. Chen, J. C. Niebles, D. Arpit, R. Xu, P. Mui, H. Wang, C. Xiong, and S. Savarese, "BOLAA: benchmarking and orchestrating llm-augmented autonomous agents," *CoRR*, vol. abs/2308.05960, 2023.
- [725] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "Agentbench: Evaluating llms as agents," *CoRR*, vol. abs/2308.03688, 2023.
- [726] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, N. Z. Gong, Y. Zhang, and X. Xie, "Promptbench: Towards evaluating the robustness of large language models on adversarial prompts," *CoRR*, vol. abs/2306.04528, 2023.
- [727] R. S. Shah, K. Chawla, D. Eidnani, A. Shah, W. Du, S. Chava, N. Raman, C. Smiley, J. Chen, and D. Yang, "WHEN FLUE MEETS FLANG: benchmarks and large pre-trained language model for financial domain," *CoRR*, vol. abs/2211.00083, 2022.
- [728] N. Guha, D. E. Ho, J. Nyarko, and C. Ré, "Legalbench: Prototyping a collaborative benchmark for legal reasoning," *CoRR*, vol. abs/2209.06120, 2022.
- [729] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-

- bench and chatbot arena," *CoRR*, vol. abs/2306.05685, 2023.
- [730] X. Wang, Z. Hu, P. Lu, Y. Zhu, J. Zhang, S. Subramaniam, A. R. Loomba, S. Zhang, Y. Sun, and W. Wang, "Scibench: Evaluating college-level scientific problem-solving abilities of large language models," *CoRR*, vol. abs/2307.10635, 2023.
- [731] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, and T. B. Hashimoto, "AlpacaEval: An automatic evaluator of instruction-following models," https://github.com/tatsu-lab/alpaca_eval, 2023.
- [732] Y. Huang, Q. Zhang, P. S. Yu, and L. Sun, "Trustgpt: A benchmark for trustworthy and responsible large language models," *CoRR*, vol. abs/2306.11507, 2023.
- [733] Y. Bai, J. Ying, Y. Cao, X. Lv, Y. He, X. Wang, J. Yu, K. Zeng, Y. Xiao, H. Lyu, J. Zhang, J. Li, and L. Hou, "Benchmarking foundation models with language-model-as-an-examiner," *CoRR*, vol. abs/2306.04181, 2023.
- [734] C. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "Chateval: Towards better llm-based evaluators through multi-agent debate," *CoRR*, vol. abs/2308.07201, 2023.
- [735] Y. Chang, X. Wang, J. Wang, Y. Wu, K. Zhu, H. Chen, L. Yang, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P. S. Yu, Q. Yang, and X. Xie, "A survey on evaluation of large language models," *CoRR*, vol. abs/2307.03109, 2023.
- [736] Z. Zhuang, Q. Chen, L. Ma, M. Li, Y. Han, Y. Qian, H. Bai, Z. Feng, W. Zhang, and T. Liu, "Through the lens of core competency: Survey on evaluation of large language models," *CoRR*, vol. abs/2308.07902, 2023.
- [737] J. H. Clark, J. Palomaki, V. Nikolaev, E. Choi, D. Garrette, M. Collins, and T. Kwiatkowski, "Tydi QA: A benchmark for information-seeking question answering in typologically diverse languages," *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 454–470, 2020.
- [738] L. Gao, J. Tow, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, K. McDonell, N. Muennighoff, J. Phang, L. Reynolds, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, "A framework for few-shot language model evaluation," Sep. 2021.
- [739] R. Shah, K. Chawla, D. Eidnani, A. Shah, W. Du, S. Chava, N. Raman, C. Smiley, J. Chen, and D. Yang, "When flue meets flang: Benchmarks and large pretrained language model for financial domain," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 2322–2335.
- [740] K. Zhou, Y. Zhu, Z. Chen, W. Chen, W. X. Zhao, X. Chen, Y. Lin, J.-R. Wen, and J. Han, "Don't make your llm an evaluation benchmark cheater," *arXiv preprint arXiv:2311.01964*, 2023.
- [741] C. Zan, K. Peng, L. Ding, B. Qiu, B. Liu, S. He, Q. Lu, Z. Zhang, C. Liu, W. Liu, Y. Zhan, and D. Tao, "Vegam: The JD explore academy machine translation system for WMT22," in *Proceedings of the Seventh Conference on Machine Translation, WMT 2022, Abu Dhabi, United Arab Emirates (Hybrid), December 7-8, 2022*, P. Koehn, L. Barrault, O. Bojar, F. Bougares, R. Chatterjee, M. R. Costa-jussà, C. Federmann, M. Fishel, A. Fraser, M. Freitag, Y. Graham, R. Grundkiewicz, P. Guzman, B. Hadrow, M. Huck, A. Jimeno-Yepes, T. Kocmi, A. Martins, M. Morishita, C. Monz, M. Nagata, T. Nakazawa, M. Negri, A. Névél, M. Neves, M. Popel, M. Turchi, and M. Zampieri, Eds. Association for Computational Linguistics, 2022, pp. 411–422.
- [742] Y. Zhao, M. Khalman, R. Joshi, S. Narayan, M. Saleh, and P. J. Liu, "Calibrating sequence likelihood improves conditional language generation," *CoRR*, vol. abs/2210.00045, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.00045>
- [743] D. Khashabi, S. Min, T. Khot, A. Sabharwal, O. Tafjord, P. Clark, and H. Hajishirzi, "Unifiedqa: Crossing format boundaries with a single QA system," in *EMNLP (Findings)*, ser. Findings of ACL, vol. EMNLP 2020. Association for Computational Linguistics, 2020, pp. 1896–1907.
- [744] X. Zhu, J. Wang, L. Zhang, Y. Zhang, R. Gan, J. Zhang, and Y. Yang, "Solving math word problem via cooperative reasoning induced language models," *arXiv preprint arXiv:2210.16257*, 2022.
- [745] A. Nguyen, N. Karampatziakis, and W. Chen, "Meet in the middle: A new pre-training paradigm," *CoRR*, vol. abs/2303.07295, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2303.07295>
- [746] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSL: decoupling schema linking and skeleton parsing for text-to-sql," *CoRR*, vol. abs/2302.05965, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2302.05965>
- [747] W. Kang and J. J. McAuley, "Self-attentive sequential recommendation," in *IEEE International Conference on Data Mining, ICDM 2018, Singapore, November 17-20, 2018*. IEEE Computer Society, 2018, pp. 197–206.
- [748] B. Yang, C. Han, Y. Li, L. Zuo, and Z. Yu, "Improving conversational recommendation systems' quality with context-aware item meta-information," in *Findings of the Association for Computational Linguistics: NAACL 2022, Seattle, WA, United States, July 10-15, 2022*, M. Carpuat, M. de Marneffe, and I. V. M. Ruíz, Eds. Association for Computational Linguistics, 2022, pp. 38–48.
- [749] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, E. Goffinet, D. Heslow, J. Lounay, Q. Malartic, B. Noune, B. Pannier, and G. Penedo, "Falcon-40B: an open large language model with state-of-the-art performance," 2023.
- [750] S. Martin, J. Liermann, and H. Ney, "Algorithms for bigram and trigram word clustering," *Speech communication*, vol. 24, no. 1, pp. 19–37, 1998.
- [751] R. Navigli, "Word sense disambiguation: A survey," *ACM computing surveys (CSUR)*, vol. 41, no. 2, pp. 1–69, 2009.
- [752] W. H. Gomaa, A. A. Fahmy et al., "A survey of text similarity approaches," *international journal of Computer Applications*, vol. 68, no. 13, pp. 13–18, 2013.
- [753] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning-based text classification: a comprehensive review," *ACM computing surveys (CSUR)*, vol. 54, no. 3, pp. 1–40, 2021.
- [754] N. Alex, E. Lifland, L. Tunstall, A. Thakur, P. Maham, C. J. Riedel, E. Hine, C. Ashurst, P. Sedille, A. Carlier, M. Noetel, and A. Stuhlmüller, "RAFT: A real-world few-shot text classification benchmark," in *NeurIPS Datasets and Benchmarks*, 2021.

- [755] C. Qin, A. Zhang, Z. Zhang, J. Chen, M. Yasunaga, and D. Yang, "Is chatgpt a general-purpose natural language processing task solver?" *CoRR*, vol. abs/2302.06476, 2023.
- [756] X. Chen, J. Ye, C. Zu, N. Xu, R. Zheng, M. Peng, J. Zhou, T. Gui, Q. Zhang, and X. Huang, "How robust is gpt-3.5 to predecessors? a comprehensive study on language understanding tasks," 2023.
- [757] D. Nadeau and S. Sekine, "A survey of named entity recognition and classification," *Linguisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.
- [758] A. Ratnaparkhi, "A maximum entropy model for part-of-speech tagging," in *Conference on empirical methods in natural language processing*, 1996.
- [759] V. Yadav and S. Bethard, "A survey on recent advances in named entity recognition from deep learning models," in *Proceedings of the 27th International Conference on Computational Linguistics*, 2018, pp. 2145–2158.
- [760] F. Souza, R. Nogueira, and R. Lotufo, "Portuguese named entity recognition using bert-crf," *arXiv preprint arXiv:1909.10649*, 2019.
- [761] S. Pawar, G. K. Palshikar, and P. Bhattacharyya, "Relation extraction: A survey," *arXiv preprint arXiv:1712.05191*, 2017.
- [762] C. Walker and et al., "Ace 2005 multilingual training corpus ldc2006t06," Philadelphia, 2006.
- [763] J. Gao, H. Zhao, C. Yu, and R. Xu, "Exploring the feasibility of chatgpt for event extraction," *CoRR*, vol. abs/2303.03836, 2023.
- [764] Y. Ma, Y. Cao, Y. Hong, and A. Sun, "Large language model is not a good few-shot information extractor, but a good reranker for hard samples!" *CoRR*, vol. abs/2303.08559, 2023.
- [765] R. Tang, X. Han, X. Jiang, and X. Hu, "Does synthetic data generation of llms help clinical text mining?" *arXiv preprint arXiv:2303.04360*, 2023.
- [766] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang et al., "Zero-shot information extraction via chatting with chatgpt," *arXiv preprint arXiv:2302.10205*, 2023.
- [767] A. Vaswani, S. Bengio, E. Brevdo, F. Chollet, A. Gomez, S. Gouws, L. Jones, L. Kaiser, N. Kalchbrenner, N. Parmar et al., "Tensor2tensor for neural machine translation," in *Proceedings of the 13th Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*, 2018, pp. 193–199.
- [768] B. Zhang, B. Haddow, and A. Birch, "Prompting large language model for machine translation: A case study," *arXiv preprint arXiv:2301.07069*, 2023.
- [769] M. Ghazvininejad, H. Gonen, and L. Zettlemoyer, "Dictionary-based phrase-level prompting of large language models for machine translation," *arXiv preprint arXiv:2302.07856*, 2023.
- [770] L. Wang, C. Lyu, T. Ji, Z. Zhang, D. Yu, S. Shi, and Z. Tu, "Document-level machine translation with large language models," *arXiv preprint arXiv:2304.02210*, 2023.
- [771] W. Jiao, J.-t. Huang, W. Wang, X. Wang, S. Shi, and Z. Tu, "Parrot: Translating during chat using large language models," *arXiv preprint arXiv:2304.02426*, 2023.
- [772] W. Yang, C. Li, J. Zhang, and C. Zong, "Bigtrans: Augmenting large language models with multilingual translation capability over 100 languages," *arXiv preprint arXiv:2305.18098*, 2023.
- [773] J. Kocon, I. Cichecki, O. Kaszyca, M. Kochanek, D. Szydlo, J. Baran, J. Bielaniec, M. Gruza, A. Janz, K. Kanclerz, A. Kocon, B. Koptyra, W. Mielewczyk-Kowszewska, P. Milkowski, M. Oleksy, M. Piasecki, L. Radlinski, K. Wojtasik, S. Wozniak, and P. Kazienko, "Chatgpt: Jack of all trades, master of none," *CoRR*, vol. abs/2302.10724, 2023.
- [774] Q. Zhong, L. Ding, J. Liu, B. Du, and D. Tao, "Can chatgpt understand too? A comparative study on chatgpt and fine-tuned BERT," *CoRR*, vol. abs/2302.10198, 2023.
- [775] D. Cheng, S. Huang, J. Bi, Y. Zhan, J. Liu, Y. Wang, H. Sun, F. Wei, D. Deng, and Q. Zhang, "Uprise: Universal prompt retrieval for improving zero-shot evaluation," *arXiv preprint arXiv:2303.08518*, 2023.
- [776] R. Ren, Y. Qu, J. Liu, W. X. Zhao, Q. She, H. Wu, H. Wang, and J.-R. Wen, "Rocketqav2: A joint training method for dense passage retrieval and passage re-ranking," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 2825–2835.
- [777] W. Sun, L. Yan, X. Ma, P. Ren, D. Yin, and Z. Ren, "Is chatgpt good at search? investigating large language models as re-ranking agent," *arXiv preprint arXiv:2304.09542*, 2023.
- [778] Z. Qin, R. Jagerman, K. Hui, H. Zhuang, J. Wu, J. Shen, T. Liu, J. Liu, D. Metzler, X. Wang et al., "Large language models are effective text rankers with pairwise ranking prompting," *arXiv preprint arXiv:2306.17563*, 2023.
- [779] S. Cho, S. Jeong, J. Seo, and J. C. Park, "Discrete prompt optimization via constrained generation for zero-shot reranker," *arXiv preprint arXiv:2305.13729*, 2023.
- [780] R. Tang, X. Zhang, X. Ma, J. Lin, and F. Ture, "Found in the middle: Permutation self-consistency improves listwise ranking in large language models," *arXiv preprint arXiv:2310.07712*, 2023.
- [781] X. Ma, X. Zhang, R. Pradeep, and J. Lin, "Zero-shot listwise document reranking with a large language model," *arXiv preprint arXiv:2305.02156*, 2023.
- [782] S. Zhuang, H. Zhuang, B. Koopman, and G. Zuccon, "A setwise approach for effective and highly efficient zero-shot ranking with large language models," *arXiv preprint arXiv:2310.09497*, 2023.
- [783] H. Zhuang, Z. Qin, K. Hui, J. Wu, L. Yan, X. Wang, and M. Berdersky, "Beyond yes and no: Improving zero-shot llm rankers via scoring fine-grained relevance labels," *arXiv preprint arXiv:2310.14122*, 2023.
- [784] N. Ziemis, W. Yu, Z. Zhang, and M. Jiang, "Large language models are built-in autoregressive search engines," *arXiv preprint arXiv:2305.09612*, 2023.
- [785] X. Ma, L. Wang, N. Yang, F. Wei, and J. Lin, "Fine-tuning llama for multi-stage text retrieval," *arXiv preprint arXiv:2310.08319*, 2023.
- [786] R. Pradeep, S. Sharifmoghaddam, and J. Lin, "Rankviciuna: Zero-shot listwise document reranking with open-source large language models," *arXiv preprint arXiv:2309.15088*, 2023.
- [787] Y. Tay, V. Q. Tran, M. Dehghani, J. Ni, D. Bahri,

- H. Mehta, Z. Qin, K. Hui, Z. Zhao, J. Gupta *et al.*, "Transformer memory as a differentiable search index," in *Advances in Neural Information Processing Systems*, 2022.
- [788] R. Ren, W. X. Zhao, J. Liu, H. Wu, J.-R. Wen, and H. Wang, "TOME: A two-stage approach for model-based retrieval," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023, pp. 6102–6114. [Online]. Available: <https://aclanthology.org/2023.acl-long.336>
- [789] Y. Qu, Y. Ding, J. Liu, K. Liu, R. Ren, W. X. Zhao, D. Dong, H. Wu, and H. Wang, "Rocketqa: An optimized training approach to dense passage retrieval for open-domain question answering," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 5835–5847.
- [790] R. Ren, S. Lv, Y. Qu, J. Liu, W. X. Zhao, Q. She, H. Wu, H. Wang, and J.-R. Wen, "Pair: Leveraging passage-centric similarity relation for improving dense passage retrieval," in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021, pp. 2173–2183.
- [791] Z. Peng, X. Wu, and Y. Fang, "Soft prompt tuning for augmenting dense retrieval with large language models," *arXiv preprint arXiv:2307.08303*, 2023.
- [792] Z. Dai, V. Y. Zhao, J. Ma, Y. Luan, J. Ni, J. Lu, A. Bakalov, K. Guu, K. Hall, and M.-W. Chang, "Promptagator: Few-shot dense retrieval from 8 examples," in *The Eleventh International Conference on Learning Representations*, 2023.
- [793] A. Askari, M. Aliannejadi, E. Kanoulas, and S. Verberne, "Generating synthetic documents for cross-encoder rerankers: A comparative study of chatgpt and human experts," *arXiv preprint arXiv:2305.02320*, 2023.
- [794] K. Mao, Z. Dou, H. Chen, F. Mo, and H. Qian, "Large language models know your contextual search intent: A prompting framework for conversational search," *arXiv preprint arXiv:2303.06573*, 2023.
- [795] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023, pp. 1762–1777.
- [796] L. Wang, N. Yang, and F. Wei, "Query2doc: Query expansion with large language models," *arXiv preprint arXiv:2303.07678*, 2023.
- [797] G. Ma, X. Wu, P. Wang, Z. Lin, and S. Hu, "Pre-training with large language model-based document expansion for dense passage retrieval," *arXiv preprint arXiv:2308.08285*, 2023.
- [798] W. Sun, Z. Chen, X. Ma, L. Yan, S. Wang, P. Ren, Z. Chen, D. Yin, and Z. Ren, "Instruction distillation makes large language models efficient zero-shot rankers," *arXiv preprint arXiv:2311.01555*, 2023.
- [799] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Large search model: Redefining search stack in the era of llms," *arXiv preprint arXiv:2310.14587*, 2023.
- [800] C. Li, Z. Gan, Z. Yang, J. Yang, L. Li, L. Wang, and J. Gao, "Multimodal foundation models: From specialists to general-purpose assistants," *CoRR*, vol. abs/2309.10020, 2023.
- [801] W. X. Zhao, S. Mu, Y. Hou, Z. Lin, Y. Chen, X. Pan, K. Li, Y. Lu, H. Wang, C. Tian, Y. Min, Z. Feng, X. Fan, X. Chen, P. Wang, W. Ji, Y. Li, X. Wang, and J. Wen, "Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms," in *CIKM*, G. Demartini, G. Zuccon, J. S. Culpepper, Z. Huang, and H. Tong, Eds. ACM, 2021, pp. 4653–4664.
- [802] K. Zhou, H. Wang, W. X. Zhao, Y. Zhu, S. Wang, F. Zhang, Z. Wang, and J. Wen, "S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization," in *CIKM*, M. d'Aquin, S. Dietze, C. Hauff, E. Curry, and P. Cudré-Mauroux, Eds. ACM, 2020, pp. 1893–1902.
- [803] W. X. Zhao, Y. Hou, X. Pan, C. Yang, Z. Zhang, Z. Lin, J. Zhang, S. Bian, J. Tang, W. Sun, Y. Chen, L. Xu, G. Zhang, Z. Tian, C. Tian, S. Mu, X. Fan, X. Chen, and J. Wen, "Recbole 2.0: Towards a more up-to-date recommendation library," in *CIKM*, M. A. Hasan and L. Xiong, Eds. ACM, 2022, pp. 4722–4726.
- [804] L. Xu, Z. Tian, G. Zhang, J. Zhang, L. Wang, B. Zheng, Y. Li, J. Tang, Z. Zhang, Y. Hou, X. Pan, W. X. Zhao, X. Chen, and J. Wen, "Towards a more user-friendly and easy-to-use benchmark library for recommender systems," in *SIGIR*, H. Chen, W. E. Duh, H. Huang, M. P. Kato, J. Mothe, and B. Poblete, Eds. ACM, 2023, pp. 2837–2847.
- [805] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: bayesian personalized ranking from implicit feedback," *CoRR*, vol. abs/1205.2618, 2012.
- [806] W. Fan, Z. Zhao, J. Li, Y. Liu, X. Mei, Y. Wang, J. Tang, and Q. Li, "Recommender systems in the era of large language models (llms)," *CoRR*, 2023.
- [807] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. Shen, C. Qin, C. Zhu, H. Zhu, Q. Liu, H. Xiong, and E. Chen, "A survey on large language models for recommendation," *CoRR*, 2023.
- [808] Y. Gao, T. Sheng, Y. Xiang, Y. Xiong, H. Wang, and J. Zhang, "Chat-rec: Towards interactive and explainable llms-augmented recommender system," *CoRR*, vol. abs/2303.14524, 2023.
- [809] S. Dai, N. Shao, H. Zhao, W. Yu, Z. Si, C. Xu, Z. Sun, X. Zhang, and J. Xu, "Uncovering chatgpt's capabilities in recommender systems," in *RecSys*, J. Zhang, L. Chen, S. Berkovsky, M. Zhang, T. D. Noia, J. Basilico, L. Pizzato, and Y. Song, Eds. ACM, 2023, pp. 1126–1132.
- [810] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. J. McAuley, and W. X. Zhao, "Large language models are zero-shot rankers for recommender systems," *CoRR*, 2023.
- [811] J. Liu, C. Liu, R. Lv, K. Zhou, and Y. Zhang, "Is chatgpt a good recommender? A preliminary study," *CoRR*, vol. abs/2304.10149, 2023.
- [812] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He, "Tallrec: An effective and efficient tuning framework to align large language model with recommendation," in *RecSys*, J. Zhang, L. Chen, S. Berkovsky, M. Zhang, T. D. Noia, J. Basilico, L. Pizzato, and Y. Song, Eds. ACM, 2023, pp. 1007–1014.
- [813] Y. Zhu, L. Wu, Q. Guo, L. Hong, and J. Li, "Collaborative large language model for recommender systems," *arXiv*

- preprint arXiv:2311.01343*, 2023.
- [814] B. Zheng, Y. Hou, H. Lu, Y. Chen, W. X. Zhao, and J.-R. Wen, "Adapting large language models by integrating collaborative semantics for recommendation," 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:265213194>
 - [815] Y. Xi, W. Liu, J. Lin, J. Zhu, B. Chen, R. Tang, W. Zhang, R. Zhang, and Y. Yu, "Towards open-world recommendation with knowledge augmentation from large language models," *CoRR*, vol. abs/2306.10933, 2023.
 - [816] Q. Liu, N. Chen, T. Sakai, and X. Wu, "A first look at llm-powered generative news recommendation," *CoRR*, vol. abs/2305.06566, 2023.
 - [817] R. Li, W. Deng, Y. Cheng, Z. Yuan, J. Zhang, and F. Yuan, "Exploring the upper limits of text-based collaborative filtering using large language models: Discoveries and insights," *CoRR*, vol. abs/2305.11700, 2023.
 - [818] W. Wei, X. Ren, J. Tang, Q. Wang, L. Su, S. Cheng, J. Wang, D. Yin, and C. Huang, "Llmrec: Large language models with graph augmentation for recommendation," *CoRR*, vol. abs/2311.00423, 2023.
 - [819] X. Li, B. Chen, L. Hou, and R. Tang, "Ctrl: Connect tabular and language model for ctr prediction," *arXiv preprint arXiv:2306.02841*, 2023.
 - [820] A. Muhamed, I. Keivanloo, S. Perera, J. Mracek, Y. Xu, Q. Cui, S. Rajagopalan, B. Zeng, and T. Chilimbi, "Ctr-bert: Cost-effective knowledge distillation for billion-parameter teacher models," in *NeurIPS Efficient Natural Language and Speech Processing Workshop*, 2021.
 - [821] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *CoRR*, vol. abs/2308.11432, 2023.
 - [822] L. Wang, J. Zhang, X. Chen, Y. Lin, R. Song, W. X. Zhao, and J. Wen, "Recagent: A novel simulation paradigm for recommender systems," *CoRR*, vol. abs/2306.02552, 2023.
 - [823] E. Ie, C. Hsu, M. Mladenov, V. Jain, S. Narvekar, J. Wang, R. Wu, and C. Boutilier, "Recsim: A configurable simulation platform for recommender systems," *CoRR*, vol. abs/1909.04847, 2019.
 - [824] J. Zhang, Y. Hou, R. Xie, W. Sun, J. J. McAuley, W. X. Zhao, L. Lin, and J. Wen, "Agentcf: Collaborative learning with autonomous language agents for recommender systems," *CoRR*, vol. abs/2310.09233, 2023.
 - [825] A. Zhang, L. Sheng, Y. Chen, H. Li, Y. Deng, X. Wang, and T. Chua, "On generative agents in recommendation," *CoRR*, vol. abs/2310.10108, 2023.
 - [826] Y. Du, Z. Liu, J. Li, and W. X. Zhao, "A survey of vision-language pre-trained models," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, L. D. Raedt, Ed. ijcai.org, 2022, pp. 5436–5443.
 - [827] Z. Gan, L. Li, C. Li, L. Wang, Z. Liu, and J. Gao, "Vision-language pre-training: Basics, recent advances, and future trends," *Found. Trends Comput. Graph. Vis.*, vol. 14, no. 3-4, pp. 163–352, 2022.
 - [828] P. K. Rubenstein, C. Asawaroengchai, D. D. Nguyen, A. Bapna, Z. Borsos, F. de Chaumont Quitry, P. Chen, D. E. Badawy, W. Han, E. Kharitonov *et al.*, "Audiopalm: A large language model that can speak and listen," *CoRR*, 2023.
 - [829] J. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan, "Flamingo: a visual language model for few-shot learning," in *NeurIPS*, 2022.
 - [830] C. Schuhmann, R. Beaumont, R. Vencu, C. Gordon, R. Wightman, M. Cherti, T. Coombes, A. Katta, C. Mullis, M. Wortsman, P. Schramowski, S. Kundurthy, K. Crowson, L. Schmidt, R. Kaczmarczyk, and J. Jitsev, "LAION-5B: an open large-scale dataset for training next generation image-text models," in *NeurIPS*, 2022.
 - [831] S. Changpinyo, P. Sharma, N. Ding, and R. Soricut, "Conceptual 12m: Pushing web-scale image-text pre-training to recognize long-tail visual concepts," in *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2021, virtual, June 19-25, 2021*. Computer Vision Foundation / IEEE, 2021, pp. 3558–3568.
 - [832] Q. Ye, H. Xu, G. Xu, J. Ye, M. Yan, Y. Zhou, J. Wang, A. Hu, P. Shi, Y. Shi, C. Li, Y. Xu, H. Chen, J. Tian, Q. Qi, J. Zhang, and F. Huang, "mplug-owl: Modularization empowers large language models with multimodality," *CoRR*, vol. abs/2304.14178, 2023.
 - [833] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-vl: A frontier large vision-language model with versatile abilities," *CoRR*, vol. abs/2308.12966, 2023.
 - [834] H. Liu, C. Li, Y. Li, and Y. J. Lee, "Improved baselines with visual instruction tuning," *CoRR*, vol. abs/2310.03744, 2023.
 - [835] P. Zhang, X. Dong, B. Wang, Y. Cao, C. Xu, L. Ouyang, Z. Zhao, S. Ding, S. Zhang, H. Duan, W. Zhang, H. Yan, X. Zhang, W. Li, J. Li, K. Chen, C. He, X. Zhang, Y. Qiao, D. Lin, and J. Wang, "Internlm-xcomposer: A vision-language large model for advanced text-image comprehension and composition," *CoRR*, vol. abs/2309.15112, 2023.
 - [836] K. Chen, Z. Zhang, W. Zeng, R. Zhang, F. Zhu, and R. Zhao, "Shikra: Unleashing multimodal llm's referential dialogue magic," *CoRR*, vol. abs/2306.15195, 2023.
 - [837] F. Liu, K. Lin, L. Li, J. Wang, Y. Yacoob, and L. Wang, "Aligning large multi-modal model with robust instruction tuning," *CoRR*, vol. abs/2306.14565, 2023.
 - [838] Y. Du, H. Guo, K. Zhou, W. X. Zhao, J. Wang, C. Wang, M. Cai, R. Song, and J.-R. Wen, "What makes for good visual instructions? synthesizing complex visual reasoning instructions for visual instruction tuning," 2023.
 - [839] D. Gurari, Q. Li, A. J. Stangl, A. Guo, C. Lin, K. Grauman, J. Luo, and J. P. Bigham, "Vizwiz grand challenge: Answering visual questions from blind people," in *CVPR*. Computer Vision Foundation / IEEE Computer Society, 2018, pp. 3608–3617.
 - [840] A. Mishra, K. Alahari, and C. V. Jawahar, "Top-down and bottom-up cues for scene text recognition," in *CVPR*. IEEE Computer Society, 2012, pp. 2687–2694.
 - [841] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu, K. Chen, and D. Lin, "Mmbench: Is your multi-modal model an all-around

- player?" *CoRR*, vol. abs/2307.06281, 2023.
- [842] C. Fu, P. Chen, Y. Shen, Y. Qin, M. Zhang, X. Lin, Z. Qiu, W. Lin, J. Yang, X. Zheng, K. Li, X. Sun, and R. Ji, "MME: A comprehensive evaluation benchmark for multimodal large language models," *CoRR*, vol. abs/2306.13394, 2023.
- [843] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the AI ocean: A survey on hallucination in large language models," *CoRR*, vol. abs/2309.01219, 2023.
- [844] A. Gunjal, J. Yin, and E. Bas, "Detecting and preventing hallucinations in large vision language models," *CoRR*, vol. abs/2308.06394, 2023.
- [845] J. Lu, J. Rao, K. Chen, X. Guo, Y. Zhang, B. Sun, C. Yang, and J. Yang, "Evaluation and mitigation of agnosia in multimodal large language models," *CoRR*, vol. abs/2309.04041, 2023.
- [846] A. Rohrbach, L. A. Hendricks, K. Burns, T. Darrell, and K. Saenko, "Object hallucination in image captioning," in *EMNLP*. Association for Computational Linguistics, 2018, pp. 4035–4045.
- [847] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen, "Evaluating object hallucination in large vision-language models," in *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. [Online]. Available: <https://openreview.net/forum?id=xozJw0kZXF>
- [848] D. A. Hudson and C. D. Manning, "GQA: A new dataset for real-world visual reasoning and compositional question answering," in *CVPR*. Computer Vision Foundation / IEEE, 2019, pp. 6700–6709.
- [849] P. Lu, S. Mishra, T. Xia, L. Qiu, K. Chang, S. Zhu, O. Tafjord, P. Clark, and A. Kalyan, "Learn to explain: Multimodal reasoning via thought chains for science question answering," in *NeurIPS*, 2022.
- [850] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Parikh, and M. Rohrbach, "Towards vqa models that can read," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8317–8326.
- [851] F. Liu, T. Guan, Z. Li, L. Chen, Y. Yacoob, D. Manocha, and T. Zhou, "Hallusionbench: You see what you think? or you think what you see? an image-context reasoning benchmark challenging for gpt-4v(ision), llava-1.5, and other multi-modality models," *CoRR*, vol. abs/2310.14566, 2023.
- [852] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, "VQA: visual question answering," in *ICCV*. IEEE Computer Society, 2015, pp. 2425–2433.
- [853] R. Vedantam, C. L. Zitnick, and D. Parikh, "Cider: Consensus-based image description evaluation," in *CVPR*. IEEE Computer Society, 2015, pp. 4566–4575.
- [854] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *CoRR*, vol. abs/2304.08485, 2023.
- [855] P. Xu, W. Shao, K. Zhang, P. Gao, S. Liu, M. Lei, F. Meng, S. Huang, Y. Qiao, and P. Luo, "Lvlm-ehub: A comprehensive evaluation benchmark for large vision-language models," *CoRR*, vol. abs/2306.09265, 2023.
- [856] Z. Li, Y. Wang, M. Du, Q. Liu, B. Wu, J. Zhang, C. Zhou, Z. Fan, J. Fu, J. Chen, X. Huang, and Z. Wei, "Reform-eval: Evaluating large vision language models via unified re-formulation of task-oriented benchmarks," *CoRR*, vol. abs/2310.02569, 2023.
- [857] B. Li, R. Wang, G. Wang, Y. Ge, Y. Ge, and Y. Shan, "Seed-bench: Benchmarking multimodal llms with generative comprehension," *CoRR*, vol. abs/2307.16125, 2023.
- [858] W. Yu, Z. Yang, L. Li, J. Wang, K. Lin, Z. Liu, X. Wang, and L. Wang, "Mm-vet: Evaluating large multimodal models for integrated capabilities," *CoRR*, vol. abs/2308.02490, 2023.
- [859] J. Wang, L. Meng, Z. Weng, B. He, Z. Wu, and Y. Jiang, "To see is to believe: Prompting GPT-4V for better visual instruction tuning," *CoRR*, vol. abs/2311.07574, 2023.
- [860] Y. Zhang, R. Zhang, J. Gu, Y. Zhou, N. Lipka, D. Yang, and T. Sun, "Llavar: Enhanced visual instruction tuning for text-rich image understanding," *arXiv preprint arXiv:2306.17107*, 2023.
- [861] X. Qi, K. Huang, A. Panda, M. Wang, and P. Mittal, "Visual adversarial examples jailbreak aligned large language models," in *The Second Workshop on New Frontiers in Adversarial Machine Learning*, 2023.
- [862] Y. Zhou, C. Cui, J. Yoon, L. Zhang, Z. Deng, C. Finn, M. Bansal, and H. Yao, "Analyzing and mitigating object hallucination in large vision-language models," *arXiv preprint arXiv:2310.00754*, 2023.
- [863] Z. Sun, S. Shen, S. Cao, H. Liu, C. Li, Y. Shen, C. Gan, L.-Y. Gui, Y.-X. Wang, Y. Yang *et al.*, "Aligning large multimodal models with factually augmented rlhf," *arXiv preprint arXiv:2309.14525*, 2023.
- [864] E. Jiménez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, and K. Srinivas, "Semtab 2019: Resources to benchmark tabular data to knowledge graph matching systems," in *The Semantic Web - 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31-June 4, 2020, Proceedings*, ser. Lecture Notes in Computer Science, vol. 12123. Springer, 2020, pp. 514–530.
- [865] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *CoRR*, vol. abs/2306.08302, 2023.
- [866] Y. Sun, S. Wang, S. Feng, S. Ding, C. Pang, J. Shang, J. Liu, X. Chen, Y. Zhao, Y. Lu, W. Liu, Z. Wu, W. Gong, J. Liang, Z. Shang, P. Sun, W. Liu, X. Ouyang, D. Yu, H. Tian, H. Wu, and H. Wang, "ERNIE 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation," *CoRR*, vol. abs/2107.02137, 2021. [Online]. Available: <https://arxiv.org/abs/2107.02137>
- [867] Z. Zhang, X. Han, Z. Liu, X. Jiang, M. Sun, and Q. Liu, "ERNIE: enhanced language representation with informative entities," in *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*. Association for Computational Linguistics, 2019, pp. 1441–1451.
- [868] X. Wang, T. Gao, Z. Zhu, Z. Zhang, Z. Liu, J. Li, and J. Tang, "KEPLER: A unified model for knowledge embedding and pre-trained language representation," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 176–194, 2021.
- [869] J. Zhang, X. Zhang, J. Yu, J. Tang, J. Tang, C. Li, and H. Chen, "Subgraph retrieval enhanced model for multi-

- hop knowledge base question answering," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022*. Association for Computational Linguistics, 2022, pp. 5773–5784.
- [870] P. Ke, H. Ji, Y. Ran, X. Cui, L. Wang, L. Song, X. Zhu, and M. Huang, "Jointgt: Graph-text joint representation learning for text generation from knowledge graphs," in *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, ser. Findings of ACL, vol. ACL/IJCNLP 2021. Association for Computational Linguistics, 2021, pp. 2526–2538.
- [871] O. Agarwal, H. Ge, S. Shakeri, and R. Al-Rfou, "Large scale knowledge graph based synthetic corpus generation for knowledge-enhanced language model pre-training," *CoRR*, vol. abs/2010.12688, 2020.
- [872] W. Chen, Y. Su, X. Yan, and W. Y. Wang, "KGPT: knowledge-grounded pre-training for data-to-text generation," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. Association for Computational Linguistics, 2020, pp. 8635–8648.
- [873] Y. Gu, X. Deng, and Y. Su, "Don't generate, discriminate: A proposal for grounding language models to real-world environments," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. Association for Computational Linguistics, 2023, pp. 4928–4949.
- [874] L. Luo, Y. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," *CoRR*, vol. abs/2310.01061, 2023.
- [875] Y. Lan and J. Jiang, "Query graph generation for answering multi-hop complex questions from knowledge bases," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, D. J. and, Ed. Association for Computational Linguistics, 2020, pp. 969–974.
- [876] P. Wang, N. Zhang, X. Xie, Y. Yao, B. Tian, M. Wang, Z. Xi, S. Cheng, K. Liu, G. Zheng, and H. Chen, "Easyedit: An easy-to-use knowledge editing framework for large language models," *CoRR*, vol. abs/2308.07269, 2023.
- [877] Y. Yao, P. Wang, B. Tian, S. Cheng, Z. Li, S. Deng, H. Chen, and N. Zhang, "Editing large language models: Problems, methods, and opportunities," *CoRR*, vol. abs/2305.13172, 2023.
- [878] S. Choi, T. Fang, Z. Wang, and Y. Song, "KCTS: knowledge-constrained tree search decoding with token-level hallucination detection," *CoRR*, vol. abs/2310.09044, 2023.
- [879] S. Zhang, L. Pan, J. Zhao, and W. Y. Wang, "Mitigating language model hallucination with interactive question-knowledge alignment," *CoRR*, vol. abs/2305.13669, 2023.
- [880] Y. Zhu, X. Wang, J. Chen, S. Qiao, Y. Ou, Y. Yao, S. Deng, H. Chen, and N. Zhang, "Llms for knowledge graph construction and reasoning: Recent capabilities and future opportunities," *CoRR*, vol. abs/2305.13168, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2305.13168>
- [881] M. Karpinska, N. Akoury, and M. Iyyer, "The perils of using mechanical turk to evaluate open-ended text generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 1265–1285.
- [882] H. Lee, S. Phatale, H. Mansoor, K. Lu, T. Mesnard, C. Bishop, V. Carbune, and A. Rastogi, "RLAIF: scaling reinforcement learning from human feedback with AI feedback," *CoRR*, vol. abs/2309.00267, 2023.
- [883] G. Cui, L. Yuan, N. Ding, G. Yao, W. Zhu, Y. Ni, G. Xie, Z. Liu, and M. Sun, "Ultrafeedback: Boosting language models with high-quality feedback," *CoRR*, vol. abs/2310.01377, 2023.
- [884] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji, "MINT: evaluating llms in multi-turn interaction with tools and language feedback," *CoRR*, vol. abs/2309.10691, 2023.
- [885] S. Saha, O. Levy, A. Celikyilmaz, M. Bansal, J. Weston, and X. Li, "Branch-solve-merge improves large language model evaluation and generation," *CoRR*, vol. abs/2310.15123, 2023.
- [886] X. Zhang, B. Yu, H. Yu, Y. Lv, T. Liu, F. Huang, H. Xu, and Y. Li, "Wider and deeper LLM networks are fairer LLM evaluators," *CoRR*, vol. abs/2308.01862, 2023.
- [887] C. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, "Chateval: Towards better llm-based evaluators through multi-agent debate," *CoRR*, vol. abs/2308.07201, 2023.
- [888] R. Li, T. Patel, and X. Du, "PRD: peer rank and discussion improve large language model based evaluations," *CoRR*, vol. abs/2307.02762, 2023.
- [889] L. Zhu, X. Wang, and X. Wang, "Judgelm: Fine-tuned large language models are scalable judges," *CoRR*, vol. abs/2310.17631, 2023.
- [890] Z. Zeng, J. Yu, T. Gao, Y. Meng, T. Goyal, and D. Chen, "Evaluating large language models at evaluating instruction following," *CoRR*, vol. abs/2310.07641, 2023.
- [891] R. Koo, M. Lee, V. Raheja, J. I. Park, Z. M. Kim, and D. Kang, "Benchmarking cognitive biases in large language models as evaluators," *CoRR*, vol. abs/2309.17012, 2023.
- [892] P. West, X. Lu, N. Dziri, F. Brahman, L. Li, J. D. Hwang, L. Jiang, J. Fisher, A. Ravichander, K. Chandu, B. Newman, P. W. Koh, A. Ettinger, and Y. Choi, "The generative AI paradox: 'what it can create, it may not understand'," *CoRR*, vol. abs/2311.00059, 2023.
- [893] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," *CoRR*, vol. abs/2310.01798, 2023.
- [894] K. Stechly, M. Marquez, and S. Kambhampati, "GPT-4 doesn't know it's wrong: An analysis of iterative prompting for reasoning problems," *CoRR*, vol. abs/2310.12397, 2023.
- [895] O. Nov, N. Singh, and D. M. Mann, "Putting chatgpt's medical advice to the (turing) test," *CoRR*, vol. abs/2301.10035, 2023.
- [896] K. Yang, S. Ji, T. Zhang, Q. Xie, and S. Anani-

- adou, "On the evaluations of chatgpt and emotion-enhanced prompting for mental health analysis," *CoRR*, vol. abs/2304.03347, 2023.
- [897] K. Jeblick, B. Schachtner, J. Dextl, A. Mittermeier, A. T. Stüber, J. Topalis, T. Weber, P. Wesp, B. O. Sabel, J. Rieke, and M. Ingrisch, "Chatgpt makes medicine easy to swallow: An exploratory case study on simplified radiology reports," *CoRR*, vol. abs/2212.14882, 2022.
- [898] K. Singhal, T. Tu, J. Gottweis, R. Sayres, E. Wulczyn, L. Hou, K. Clark, S. Pfohl, H. Cole-Lewis, D. Neal, M. Schaekermann, A. Wang, M. Amin, S. Lachgar, P. A. Mansfield, S. Prakash, B. Green, E. Dominowska, B. A. y Arcas, N. Tomasev, Y. Liu, R. Wong, C. Semturs, S. S. Mahdavi, J. K. Barral, D. R. Webster, G. S. Corrado, Y. Matias, S. Azizi, A. Karthikesalingam, and V. Natarajan, "Towards expert-level medical question answering with large language models," *CoRR*, vol. abs/2305.09617, 2023.
- [899] S. Yang, H. Zhao, S. Zhu, G. Zhou, H. Xu, Y. Jia, and H. Zan, "Zhongjing: Enhancing the chinese medical capabilities of large language model through expert feedback and real-world multi-turn dialogue," *CoRR*, vol. abs/2308.03549, 2023.
- [900] S. Chen, B. H. Kann, M. B. Foote, H. J. Aerts, G. K. Savova, R. H. Mak, and D. S. Bitterman, "The utility of chatgpt for cancer treatment information," *medRxiv*, 2023.
- [901] K. Malinka, M. Peresini, A. Firc, O. Hujnak, and F. Janus, "On the educational impact of chatgpt: Is artificial intelligence ready to obtain a university degree?" *CoRR*, vol. abs/2303.11146, 2023.
- [902] T. Susnjak, "Chatgpt: The end of online exam integrity?" *CoRR*, vol. abs/2212.09292, 2022.
- [903] K. Tan, T. Pang, and C. Fan, "Towards applying powerful large ai models in classroom teaching: Opportunities, challenges and prospects," 2023.
- [904] F. Kamalov and I. Gurrib, "A new era of artificial intelligence in education: A multifaceted revolution," *CoRR*, vol. abs/2305.18303, 2023.
- [905] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [906] A. Blair-Stanek, N. Holzenberger, and B. V. Durme, "Can GPT-3 perform statutory reasoning?" *CoRR*, vol. abs/2302.06100, 2023.
- [907] D. Trautmann, A. Petrova, and F. Schilder, "Legal prompt engineering for multilingual legal judgement prediction," *CoRR*, vol. abs/2212.02199, 2022.
- [908] J. H. Choi, K. E. Hickman, A. Monahan, and D. Schwarcz, "Chatgpt goes to law school," *Available at SSRN*, 2023.
- [909] J. J. Nay, "Law informs code: A legal informatics approach to aligning artificial intelligence with humans," *CoRR*, vol. abs/2209.13020, 2022.
- [910] F. Yu, L. Quartey, and F. Schilder, "Legal prompting: Teaching a language model to think like a lawyer," *CoRR*, vol. abs/2212.01326, 2022.
- [911] D. Trautmann, A. Petrova, and F. Schilder, "Legal prompt engineering for multilingual legal judgement prediction," *CoRR*, vol. abs/2212.02199, 2022.
- [912] A. Tamkin, M. Brundage, J. Clark, and D. Ganguli, "Understanding the capabilities, limitations, and societal impact of large language models," *CoRR*, vol. abs/2102.02503, 2021.
- [913] Z. Sun, "A short survey of viewing large language models in legal aspect," *CoRR*, vol. abs/2303.09136, 2023.
- [914] A. Abid, M. Farooqi, and J. Zou, "Persistent anti-muslim bias in large language models," in *AIES '21: AAAI/ACM Conference on AI, Ethics, and Society, Virtual Event, USA, May 19-21, 2021*, M. Fourcade, B. Kuipers, S. Lazar, and D. K. Mulligan, Eds. ACM, 2021, pp. 298–306.
- [915] A. Shah and S. Chava, "Zero is not hero yet: Benchmarking zero-shot performance of llms for financial tasks," *CoRR*, vol. abs/2305.16633, 2023.
- [916] D. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," *CoRR*, vol. abs/1908.10063, 2019.
- [917] J. C. S. Alvarado, K. Verspoor, and T. Baldwin, "Domain adaption of named entity recognition to support credit risk assessment," in *Proceedings of the Australasian Language Technology Association Workshop, ALTA 2015, Parramatta, Australia, December 8 - 9, 2015*, B. Hachey and K. Webster, Eds. ACL, 2015, pp. 84–90.
- [918] G. Son, H. Jung, M. Hahm, K. Na, and S. Jin, "Beyond classification: Financial reasoning in state-of-the-art language models," *CoRR*, vol. abs/2305.01505, 2023.
- [919] X. Zhang, Q. Yang, and D. Xu, "Xuanyuan 2.0: A large chinese financial chat model with hundreds of billions parameters," *arXiv preprint arXiv:2305.12002*, 2023.
- [920] H. Yang, X.-Y. Liu, and C. D. Wang, "Fingpt: Open-source financial large language models," *CoRR*, vol. abs/2306.06031, 2023.
- [921] Q. Jin, B. Dhingra, Z. Liu, W. W. Cohen, and X. Lu, "Pubmedqa: A dataset for biomedical research question answering," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, 2019, pp. 2567–2577.
- [922] A. Krithara, A. Nentidis, K. Bougiatiotis, and G. Paliouras, "Bioasq-qa: A manually curated corpus for biomedical question answering," 2022.
- [923] Z. Bi, N. Zhang, Y. Xue, Y. Ou, D. Ji, G. Zheng, and H. Chen, "Oceangpt: A large language model for ocean science tasks," *CoRR*, vol. abs/2310.02031, 2023.
- [924] C. Zhang, C. Zhang, C. Li, Y. Qiao, S. Zheng, S. K. Dam, M. Zhang, J. U. Kim, S. T. Kim, J. Choi, G. Park, S. Bae, L. Lee, P. Hui, I. S. Kweon, and C. S. Hong, "One small step for generative ai, one giant leap for AGI: A complete survey on chatgpt in AIGC era," *CoRR*, vol. abs/2304.06488, 2023.
- [925] M. Haman and M. Skolnik, "Using chatgpt to conduct a literature review," *Accountability in research*, 2023.
- [926] Ö. Aydın and E. Karaarslan, "Openai chatgpt generated literature review: Digital twin in healthcare," *SSRN Electronic Journal*, 2022.
- [927] Y. J. Park, D. Kaplan, Z. Ren, C. Hsu, C. Li, H. Xu, S. Li, and J. Li, "Can chatgpt be used to generate scientific

- hypotheses?" *CoRR*, vol. abs/2304.12208, 2023.
- [928] M. M. Hassan, R. A. Knipper, and S. K. K. Santu, "Chatgpt as your personal data scientist," *CoRR*, vol. abs/2305.13657, 2023.
- [929] L. Cheng, X. Li, and L. Bing, "Is GPT-4 a good data analyst?" *CoRR*, vol. abs/2305.15038, 2023.
- [930] S. I. M. Hussam Alkaissi, "Artificial hallucinations in chatgpt: Implications in scientific writing," *PubMed*, 2023.
- [931] A. Azaria, R. Azoulay, and S. Reches, "Chatgpt is a remarkable tool – for experts," *CoRR*, vol. abs/2306.03102, 2023.
- [932] O. O. Buruk, "Academic writing with GPT-3.5: reflections on practices, efficacy and transparency," *CoRR*, vol. abs/2304.11079, 2023.
- [933] R. Liu and N. B. Shah, "Reviewergpt? an exploratory study on using large language models for paper reviewing," *CoRR*, vol. abs/2306.00622, 2023.
- [934] M. Kosinski, "Theory of mind may have spontaneously emerged in large language models," *CoRR*, vol. abs/2302.02083, 2023.
- [935] M. M. Amin, E. Cambria, and B. W. Schuller, "Will affective computing emerge from foundation models and general ai? A first evaluation on chatgpt," *CoRR*, vol. abs/2303.03186, 2023.
- [936] G. Sridhara, R. H. G., and S. Mazumdar, "Chatgpt: A study on its utility for ubiquitous software engineering tasks," *CoRR*, vol. abs/2305.16837, 2023.
- [937] W. Sun, C. Fang, Y. You, Y. Miao, Y. Liu, Y. Li, G. Deng, S. Huang, Y. Chen, Q. Zhang, H. Qian, Y. Liu, and Z. Chen, "Automatic code summarization via chatgpt: How far are we?" *CoRR*, vol. abs/2305.12865, 2023.
- [938] C. S. Xia and L. Zhang, "Conversational automated program repair," *CoRR*, vol. abs/2301.13246, 2023.
- [939] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy, "The impact of positional encoding on length generalization in transformers," *CoRR*, vol. abs/2305.19466, 2023.
- [940] W. Xiong, J. Liu, I. Molybog, H. Zhang, P. Bhargava, R. Hou, L. Martin, R. Rungta, K. A. Sankararaman, B. Oguz, M. Khabsa, H. Fang, Y. Mehdad, S. Narang, K. Malik, A. Fan, S. Bhosale, S. Edunov, M. Lewis, S. Wang, and H. Ma, "Effective long-context scaling of foundation models," *CoRR*, vol. abs/2309.16039, 2023.
- [941] kaiokendev, "Things I'm learning while training super-hot." 2023.
- [942] Z. Dong, T. Tang, J. Li, W. X. Zhao, and J. Wen, "BAMBOO: A comprehensive benchmark for evaluating long text modeling capacities of large language models," *CoRR*, vol. abs/2309.13345, 2023.
- [943] J. Su. (2023) Transformer upgrade path: 12, infinite extrapolation of roper?
- [944] A. Pal, D. Karkhanis, M. Roberts, S. Dooley, A. Sundararajan, and S. Naidu, "Giraffe: Adventures in expanding context lengths in llms," *CoRR*, vol. abs/2308.10882, 2023.
- [945] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume, EACL 2021, Online, April 19* - 23, 2021. Association for Computational Linguistics, 2021, pp. 874–880.
- [946] N. Ratner, Y. Levine, Y. Belinkov, O. Ram, I. Magar, O. Abend, E. Karpas, A. Shashua, K. Leyton-Brown, and Y. Shoham, "Parallel context windows for large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*. Association for Computational Linguistics, 2023, pp. 6383–6402.
- [947] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *CoRR*, vol. abs/2004.05150, 2020.
- [948] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *CoRR*, vol. abs/2309.17453, 2023.
- [949] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [950] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, and S. Wang, "Lm-infinite: Simple on-the-fly length generalization for large language models," *CoRR*, vol. abs/2308.16137, 2023.
- [951] A. Bertsch, U. Alon, G. Neubig, and M. R. Gormley, "Unlimiformer: Long-range transformers with unlimited length input," *CoRR*, vol. abs/2305.01625, 2023.
- [952] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, "Memorizing transformers," in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022.
- [953] Y. Lu, X. Zhou, W. He, J. Zhao, T. Ji, T. Gui, Q. Zhang, and X. Huang, "Longheads: Multi-head attention is secretly a long context processor," *CoRR*, vol. abs/2402.10685, 2024.
- [954] C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, S. Han, and M. Sun, "Infillm: Unveiling the intrinsic capacity of llms for understanding extremely long sequences with training-free memory," *CoRR*, vol. abs/2402.04617, 2024.
- [955] Y. Fu, R. Panda, X. Niu, X. Yue, H. Hajishirzi, Y. Kim, and H. Peng, "Data engineering for scaling language models to 128k context," *CoRR*, vol. abs/2402.10171, 2024.
- [956] K. Lv, X. Liu, Q. Guo, H. Yan, C. He, X. Qiu, and D. Lin, "Longwanjuan: Towards systematic measurement for long text quality," *CoRR*, vol. abs/2402.13583, 2024.
- [957] H. Chen, R. Pasunuru, J. Weston, and A. Celikyilmaz, "Walking down the memory maze: Beyond context limit through interactive reading," *CoRR*, vol. abs/2310.05029, 2023.
- [958] W. Zhou, Y. E. Jiang, P. Cui, T. Wang, Z. Xiao, Y. Hou, R. Cotterell, and M. Sachan, "Recurrentgpt: Interactive generation of (arbitrarily) long text," *CoRR*, vol. abs/2305.13304, 2023.
- [959] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "Memgpt: Towards llms as operating systems," *CoRR*, vol. abs/2310.08560, 2023.
- [960] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoenybi, and B. Catanzaro,

- "Retrieval meets long context large language models," *CoRR*, vol. abs/2310.03025, 2023.
- [961] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. [Online]. Available: <http://aima.cs.berkeley.edu/>
- [962] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, "Building machines that learn and think like people," *CoRR*, vol. abs/1604.00289, 2016.
- [963] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *CoRR*, vol. abs/2210.03629, 2022.
- [964] 2023. [Online]. Available: <https://github.com/AntonOsika/gpt-engineer>
- [965] X. Team, "Xagent: An autonomous agent for complex task solving," 2023.
- [966] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: communicative agents for "mind" exploration of large scale language model society," *CoRR*, vol. abs/2303.17760, 2023.
- [967] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu, "Metagpt: Meta programming for multi-agent collaborative framework," *CoRR*, vol. abs/2308.00352, 2023.
- [968] C. Pham, B. Liu, Y. Yang, Z. Chen, T. Liu, J. Yuan, B. A. Plummer, Z. Wang, and H. Yang, "Let models speak ciphers: Multiagent debate through embeddings," *CoRR*, vol. abs/2310.06272, 2023.
- [969] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C. Qian, C.-M. Chan, Y. Qin, Y. Lu, R. Xie et al., "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents," *arXiv preprint arXiv:2308.10848*, 2023.
- [970] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation framework," 2023.
- [971] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," *CoRR*, vol. abs/2305.14325, 2023.
- [972] Y. Shao, L. Li, J. Dai, and X. Qiu, "Character-llm: A trainable agent for role-playing," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Association for Computational Linguistics, 2023, pp. 13 153–13 187.
- [973] W. Hua, X. Yang, Z. Li, W. Cheng, and Y. Zhang, "Trustagent: Towards safe and trustworthy llm-based agents through agent constitution," *CoRR*, vol. abs/2402.01586, 2024.
- [974] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *CoRR*, vol. abs/2311.05232, 2023.
- [975] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *ICLR (Poster)*. OpenReview.net, 2019.
- [976] V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoenybi, and B. Catanzaro, "Reducing activation recomputation in large transformer models," in *MLSys*. mlsys.org, 2023.
- [977] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "Zero: memory optimizations toward training trillion parameter models," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, C. Cuicchi, I. Qualters, and W. T. Kramer, Eds. IEEE/ACM, 2020, p. 20.
- [978] J. Ren, S. Rajbhandari, R. Y. Aminabadi, O. Ruwase, S. Yang, M. Zhang, D. Li, and Y. He, "Zero-offload: Democratizing billion-scale model training," in *2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021*, I. Calciu and G. Kuenning, Eds. USENIX Association, 2021, pp. 551–564.
- [979] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "Zero-infinity: breaking the GPU memory wall for extreme scale deep learning," in *SC*. ACM, 2021, p. 59.
- [980] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *NeurIPS*, 2022.
- [981] S. A. Jacobs, M. Tanaka, C. Zhang, M. Zhang, S. L. Song, S. Rajbhandari, and Y. He, "Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models," *CoRR*, vol. abs/2309.14509, 2023.
- [982] H. Liu, M. Zaharia, and P. Abbeel, "Ring attention with blockwise transformers for near-infinite context," *CoRR*, vol. abs/2310.01889, 2023.
- [983] Y. Chen, T. Tang, E. Xiang, L. Li, W. X. Zhao, J. Wang, Y. Chai, and J. Wen, "Towards coarse-to-fine evaluation of inference efficiency for large language models," *CoRR*, vol. abs/2404.11502, 2024.
- [984] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single GPU," in *ICML*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 31 094–31 116.
- [985] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-decoding for long-context inference," 2023. [Online]. Available: <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>
- [986] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko, and Y. He, "Deepspeed-fastgen: High-throughput text generation for llms via MII and deepspeed-inference," *CoRR*, vol. abs/2401.08671, 2024.
- [987] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*, 2023.
- [988] C. Chen, S. Borgeaud, G. Irving, J. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *CoRR*, vol. abs/2302.01318, 2023.
- [989] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, R. Y. Y. Wong, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating generative LLM serving

- with speculative inference and token tree verification,” *CoRR*, vol. abs/2305.09781, 2023.
- [990] B. Spector and C. Ré, “Accelerating LLM inference with staged speculative decoding,” *CoRR*, vol. abs/2308.04623, 2023.
- [991] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: How to use large language models while reducing cost and improving performance,” *CoRR*, vol. abs/2305.05176, 2023.
- [992] M. Yue, J. Zhao, M. Zhang, L. Du, and Z. Yao, “Large language model cascades with mixture of thoughts representations for cost-efficient reasoning,” *CoRR*, vol. abs/2310.03094, 2023.
- [993] J. Gu, J. Bradbury, C. Xiong, V. O. K. Li, and R. Socher, “Non-autoregressive neural machine translation,” in *ICLR (Poster)*. OpenReview.net, 2018.
- [994] C. Wang, J. Zhang, and H. Chen, “Semi-autoregressive neural machine translation,” in *EMNLP*. Association for Computational Linguistics, 2018, pp. 479–488.
- [995] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, “Medusa: Simple LLM inference acceleration framework with multiple decoding heads,” *CoRR*, vol. abs/2401.10774, 2024.
- [996] S. Teerapittayanon, B. McDanel, and H. T. Kung, “Branchynet: Fast inference via early exiting from deep neural networks,” in *ICPR*. IEEE, 2016, pp. 2464–2469.
- [997] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, “Multi-scale dense networks for resource efficient image classification,” in *ICLR*. OpenReview.net, 2018.
- [998] D. Raposo, S. Ritter, B. A. Richards, T. P. Lillicrap, P. C. Humphreys, and A. Santoro, “Mixture-of-depths: Dynamically allocating compute in transformer-based language models,” *CoRR*, vol. abs/2404.02258, 2024.
- [999] Z. Wan, X. Wang, C. Liu, S. Alam, Y. Zheng, J. Liu, Z. Qu, S. Yan, Y. Zhu, Q. Zhang, M. Chowdhury, and M. Zhang, “Efficient large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.03863>
- [1000] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” *CoRR*, vol. abs/2103.13630, 2021. [Online]. Available: <https://arxiv.org/abs/2103.13630>
- [1001] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” *CoRR*, vol. abs/2208.07339, 2022.
- [1002] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” 2023.
- [1003] Y. Shang, Z. Yuan, Q. Wu, and Z. Dong, “PB-LLM: partially binarized large language models,” *CoRR*, vol. abs/2310.00034, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.00034>
- [1004] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, “Spqr: A sparse-quantized representation for near-lossless LLM weight compression,” *CoRR*, vol. abs/2306.03078, 2023.
- [1005] Z. Guan, H. Huang, Y. Su, H. Huang, N. Wong, and H. Yu, “APTQ: attention-aware post-training mixed-precision quantization for large language models,” *CoRR*, vol. abs/2402.14866, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2402.14866>
- [1006] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, “OWQ: outlier-aware weight quantization for efficient fine-tuning and inference of large language models,” in *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, M. J. Wooldridge, J. G. Dy, and S. Natarajan, Eds. AAAI Press, 2024, pp. 13 355–13 364. [Online]. Available: <https://doi.org/10.1609/aaai.v38i12.29237>
- [1007] G. Xiao, J. Lin, M. Seznec, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” *CoRR*, vol. abs/2211.10438, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2211.10438>
- [1008] Z. Yao, R. Y. Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, “Zeroquant: Efficient and affordable post-training quantization for large-scale transformers,” in *NeurIPS*, 2022.
- [1009] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “Gptq: Accurate post-training quantization for generative pre-trained transformers,” *arXiv preprint arXiv:2210.17323*, 2022.
- [1010] E. Frantar and D. Alistarh, “Optimal brain compression: A framework for accurate post-training quantization and pruning,” in *NeurIPS*, 2022.
- [1011] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *arXiv preprint arXiv:2305.14314*, 2023.
- [1012] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, “Llm-qat: Data-free quantization aware training for large language models,” 2023.
- [1013] Z. Yao, X. Wu, C. Li, S. Youn, and Y. He, “Zeroquant-v2: Exploring post-training quantization in llms from comprehensive study to low rank compensation,” 2023.
- [1014] T. Dettmers and L. Zettlemoyer, “The case for 4-bit precision: k-bit inference scaling laws,” *CoRR*, vol. abs/2212.09720, 2022.
- [1015] L. Peiyu, L. Zikang, G. Ze-Feng, G. Dawei, Z. W. Xin, L. Yaliang, D. Bolin, and W. Ji-Rong, “Do emergent abilities exist in quantized large language models: An empirical study,” *arXiv preprint arXiv:2307.08072*, 2023.
- [1016] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian, “Qa-lora: Quantization-aware low-rank adaptation of large language models,” *CoRR*, vol. abs/2309.14717, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2309.14717>
- [1017] Y. Li, Y. Yu, C. Liang, P. He, N. Karampatziakis, W. Chen, and T. Zhao, “Loftq: Lora-fine-tuning-aware quantization for large language models,” *CoRR*, vol. abs/2310.08659, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.08659>
- [1018] Y. Gu, L. Dong, F. Wei, and M. Huang, “Knowledge distillation of large language models,” *CoRR*, vol. abs/2306.08543, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2306.08543>
- [1019] C. Hsieh, C. Li, C. Yeh, H. Nakhost, Y. Fujii,

- A. Ratner, R. Krishna, C. Lee, and T. Pfister, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," in *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023*, A. Rogers, J. L. Boyd-Graber, and N. Okazaki, Eds. Association for Computational Linguistics, 2023, pp. 8003–8017. [Online]. Available: <https://doi.org/10.18653/v1/2023.findings-acl.507>
- [1020] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 323–10 337.
- [1021] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.
- [1022] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared llama: Accelerating language model pre-training via structured pruning," *arXiv preprint arXiv:2310.06694*, 2023.
- [1023] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, "8-bit optimizers via block-wise quantization," *9th International Conference on Learning Representations, ICLR*, 2022.
- [1024] Y. Ding, W. Fan, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A survey on rag meets llms: Towards retrieval-augmented large language models," *arXiv preprint arXiv:2405.06211*, 2024.
- [1025] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [1026] S. Robertson and H. Zaragoza, *The probabilistic relevance framework: BM25 and beyond*, 2009.
- [1027] Y. Wang, R. Ren, J. Li, W. X. Zhao, J. Liu, and J.-R. Wen, "Rear: A relevance-aware retrieval-augmented framework for open-domain question answering," *arXiv preprint arXiv:2402.17497*, 2024.
- [1028] D. Rau, S. Wang, H. Déjean, and S. Clinchant, "Context embeddings for efficient answer generation in rag," *arXiv preprint arXiv:2407.09252*, 2024.
- [1029] F. Xu, W. Shi, and E. Choi, "Recomp: Improving retrieval-augmented lms with context compression and selective augmentation," in *The Twelfth International Conference on Learning Representations*, 2024.
- [1030] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 9248–9274.
- [1031] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, D. Yu, and H. Zhang, "Dense x retrieval: What retrieval granularity should we use?" *arXiv preprint arXiv:2312.06648*, 2023.
- [1032] X. Huang, S. Cheng, Y. Shu, Y. Bao, and Y. Qu, "Question decomposition tree for answering complex questions over knowledge bases," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, 2023, pp. 12 924–12 932.
- [1033] Y. He, J. Tang, H. Ouyang, C. Kang, D. Yin, and Y. Chang, "Learning to rewrite queries," in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, 2016, pp. 1443–1452.
- [1034] J. Liu and B. Mozafari, "Query rewriting via large language models," *arXiv preprint arXiv:2403.09060*, 2024.
- [1035] F. Ye, M. Fang, S. Li, and E. Yilmaz, "Enhancing conversational search: Large language model-aided informative query rewriting," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 5985–6006.
- [1036] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity," *arXiv preprint arXiv:2403.14403*, 2024.
- [1037] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llm-lingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 13 358–13 376.
- [1038] T. Xu, S. Wu, S. Diao, X. Liu, X. Wang, Y. Chen, and J. Gao, "Sayself: Teaching llms to express confidence with self-reflective rationales," *arXiv preprint arXiv:2405.20974*, 2024.
- [1039] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.
- [1040] H. Luo, Y.-S. Chuang, Y. Gong, T. Zhang, Y. Kim, X. Wu, D. Fox, H. Meng, and J. Glass, "Sail: Search-augmented instruction learning," *arXiv preprint arXiv:2305.15225*, 2023.
- [1041] X. V. Lin, X. Chen, M. Chen, W. Shi, M. Lomeli, R. James, P. Rodriguez, J. Kahn, G. Szilvasy, M. Lewis et al., "Ra-dit: Retrieval-augmented dual instruction tuning," *arXiv preprint arXiv:2310.01352*, 2023.
- [1042] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang, "Retrieval augmented language model pre-training," in *International conference on machine learning*. PMLR, 2020, pp. 3929–3938.
- [1043] K. Lee, M.-W. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 6086–6096.
- [1044] J. Li, J. Chen, R. Ren, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "The dawn after the dark: An empirical study on factuality hallucination in large language models," *arXiv preprint arXiv:2401.03205*, 2024.
- [1045] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, 2023.
- [1046] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's song in the AI ocean: A survey on hallucination in large language models," *arXiv preprint arXiv:2309.01219*, 2023.
- [1047] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *NIPS*, 2015, pp. 1171–1179.
- [1048] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askeel, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish,

- K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, and E. Perez, "Towards understanding sycophancy in language models," *CoRR*, vol. abs/2310.13548, 2023.
- [1049] V. Rawte, P. Priya, S. M. T. I. Tonmoy, S. M. M. Zaman, A. P. Sheth, and A. Das, "Exploring the relationship between LLM hallucinations and prompt linguistic nuances: Readability, formality, and concreteness," *CoRR*, vol. abs/2309.11064, 2023.
- [1050] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," *CoRR*, vol. abs/2309.11495, 2023.
- [1051] P. Manakul, A. Liusie, and M. J. F. Gales, "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models," in *EMNLP*. Association for Computational Linguistics, 2023, pp. 9004–9017.
- [1052] N. Varshney, W. Yao, H. Zhang, J. Chen, and D. Yu, "A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation," *CoRR*, vol. abs/2307.03987, 2023.
- [1053] Y. Yehuda, I. Malkiel, O. Barkan, J. Weill, R. Ronen, and N. Koenigstein, "In search of truth: An interrogation approach to hallucination detection," *CoRR*, vol. abs/2403.02889, 2024.
- [1054] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "Factscore: Fine-grained atomic evaluation of factual precision in long form text generation," 2023.
- [1055] I. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu, "Factool: Factuality detection in generative AI - A tool augmented framework for multi-task and multi-domain scenarios," *CoRR*, vol. abs/2307.13528, 2023.
- [1056] X. Cheng, J. Li, W. X. Zhao, H. Zhang, F. Zhang, D. Zhang, K. Gai, and J.-R. Wen, "Small agent can also rock! empowering small language models as hallucination detector," *CoRR*, vol. abs/2406.11277, 2024.
- [1057] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, and E. Perez, "Towards understanding sycophancy in language models," in *ICLR*. OpenReview.net, 2024.
- [1058] J. W. Wei, D. Huang, Y. Lu, D. Zhou, and Q. V. Le, "Simple synthetic data reduces sycophancy in large language models," *CoRR*, vol. abs/2308.03958, 2023.
- [1059] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Y. Zhao, N. Lao, H. Lee, D. Juan, and K. Guu, "RARR: researching and revising what language models say, using language models," in *ACL (I)*. Association for Computational Linguistics, 2023, pp. 16 477–16 508.
- [1060] R. Zhao, X. Li, S. Joty, C. Qin, and L. Bing, "Verify-and-edit: A knowledge-enhanced chain-of-thought framework," in *ACL (I)*. Association for Computational Linguistics, 2023, pp. 5823–5840.
- [1061] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," *CoRR*, vol. abs/2212.10509, 2022.
- [1062] K. Li, O. Patel, F. B. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model," in *NeurIPS*, 2023.
- [1063] W. Shi, X. Han, M. Lewis, Y. Tsvetkov, L. Zettlemoyer, and S. W. Yih, "Trusting your evidence: Hallucinate less with context-aware decoding," *CoRR*, vol. abs/2305.14739, 2023.
- [1064] D. Kahneman, "Thinking, fast and slow," *Farrar, Straus and Giroux*, 2011.
- [1065] S. Wu, Z. Peng, X. Du, T. Zheng, M. Liu, J. Wu, J. Ma, Y. Li, J. Yang, W. Zhou *et al.*, "A comparative study on reasoning patterns of openai's o1 model," *arXiv preprint arXiv:2410.13639*, 2024.
- [1066] T. Zhong, Z. Liu, Y. Pan, Y. Zhang, Y. Zhou, S. Liang, Z. Wu, Y. Lyu, P. Shu, X. Yu *et al.*, "Evaluation of openai o1: Opportunities and challenges of agi," *arXiv preprint arXiv:2409.18486*, 2024.
- [1067] Y. Min, Z. Chen, J. Jiang, J. Chen, J. Deng, Y. Hu, Y. Tang, J. Wang, X. Cheng, H. Song *et al.*, "Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems," *arXiv preprint arXiv:2412.09413*, 2024.
- [1068] D. Team, "Deepseek-r1-lite-preview is now live: unleashing supercharged reasoning power," 2024.
- [1069] Q. Team, "Qwq: Reflect deeply on the boundaries of the unknown, november 2024," URL <https://qwenlm.github.io/blog/qwq-32b-preview>.
- [1070] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025.
- [1071] J. Jiang, Z. Chen, Y. Min, J. Chen, X. Cheng, J. Wang, Y. Tang, H. Sun, J. Deng, W. X. Zhao, Z. Liu, D. Yan, J. Xie, Z. Wang, and J.-R. Wen, "Enhancing llm reasoning with reward-guided tree search," 2024.
- [1072] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, "Encouraging divergent thinking in large language models through multi-agent debate," *arXiv preprint arXiv:2305.19118*, 2023.
- [1073] Y. Du, Z. Liu, Y. Li, W. X. Zhao, Y. Huo, B. Wang, W. Chen, Z. Liu, Z. Wang, and J.-R. Wen, "Virgo: A preliminary exploration on reproducing o1-like mllm," *arXiv preprint arXiv:2501.01904*, 2025.
- [1074] K. Team, "Kimi k1.5: Scaling reinforcement learning with llms," 2025. [Online]. Available: <https://arxiv.org/abs/2501.12599>
- [1075] OpenAI, "Openai's reinforcement fine-tuning research program," *OpenAI Blog*, 2024.
- [1076] Z. Zeng, Q. Cheng, Z. Yin, B. Wang, S. Li, Y. Zhou, Q. Guo, X. Huang, and X. Qiu, "Scaling of search and learning: A roadmap to reproduce o1 from reinforcement learning perspective," *arXiv preprint arXiv:2412.14135*, 2024.
- [1077] Z. Chen, Y. Min, B. Zhang, J. Chen, J. Jiang, D. Cheng, W. X. Zhao, Z. Liu, X. Miao, Y. Lu, L. Fang, Z. Wang, and J.-R. Wen, "An empirical study on eliciting and improving r1-like reasoning models," 2025.
- [1078] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu *et al.*, "Deepseek-math: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.
- [1079] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE samples, get a baseline for free!" in *Deep Reinforcement Learning Meets Structured Prediction, ICLR*

2019 Workshop, New Orleans, Louisiana, United States, May 6, 2019. OpenReview.net, 2019.

- [1080] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling llm test-time compute optimally can be more effective than scaling model parameters," 2024. [Online]. Available: <https://arxiv.org/abs/2408.03314>
- [1081] W. Kuang, B. Qian, Z. Li, D. Chen, D. Gao, X. Pan, Y. Xie, Y. Li, B. Ding, and J. Zhou, "Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning," 2023.